

Specification of
RobotFramework AIO
v. "1.0.1.16"
(extended core)

Implemented by:

Thomas Pollerspöck

Nguyen Huynh Tri Cuong

Tran Duy Ngoan

Mai Dinh Nam Son

Tran Hoang Nguyen

Holger Queckenstedt

"09.2026"

Contents

1	Introduction	1
2	Installation and start	2
3	GitHub Copilot Extension	5
3.1	Introduction	5
3.2	Proxy Configuration	5
3.3	Installation	5
3.4	Configuration	7
4	Components	9
5	Environment	12
5.1	Paths and Variables for Framework Setup	12
5.2	Proxy configuration	13
6	Code analysis	14
7	Logging	16
7.1	Background	16
7.2	Summary	16
8	Test result 'UNKNOWN'	17
9	Threading in Robot Framework	20
9.1	Background	20
9.2	Threading Implementation	20
9.3	Threading Keywords	20
9.3.1	Send Thread Notification Keyword	20
9.3.2	Wait Thread Notification Keyword	21
9.3.3	Thread RLock Acquire Keyword	21
9.3.4	Thread RLock Release Keyword	22
9.4	Summary	22
10	Library documentation	23
10.1	GenPackageDoc	26
10.2	PythonExtensionsCollection	70
10.3	RobotframeworkExtensions	110
10.4	JsonPreprocessor	126
10.5	RobotFramework_TestsuitesManagement	166

10.6	QConnectBase	233
10.7	QConnectWinapp	301
10.8	RobotFramework_DBus	320
10.9	PrometheusInterface	362
10.10	RobotFramework_UDS	377
10.11	RobotLog2RQM	416
10.12	TestResultDBAccess	456
10.13	RobotLog2DB	492
10.14	RobotFramework_DoIP	517
10.15	PyTestLog2DB	540
10.16	TestResultWebApp	561
10.17	VSCodeWelcome	581
10.18	VSCodeJsonp	588
11	Appendix	594
11.1	Installed Python Modules	594

Chapter 1

Introduction

The Robot Framework is an automation framework that can be used to execute automated software tests. It's open source, keyword driven, and extensible. Tests are implemented in simple text files.

The extensibility of the Robot Framework gives every test developer the possibility to add missing functionality easily. But when thinking about missing functionality, some aspects urgently need to be considered:

- Additions should be designed carefully to make them as much generic as possible. This will make the additions useful for a wide range of test developers.
- Additions have to be bundled in a meaningful way, and they have to be set under version control.
- Tests need to be reproduced, and therefore also the entire test system (Robot Framework together with all additions) must be reproducible.

And these deliberations are not only related to the Robot Framework itself. Developing software in an efficient way also requires a development environment. And this development environment most probably needs to be configured - at least to be able to handle the additions also. It has to be ensured to use open source software only, to avoid any license fee. And finally it must be ensured that every test developer within a team of developers uses the same development environment with the same settings.

To cover all aspects mentioned above, the following things have been done:

- Download a certain Python version (because the Robot Framework is a Python based application).
- Add further useful Python modules to the Python installation.
- Download a certain version of the Robot Framework. This is the base.
- Implement (carefully) some changes within the core source code of the Robot Framework.
- Add further components to provide useful features that are currently missing in Robot Framework (like a management of test configuration values and a version control mechanism).
- Select a development environment (here: VSCodium).
- Configure this development environment to handle especially the Robot Framework code together with all additions.
- Put all things together in a separate installer.

The outcome is one single installer, that installs all together: the framework, additional libraries, the development environment and all settings.

This is what we call "**RobotFramework AIO**", with **AIO** means: **All In One**.

With this solution the entire test setup is reproducible by every developer on every computer.

And there is no need any more for any user, to install all affected components manually, and there is also no need any more for any user, to spend effort on the configuration of the test setup. After using the RobotFramework AIO installer, test developers immediately can start to implement tests.

Chapter 2

Installation and start

The **RobotFramework AIO** can be installed under Windows and under Linux in this way:

- **Win10**

Double-click the `setup_*.exe` file and follow the instructions.

- **Linux**

If already installed, remove the previous version (this will not remove your own data):

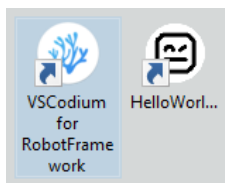
```
sudo apt-get remove robotframework-aio
```

Install/Update now the current version (assumed you are in the download directory):

```
sudo apt-get install ./setup_RobotFramework_AIO_OSD6Linux\/*.deb
```

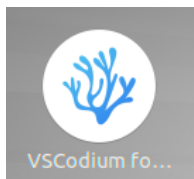
After the installation is finished, a new **VSCodium** icon is present on desktop:

- **Win10**



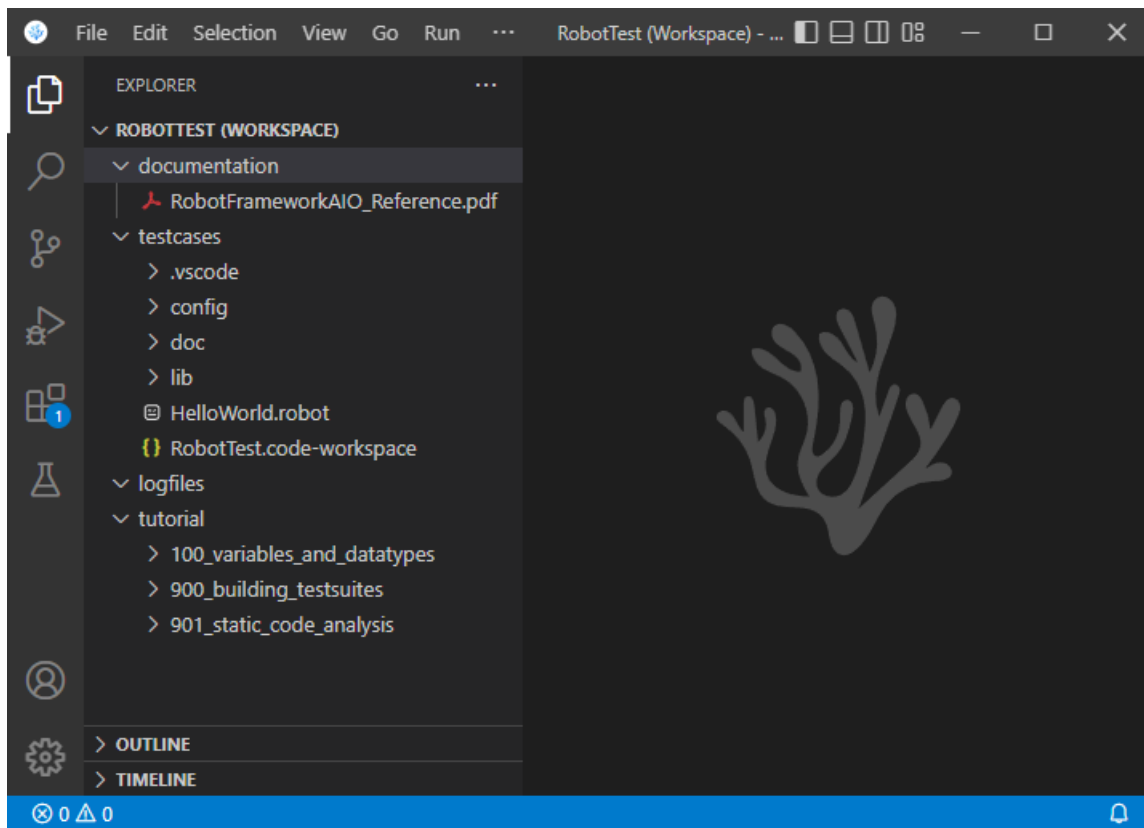
(together with a link to the example robot file `HelloWorld.robot`)

- **Linux**



A double-click starts the application.

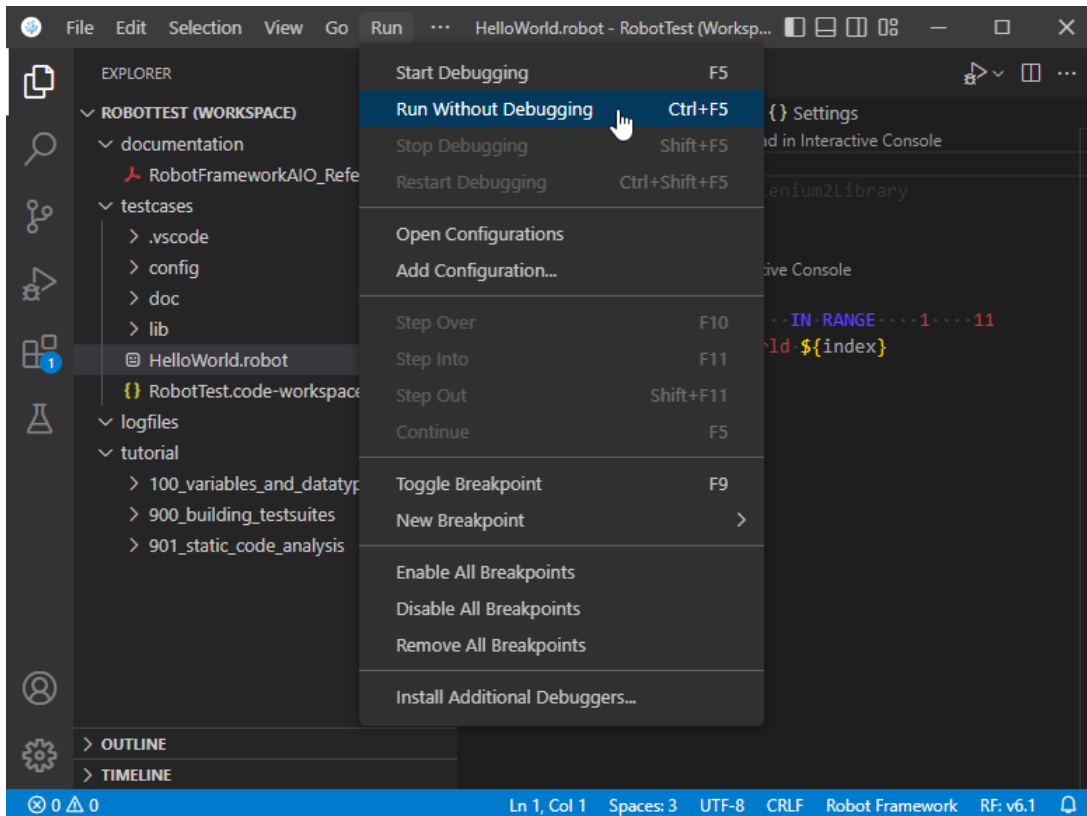
On first start the **VSCodium** main window appears like this:



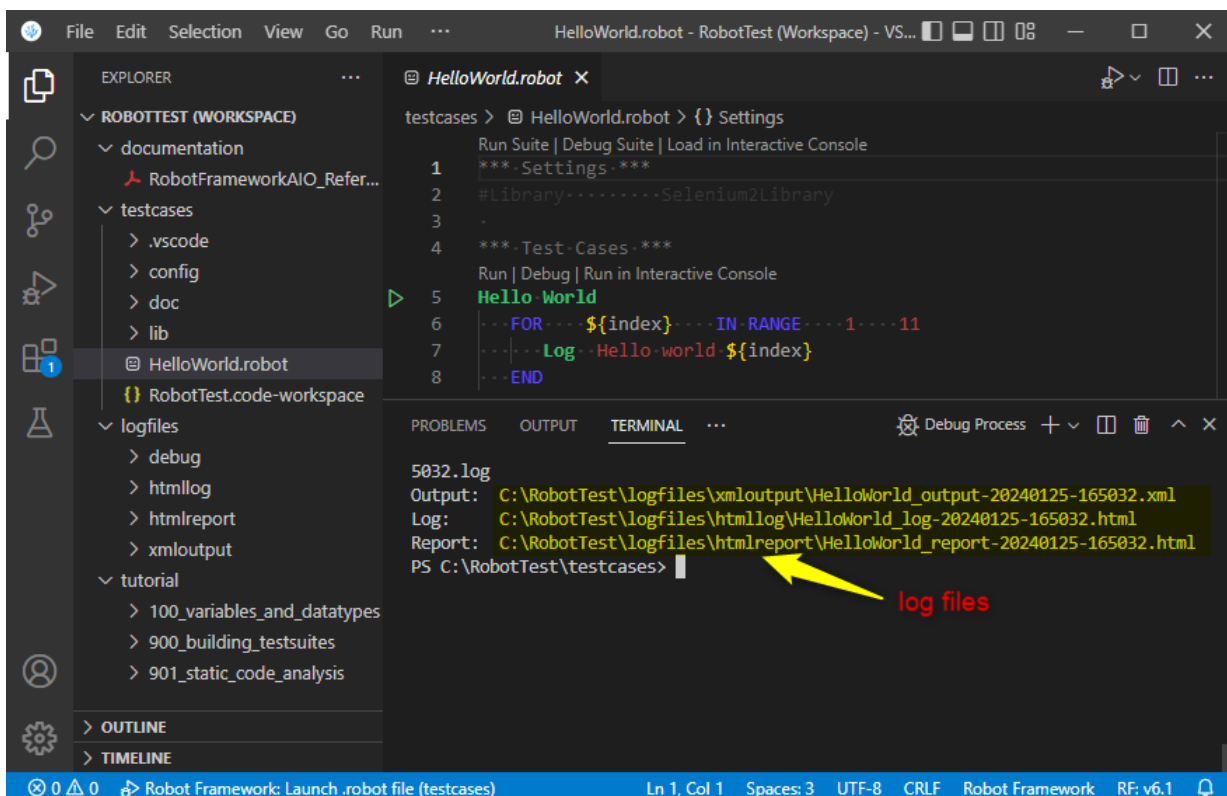
The preconfigured workspace contains the following parts:

- The main documentation `RobotFrameworkAIO_Reference.pdf` (including the documentation of all components that are part of the **RobotFramework AIO**)
- A `testcases` folder containing (beneath other elements) the example robot file `HelloWorld.robot`
- A `logfiles` folder in which all log files and reports will be placed (from tests that are executed with **VSCodium**)
- A `tutorial` folder containing a large bunch of testfiles that can be used for self-dependent learning

For the usage of the predefined infrastructure (e.g. the `logfiles` folder), the **VSCodium** provides an adapted configuration of the **Run** button. This button executes the robot file selected in project explorer of **VSCodium**.



After test execution, the links to log files and reports can be found in the **TERMINAL** window.



Every file there can be opened by CTRL+click on the selected file.

Chapter 3

GitHub Copilot Extension

3.1 Introduction

GitHub Copilot is an AI-powered code completion tool that helps you write code faster by suggesting whole lines or blocks of code as you type.

3.2 Proxy Configuration

GitHub Copilot requires internet access to function properly.

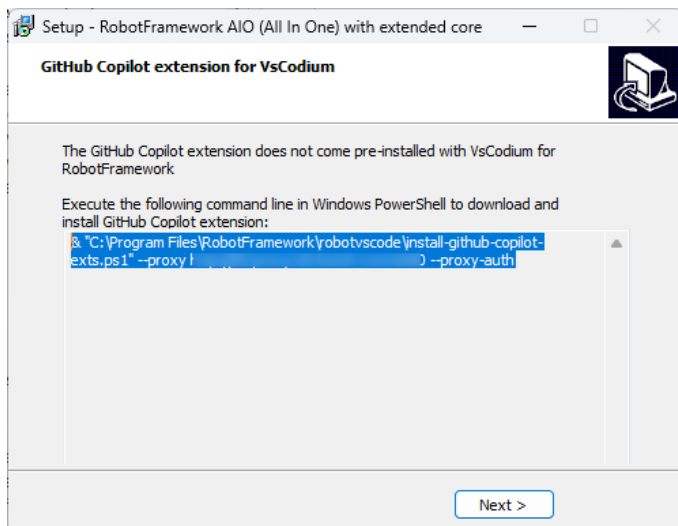
If you are behind a proxy, you need to configure the proxy settings for GitHub Copilot to ensure it can connect to the internet.

Details about how to do this can be found [here](#).

3.3 Installation

Due to licensing restrictions, GitHub Copilot Extension cannot be pre-installed in the **Robot Framework AIO** package.

You need to install it manually by running the script that is offered during the RobotFramework AIO post-installation step, as shown below:



In case you missed above instruction during the setup, you can still install the GitHub Copilot extension later by following these steps:

Open a terminal (**Windows PowerShell** for Windows or Linux Terminal) and run the appropriate script to install the GitHub Copilot extension in your **VSCodium For RobotFramework**.

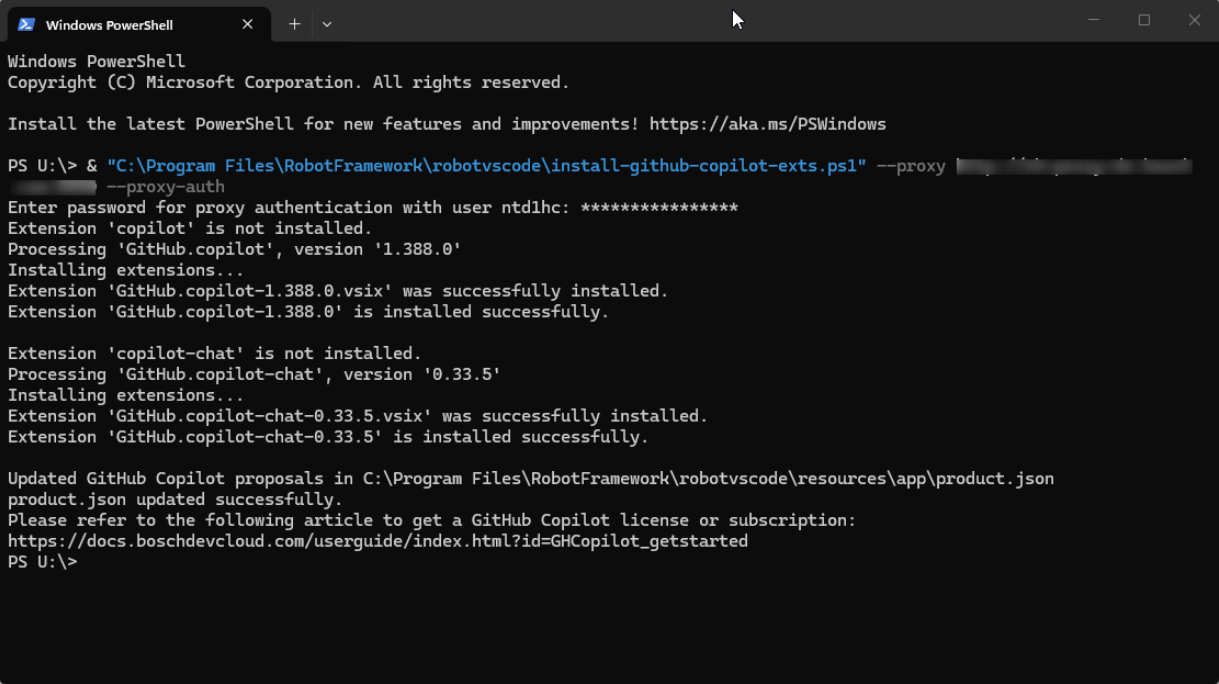
- Windows PowerShell:

```
& "$env:RobotVsCode\install-github-copilot-exts.ps1" --proxy <your-proxy-address:port> --proxy-auth
```

- Linux Terminal:

```
$RobotVsCode/install-github-copilot-exts.sh --proxy <your-proxy-address:port> --proxy-auth
```

The installation script will download and install the GitHub Copilot extension in your **VSCodium For RobotFramework** environment as below:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS U:\> & "C:\Program Files\RobotFramework\robotvscode\install-github-copilot-exts.ps1" --proxy <your-proxy-address:port> --proxy-auth
Enter password for proxy authentication with user ntdlhc: *****
Extension 'copilot' is not installed.
Processing 'GitHub.copilot', version '1.388.0'
Installing extensions...
Extension 'GitHub.copilot-1.388.0.vsix' was successfully installed.
Extension 'GitHub.copilot-1.388.0' is installed successfully.

Extension 'copilot-chat' is not installed.
Processing 'GitHub.copilot-chat', version '0.33.5'
Installing extensions...
Extension 'GitHub.copilot-chat-0.33.5.vsix' was successfully installed.
Extension 'GitHub.copilot-chat-0.33.5' is installed successfully.

Updated GitHub Copilot proposals in C:\Program Files\RobotFramework\robotvscode\resources\app\product.json
product.json updated successfully.
Please refer to the following article to get a GitHub Copilot license or subscription:
https://docs.boschdevcloud.com/userguide/index.html?id=GHCopilot_getstarted
PS U:\>
```

3.4 Configuration

Before using GitHub Copilot, you need to sign in with your GitHub account and enable the extension. To do this, follow these steps:

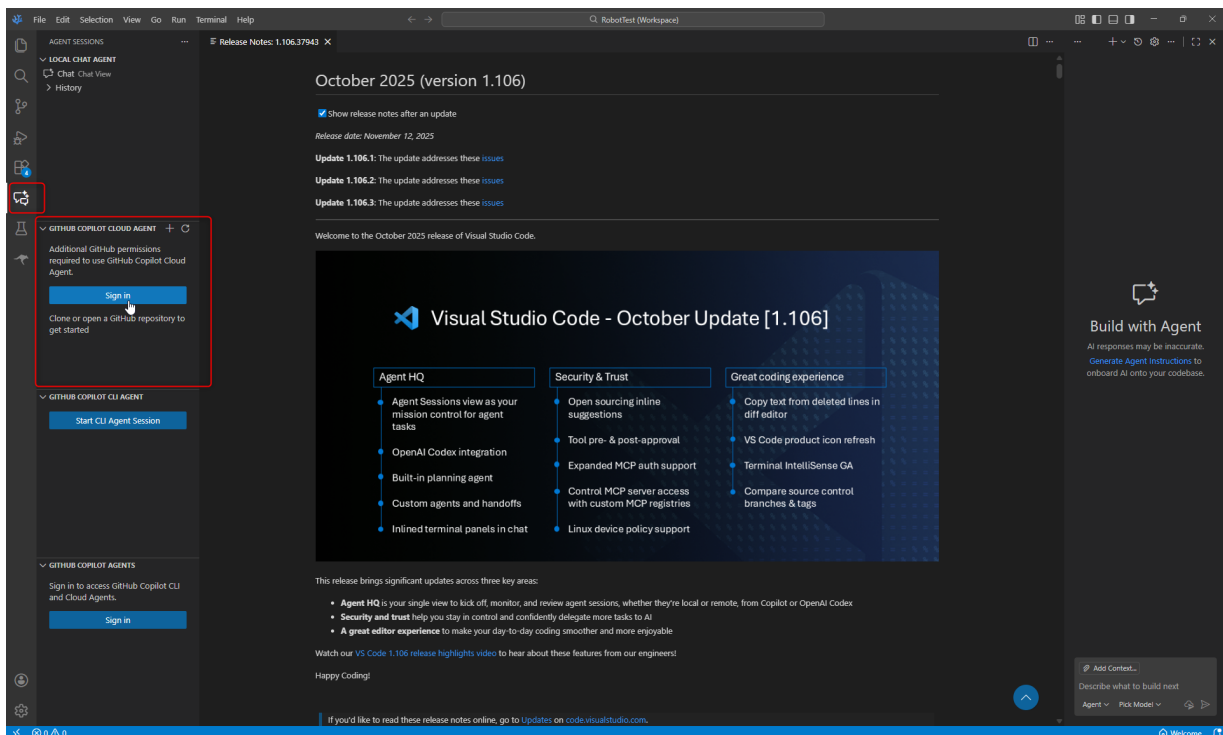
- Click on the **GitHub Copilot** icon in the Activity Bar.
- Then Click on the **Sign in** button of **GitHub Copilot Cloud Agent** section as below:

Warning

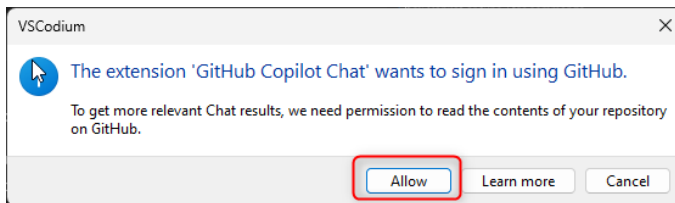


The **Sign in** button of **GitHub Copilot Cloud Agent** may not be visible in some cases due to limited vertical space, as the **GitHub Copilot CLI Agent** may overlap and hide the **Sign in** button.

In this case, scroll down within the **GitHub Copilot Cloud Agent** panel or resize the panel to reveal the **Sign in** button.



- The pop-up will appear to ask the permission for **GitHub Copilot Chat** extension.

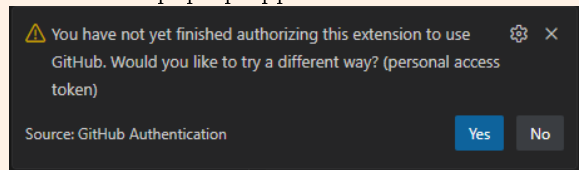


Click **Allow** to move to next step.

Warning



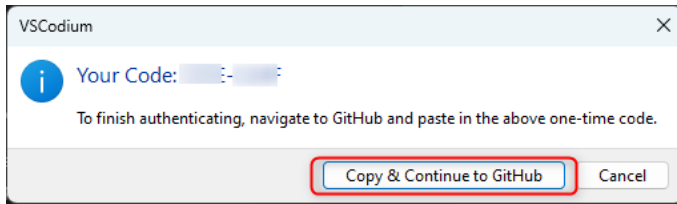
If the below pop-up appears instead:



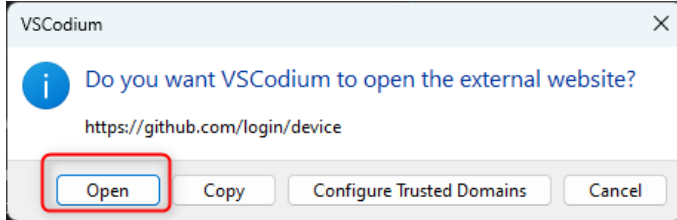
It may indicate that the internet access is not available for GitHub Copilot extension, please check your network connection and proxy settings if applicable.

Details about how to configure proxy for **VsCodium For RobotFramework** can be found [here](#).

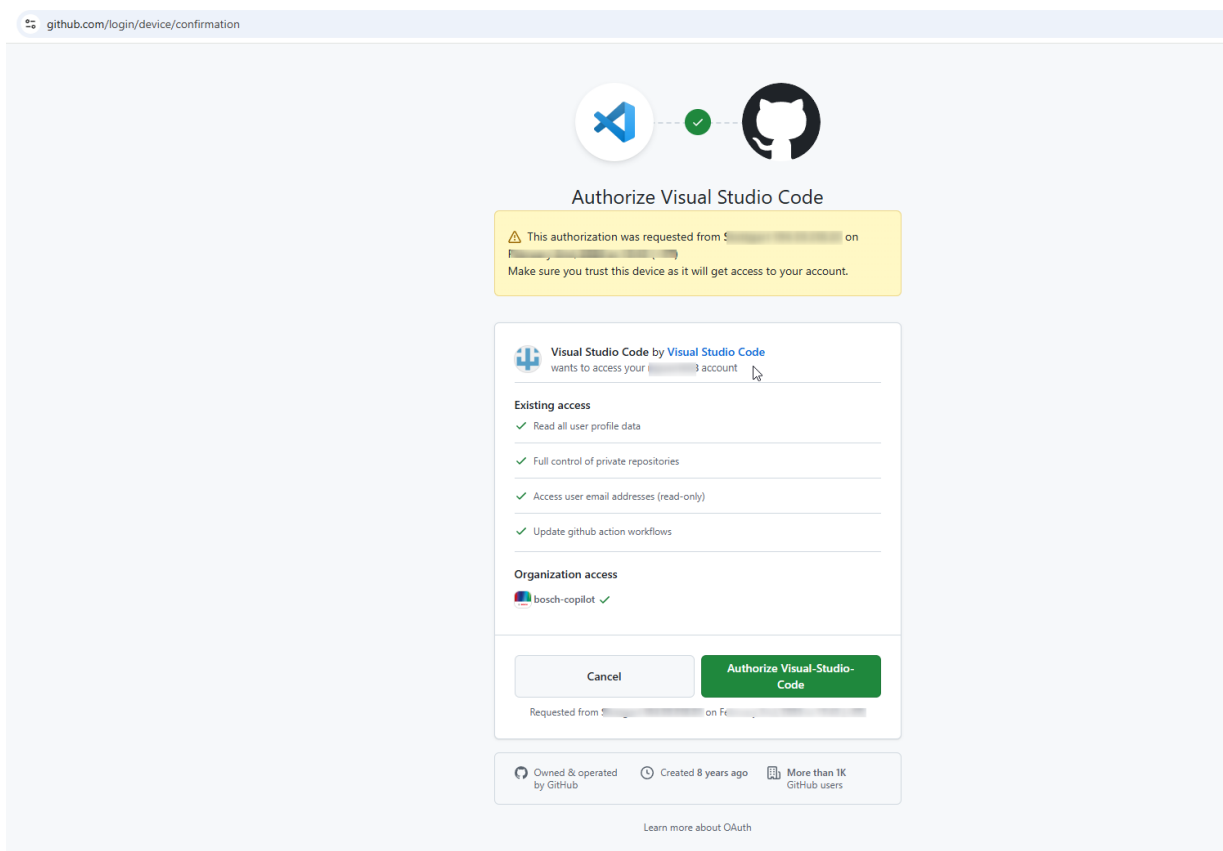
- The authentication code will be displayed in the pop-up window as below:



Click **Copy & Continue to Github** button then click **Open** the GitHub sign-in page in your web browser.



- In the GitHub sign-in page, paste the authentication code and click **Continue** button.
- Then, the last confirmation page will appear as below:



Click **Authorize Visual Studio Code** button to complete the sign-in process.

- Now back to **VSCodium For RobotFramework**, the GitHub Copilot extension is enabled and ready to use.

With GitHub Copilot enabled, you can enhance your coding experience and boost your productivity in **VSCodium For RobotFramework**.

Chapter 4

Components

Compared with the core Robot Framework, the RobotFramework AIO contains some additional components, that extend the functionality by useful features. All components are hosted on GitHub. Most of the components can also be used stand-alone (= outside the context of RobotFramework AIO). But in case of a stand-alone usage, the user is responsible for the installation and configuration. Further hints can be found in the readme files of the corresponding GitHub repository.

The following is an overview about all additional components, that are part of the RobotFramework AIO:

1. RobotFramework_TestsuitesManagement

The **RobotFramework_TestsuitesManagement** enables users to define dynamic configuration values within separate configuration files in JSON format. These configuration values are available during test execution - but under certain conditions that can be defined by the user (e.g. to realize a variant handling or to define parameters that are test bench specific).

The **RobotFramework_TestsuitesManagement** also provides a version control mechanism to ensure that the developed tests fit to the test environment.

Homepage: [RobotFramework_TestsuitesManagement](#)

2. JsonPreprocessor

The **JsonPreprocessor** provides additional features in JSON files. These features extend the standard JSON format and make it easier to use JSON files for e.g. configuring tests (see also **RobotFramework_TestsuitesManagement**). The additional features are:

- Add comments
- Let a JSON file import other JSON files (nested imports; e.g. to avoid redundancy)
- Allow also pure Python keywords like `True`, `False` and `None`
- Define new parameters and overwrite already existing ones

Homepage: [JsonPreprocessor](#)

3. QConnectBase

QConnectBase is a connection testing library for the Robot Framework. It provides a mechanism to handle trace log continuously receiving from a connection (such as Raw TCP, SSH, Serial, etc.) besides sending data back to the other side. It's especially efficient for monitoring the overflowed response trace log from asynchronous trace systems.

Homepage: [QConnectBase](#)

4. RobotLog2RQM

RobotLog2RQM writes Robot Framework test results (available in XML format) to the IBM® Rational® Quality Manager (RQM). This includes:

- Create all required resources (Test Case Execution Record, Test Case Execution Result, ...) for new testcase on RQM
- Link all testcases to provided testplan

- Add new test result for existing testcase on RQM
- Update existing testcase on RQM

Homepage: [RobotLog2RQM](#)

5. RobotLog2DB

RobotLog2DB writes Robot Framework test results (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: [RobotLog2DB](#)

6. PyTestLog2DB

PyTestLog2DB writes test results of the Python module pytest (available in XML format) to a database. This database is provided by another component called **TestResultWebApp**. Based on the content of the database the **TestResultWebApp** presents an overview about the whole test execution and details about each test result.

Homepage: [PyTestLog2DB](#)

7. TestResultWebApp

TestResultWebApp is a web-based application and is used for visualizing and tracking test execution results. It provides charts from an overview of the test result to the detail of all included test cases. It also provides tools to control the quality of developing software version (under testing) by the a graphical comparison of test results from different test executions

Homepage: [TestResultWebApp](#)

8. GenPackageDoc

GenPackageDoc provides a toolchain to generate a documentation in PDF format out of Python sources that are stored within a repository. The content of this documentation is taken out of the docstrings of functions, classes and their methods. All docstrings have to be written in reStructuredText (RST) format, that is a certain markdown dialect. It is possible to extend the documentation by the content of additional files either in RST format or in LaTeX format. **GenPackageDoc** is also designed to consider setup informations of a repository.

Homepage: [GenPackageDoc](#)

9. PythonExtensionsCollection

The **PythonExtensionsCollection** extends the functionality of Python by some useful functions that are not available in Python immediately. This covers for example file and folder operations, string operations like normalizing a path and a pretty print method. Also a file comparison feature is available.

Homepage: [PythonExtensionsCollection](#)

10. RobotframeworkExtensions

The **RobotframeworkExtensions** extend the functionality of the Robot Framework by some useful keywords. This covers for example string operations like normalizing a path and a pretty print keyword (especially for composite Python data types). The **RobotframeworkExtensions** keywords are implemented in Python (see **PythonExtensionsCollection**).

Homepage: [RobotframeworkExtensions](#)

11. Tutorial

The RobotFramework AIO tutorial contains robot code examples together with the documentation explaining how to use these examples and how they work. The basic idea behind this tutorial is not to have theory only but additionally to this theory also the possibility to experience what is explained inside. The main focus of this tutorial is on some feature extensions available for the Robot Framework.

Homepage: [Tutorial](#)

12. Development environment

The RobotFramework AIO installer also installs VSCodium, that is an open source editor. VSCodium is preconfigured especially for the usage together with the RobotFramework AIO.

Features:

- Workspace with included testcases folder, documentation and tutorial
- Previews for certain formats like, MD, RST, PDF, PlantUML
- Support of the extended JSON syntax of **RobotFramework__TestsuitesManagement**
- Static code analysis by Robocop, that is an also included static code checker

Notice



If you are working behind a company proxy, you need to configure the proxy settings in VSCodium to access the extension marketplace. Details about how to do this can be found [here](#).

Chapter 5

Environment

5.1 Paths and Variables for Framework Setup

During an installation of the RobotFramework AIO all content is placed at two different main locations:

- `RobotFramework`
Contains the framework installation files itself
- `RobotTest`
Contains the *playground* for the user, including default folder for testcases and logfiles

The root path to these folders can be set by the user during the installation process (called `<root>` in listings below).

The following overview contains a list of environment variables that are introduced by the installer. They can be used to refer to locations inside these folders.

- `RobotLogPath`
Log files folder in which the RobotFramework AIO places the log files per default.
Location: `<root>/RobotTest/logfiles`
- `RobotTestPath`
Folder in which the user can place the testcases.
Location: `<root>/RobotTest/testcases`
- `RobotTutorialPath`
Folder containing a tutorial of the RobotFramework AIO (useful for self learning).
Location: `<root>/RobotTest/tutorial`
- `RobotPythonPath`
Folder containing the Python installation.
Location: `<root>/RobotFramework/python39`
- `RobotScriptPath`
Folder containing scripts of the Python installation.
Location: `<root>/RobotFramework/python39/scripts`
- `RobotVsCode`
Folder containing the installation of *Visual Studio Code* that is intended to be the main development environment for the RobotFramework AIO.
Location: `<root>/RobotFramework/robotvscode`
- `RobotToolsPath`
Tools which require and extend functionality of the Robot Framework.
Location: `<root>/RobotFramework/tools`
- `RobotDevtools`
Tools which are independent of Robot Framework, but are bundled and tested with RobotFramework AIO.
Location: `<root>/RobotFramework/devtools`

- **RobotAppium**
Location of Appium tested and distributed with RobotFramework AIO. More information about Appium you find here: [appium](#)
Location: `<root>/RobotFramework/devtools/Windows/Appium`
- **RobotNodeJS**
Root folder of `nodjs` tested and distributed with RobotFramework AIO. More information about `nodjs` you find here: [nodejs.org](#)
Location: `<root>/RobotFramework/devtools/Windows/nodejs`

5.2 Proxy configuration

To configure the proxy settings in VSCodium, follow these steps:

1. Open the `settings.json` file in VSCodium. You can do this by:
 - Pressing `Ctrl + ,` (or `Cmd + ,` on macOS) to open the settings UI.
 - Clicking on the **Open Settings (JSON)** icon in the top-right corner of the settings UI.
2. Locate the `http.proxy` setting. If it does not exist, add the following entry with your company's proxy information:

```
"http.proxy": "http://your-proxy-address:port"
```

3. If your proxy requires authentication, include the username and password in the URL:

```
"http.proxy": "http://username:password@your-proxy-address:port"
```

4. Save the changes to the `settings.json` file.
5. Restart VSCodium to apply the changes.

Chapter 6

Code analysis

The RobotFramework AIO installation provides a static code analyser to detect potential errors and violations to coding conventions. The name of the analyser is **Robocop**.

Robocop is integrated in Visual Studio Code and is triggered automatically in case of

1. a file is opened in editor,
2. a file is changed and saved.

The outcome will look like this:

```

suite02.robot 3 X
robotframework-tutorial > 901_static_code_analysis > suite02.robot > ...
31 Test Case 0201
32     ...[Documentation]...Documentation of test case
33     ...Log...I am test '${TEST_NAME}' of suite '${SUITE NAME}'...console=yes
34     ...#TODO::Add return value to keyword implementation; check return value here
35     ...test_keyword_1
36
37 Test Case 0201
38     ...[Documentation]...Documentation of test case
39     ...Log...I am test '${TEST_NAME}' of suite '${SUITE NAME}'...console=yes
40
41
42
43

```

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL Filter (e.g. text, **/*.ts, !**/node_modules/**)

- ✖ suite02.robot robotframework-tutorial • 901_static_code_analysis 3
 - Multiple test cases with name 'Test Case 0201' (first occurrence in line 31) robocop(0801) [37, 1]
 - ⚠ Found TODO in comment robocop(0701) [34, 7]
 - ⚠ Too many blank lines at the end of file robocop(1010) [43, 1]

Further examples can be found in the RobotFramework AIO tutorial, chapter `901_static_code_analysis`.

How to configure?

Robocop contains a set of rules that are used to check the source files of the RobotFramework AIO. Not all of them are really useful and some of them are also against our company internal development rules. Therefore we have the need to exclude rules from Robocop execution.

Some rules can be configured (e.g. the rule checking the maximum number of characters within a line of code). Also here we have the need for careful adaptations.

When Robocop is triggered automatically within Visual Studio Code, then the configuration is done with the help of a `.robocop` file. This file has to be placed within the projects folder (see tutorial). To make Robocop results comparable this configuration file should not be modified - or at least should be kept aligned with the version all other project team members use.

Command line

In addition to the automated triggering within Visual Studio Code Robocop can also be called in command line. This is useful

1. in case of a complete folder has to be checked in one step (to avoid the need to open every single source file inside separately),
2. in case of an alternative configuration is wanted (temporarily; to avoid to manipulate the standard configuration file),
3. in case of Robocop shall be triggered by automats like Jenkins,
4. in case of the Robocop results are needed within a log file (output to a log file needs to be configured separately).

To give it a try make a copy of the configuration file `.robocop` available within the tutorial, and give this copy any other name (e.g. `robocop.arg`).

Now Robocop can be called with command lines like this:

1.) *Simple check of single file with default configuration:*

```
"%RobotPythonPath%/python.exe" -m robocop "<path>/mytestfile.robot"
```

2.) *Check of single file with individual configuration taken from argument file:*

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "<path>/robocop.arg" ↔  
↔ "<path>/mytestfile.robot"
```

3.) *Check of entire folder with individual configuration taken from argument file:*

```
"%RobotPythonPath%/python.exe" -m robocop --argumentfile "<path>/robocop.arg" "<folder>"
```

How to activate?

In Visual Studio Code the automated triggering of Robocop is activated per default within

```
RobotFramework\robotvscode\data\user-data\User\settings.json
```

with the following switches set to `true` :

```
"robotcode.robocop.enabled": true,
```

Chapter 7

Logging

7.1 Background

The Robot Framework provides several log levels (also known as trace levels) to give users the possibility to control the amount of output in log files. The build in keyword `log` writes content to the log files. The optional parameter `level` of this keyword defines under which conditions (under which log level) the `log` keyword shall write the log message.

With command line parameter `--loglevel` a user defines this condition. If the log level of an executed `log` keyword matches to the log level defined for test execution, the log message is logged, otherwise not.

Available log levels in Robot Framework are: `ERROR`, `WARN`, `INFO`, `DEBUG` and `TRACE`. The default log level in Robot Framework is `INFO`. This level becomes active, if nothing else is specified by a user.

The log level `INFO` is also used by the Robot Framework itself, e.g. for the logging of informations about executed keywords. The impact is that every log file contains both the users output and the Robot Framework output. For larger tests this might cause unfavorable large log files.

Therefore the RobotFramework AIO provides an additional log level `USER`. Tests that are executed under this log level, do not contain the Robot Framework `INFO` messages any more (but for sure still contain errors and warnings).

Example code:

```
log    my log message    USER
```

Example execution:

```
python -m robot --loglevel USER -b logfile.log ./testfile.robot
```

7.2 Summary

In opposite to the Robot Framework (core), the RobotFramework AIO provides the following log levels to control the amount of logged content:

		log level in log keyword					
		ERROR	WARN	USER	INFO	DEBUG	TRACE
log level in command line --loglevel	ERROR	logged	-----	-----	-----	-----	-----
	WARN	logged	logged	-----	-----	-----	-----
	USER	logged	logged	logged	-----	-----	-----
	INFO	logged	logged	logged	logged	-----	-----
	DEBUG	logged	logged	logged	logged	logged	-----
	TRACE	logged	logged	logged	logged	logged	logged

The log level `USER` has been added to give users the possibility to log own content without all `INFO` messages logged by the Robot Framework (core).

Chapter 8

Test result 'UNKNOWN'

The usual test results `PASS` and `FAIL` are statements about the *system under test*. The precondition for such test results is a proper test execution. For some reasons this is not possible:

1. The preconditions of a test cannot be fulfilled.
2. Libraries that cannot be loaded.
3. Errors in the code of the test itself.
4. Broken connection to the *system under test*.

The impact in this case is: A conclusive statement about the *system under test* is not possible. The test itself is the one who has the issue. This is announced by an additional test result `UNKNOWN`. This enhances the expressiveness and accuracy of Robot Framework's test results by clearly indicating, that an error prevented the test.

This additional precision will not only help clarify the nature of issues encountered but also prevent unnecessary analyzing and debugging of features that weren't even tested due to unmet preconditions. Finally it brings clarity and saves time and money.

The test result `UNKNOWN` can be caused by the following events:

Syntax error in robot code (*typing mistake*):

```
ExampleTest
    Lo**g    ${PORT}
```

The result in console is:

```
ExampleTest                                     | UNKNOWN |
No keyword with name 'Lo**g' found. Did you mean:
    BuiltIn.Log
```

Syntax error in keyword library (*missing bracket*):

```
@keyword
def went_wrong(self):
    BuiltIn().log(f"This went wrong", "INFO"
```

Additionally to unexpected events, the test result `UNKNOWN` can also be forced explicitly.

In robot code by keyword `UNKNOWN`:

```
ExampleTest
    UNKNOWN
```


It's possible to add a message:

```
ExampleTest
```

```
    UNKNOWN    "This went wrong"
```

The result in console is:

```
ExampleTest          | UNKNOWN |  
"This went wrong"
```

Raise an exception in keyword library:

```
@keyword  
def went_wrong(self):  
    raise Exception("This went wrong")
```

Raised exceptions within Python code are mapped to test result `UNKNOWN` internally.

Chapter 9

Threading in Robot Framework

9.1 Background

The Robot Framework core (Robot Framework) now includes advanced threading features. This enhancement enables the simulation of concurrent processes and interactions within test environments.

9.2 Threading Implementation

The threading functionality in Robot Framework introduces the `THREAD` keyword, which allows for the execution of parallel processes within tests. Here are some key aspects of how `THREAD` works:

Keyword Usage: The `THREAD` keyword functions similarly to `FOR`, `WHILE`, or `TRY` keywords. It can be utilized within a test case or included in a User Defined Keyword. This flexibility allows for a wide range of multi-threaded test scenarios.

Immediate Thread Start: A thread starts executing immediately after its declaration. This means that as soon as the line `THREAD ThreadNameIsDaemon` is executed, the thread begins its operation. This immediate start is crucial for real-time testing and synchronization.

Variable Scope in Threads: Each thread has its own variable scope. Variables declared or modified within a thread do not affect the variables in other threads or the main test flow. This separation ensures that threads operate independently and reduces the risk of data conflicts.

Threading Syntax:

```
THREAD    ${ThreadName}    ${IsDaemon}
          [Thread's actions and keywords]
END
```

This syntax is used to define a thread in Robot Framework tests. Here's a breakdown of the parameters:

`${ThreadName}` : This is a required parameter that specifies the name of the thread. It's used for identification and logging purposes. The thread name should be unique within the context of the test suite to avoid confusion. Thread name is case-sensitive.

`${IsDaemon}` : This is an optional parameter that determines the nature of the thread. When set to true, the thread is treated as a daemon thread. Daemon threads are typically background tasks that exit when all non-daemon threads have completed. If not specified, the thread is considered a non-daemon thread by default.

The use of named threads and the option to set them as daemon threads allow for precise control over the threading behavior in test cases, facilitating complex test scenarios.

9.3 Threading Keywords

9.3.1 Send Thread Notification Keyword

This keyword is utilized to send a notification from one thread to another, either to a specific thread or broadcasted to all threads.

Syntax:

```
Send Thread Notification    notification_name    params=None    dst_thread=None
```

Parameters:

`notification_name` : Mandatory. The unique name of the notification to be sent. This name is used by receiving threads to identify the notification.

`params` : Optional. The payload of the notification. It can be a string or any structured data and is passed to the receiving thread along with the notification. Default is None, which means no payload is sent.

`dst_thread` : Optional. The specific target thread to which the notification should be sent. If not specified or set to None, the notification is broadcasted to all threads.

9.3.2 Wait Thread Notification Keyword

Allows a thread to wait for a specific notification, providing a mechanism for synchronization and response to inter-thread communication.

Syntax:

```
Wait Thread Notification    notification_name    condition=None    timeout=5
```

Parameters:

`notification_name` : Mandatory. The name of the notification the thread is waiting for. This should match the name given in `Send Thread Notification`.

`condition` : Optional. This parameter allows specifying a Python expression that acts as a condition the notification's payload must meet for the notification to be acknowledged. The condition is provided as a Python expression and evaluated at runtime. In this context, the variable `payloads` refers to the payload of the received notification, and it can be used within the condition to apply filters or checks based on the content of the notification.

For example, setting `condition=$payloads=='test'` means that the thread will continue execution only if it receives a notification named **notification_name** whose payload is equal to the string "test". This allows for selective synchronization based on the content of notifications.

Example:

In this example, the thread waits for a notification named "DataUpdate" and proceeds only if the payload of this notification is "test".

```
Wait Thread Notification    DataUpdate    condition=$payloads=='test'    timeout=10
```

`timeout` : Optional. The maximum time in seconds to wait for the notification. If the notification is not received within this period, the keyword will fail. Default is 5 seconds.

9.3.3 Thread RLock Acquire Keyword

This keyword allows a thread to acquire a re-entrant lock (RLock) with a specified name, providing a mechanism for thread synchronization with optional blocking and timeout features.

Syntax:

```
Thread RLock Acquire    rlock_name    blocking=True    timeout=-1
```

Parameters:

`rlock_name` : Mandatory. The name of the re-entrant lock to be acquired.

`blocking` : Optional. Determines whether the thread should wait for the lock to become available if it's currently held by another thread. Default is **True**, meaning the thread will wait.

`timeout` : Optional. The maximum time in seconds to wait for the lock if blocking is set to **True**. The default value is **-1**, meaning the thread will wait indefinitely.

This keyword will fail if the lock could not be acquired within the specified timeout or if **blocking** is set to **False** and the lock is already held by another thread.

Examples:

```
Thread RLock Acquire    MyLock
Thread RLock Acquire    MyLock    blocking=False
Thread RLock Acquire    MyLock    blocking=True    timeout=5
```

9.3.4 Thread RLock Release Keyword

This keyword releases a previously acquired re-entrant lock (RLock) specified by the given lock name.

Syntax:

```
Thread RLock Release    rlock_name
```

Parameters:

`rlock_name` : Mandatory. The name of the re-entrant lock to be released.

The lock must have been acquired by the same thread attempting to release it. Releasing a lock not held by the current thread will result in an error.

Examples:

```
Thread RLock Release    MyLock
```

This keyword will fail if the lock cannot be released, such as when the current thread did not acquire the lock or if the lock was never acquired in the first place.

9.4 Summary

With the introduction of threading and the new keywords

```
Send Thread Notification
```

and

```
Wait Thread Notification
```

RobotFramework AIO (Robot Framework) now offers advanced multi-threaded testing capabilities. These enhancements allow for more sophisticated, parallel testing scenarios and improve synchronization and communication between threads.

In addition, the keywords

```
Thread RLock Acquire
```

and

```
Thread RLock Release
```

provide powerful mechanisms for controlling shared resources between threads using re-entrant locks (RLocks). These keywords enable precise control over concurrent access to resources, ensuring that test cases can safely handle critical sections without risking race conditions or data conflicts.

Together, these new threading features give users the ability to design complex, real-world testing scenarios involving multiple threads, inter-thread communication, and synchronization. This brings greater flexibility and robustness to automated testing in RobotFramework AIO, making it well-suited for environments where concurrency and parallel execution are required.

Chapter 10

Library documentation

The following sections contain the documentation of additional libraries that are part of the RobotFramework AIO.

RobotFramework AIO bundle

Version "1.0.1.16" (from "09.2026")

Underlying Robot Framework (core)

Robot Framework 7.4.2 (Python 3.13.2 on linux)
--

Included libraries

GenPackageDoc

Version 0.44.0 (from 05.06.2026)

https://github.com/test-fullautomation/python-genpackagedoc

<i>Documentation builder for Python packages</i>
--

PythonExtensionsCollection

Version 0.17.1 (from 22.06.2026)

https://github.com/test-fullautomation/python-extensions-collection

<i>Additional Python functions</i>

robotframework-extensions-collection

Version 0.12.0 (from 17.02.2026)

https://github.com/test-fullautomation/robotframework-extensions-collection

<i>Additional Robot Framework keywords</i>
--

JsonPreprocessor

Version 0.12.0 (from 02.02.2026)

https://github.com/test-fullautomation/python-jsonpreprocessor

<i>Preprocessor for JSON files with extended syntax</i>

RobotFramework _TestsuitesManagement

Version 0.11.0 (from 27.02.2026)

<https://github.com/test-fullautomation/robotframework-testsuitesmanagement>*Functionality to manage Robot Framework testsuites***robotframework-qconnect-base**

Version 1.4.0 (from 11.02.2026)

<https://github.com/test-fullautomation/robotframework-qconnect-base>*Robot Framework library for TCP, SSH, serial connection***robotframework-qconnect-winapp**

Version 1.1.0 (from 24.02.2026)

<https://github.com/test-fullautomation/robotframework-qconnect-winapp>*Extension for Robot Framework QConnectBase library to test Winapp GUIs***robotframework-dbus**

Version 0.2.0 (from 23.02.2026)

<https://github.com/test-fullautomation/robotframework-dbus>*Robot Framework library for automated testing of DBus services***robotframework-prometheus**

Version 0.9.0 (from 23.02.2026)

<https://github.com/test-fullautomation/robotframework-prometheus>*Robot Framework keywords to communicate with the monitoring system Prometheus***RobotFramework-UDS**

Version 0.2.0 (from 23.02.2026)

<https://github.com/test-fullautomation/robotframework-uds>*Robot Framework keywords for UDS (Unified Diagnostic Services) communication***robotframework-robotlog2rqm**

Version 1.8.1 (from 09.06.2026)

<https://github.com/test-fullautomation/robotframework-robotlog2rqm>*Imports robot results to IBM Rational Quality Manager (RQM)***TestResultDBAccess**

Version 0.2.1 (from 30.06.2026)

<https://github.com/test-fullautomation/python-testresultdbaccess>*Interfaces to access TestResultWebApp database*

robotframework-robotlog2db

Version 1.6.0 (from 24.02.2026)

<https://github.com/test-fullautomation/robotframework-robotlog2db>*Imports robot results to TestResultWebApp database***robotframework-doip**

Version 0.2.0 (from 23.02.2026)

<https://github.com/test-fullautomation/robotframework-doip>*Robot Framework library for interacting with Electronic Control Units (ECUs)***PyTestLog2DB**

Version 0.4.2 (from 01.07.2026)

<https://github.com/test-fullautomation/python-pytestlog2db>*Imports pytest result(s) to TestResultWebApp database***TestResultWebApp**

Version 0.1.3 (from 18.10.2022)

<https://github.com/test-fullautomation/testresultwebapp>*Web based display of test results***VSCoDeWelcome**

Version 0.0.2 (from 06.2025)

<https://github.com/test-fullautomation/vscode-welcome>*RobotFramework_AIO Welcome Page for VSCode***VSCoDeJsonp**

Version 0.0.3 (from 04.2026)

<https://github.com/test-fullautomation/vscode-jsonp>*JSONP extension for VSCode*

10.1 GenPackageDoc

GenPackageDoc

v. 0.44.0

Holger Queckenstedt

05.06.2026

Contents

1	Introduction	1
2	Repository Structure	2
3	Documentation Build Process	3
4	Command Line	5
5	Documents Structure	7
5.1	Example: reST file	8
5.2	Example: Python module	11
6	Interface and Module Descriptions	13
7	Runtime Variables	15
8	Syntax Aspects	16
8.1	Common rules	16
8.2	Syntax extensions	17
9	Listings	20
10	Pictures and Diagrams	26
10.1	Pictures Import	26
10.2	Diagrams Rendering and Import	26
10.3	Example: Sequence diagram	27
10.4	Example: JSON diagram	28
11	CDocBuilder.py	29
11.1	Class: CDocBuilder	29
11.1.1	Method: Build	29
12	CInterface.py	30
12.1	Class: CInterface	30
12.1.1	Method: GetLaTeXStyles	30
13	CPackageDocConfig.py	31
13.1	Class: CPackageDocConfig	31
13.1.1	Method: PrintConfig	31
13.1.2	Method: PrintConfigKeys	31
13.1.3	Method: Get	31

CONTENTS	CONTENTS
13.1.4 Method: <code>GetConfig</code>	31
14 CPatterns.py	32
14.1 Class: <code>CPatterns</code>	32
14.1.1 Method: <code>GetHeader</code>	32
14.1.2 Method: <code>GetChapter</code>	32
14.1.3 Method: <code>GetFooter</code>	33
14.1.4 Method: <code>GetAutodefinedHeader</code>	33
15 CSourceParser.py	34
15.1 Class: <code>CSourceParser</code>	34
15.1.1 Method: <code>ParseSourceFile</code>	34
16 Appendix	35
17 History	36
B	

Chapter 1

Introduction

What is the meaning of this Python package?

The Python package **GenPackageDoc** generates the documentation of Python modules. The content of this documentation is taken out of the docstrings of functions, classes and their methods. All docstrings have to be written in reStructuredText (reST) format, that is a certain markdown dialect.

It is possible to extend the documentation by the content of additional files either in reST format or in LaTeX format.

The documentation is generated in the following way:

1. Files in LaTeX format are taken over immediately.
2. Files in reST format are converted to LaTeX files and to HTML files.
3. All docstrings of all Python modules in the package are converted to LaTeX files and to HTML files.
4. All LaTeX files together are converted to a single PDF document. This requires a separately installed LaTeX distribution (recommended: TeX Live). A LaTeX distribution is **not** part of **GenPackageDoc** and has to be installed separately!
5. All HTML files are collected in a separate folder together with a `index.html` file as entry point.

Caution: LaTeX files are not converted to HTML files! Therefore, the content of LaTeX files are not part of the Documentation in html format.

The sources of **GenPackageDoc** are available in the following GitHub repository:

[python-genpackagedoc](https://github.com/python-genpackagedoc/python-genpackagedoc)

The repository **python-genpackagedoc** uses it's own functionality to document itself and the contained Python package **GenPackageDoc**.

Therefore, the complete repository can be used as an example about writing a package documentation.

It has to be considered, that the main goal of **GenPackageDoc** is to provide a toolchain to generate documentation out of Python sources that are stored within a repository, and therefore we have dependencies to the structure of the repository. For example: Configuration files with values that are specific for a repository, should not be installed. Such a specific configuration value is e.g. the name of the package or the name of the PDF document.

The impact is: There is a deep relationship between the repository containing the sources to be documented, and the sources and the configuration of **GenPackageDoc** itself. Therefore some manual preparations are necessary to use **GenPackageDoc** also in other repositories.

How to do this is explained in detail in the next chapters.

The outcome of all preparations of **GenPackageDoc** in your own repository is a document like the one you are currently reading.

Chapter 2

Repository Structure

What is the structure of the application repository?

- Folder `GenPackageDoc`
Contains the package code.
This folder is specific for the package.
- Folder `packagedoc`
Contains all package documentation related files, e.g. the **GenPackageDoc** configuration, additional input files and the generated documentation itself.
This folder is specific for the documentation.
- Repository root folder
 - `genpackagedoc.py`
Python script to start the documentation build
 - `build_backend.py`
Custom build backend to execute additional installation steps. Currently this is a pattern only reserved for future development.
 - `pyproject.toml`
Main configuration file for the installation of the component
 - `dump_repository_config.py`
Little helper to dump the repository configuration to console

Chapter 3

Documentation Build Process

How do the files and folders listed above relate to each other? What is the flow of information when the documentation is generated?

- The process starts with the execution of `genpackagedoc.py` within the repository root folder.
- `genpackagedoc.py` creates a repository configuration object

```
config/CRepositoryConfig.py
```

- The repository configuration object reads the static repository configuration values out of the TOML file

```
pyproject.toml
```

Exception: The component version is taken from the Python source code

```
GenPackageDoc/version.py
```

- The repository configuration object adds dynamic values (like operating system specific settings and paths) to the repository configuration.
- The configuration file `packagedoc_config.json` contains settings like
 - Paths to Python packages to be documented
 - Paths and names of additional reST files
 - Path and name of output folder (LaTeX files and output PDF file)
 - User defined parameter (that can be defined here as global runtime variables and can be used in any reST code)
 - Basic settings related to the output PDF file (like document name, name of author, ...)
 - Path to LaTeX compiler/ (*a LaTeX distribution is not part of GenPackageDoc*)

Be aware of that the within `packagedoc_config.json` specified output folder

```
"OUTPUT" : "../build"
```

will be deleted at the beginning of the documentation build process! Make sure that you do not have any files inside this folder opened when you start the process. In case of the path is relative, the reference is the position of `genpackagedoc.py`. The complete path is created recursively.

Further details are explained within the json file itself.

- `genpackagedoc.py` also creates an own configuration object

```
GenPackageDoc/CPackageDocConfig.py
```

`CPackageDocConfig.py` takes over all repository configuration values, reads in the static **GenPackageDoc** configuration (`packagedoc_config.json`) and adds dynamically computed values like the full absolute paths belonging to the documentation build process. Also all command line parameters are resolved and checked.

The reference for all relative paths is the position of `genpackagedoc.py` (that is the repository root folder).

CHAPTER 3. DOCUMENTATION BUILD PROCESS

After the execution of `genpackagedoc.py` the resulting PDF document can be found under the specified name within the specified output folder (`"OUTPUT"`). This folder also contains all temporary files generated during the documentation build process.

Because the output folder is a temporary one, the PDF document is copied to the folder containing the package sources and therefore is included in the package installation. This is defined in the **GenPackageDoc** configuration, section `"PDFDEST"`.

Chapter 4

Command Line

Which command line parameters are available to influence the behavior of the application?

Some configuration parameter predefined within `packagedoc_config.json`, can be overwritten in command line.

`--output`

Path and name of folder containing all output files.

Caution: GenPackageDoc deletes this folder!

`--pdfdest`

Path and name of folder in which the generated PDF file will be copied to (after this file has been created within the output folder).

Caution: The generated PDF file will per default be copied to the package folder within the repository. This is defined in `packagedoc_config.json`. The version of the PDF file within the package folder will be part of the installation. When you change the PDF destination, then you get this file at another location - but this file will not be part of the installation any more. Installed will be the version, that is still present within the package folder of the repository.

`--htmldest`

Path and name of folder in which the generated HTML files will be copied to (after this file has been created within the output folder).

Caution: GenPackageDoc deletes this folder!

`--configdest`

Path and name of folder in which a dump of the current configuration will be copied to.

The configuration dump is part of the build output (section `'OUTPUT'`) and available in txt and in json format. It might be useful for further processes to have access to all details regarding the current documentation build.

`--strict`

If `True`, a missing LaTeX compiler aborts the process, otherwise the process continues.

CHAPTER 4. COMMAND LINE

```
--simulateonly
```

If `True`, the LaTeX compiler is switched off. No new PDF output will be generated. Already existing PDF output will not be updated. This is not handled as error and also not handled as warning. Only the source files will be parsed. This switch is useful to do a pre check for possible syntax issues within the source files without spending time for rendering PDF files.

Example

```
genpackagedoc.py --output="../any/other/location" --pdfdest="../any/other/location" --↩↪  
↩↪ htmldest="../any/other/location" --configdest="../any/other/location" --strict=True
```

Chapter 5

Documents Structure

How are the generated documents structured? What causes an entry within the table of content and how does the table of content look like?

In the following we use terms taken over from the LaTeX world: *chapter*, *section* and *subsection*.

A *chapter* is the top level within the PDF document; a *section* is the level below *chapter*, a *subsection* is the level below *section*.

The following assignments happen during the generation of a PDF document:

- The content of every additionally included separate reST file is a *chapter*.
 - In case of you want to add another chapter to your documentation, you have to include another reST file.
 - The headline of the chapter is the name of the reST file (automatically).
Therefore, it is not necessary to repeat the headline inside the file.
- The content of every included Python module is also a *chapter*.
 - The headline of the chapter is the name of the Python module (automatically).
This means also that within the PDF document structure every Python module is at the same level as additionally included reST files.
- Within additionally included separate reST files sections and subsections can be defined by the usual reST syntax elements for headings:
 - A line underlined with " `=` "
 - A line underlined with " `-` "
- Within the docstrings of Python modules the headings are added automatically (for functions, classes and methods)
 - Classes and functions are listed at section level (both classes and functions are assumed to be at the same level).
 - Class methods are listed at subsection level.

Further nestings of headings are not supported (because we do not want to overload the table of content).

5.1 Example: reST file

This can be the content of a file `Example Chapter.rst` in reST format:

```
.. highlight::

    This is an example chapter to demonstrate the structure of a PDF document.

Example section 1
=====

*text text text text text text text text text text text text text text*

Example subsection 1.1
-----

*text text text text text text text text text text text text text text*

Example subsection 1.2
-----

*text text text text text text text text text text text text text text*

Example section 2
=====

*text text text text text text text text text text text text text text*

Example subsection 2.1
-----

*text text text text text text text text text text text text text text*
```

In PDF this causes an entry in the table of content:

2	Example Chapter	11
2.1	Example section 1	11
2.1.1	Example subsection 1.1	11
2.1.2	Example subsection 1.2	11
2.2	Example section 2	11
2.2.1	Example subsection 2.1	11

And this is the content itself:

Chapter 2

Example Chapter

This is an example chapter to demonstrate the structure of a PDF document.

2.1 Example section 1

text text text text text text text text text text text text text text text text

2.1.1 Example subsection 1.1

text text text text text text text text text text text text text text text text

2.1.2 Example subsection 1.2

text text text text text text text text text text text text text text text text

2.2 Example section 2

text text text text text text text text text text text text text text text text

2.2.1 Example subsection 2.1

text text text text text text text text text text text text text text text text

The table of contents of the documentation in HTML format is currently limited to the top level (*chapters*):

GenPackageDoc

Examples

Example Chapter

Introduction

Repository Structure

Documentation Build Process

Command Line

Documents Structure

Interface and Module Descriptions

Runtime Variables

Syntax Aspects

Listings

This is an example chapter to demonstrate the structure of a PDF document.

Example section 1

text text text text text text text text text text text text text text text text

Example subsection 1.1

text text text text text text text text text text text text text text text text

Example subsection 1.2

text text text text text text text text text text text text text text text text

Example section 2

text text text text text text text text text text text text text text text text

Example subsection 2.1

text text text text text text text text text text text text text text text text

5.2 Example: Python module

This can be the content of some docstrings within a Python module `example_module.py`:

```
"""
.. highlight::

    The example module is a Python module containing example docstrings
"""

def example_function():
    """
    :acontent:`example_function` is a simple example function.
    """

class CExample():
    """
    :acontent:`CExample` is the main class of the example module.
    """

    def example_method_1(self):
        """
        This is the example method :acontent:`example_method_1` of the example module :fssystem:`↔
        ↪ example_module.py`.
        """
        pass

    def example_method_2(self):
        """
        This is the example method :acontent:`example_method_2` of the example module :fssystem:`↔
        ↪ example_module.py`.
        """
        pass
```

In PDF this causes an entry in the table of content:

18	example_module.py	42
18.1	Function: example_function	42
18.2	Class: CExample	42
18.2.1	Method: example_method_1	42
18.2.2	Method: example_method_2	42

And this is the content itself:

Chapter 18

example_module.py

The example module is a Python module containing example docstrings

18.1 Function: example_function

`example_function` is a simple example function.

18.2 Class: CExample

`CExample` is the main class of the example module.

18.2.1 Method: example_method_1

This is the example method `example_method_1` of the example module `example_module.py`.

18.2.2 Method: example_method_2

This is the example method `example_method_2` of the example module `example_module.py`.

The name of the Python module is placed at *chapter* level. The names of functions and classes are placed at *section* level. The names of class methods are placed at *subsection* level.

Caution: Headings within docstrings of Python module are not supported! Nested function definitions are skipped also.

The table of contents of the documentation in HTML format is currently limited to the top level (that is the name of the Python module). The functions classes and methods are not yet included in the HTML table of content.

Chapter 6

Interface and Module Descriptions

How to describe an interface of a function or a method? How to describe a Python module?

To have a unique look and feel of all interface descriptions, the following style is recommended:

Example

```

def ExampleInterface(self):
    """
Description of ExampleInterface

**Arguments:**

* :acontent: `arg1`

/ *Condition*: mandatory / *Type*: :acontent: `int` /

arg1 description

* :acontent: `arg2`

/ *Condition*: optional / *Type*: :acontent: `bool` / *Default*: :acontent: `True` /

arg2 description

**Returns:**

* :acontent: `ack`

/ *Type*: :acontent: `bool` /

Indicates if the computation was successful or not.

* :acontent: `result`

/ *Type*: :acontent: `str` /

Returned value
    """

```


CHAPTER 6. INTERFACE AND MODULE DESCRIPTIONS

Within the rendered documents this interface description looks like this:

7.3.2 Method: ExampleInterface

Description of ExampleInterface

Arguments:

- `arg1`
/ Condition: mandatory / Type: `int` /
 arg1 description
- `arg2`
/ Condition: optional / Type: `bool` / Default: `True` /
 arg2 description

Returns:

- `ack`
/ Type: `bool` /
 Indicates if the computation was successful or not.
- `result`
/ Type: `str` /
 Returned value

The docstrings containing the description, have to be placed directly in the next line after the `def` or `class` statement.

It is also possible to place a docstring at the top of a Python module. The exact position doesn't matter - but it has to be the first constant expression within the code. Within the documentation the content of this docstring is placed before the interface description and should contain general information belonging to the entire module.

The usage of such a docstring is an option.

Chapter 7

Runtime Variables

What are "runtime variables" and how to use them in reST content?

All configuration parameters of **GenPackageDoc** are taken out of four sources:

1. the static repository configuration
`pyproject.toml`
2. the dynamic repository configuration
`config/CRepositoryConfig.py`
3. the static **GenPackageDoc** configuration
`packagedoc/packagedoc_config.json`
4. the dynamic **GenPackageDoc** configuration
`GenPackageDoc/CPackageDocConfig.py`

Some of them are runtime variables and can be accessed within reST content (within docstrings of Python modules and also within separate reST files).

This means it is possible to add configuration values automatically to the documentation.

This happens by encapsulating the runtime variable name in triple hashes. This "triple hash" syntax is introduced to make it easier to distinguish between the json syntax (mostly based on curly brackets) and additional syntax elements used within values of json keys.

The name of the repository e.g. can be added to the documentation with the following reST content:

The name of the repository is `###REPOSITORYNAME###`.

This document contains a chapter "Appendix" at the end. This chapter is used to make the repository configuration a part of this documentation and can be used as example.

Additionally to the predefined runtime variables a user can add own ones.

See `"PARAMS"` within `packagedoc_config.json`.

All predefined runtime variables are written in capital letters. To make it easier for a developer to distinguish between predefined and user defined runtime variables, all user defined runtime variables have to be written in small letters completely.

Also the `"DOCUMENT"` keys within `packagedoc_config.json` are runtime variables.

Also within `packagedoc_config.json` the triple hash syntax can be used to access repository configuration values.

With this mechanism it is e.g. possible to give the output PDF document automatically the name of the package:

```
"DOCUMENT" : {
    "OUTPUTFILENAME" : "###PACKAGENAME###.tex",
```

Chapter 8

Syntax Aspects

What specific syntax rules must be observed when writing separate documents and the docstrings of Python modules in reST format?

8.1 Common rules

Important to know about the syntax of Python and reST is:

- In both Python and reST the indentation of text is part of the syntax!
- The indentation of the triple quotes indicating the beginning and the end of a docstring has to follow the Python syntax rules.
- The indentation of the content of the docstring (= the interface description in reST format) has to follow the reST syntax rules. To avoid a needless indentation of the text within the resulting PDF document and to avoid further unwanted side effects caused by improper indentations, it is strongly required to start at least the first line of a docstring text within the first column! And this first line is the reference for the indentation of further lines of the current docstring. The indentation of these further lines depends on the reST syntax element that is used here.
- In reSTreST also blank lines are part of the syntax!

Why is a proper indentation of the docstrings so much important?

The contents of all docstrings of a Python module will be merged to one single reST document (internally by **GenPackageDoc**). In this single reST document we do not have separated docstring lines any more. We have one text! And we have a relationship between previous lines and following lines in this text. The indentation of these previous and following lines must fit together – accordingly to the reST syntax rules. Otherwise we either get syntax issues during computation or we get text with a layout that does not fit to our expectation.

8.2 Syntax extensions

Like mentioned above, **GenPackageDoc** converts the documentation content from reST format to LaTeX format and to HTML format. Every format defines its own special characters as part of the syntax. For example: *underscores* in reST, *backslashes* in LaTeX and horizontal and vertical distances in HTML.

It is not possible, with acceptable effort, to use all special characters directly as literals in reST in such a way that they are displayed as literals in both LaTeX and HTML. The differences between the involved formats and the resulting requirements for escaping are simply too great. To enable the use of these special characters nonetheless, **GenPackageDoc** provides its own syntax elements. This allows **GenPackageDoc** to handle these special cases specifically for each output format.

Every syntax element starts with a slash, followed by an abbreviation in capital letters.

The following syntax elements are available:

RST		LaTeX/PDF	HTML
/NL	(newline)	\newline	
/NP	(newpage)	\newpage	" "
/VS	(vertical space 1)	\vspace{1ex}	display:block; height:1ex;
/VVS	(vertical space 2)	\vspace{2ex}	display:block; height:2ex;
/VVVS	(vertical space 3)	\vspace{3ex}	display:block; height:3ex;
/BS	(backslash)	\textbackslash{}	\\
/IBS	(inline backslash)	\\	\\
/HS	(horizontal space)	{\ttfamily\hspace{0.6em}}	
/IHS	(inline horizontal space)	{\ttfamily\hspace{0.6em}}	
/US	(underscore)	_	—

The following example demonstrates how the syntax extensions work:

```

**Distances:**

This Robot Framework code prints a string to log: :acontent:`log    Hello`.
This Robot Framework code prints another string to log: :acontent:`log    World`.

This Robot Framework code prints a string to log: :acontent:`log/IHS/IHS/IHS/IHSHello`./NL
This Robot Framework code prints another string to log: :acontent:`log/IHS/IHS/IHS/IHSWorld`./VVVS

Some distances between  strings/HS/HSstrings/HS/HS/HS/HSstrings/VS
and lines/VS
and lines/VVS
and lines/VVVS
and lines

```

Output in HTML format:

Distances:

This Robot Framework code prints a string to log: `log Hello`. This Robot Framework code prints another string to log: `log World`.

This Robot Framework code prints a string to log: `log   Hello`.
This Robot Framework code prints another string to log: `log   World`.

Some distances between strings strings strings
and lines
and lines
and lines
and lines

Insights:

- A line break in editor does not cause a line break in output files.
- A line break in output files must be set explicitly by `/NL` (= *newline*) at the end of a line.
- Without additional measures, multiple spaces in standard text and in inline literals are each reduced to a single space.
- A space that is to be preserved is defined by `/HS` (= *horizontal space*) in standard text and by `/IHS` (= *inline horizontal space*) in inline literals.
- Additional to the possibility to use blank lines for vertical distances in standard text, several *vertical space* elements (`/VS`, `/VVS`, `/VVVS`) are available to define vertical distances in higher granularity.
- User defined vertical distances are not possible in inline literals (and makes no sense here).

Recommendations:

Like shown in the example above, the usage of syntax elements should follow these rules:

- Prefer to put the syntax element for a line break at the end of the line - and combine this with a line break of the text in editor. This eases the readability of the text.
- Prefer to put syntax elements for vertical distances at the end of a line.
- Prefer to put syntax elements for horizontal distances immediately between strings inside a line.

Reasons:

This would increase the vertical distance between lines caused by additional blank lines:

```
.. anycontent::

    text text text text

/VVS

    text text text text
```

This would increase the horizontal distance between two strings within a line by additional blanks:

```
.. anycontent::  
  
    text /HS text
```

This style should be avoided (but is for sure not forbidden).

In block literals (e.g. like `anycontent`), the extended syntax is not allowed - and not necessary. The content is printed out **as is** (*without any interpretation*).

Hint: It is good practise to put a `/NP` at the end of each section. Every following section will start at a new page in PDF output (in HTML output, `/NP` has no effect).

Chapter 9

Listings

How to realize listings of code written in Python, Robot Framework and JSON? How to realize listings of console content and log file content?

GenPackageDoc supports several code listings and console listings. These listings are available as text block and as inline text also. You can use these listings in reST files and also in LaTeX files immediately. The reST elements for listings are mapped to the corresponding LaTeX commands when **GenPackageDoc** internally converts the reST files in LaTeX files.

Text blocks are realized by reST directives. Inline text is realized by reST roles.

Examples:

This is a reST directive `pythoncode` code example:

```
for index in list:
    print("index")
```

Written in reST format:

```
.. pythoncode::

    for index in list:
        print("index")
```

Written in LaTeX format:

```
\begin{pythoncode}
    for index in list:
        print("index")
\end{pythoncode}
```

★

This is a reST directive `robotcode` code example:

```
*** Test Cases ***

Example

    FOR      ${index}      IN RANGE      0      ${max}
        log      index:  ${index}      console=yes
    END
```

CHAPTER 9. LISTINGS

Written in reST format:

```
.. robotcode::

    *** Test Cases ***

    Example

        FOR      ${index}      IN RANGE      0      ${max}
            log      index: ${index}      console=yes
        END
```

Written in LaTeX format:

```
\begin{robotcode}
    *** Test Cases ***

    Example

        FOR      ${index}      IN RANGE      0      ${max}
            log      index: ${index}      console=yes
        END
\end{robotcode}
```

★

This is a reST directive `jsoncode` code example:

```
{
    "param_1" : "ABC",
    "param_2" : true,
    "param_3" : [1,2,3],
    "param_4" : {"A" : 1, "B" : 2}
}
```

Written in reST format:

```
.. jsoncode::

{
    "param_1" : "ABC",
    "param_2" : true,
    "param_3" : [1,2,3],
    "param_4" : {"A" : 1, "B" : 2}
}
```

Written in LaTeX format:

```
\begin{jsoncode}
{
    "param_1" : "ABC",
    "param_2" : true,
    "param_3" : [1,2,3],
    "param_4" : {"A" : 1, "B" : 2}
}
\end{jsoncode}
```

★

This is a reST directive `anycontent` code example:

```
Any content, no matter if Python, Robot Framework, JSON or something else.
Any content, no matter if Python, Robot Framework, JSON or something else.
Any content, no matter if Python, Robot Framework, JSON or something else.
```

CHAPTER 9. LISTINGS

Written in reST format:

```

.. anycontent::

    Any content, no matter if Python, Robot Framework, JSON or something else.
    Any content, no matter if Python, Robot Framework, JSON or something else.
    Any content, no matter if Python, Robot Framework, JSON or something else.

```

Written in LaTeX format:

```

\begin{anycontent}
    Any content, no matter if Python, Robot Framework, JSON or something else.
    Any content, no matter if Python, Robot Framework, JSON or something else.
    Any content, no matter if Python, Robot Framework, JSON or something else.
\end{anycontent}

```

★

This is a reST directive `consolelog` code example:

```

log index: 0
log index: 1
log index: 2

```

Written in reST format:

```

.. consolelog::

    log index: 0
    log index: 1
    log index: 2

```

Written in LaTeX format:

```

\begin{consolelog}
    log index: 0
    log index: 1
    log index: 2
\end{consolelog}

```

★

This is a reST directive `filesystem` code example:

```

C:\folder_1\file_1.txt
C:\folder_2\file_2.txt
C:\folder_3\file_3.txt

```

Written in reST format:

```

.. filesystem::

    C:\folder_1\file_1.txt
    C:\folder_2\file_2.txt
    C:\folder_3\file_3.txt

```

CHAPTER 9. LISTINGS

Written in LaTeX format:

```
\begin{filesystem}
  C:\folder_1\file_1.txt
  C:\folder_2\file_2.txt
  C:\folder_3\file_3.txt
\end{filesystem}
```

Hint: `consolelog` and `filesystem` currently have the same layout. Nevertheless, using individual and specific names eases the readability. Maybe later different layouts will be realized.

This is inline Python code: `print("Hello Python")`

Written in reST format:

```
:pcode:`print("Hello Python")`
```

Written in LaTeX format:

```
\pcode{print("Hello Python")}
```

This is inline Robot Framework code: `log Hello RobotFramework AIO`

Written in reST format:

```
:rcode:`log/IHS/IHS/IHS/IHSHello RobotFramework AIO`
```

Written in LaTeX format:

```
\rcode{log\ \ \ \ Hello RobotFramework AIO}
```

Hint: Blanks must be announced explicitly in inline code, because under normal circumstances multiple blanks are collapsed to a single blank (in LaTeX and in HTML also).

This is inline JSON code: `"param" : [1,2,3]`

Written in reST format:

```
:jcode:`"param" : [1,2,3]`
```

Written in LaTeX format:

```
\jcode{"param" : [1,2,3]}
```

This is any inline content: `any content`

Written in reST format:

```
:acontent:`any content`
```

Written in LaTeX format:

```
\acontent{any content}
```

The horizontal separator line with the star in the middle can be inserted with the reST directive `.. hrstar::`. This is a nice possibility to separate smaller parts of the documentation from each other - without introducing a new headline for every part.

CHAPTER 9. LISTINGS

When we use all of these reST syntax elements to realize a concrete documentation - how does this look like? The following is a full example:

In the system configuration file, the parameter :acontent:`aparam` and :acontent:`bparam` ↩↩ are defined in the following way:

```
.. jsoncode::

    {
        // system parameter
        "aparam" : 1,
        "bparam" : "hello world"
    }
```

This configuration file can be found here:

```
.. filesystem::

    ${systemroot}/packages/labels/alabel/config.jsonp
```

Make sure to select the proper label: :acontent:`alabel`.

The **Robot Framework** keyword :acontent:`compute\_system\_parameter` computes these ↩↩ system parameters in the following way:

- * The parameter :acontent:`aparam` is increased by value :acontent:`1`.
- * The parameter :acontent:`bparam` is concatenated with string :acontent:`today` (with a ↩↩ blank in between).

This is the corresponding Python implementation of the keyword :acontent:`↩↩ compute\_system\_parameter`:

```
.. pythoncode::

    system_config = system.get_config()
    system_config['aparam'] = system_config['aparam'] + 1
    system_config['bparam'] = system_config['bparam'] + " today"
    number_of_parameters = len(system_config)
    return system_config, number_of_parameters
```

Additionally to the system configuration :acontent:`system\_config`, another variable :↩↩ ↩↩ acontent:`number\_of\_parameters` containing the number of system parameters, is returned.

Within a **Robot Framework** test you call the keyword :acontent:`↩↩ compute\_system\_parameter` in the following way:

```
.. robotcode::

    *** Test Cases ***

    SystemTest

        ${system_config}    ${number_of_parameters}    compute_system_parameter
        log    Number of parameters: ${number_of_parameters}
        log    aparam.....: ${system_config}[aparam]
```

In the log file it can be observed that the value of parameter :acontent:`aparam` is :↩↩ ↩↩ acontent:`2` now.

The value of parameter :acontent:`bparam` is :acontent:`hello world today` now. With 2 parameters in total.

CHAPTER 9. LISTINGS

Result:

★

In the system configuration file, the parameter `aparam` and `bparam` are defined in the following way:

```
{
  // system parameter
  "aparam" : 1,
  "bparam" : "hello world"
}
```

This configuration file can be found here:

```
${systemroot}/packages/labels/alabel/config.jsonp
```

Make sure to select the proper label: `alabel`.

The **Robot Framework** keyword `compute_system_parameter` computes these system parameters in the following way:

- The parameter `aparam` is increased by value `1`.
- The parameter `bparam` is concatenated with string `today` (with a blank in between).

This is the corresponding Python implementation of the keyword `compute_system_parameter`:

```
system_config = system.get_config()
system_config['aparam'] = system_config['aparam'] + 1
system_config['bparam'] = system_config['bparam'] + " today"
number_of_parameters = len(system_config)
return system_config, number_of_parameters
```

Additionally to the system configuration `system_config`, another variable `number_of_parameters` containing the number of system parameters, is returned.

Within a **Robot Framework** test you call the keyword `compute_system_parameter` in the following way:

```
*** Test Cases ***

SystemTest

    ${system_config}    ${number_of_parameters}    compute_system_parameter
    log    Number of parameters: ${number_of_parameters}
    log    aparam.....: ${system_config}[aparam]
```

In the log file it can be observed that the value of parameter `aparam` is `2` now. The value of parameter `bparam` is `hello world today` now. With 2 parameters in total.

★

Chapter 10

Pictures and Diagrams

How to import pictures? How to render and import diagrams?

10.1 Pictures Import

Pictures can be imported in the following way:

1. Import in reST content:

```
.. image:: ./pictures/AnyPicture.png
```

2. Import in LaTeX content:

```
\includegraphics[scale=0.7]{./pictures/AnyPicture.png}
```

The user needs to adapt the scaling to make the rendered diagrams fit to a page of a PDF document in best way. But this scaling only works in LaTeX code, but not in reST code.

10.2 Diagrams Rendering and Import

A *diagram* in this context is a picture that is rendered out of source code. **GenPackageDoc** supports **PlantUML** that supports a wide range of diagrams.

To use the **PlantUML** functionality with **GenPackageDoc**, some preconditions have to be fulfilled:

1. **PlantUML** is installed (either as stand-alone installation or as VSCodium extension)
2. **GenPackageDoc** is configured (`packagedoc_config.json`):
 - a. In section `"DIAGRAMS"` a path to a diagrams folder is defined (containing the diagrams source code).
 - b. In section `"JAVA"` path and name of the java interpreter is defined (because **PlantUML** is a java application).
 - c. In section `"PLANT_UML"` path and name of the **PlantUML** application is defined.

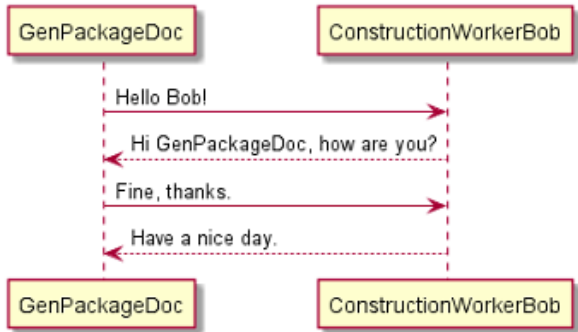
All **PlantUML** source code files within the `"DIAGRAMS"` folder need to have the extension `puml`.

10.3 Example: Sequence diagram

The following code of a `puml` file produces a sequence diagram:

```
@startuml
GenPackageDoc -> ConstructionWorkerBob: Hello Bob!
ConstructionWorkerBob --> GenPackageDoc: Hi GenPackageDoc, how are you?
GenPackageDoc -> ConstructionWorkerBob: Fine, thanks.
ConstructionWorkerBob --> GenPackageDoc: Have a nice day.
@enduml
```

Result:



10.4 Example: JSON diagram

The following code of a `puml` file produces a sequence diagram:

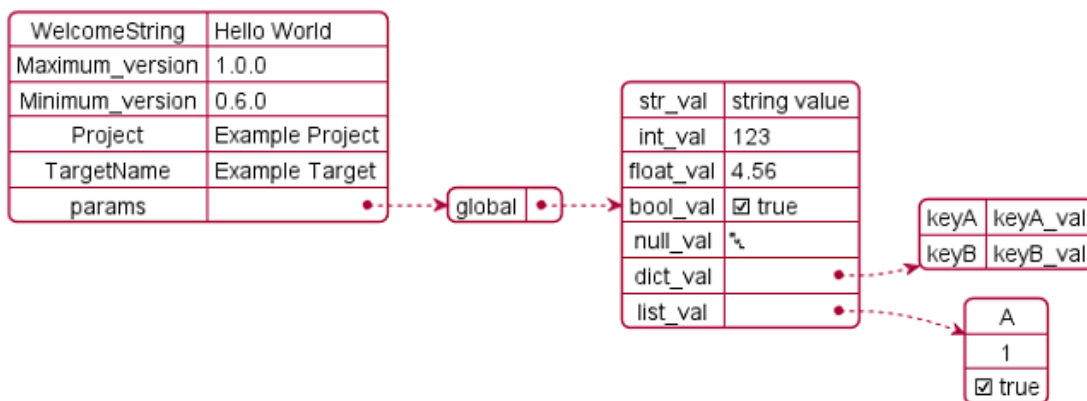
```
@startjson
{
  "WelcomeString" : "Hello World",

  "Maximum_version" : "1.0.0",
  "Minimum_version" : "0.6.0",

  "Project" : "Example Project",
  "TargetName" : "Example Target",

  "params" : {
    "global" : {
      "str_val" : "string value",
      "int_val" : 123,
      "float_val" : 4.56,
      "bool_val" : true,
      "null_val" : null,
      "dict_val" : {"keyA" : "keyA_val", "keyB" : "keyB_val"},
      "list_val" : ["A", 1, true]
    }
  }
}
@endjson
```

Result:



In case of **PlantUML** is configured in the **GenPackageDoc** configuration and in case of `puml` files are available within the diagrams folder, **GenPackageDoc** automatically calls **PlantUML** to render the diagrams. They can be imported in the following way:

1. Import in reST content:

```
.. image:: ../diagrams/SequenceDiagram.png
```

2. Import in LaTeX content:

```
\includegraphics[scale=0.7]{../diagrams/SequenceDiagram.png}
```

Chapter 11

CDocBuilder.py

CDocBuilder is a Python module containing all methods to generate files in TEX and HTML format.

11.1 Class: CDocBuilder

`CDocBuilder` is the main class to build TEX sources out of docstrings of Python modules and separate text files in reST format.

This depends on a JSON configuration file, provided by a `oPackageDocConfig` object (that includes the Repository configuration).

Method to execute: `Build()`

11.1.1 Method: Build

Arguments:

(no arguments)

Returns:

- `bSuccess`
/ Type: bool /
Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
/ Type: str /
The result of the computation of the method `sMethod`.

Chapter 12

CInterface.py

Python module containing an interface for **GenPackageDoc**. This interface can be used to get access to the LaTeX stylesheets that are part of the **GenPackageDoc** installation.

12.1 Class: CInterface

12.1.1 Method: GetLaTeXStyles

The LaTeX stylesheets are part of the installation of **GenPackageDoc**. In case of anyone else than **GenPackageDoc** needs these stylesheets, this method can be used to copy them to any other folder.

Arguments:

- `sDestination`
/ *Condition*: required / *Type*: str /
Path and name of a folder in which the styles folder from **GenPackageDoc** will be copied.

Returns:

- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
/ *Type*: str /
The result of the computation of the method `sMethod`.

Chapter 13

CPackageDocConfig.py

Python module containing the configuration for **GenPackageDoc**. This includes the repository configuration and command line values.

13.1 Class: CPackageDocConfig

13.1.1 Method: PrintConfig

Prints all configuration values to console.

13.1.2 Method: PrintConfigKeys

Prints all configuration key names to console.

13.1.3 Method: Get

Returns the configuration value belonging to a key name.

13.1.4 Method: GetConfig

Returns the complete configuration dictionary.

Chapter 14

CPatterns.py

Python module containing source patterns used to generate the tex file output.

14.1 Class: CPatterns

The CPatterns class provides a set of LaTeX source patterns used to generate the tex file output.

All source patterns are accessible by corresponding `Get` methods. Some source patterns contain placeholder that will be replaced by input parameter of the `Get` method.

14.1.1 Method: GetHeader

Defines the header of the main tex file.

Arguments:

- `sTitle`
/ *Condition*: required / *Type*: str /
The title of the output document (name of the described package)
- `sVersion`
/ *Condition*: required / *Type*: str /
The version of the output document (version of the described package)
- `sAuthor`
/ *Condition*: required / *Type*: str /
The author of the output document (author of the described package)
- `sDate`
/ *Condition*: required / *Type*: str /
The date of the output document (date of the described package)

Returns:

- `sHeader`
/ *Type*: str /
LaTeX code containing the header of main tex file.

14.1.2 Method: GetChapter

Defines single chapter of the main tex file.

A single chapter is equivalent to an additionally imported text file in rst format or equivalent to a single Python module within a Python package.

Arguments:

- `sHeadline`
/ Condition: required / Type: str /
 The chapter headline (that is either the name of an additional rst file or the name of a Python module).
- `sLabel`
/ Condition: required / Type: str /
 The chapter label (to enable linking to this chapter)
- `sDocumentName`
/ Condition: required / Type: str /
 The name of a single tex file containing the chapter content. This file is imported in the main text file after the chapter headline that is set by `sHeadline`.

Returns:

- `sHeader`
/ Type: str /
 LaTeX code containing the headline and the input of a single tex file.

14.1.3 Method: GetFooter

Defines the footer of the main tex file.

Arguments:

(no arguments)

Returns:

- `sFooter`
/ Type: str /
 LaTeX code containing the footer of the main tex file.

14.1.4 Method: GetAutodefinedHeader

Defines the header of the autodefined LaTeX sty file.

Arguments:

(no arguments)

Returns:

- `sAutodefinedHeader`
/ Type: str /
 LaTeX code containing the header of the autodefined LaTeX sty file.

Chapter 15

CSourceParser.py

Python module containing all methods to parse the documentation content of Python source files.

15.1 Class: CSourceParser

The CSourceParser class provides a method to parse the functions, classes and their methods together with the corresponding docstrings out of Python modules. The docstrings have to be written in rst syntax.

15.1.1 Method: ParseSourceFile

The method ParseSourceFile parses the content of a Python module.

Arguments:

- `source_file`
/ *Condition*: required / *Type*: str /
Path and name of a single Python module.
- `include_private`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If False: private methods are skipped, otherwise they are included in documentation.
- `include_undocumented`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True: also classes and methods without docstring are listed in the documentation (together with a hint that information is not available), otherwise they are skipped.

Returns:

- `dictContent`
/ *Type*: dict /
A dictionary containing all the information parsed out of `source_file`.
- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
/ *Type*: str /
The result of the computation of the method `sMethod`.

Chapter 16

Appendix

About GenPackageDoc:

Author

Holger Queckenstedt (Holger.Queckenstedt@de.bosch.com)

Version

0.44.0 (05.06.2026)

Short Description

Documentation builder for Python packages

Homepage

[python-genpackagedoc](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 17

History

Version 0.44.0 (05.06.2026)

Added:

- Search function
- GenPackageDoc decorator
- Tags included in PDF and HTML output

Maintenance:

- LaTeX preamble (`xcolor` fix)
- Robot Framework decorator parsing

Version 0.43.0 (18.05.2026)

Maintenance:

- Usage of **Setuptools** replaced by usage of **PIP/TOML/Setuptools**
- History reworked (youngest version on top now) and migrated from LaTeX format to RST format

Added:

- Possibility to exclude interface files from computation
- Markdown directives to support Python and **Robot Framework** code listings
- Markdown directives to support JSONP code listings
- Markdown directives to support common console and log listings
- Markdown directives to support listings of file system contents
- Markdown directives to support a simple highlighting in gray color
- Syntax extensions for vertical and horizontal distances
- Support of **async** functions and methods
- Output of documentation in HTML-Format

Version 0.42.0 (12.01.2026)

Maintenance:

- Usage of **pandoc/py pandoc** replaced by usage of **docutils** (conversion of RST content to LaTeX format)

Version 0.41.0 (11.07.2023)

Added:

CHAPTER 17. HISTORY

- Text macro `\q{ }` to set double quotes

Version 0.40.0 (09.05.2023)

Added:

- **PlantUML** support
- Possibility to render diagrams directly from text source code

Version 0.39.0 (06.01.2023)

Added:

- Masking of underlines in case of the content of `\repo` or `\pkg` contains underlines (masking required by LaTeX)

Version 0.38.0 (30.11.2022)

Removed:

- Harming ligatures that were added by **Pandoc** automatically to LaTeX code in case of multiple minus characters in names

Version 0.37.0 (21.11.2022)

Added:

- LaTeX commands `pythonlog` and `plog`
- keyword decorator detection

Maintenance:

- LaTeX style adaptations
- `INCLUDEPRIVATE` temporarily switched off (requires bugfixes)

Version 0.36.0 (17.11.2022)

Maintenance:

- Brightness of all colors of listings reduced to 45% (both text boxes and inline)

Version 0.35.0 (16.11.2022)

Maintenance:

- Layout settings of some LaTeX commands adapted
- Repository name and package name in bold
- Inline code and inline listings in clearer colors

Version 0.34.0 (07.11.2022)

Added:

- Auto defined LaTeX style file containing mnemotechnical commands to type the repository name and the package name

Version 0.33.0 (19.09.2022)

Maintenance:

CHAPTER 17. HISTORY

- Rework of label mechanism (to enable unique links to functions, classes and methods with names that are not unique over all Python modules within a package)

Version 0.32.0 (16.09.2022)*Added:*

- Labels at chapter level

Maintenance:

- Partial rework of label mechanisms

Version 0.31.0 (12.09.2022)*Fixes:*

- Import path of a module in PDF file

Version 0.30.0 (31.08.2022)*Added:*

- `simulateonly` mode (command line switch to skip the PDF generation)

Version 0.29.0 (24.08.2022)*Changes:*

- Layout of import path of a module (in PDF file)

Version 0.28.0 (23.08.2022)*Added:*

- LaTeX environment variable `GENDOC_LATEXPAT`H

Maintenance:

- File `robotframeworkaio.sty` aligned to version of this file in `GenMainDoc`

Version 0.27.0 (17.08.2022)*Added:*

- LaTeX style definition for Python syntax highlighting

Version 0.26.0 (27.07.2022)*Maintenance:*

- History reworked

Added:

- File `common.sty` (common LaTeX commands e.g. to support the history)

Version 0.25.0 (27.07.2022)*Maintenance:*

- Layout of `RobotFramework AIO` syntax highlighting (in `.sty` files)

CHAPTER 17. HISTORY

Version 0.24.0 (25.07.2022)*Maintenance:*

- Line breaks in listings (in `robotframeworkaio.sty`)

Version 0.23.0 (13.07.2022)*Maintenance:*

- File `preamble.tex`

Version 0.22.0 (13.07.2022)*Maintenance:*

- File `preamble.tex`
- Folder `styles`

Fix:

- `setup.py` installs tex files also

Version 0.21.0 (12.07.2022)*Maintenance:*

- Separated file `preamble.tex`

Version 0.20.0 (29.06.2022)*Fix:*

- Added missing masking of underlines in PDF document title (required for LaTeX)

Version 0.19.0 (28.06.2022)*Added:*

- Method `GetLaTeXStyles`

Maintenance:

- `PythonExtensionsCollection` updated to version 0.8.0

Version 0.18.0 (20.06.2022)*Added:*

- Parameter to define an output folder for a dump of final configuration

Version 0.17.0 (17.06.2022)*Added:*

- Possibility to dump the configuration

Maintenance:

- Code and error handling

Version 0.16.0 (01.06.2022)

CHAPTER 17. HISTORY

Maintenance:

- Path computation reworked

Version 0.15.0 (31.05.2022)*Added:*

- **GenPackageDoc** command line
- Separate **GenPackageDoc** configuration class

Version 0.14.0 (27.05.2022)*Added:*

- LaTeX compiler check
- Control parameter `STRICT` added to **GenPackageDoc** configuration

Version 0.13.0 (24.05.2022)*Maintenance:*

- LaTeX style definitions moved to a separate folder

Version 0.12.0 (19.05.2022)*Added:*

- Admonitions based on LaTeX environment `tcolorbox`

Maintenance:

- Layout adaption in titlepage

Fix:

- Page numbering in TOC

Version 0.11.0 (18.05.2022)*Added:*

- Possibility to import `tex` files

Version 0.10.0 (17.05.2022)*Added:*

- Postprocessing for `rst` and `tex` source files added

Fix:

- multiply-defined labels

Version 0.9.2 (16.05.2022)*Fix:*

- Automated line breaks within code blocks

Version 0.9.1 (11.05.2022)*Maintenance:*

CHAPTER 17. HISTORY

- Documentation

Version 0.9.0 (10.05.2022)*Maintenance:*

- Layout maintenance and syntax extensions for newline, newpage and vspace

Version 0.8.0 (10.05.2022)*Changes:*

- Bugfixes and code maintenance

Added:

- Version history

Version 0.7.0 (10.05.2022)*Added:*

- Setup process and file **README.rst**

Maintenance:

- Code

Version 0.6.0 (09.05.2022)*Added:*

- Parameter `INCLUDEUNDOCUMENTED`

Version 0.5.0 (09.05.2022)*Added:*

- Parameter `INCLUDEPRIVATE`

Version 0.4.0 (06.05.2022)*Added:*

- Possibility to describe complete Python modules

Version 0.3.0 (06.05.2022)*Added:*

- Automated headings for functions, classes and methods

Version 0.2.0 (05.05.2022)*Added:*

- Python syntax highlighting within code blocks

Version 0.1.0 (04/2022)**Initial version**

[GenPackageDoc in GitHub](#)

[GenPackageDoc in PyPi](#)

GenPackageDoc.pdf

Created at 07.09.2026 - 09:46:46

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.2 PythonExtensionsCollection

PythonExtensionsCollection

v. 0.17.1

Holger Queckenstedt

22.06.2026

Contents

1	Introduction	1
2	Description	2
2.1	Modules	2
2.2	Methods	2
2.2.1	String operations with CString	2
2.2.2	File access with CFile	11
2.2.3	Utilities	15
3	CComparison.py	16
3.1	Method: Compare	16
4	CFile.py	17
4.1	Class: enFileStatType	17
4.2	Class: CFile	17
4.2.1	Method: Close	17
4.2.2	Method: Delete	18
4.2.3	Method: Write	18
4.2.4	Method: Append	19
4.2.5	Method: ReadLines	19
4.2.6	Method: GetFileInfo	21
4.2.7	Method: CopyTo	21
4.2.8	Method: MoveTo	22
5	CFolder.py	23
5.1	Function: rm_dir_readonly	23
5.2	Class: CFolder	23
5.2.1	Method: Delete	23
5.2.2	Method: Create	24
5.2.3	Method: CopyTo	24
6	CString.py	26
6.1	Method: NormalizePath	26
6.2	Method: DetectParentPath	27
6.3	Method: StringFilter	28
6.4	Method: FormatResult	30
7	CUtills.py	31
7.1	Function: PrettyPrint	31

CONTENTS	CONTENTS
7.2 Class: CTypePrint	32
7.2.1 Method: TypePrint	32
7.3 Class: CUtils	32
7.3.1 Method: GetInstalledPackages	32
7.3.2 Method: GetComponentName	33
8 Appendix	34
9 History	35

B

Chapter 1

Introduction

The **PythonExtensionsCollection** extends the functionality of Python by some useful functions that are not available in Python immediately.

This covers for example file and folder operations, string operations like normalizing a path and a pretty print method.

The **PythonExtensionsCollection** contains several Python modules and every module has to be imported separately in case of the functions inside are needed.

The sources of the **PythonExtensionsCollection** are available in [GitHub](#).

Informations about how to install the **PythonExtensionsCollection** can be found in the [README](#).

Chapter 2

Description

2.1 Modules

The **PythonExtensionsCollection** contains the following modules:

1. **CComparison**
Compare two text files based on regular expressions.
Import: `from PythonExtensionsCollection.Comparison.CComparison import CComparison`
2. **CFile**
File operations like read, write, copy, move, ...
Import: `from PythonExtensionsCollection.File.CFile import CFile`
3. **CFolder**
Folder operations like create, delete, copy, ...
Import: `from PythonExtensionsCollection.Folder.CFolder import CFolder`
4. **CString**
String operations like normalize a path, string filter, format results, ...
Import: `from PythonExtensionsCollection.String.CString import CString`
5. **CUtills**
Pretty print of Python data types
Import: `from PythonExtensionsCollection.Utills.CUtills import CTypePrint`

2.2 Methods

Additionally to the interface descriptions in the second part of this document this section contains a more detailed description of some assorted methods together with examples how to use.

2.2.1 String operations with CString

NormalizePath

It's not easy to handle paths - and especially the path separators - independent from the operating system.

Under Linux it is obvious that single slashes are used as separator within paths. Whereas the Windows explorer uses single backslashes. In both operating systems web addresses contains single slashes as separator when displayed in web browsers.

Using single backslashes within code - as content of string variables - is dangerous because the combination of a backslash and a letter can be interpreted as escape sequence - and this is maybe not the effect a user wants to have.

To avoid unwanted escape sequences backslashes have to be masked (by the usage of two of them: `"\\"`). But also this could not be the best solution because there are also applications (like the Windows explorer) that are not able to handle masked backslashes. They expect to get single backslashes within a path.

Preparing a path for best usage within code also includes collapsing redundant separators and up-level references. Python already provides functions to do this, but the outcome (path contains slashes or backslashes) depends on the

CHAPTER 2. DESCRIPTION

2.2. METHODS

operating system. And like already mentioned above also under Windows backslashes might not be the preferred choice.

It also has to be considered that redundant separators at the beginning of an address of a local network resource (like `\\server.com`) and or inside an internet address (like `https:\\server.com`) must **not** be collapsed! Unfortunately the Python function `normpath` does not consider this context.

To give the user full control about the format of a path, independent from the operating system and independent if it's a local path, a path to a local network resource or an internet address, the method `NormalizePath()` provides lot's of parameters to influence the result.

Example 1 (*file system path*)

```
path = r"C:\\subfolder1\\../subfolder2\\\\../subfolder3\\"
path = CString.NormalizePath(path)
print(path)
```

Result (*output contains slashes*)

```
C:/subfolder3
```

Example 2 (*file system path*)

```
path = r"C:\\subfolder1\\../subfolder2\\\\../subfolder3\\"
path = CString.NormalizePath(path, bWin=True)
print(path)
```

Result (*output contains masked backslashes*)

```
C:\\subfolder3
```

Example 3 (*path of a local network resource*)

```
path = r"\\anyserver.com\\part1/part2\\\\part3/part4"
path = CString.NormalizePath(path)
print(path)
```

Result

```
//anyserver.com/part1/part2/part3/part4
```

Example 4 (*internet address*)

```
path = r"http:\\anyserver.com\\part1/part2\\\\part3/part4"
path = CString.NormalizePath(path)
print(path)
```

Result

```
http://anyserver.com/part1/part2/part3/part4
```

DetectParentPath

The method `DetectParentPath` computes the path to any parent folder inside a given path. Optionally `DetectParentPath` is able to search for files inside the identified parent folder.

In the following examples we assume to have the following file system structure:

```
D:\pathtest\ABC\DEF\GHI\FILE.txt
D:\pathtest\ABC\FILE.txt
D:\pathtest\RST\UVW\XYZ\FILE.txt
```

We are inside folder `GHI` and want to know the path to folder `ABC`.

Python code

```
sStartPath = r"D:\pathtest\ABC\DEF\GHI"
sFolderName = "ABC"
sDestPath, listDestPaths, sDestFile, listDestFiles, sDestPathParent = CString.<↵
    ↳ DetectParentPath(sStartPath, sFolderName)
print(f"sDestPath      = {sDestPath}")
print(f"listDestPaths  = {listDestPaths}")
print(f"sDestFile      = {sDestFile}")
print(f"listDestFiles   = {listDestFiles}")
print(f"sDestPathParent = {sDestPathParent}")
```

Outcome

```
sDestPath      = D:/pathtest/ABC
listDestPaths   = ['D:/pathtest/ABC']
sDestFile      = None
listDestFiles   = None
sDestPathParent = D:/pathtest
```

`sDestPath` contains the path to `sFolderName` within `sStartPath`. `sDestPathParent` contains the path to the parent folder of `sDestPath`. The meaning of the remaining return parameter are handled in the next examples.

It is possible to search for several folders.

We are inside folder `GHI` and want to know the path to folders `ABC` and `DEF`.

```
sStartPath = r"D:\pathtest\ABC\DEF\GHI"
sFolderName = "ABC;DEF"
sDestPath, listDestPaths, sDestFile, listDestFiles, sDestPathParent = CString.<↵
    ↳ DetectParentPath(sStartPath, sFolderName)
```

Outcome

```
sDestPath      = D:/pathtest/ABC/DEF
listDestPaths   = ['D:/pathtest/ABC/DEF', 'D:/pathtest/ABC']
sDestFile      = None
listDestFiles   = None
sDestPathParent = D:/pathtest/ABC
```

`sFolderName` is a semicolon separated list of folder names. Accordingly to this list of folder names `listDestPaths` contains the paths to these folders. `DetectParentPath` searches in the path from right to left (from bottom level up to top level). Therefore the folder `DEF` is the first one who is found. The order of elements in `listDestPaths` is not synchronized with the order of folder names in `sFolderName`! In every case `sDestPath` contains the first element in the list. `sDestPathParent` contains the path to the parent folder of `sDestPath`.

It is possible to search for a file.

We are inside folder `GHI` and want to know the path to folder `DEF` and within this folder we want to know the path to file `FILE.txt`.

```
sStartPath = r"D:\pathtest\ABC\DEF\GHI"
sFolderName = "DEF"
sFileName = "FILE.txt"
sDestPath, listDestPaths, sDestFile, listDestFiles, sDestPathParent = CString.<↵
    ↳ DetectParentPath(sStartPath, sFolderName, sFileName)
```

Outcome

```
sDestPath      = D:/pathtest/ABC/DEF
listDestPaths  = ['D:/pathtest/ABC/DEF']
sDestFile      = D:/pathtest/ABC/DEF/GHI/FILE.txt
listDestFiles  = ['D:/pathtest/ABC/DEF/GHI/FILE.txt']
sDestPathParent = D:/pathtest/ABC
```

`listDestPaths` contains a list of all paths to `sFolderName` within `sStartPath`. `sDestPath` contains the first element of `listDestPaths`. `listDestFiles` contains a list of all files with name `sFileName` found within `listDestPaths`. `sDestFile` contains the first element of `listDestFiles`.

A semicolon separated list of file names in `sFileName` (like for `sFolderName`) is not supported.

Providing more than one folder together with a file name may cause overlapping results. But `listDestFiles` will not contain any redundant paths to files.

```
sStartPath = r"D:\pathtest\ABC\DEF\GHI"
sFolderName = "ABC;DEF"
sFileName = "FILE.txt"
sDestPath, listDestPaths, sDestFile, listDestFiles, sDestPathParent = CString.↵
    ↪ DetectParentPath(sStartPath, sFolderName, sFileName)
```

Outcome

```
sDestPath      = D:/pathtest/ABC/DEF
listDestPaths  = ['D:/pathtest/ABC/DEF', 'D:/pathtest/ABC']
sDestFile      = D:/pathtest/ABC/DEF/GHI/FILE.txt
listDestFiles  = ['D:/pathtest/ABC/DEF/GHI/FILE.txt', 'D:/pathtest/ABC/FILE.txt']
sDestPathParent = D:/pathtest/ABC
```

In the last example we go one further level up (`pathtest`). Because of the file search is recursive, also files in parallel trees are found now.

```
sStartPath = r"D:\pathtest\ABC\DEF\GHI"
sFolderName = "pathtest"
sFileName = "FILE.txt"
sDestPath, listDestPaths, sDestFile, listDestFiles, sDestPathParent = CString.↵
    ↪ DetectParentPath(sStartPath, sFolderName, sFileName)
```

Outcome

```
sDestPath      = D:/pathtest
listDestPaths  = ['D:/pathtest']
sDestFile      = D:/pathtest/ABC/DEF/GHI/FILE.txt
listDestFiles  = ['D:/pathtest/ABC/DEF/GHI/FILE.txt', 'D:/pathtest/ABC/FILE.txt', 'D:/↵
    ↪ pathtest/RST/UVW/XYZ/FILE.txt']
sDestPathParent = D:/
```

StringFilter

During the computation of strings there might occur the need to get to know if this string fulfils certain criteria or not. Such a criterion can e.g. be that the string contains a certain substring. Also an inverse logic might be required: In this case the criterion is that the string does **not** contain this substring.

It might also be required to combine several criteria to a final conclusion if in total the criterion for a string is fulfilled or not. For example: The string must start with the string `prefix` and must also contain either the string `substring1` or the string `substring2` but must also **not** end with the string `suffix`.

This method provides a bunch of predefined filters that can be used singly or combined to come to a final conclusion if the string fulfils all criteria or not.

These filters can be e.g. used to select or exclude lines while reading from a text file. Or they can be used to select or exclude files or folders while walking through the file system. The filters are divided into three different types:

1. Filters that are interpreted as raw strings (called 'standard filters'; no wild cards supported)
2. Filters that are interpreted as regular expressions (called 'regular expression based filters'; the syntax of regular expressions has to be considered)
3. Boolean switches (e.g. indicating if also an empty string is accepted or not)

The input string might contain leading and trailing blanks and tabs. This kind of horizontal space is removed from the input string before the standard filters start their work (except the regular expression based filters).

The regular expression based filters consider the original input string (including the leading and trailing space).

The outcome is that in case of the leading and trailing space shall be part of the criterion, the regular expression based filters can be used only.

It is possible to decide if the standard filters shall work case sensitive or not. This decision has no effect on the regular expression based filters.

The regular expression based filters always work with the original input string that is not modified in any way.

Except the regular expression based filters it is possible to provide more than one string for every standard filter (must be a semikolon separated list in this case). A semicolon that shall be part of the search string, has to be masked in this way: `"\";`.

This method returns a boolean value that is `True` in case of all criteria are fulfilled, and `False` in case of some or all of them are not fulfilled.

The default value for all filters is `None` (except `bSkipBlankStrings`). In case of a filter value is `None` this filter has no influence on the result.

In case of all filters are `None` (default) the return value is `True` (except the string itself is `None` or the string is empty and `bSkipBlankStrings` is `True`).

In case of the string is `None`, the return value is `False`, because nothing concrete can be done with `None` strings.

Internally every filter has his own individual acknowledge that indicates if the criterion of this filter is fulfilled or not.

The meaning of *criterion fulfilled* of a filter is that the filter supports the final return value `bAck` of this method with `True`.

The final return value `bAck` of this method is a logical join (AND) of all individual acknowledges (except `bSkipBlankStrings` and `sComment`; in case of their criteria are fulfilled, immediately `False` is returned).

Summarized:

- Filters are used to define *criteria*
- The return value of this method provides the *conclusion* - indicating if all criteria are fulfilled or not

All available filters are described in more detail in the interface description of `StringFilter`. Here we continue with some code examples.

Example 1

`sString` has to start with `sStartsWith` and has to contain `sContains`.

That's true. Therefore `StringFilter` returns `True`.

```
StringFilter(sString      = "Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = "Sp",
             sEndsWith     = None,
             sStartsNotWith = None,
             sEndsNotWith  = None,
             sContains     = "beats",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 2

`sString` must not end with `sEndsNotWith`. But does. Therefore `StringFilter` returns `False` - even in case of other criterions are fulfilled.

```
StringFilter(sString      = "Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = "Sp",
             sEndsWith     = None,
             sStartsNotWith = None,
             sEndsNotWith  = "minute",
             sContains     = "beats",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 3

`sString` must not contain `sContainsNot`. Because `bCaseSensitive` is `True` the spelling does not fit. Therefore `StringFilter` returns `True`.

```
StringFilter(sString      = "Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsNotWith = None,
             sEndsNotWith  = None,
             sContains     = None,
             sContainsNot  = "Beats",
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 4

`sString` must contain exactly two digits (postulated by regular expression based `sInclRegex`). That's true. Therefore `StringFilter` returns `True`.

```
StringFilter(sString      = "Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = None,
             sContainsNot  = None,
             sInclRegex    = r"\d{2}",
             sExclRegex    = None)
```

Example 5

`sString` must contain exactly three digits (postulated by regular expression based `sInclRegex`). That's not true. Therefore `StringFilter` returns `False` - even in case of other criterions are fulfilled.

```
StringFilter(sString      = "Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = "Speed",
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = None,
             sContainsNot  = None,
             sInclRegex    = r"\d{3}",
             sExclRegex    = None)
```

Example 6

Leading and trailing spaces are removed from the input string `sString` at the beginning. In this example the result is an empty input string. `bSkipBlankStrings` is set to `True`. In this case `StringFilter` immediately returns `False` and all other filters are ignored.

```
StringFilter(sString      = "      ",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = None,
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 7

The input string `sString` starts with a character that is defined to be a comment character (`sComment`). Therefore `StringFilter` immediately returns `False` and all other filters are ignored.

```
StringFilter(sString      = "# Speed is 25 beats per minute",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = "#",
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = "beats",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 8

Blanks around search strings (here `sContains` is `" Alpha "`) are considered, whereas the blanks around the input string are removed before computation. Therefore `" Alpha "` cannot be found within the (shortened) input string and `StringFilter` returns `False`.

```
StringFilter(sString      = " Alpha is not beta; and beta is not gamma ",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = " Alpha ",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 9

In case of blanks around search strings have to be considered, a regular expression based filter has to be used (here `sInclRegex`).

This is possible because the regular expression based filters `sInclRegex` and `sExclRegex` work **with the original value** of `sString`, and not with the shortened version with leading and trailing blanks removed! The shortened version is applied to standard filters only.

In this example `StringFilter` returns `True`.

```
StringFilter(sString      = " Alpha is not beta; and beta is not gamma ",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = None,
             sContainsNot  = None,
             sInclRegex    = r"\s{3}Alpha",
             sExclRegex    = None)
```


Example 10

The meaning of `"beta; and"` in this example is: The criterion is fulfilled in case of either `"beta"` or `" and"` can be found. That's true - but this has nothing to do with the fact, that also this string `"beta; and"` can be found. The semikolon is a separator character and therefore part of the syntax.

Nevertheless `StringFilter` returns `True`.

```
StringFilter(sString      = "Alpha is not beta; and beta is not gamma",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = "beta; and",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

Example 11

In this example the semicolon is masked with `"\";`" and therefore part of the search string `sContains` - and not part of the syntax any more.

The meaning of `"beta\; not"` in this example is: The criterion is fulfilled in case of `"beta; not"` can be found. That's **not** `True`. Therefore `StringFilter` returns `False`.

```
StringFilter(sString      = "Alpha is not beta; and beta is not gamma",
             bCaseSensitive = True,
             bSkipBlankStrings = True,
             sComment      = None,
             sStartsWith   = None,
             sEndsWith     = None,
             sStartsWithNot = None,
             sEndsWithNot  = None,
             sContains     = r"beta\; not",
             sContainsNot  = None,
             sInclRegex    = None,
             sExclRegex    = None)
```

2.2.2 File access with CFile

The motivation for the `CFile` module contains two main topics:

1. Extended user control by introducing further parameter for file access functions. With high priority `CFile` enables the user to take care about that nothing existing is overwritten accidentally.
2. Hide the file handles und use the mechanism of class variables to avoid access violations independent from the way different operation systems like Windows and Unix are handling this.

This shortens the code, eases the implementation and makes tests (in which this module is used) more stable.

Examples

Define two variables with path and name of test files.

Under Windows:

```
sFile_1 = r"%TMP%\CFile_TestFile_1.txt"
sFile_2 = r"%TMP%\CFile_TestFile_2.txt"
```

Or under Linux:

```
sFile_1 = r"/tmp/CFile_TestFile_1.txt"
sFile_2 = r"/tmp/CFile_TestFile_2.txt"
```

The first class instance:

```
oFile_1 = CFile(sFile_1)
```

`oFile_1` is the instance of a class - *and not the file handle*. The file handle is hidden, the user has nothing to do with it.

Every class instance can work with one single file only (during the complete instance lifetime) and has exclusive access to this file.

No other class instance is allowed to use this file. Therefore the second line in the following code throws an exception:

```
oFile_1_A = CFile(sFile_1)
oFile_1_B = CFile(sFile_1)
```

It's more save to implement in this way:

```
try:
    oFile_1 = CFile(sFile_1)
except Exception as reason:
    print(str(reason))
```

For writing content to files two methods are available: `Write()` and `Append()`.

- Using `Write()` causes the class to open the file for writing (`w`) - in case of the file is not already opened for writing.
- Using `Append()` causes the class to open the file for appending (`a`) - in case of the file is not already opened for appending.

Switching between `Write()` and `Append()` causes an intermediate file handle `close()` internally!

Write some content to file:

```
bSuccess, sResult = oFile_1.Write("A B C")
print(f"sResult oFile_1.Write : '{sResult}' / bSuccess : {bSuccess}")
```

Most of the functions return at least `bSuccess` and `sResult`.

- `bSuccess` is `True` in case of no error occurred.

CHAPTER 2. DESCRIPTION

2.2. METHODS

- `bSuccess` is `False` in case of an error occurred.
- `bSuccess` is `None` in case of a very fatal error occurred - like an exception.
- `sResult` contains details about what happens during computation.

It is possible now to continue with using `oFile_1.Write("...")`; the content will be appended - as long as the file is still open for writing.

Some functions close the file handle (e.g. `ReadLines()`). Therefore sequences like

```
oFile_1.Write("...")
oFile_1.ReadLines("...")
oFile_1.Write("...")
```

should be avoided - because `Write()` after `ReadLines()` starts the file from scratch and the file content written by the previous `Write()` calls is lost.

For appending content to a file use the function `Append()`.

Append content to file:

```
bSuccess, sResult = oFile_1.Append("A B C")
```

For reading content from a file use the function `ReadLines()`.

Read from file:

```
listLines_1, bSuccess, sResult = oFile_1.ReadLines()
for sLine in listLines_1:
    print(f"{sLine}")
```

Additionally to `bSuccess` and `sResult` the function returns a list of lines.

Internally `ReadLines()` takes care about:

- Closing the file - in case the file is still opened
- Opening the file for reading
- Reading the content line by line until the end of file is reached
- Closing the file

To avoid code like this

```
for sLine in listLines_1:
    print(f"{sLine}")
```

it is also possible to let `ReadLines()` do this:

```
listLines_1, bSuccess, sResult = oFile_1.ReadLines(bToScreen=True)
```

A function to read only a single line from a file is not available, but it is possible to use some filter parameter of `ReadLines()` to reduce the amount of content already during the file is read. This prevents the user from implementing further loops.

Internally `ReadLines()` uses the string filter method `StringFilter()`. All filter related input parameter of `ReadLines()` and `StringFilter()` are the same.

Let's assume the following:

- The file `sFile_1` contains empty lines
- The file `sFile_1` contains also lines, that are commented out (with a hash (`#`) at the beginning)
- We want `ReadLines()` to skip empty lines and lines that are commented out

CHAPTER 2. DESCRIPTION

2.2. METHODS

This can be implemented in the following way.

Read a subset of file content:

```
listLines_1, bSuccess, sResult = oFile_1.ReadLines(bSkipBlankLines=True,
                                                    sComment='#')
```

It is a good practice to close file handles as soon as possible. Therefore `CFile` provides the possibility to do this explicitly.

Close a file handle:

```
bSuccess, sResult = oFile_1.Close()
```

This makes sense in case of later again access to this file is needed (the class object `oFile_1` still exists).

Additionally to that the file handle is closed implicitly:

- in case of it is required (e.g. when switching between read and write access),
- in case of the class instance is destroyed.

Therefore an alternative to the `Close()` function is the deletion of the class instance:

```
del oFile_1
```

This makes sense in case of access to this file is not needed any more (therefore we also do not need the class object any more).

It is recommended to prefer `del` (instead of `Close()`) to avoid to keep too much not used objects for a too long length of time in memory.

A file can be copied to another file.

Copy a file:

```
bSuccess, sResult = oFile_1.CopyTo(sFile_2)
```

The destination (`sFile_2` in the example above) can either be a full path and name of a file or the path only.

It makes a difference if the destination file exists or not. The optional parameter `bOverwrite` controls the behavior of `CopyTo()`.

The default is that it is not allowed to overwrite an existing destination file: `bOverwrite` is `False`. `CopyTo()` returns `bSuccess = False` in this case.

In case the user want to allow `CopyTo()` to overwrite existing destination files, it has to be coded explicitly:

```
bSuccess, sResult = oFile_1.CopyTo(sFile_2, bOverwrite=True)
```

A file can be moved to another file.

Move a file:

```
bSuccess, sResult = oFile_1.MoveTo(sFile_2)
```

Also `MoveTo()` supports `bOverwrite`. The behavior is the same as `CopyTo()`.

A file can be deleted.

Delete a file:

```
bSuccess, sResult = oFile_1.Delete()
```

It is possible to distinguish between two different motivations to delete a file:

1. **Explicitly do a deletion**
This requires that the file to be deleted, does exist.
2. **Making sure only that the files does not exist**
In this case it doesn't matter that maybe there is nothing to delete because the file already does not exist.

The optional parameter `bConfirmDelete` controls this behavior.

Default is that `Delete()` requires an existing file to delete:

```
bSuccess, sResult = oFile_1.Delete(bConfirmDelete=True)
```

In case of the file does not exist, `Delete()` returns `bSuccess = False`.

`Delete()` also returns `bSuccess = False|None` in case of an existing file cannot be deleted (e.g. because of an access violation).

If it doesn't matter if the file exists or not, it has to be coded explicitly:

```
bSuccess, sResult = oFile_1.Delete(bConfirmDelete=False)
```

In this case `Delete()` only returns `bSuccess = False|None` in case of an existing file cannot be deleted (e.g. because of an access violation).

Avoid access violations

Like already mentioned above every instance of `CFile` has an exclusive access to it's own file.

Only in case of `CopyTo()` and `MoveTo()` other files are involved: the destination files.

To avoid access violations it is not possible to copy or move a file to another file, that is under access of another instance of `CFile`.

In the following example `oFile_1.CopyTo(sFile_2)` returns `bSuccess = False` because `sFile_2` is already in access by `oFile_2`.

```
oFile_1 = CFile(sFile_1)
bSuccess, sResult = oFile_1.Write("A B C")

oFile_2 = CFile(sFile_2)
listLines_2, bSuccess, sResult = oFile_2.ReadLines()

bSuccess, sResult = oFile_1.CopyTo(sFile_2)

del oFile_1
del oFile_2
```

The solution is to delete the class instances as early as possible.

In the following example the copying is successful:

```
oFile_1 = CFile(sFile_1)
bSuccess, sResult = oFile_1.Write("A B C")

oFile_2 = CFile(sFile_2)
listLines_2, bSuccess, sResult = oFile_2.ReadLines()
del oFile_2

bSuccess, sResult = oFile_1.CopyTo(sFile_2)
del oFile_1
```

2.2.3 Utilities

PrettyPrint

The idea behind the `PrettyPrint()` function is to resolve the content of composite data types and provide for every parameter inside:

- the type
- the total number of elements inside (e.g. the number of keys inside a dictionary)
- the counter number of the current element
- the value

Example

The following Python code defines a composite data type and prints the content with `PrettyPrint()`:

```
dictTest = {'K1' : 'value',
            'K2' : ["A", 22, True, (33, 'XYZ')],
            'K3' : 10,
            'K4' : {'A' : 1,
                    'B' : 2}}

PrettyPrint(dictTest)
```

Result

```
[DICT] (4/1) > {K1} [STR] : 'value'
[DICT] (4/2) > {K2} [LIST] (4/1) > [STR] : 'A'
[DICT] (4/2) > {K2} [LIST] (4/2) > [INT] : 22
[DICT] (4/2) > {K2} [LIST] (4/3) > [BOOL] : True
[DICT] (4/2) > {K2} [LIST] (4/4) > [TUPLE] (2/1) > [INT] : 33
[DICT] (4/2) > {K2} [LIST] (4/4) > [TUPLE] (2/2) > [STR] : 'XYZ'
[DICT] (4/3) > {K3} [INT] : 10
[DICT] (4/4) > {K4} [DICT] (2/1) > {A} [INT] : 1
[DICT] (4/4) > {K4} [DICT] (2/2) > {B} [INT] : 2
```

Every line of output has to be interpreted strictly from left to right.

For example the meaning of the fifth line of output

```
[DICT] (4/2) > {K2} [LIST] (4/4) > [TUPLE] (2/1) > [INT] : 33
```

is:

- The type of input parameter `oData` is `dict`
- The dictionary contains 4 keys
- The current line gives information about the second key of the dictionary
- The name of the second key is `K2`
- The value of the second key is of type `list`
- The list contains 4 elements
- The current line gives information about the fourth element of the list
- The fourth element of the list is of type `tuple`
- The tuple contains 2 elements
- The current line gives information about the first element of the tuple
- The first element of the tuple is of type `int` and has the value `33`

Types are encapsulated in square brackets, counter in round brackets and key names are encapsulated in curly brackets.

Chapter 3

CComparison.py

The class `CComparison` contains mechanisms to compare two files either based on the original version of these files or based on an extract (made with regular expressions) to ensure that only relevant parts of the files are compared.

3.1 Method: Compare

Compares two files. While reading in all files empty lines are skipped.

Arguments:

- `sFile_1`
/ *Condition*: required / *Type*: str /
First file used for comparison.
- `sFile_2`
/ *Condition*: required / *Type*: str /
Second file used for comparison.
- `sPatternFile`
/ *Condition*: optional / *Type*: str / *Default*: None /
Pattern file containing a set of regular expressions (line by line). The regular expressions are used to make an extract of both input files. In this case the extracts are compared (instead of the original file content).
- `sIgnorePatternFile`
/ *Condition*: optional / *Type*: str / *Default*: None /
Pattern file containing a set of strings (**not** regular expressions; line by line). Every line containing one of the strings, is skipped.

Returns:

- `bIdentical`
/ *Type*: bool /
Indicates if the two input files (or their extracts) have the same content or not.
- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method was successful or not.
- `sResult`
/ *Type*: str /
The result of the computation of the method.

Chapter 4

CFile.py

4.1 Class: `enFileStatType`

The class `enFileStatType` defines the following file states:

- `closed`
- `openedforwriting`
- `openedforappending`
- `openedforreading`

4.2 Class: `CFile`

The class `CFile` provides a small set of file functions with extended parametrization (like switches defining if a file is allowed to be overwritten or not).

Most of the functions at least returns `bSuccess` and `sResult`.

- `bSuccess` is `True` in case of no error occurred.
- `bSuccess` is `False` in case of an error occurred.
- `bSuccess` is `None` in case of a very fatal error occurred (exceptions).
- `sResult` contains details about what happens during computation.

Every instance of `CFile` handles one single file only and forces exclusive access to this file.

It is not possible to create an instance of this class with a file that is already in use by another instance.

It is also not possible to use `CopyTo` or `MoveTo` to overwrite files that are already in use by another instance. This makes the file handling more save against access violations.

4.2.1 Method: `Close`

Closes the opened file.

Arguments:

(no args)

Returns:

- `bSuccess`
/ *Type*: `bool` /
Indicates if the computation of the method was successful or not.
- `sResult`
/ *Type*: `str` /
The result of the computation of the method.

4.2.2 Method: Delete

Deletes the current file.

Arguments:

- `bConfirmDelete`
/ Condition: optional / Type: bool / Default: True /
 Defines if it will be handled as error if the file does not exist.
 If `True`: If the file does not exist, the method indicates an error (`bSuccess = False`).
 If `False`: It doesn't matter if the file exists or not.

Returns:

- `bSuccess`
/ Type: bool /
 Indicates if the computation of the method was successful or not.
- `sResult`
/ Type: str /
 The result of the computation of the method.

4.2.3 Method: Write

Writes the content of a variable `Content` to file.

Arguments:

- `Content`
/ Condition: required / Type: one of: str, list, tuple, set, dict, dict /
 If `Content` is not a string, the `Write` method resolves the data structure before writing the content to file.
- `nVSpaceAfter`
/ Condition: optional / Type: int / Default: 0 /
 Adds vertical space `nVSpaceAfter` (= number of blank lines) after `Content`.
- `sPrefix`
/ Condition: optional / Type: str / Default: None /
`sPrefix` is added to every line of output (in case of `sPrefix` is not `None`).
- `bToScreen`
/ Condition: optional / Type: bool / Default: False /
 Prints `Content` also to screen (in case of `bToScreen` is `True`).

Returns:

- `bSuccess`
/ Type: bool /
 Indicates if the computation of the method was successful or not.
- `sResult`
/ Type: str /
 The result of the computation of the method.

4.2.4 Method: Append

Appends the content of a variable `Content` to file.

Arguments:

- `Content`
/ *Condition*: required / *Type*: one of: str, list, tuple, set, dict, dict /
If `Content` is not a string, the `Write` method resolves the data structure before writing the content to file.
- `nVSpaceAfter`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Adds vertical space `nVSpaceAfter` (= number of blank lines) after `Content`.
- `sPrefix`
/ *Condition*: optional / *Type*: str / *Default*: None /
`sPrefix` is added to every line of output (in case of `sPrefix` is not None).
- `bToScreen`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Prints `Content` also to screen (in case of `bToScreen` is True).

Returns:

- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method was successful or not.
- `sResult`
/ *Type*: str /
The result of the computation of the method.

4.2.5 Method: ReadLines

Reads content from current file. Returns an array of lines together with `bSuccess` and `sResult` (feedback).

The method takes care of opening and closing the file. The complete file content is read by `ReadLines` in one step, but with the help of further parameters it is possible to reduce the content by including and excluding lines.

Internally `ReadLines` uses the string filter method `StringFilter`. All filter related input parameter of `ReadLines` and `StringFilter` are the same.

The logical join of all filter is: AND.

Arguments:

- `bCaseSensitive`
/ *Condition*: optional / *Type*: bool / *Default*: True /
– If True, the standard filters work case sensitive, otherwise not.
– This has no effect to the regular expression based filters `sInclRegex` and `sExclRegex`.
- `bSkipBlankLines`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If True, blank lines will be skipped, otherwise not.
- `sComment`
/ *Condition*: optional / *Type*: str / *Default*: None /
In case of a line starts with the string `sComment`, this line is skipped.

- `sStartsWith`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string starts with the string `sStartsWith`
 - More than one string can be provided (semicolon separated; logical join: OR)
- `sEndsWith`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string ends with the string `sEndsWith`
 - More than one string can be provided (semicolon separated; logical join: OR)
- `sStartsNotWith`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string starts not with the string `sStartsNotWith`
 - More than one string can be provided (semicolon separated; logical join: AND)
- `sEndsNotWith`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string ends not with the string `sEndsNotWith`
 - More than one string can be provided (semicolon separated; logical join: AND)
- `sContains`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string contains the string `sContains` at any position
 - More than one string can be provided (semicolon separated; logical join: OR)
- `sContainsNot`
/ Condition: optional / Type: str / Default: None /
 - The criterion of this filter is fulfilled in case of the input string does **not** contain the string `sContainsNot` at any position
 - More than one string can be provided (semicolon separated; logical join: AND)
- `sInclRegex`
/ Condition: optional / Type: str / Default: None /
 - *Include* filter based on regular expressions (consider the syntax of regular expressions!)
 - The criterion of this filter is fulfilled in case of the regular expression `sInclRegex` matches the input string
 - Leading and trailing blanks within the input string are considered
 - `bCaseSensitive` has no effect
 - A semicolon separated list of several regular expressions is **not** supported
- `sExclRegex`
/ Condition: optional / Type: str / Default: None /
 - *Exclude* filter based on regular expressions (consider the syntax of regular expressions!)
 - The criterion of this filter is fulfilled in case of the regular expression `sExclRegex` does **not** match the input string
 - Leading and trailing blanks within the input string are considered
 - `bCaseSensitive` has no effect
 - A semicolon separated list of several regular expressions is **not** supported

- `bLStrip`
/ Condition: optional / Type: bool / Default: False /
 If `True`, leading spaces are removed from line before the filters are used, otherwise not.
- `bRStrip`
/ Condition: optional / Type: bool / Default: True /
 If `True`, trailing spaces are removed from line before the filters are used, otherwise not.
- `bToScreen`
/ Condition: optional / Type: bool / Default: False /
 If `True`, the content read from file is also printed to screen, otherwise not.

4.2.6 Method: GetFileInfo

Returns the following informations about the file (encapsulated within a dictionary `dFileInfo`):

Returns:

- Key `sFile`
/ Type: str /
 Path and name of current file
- Key `bFileIsExisting`
/ Type: bool /
 True if file is existing, otherwise `False`
- Key `sFileName`
/ Type: str /
 The name of the current file (incl. extension)
- Key `sFileExtension`
/ Type: str /
 The extension of the current file
- Key `sFileNameOnly`
/ Type: str /
 The pure name of the current file (without extension)
- Key `sFilePath`
/ Type: str /
 The the path to current file
- Key `bFilePathIsExisting`
/ Type: bool /
 True if file path is existing, otherwise `False`

4.2.7 Method: CopyTo

Copies the current file to `sDestination`, that can either be a path without file name or a path together with a file name.

In case of the destination file already exists and `bOverwrite` is `True`, than the destination file will be overwritten.

In case of the destination file already exists and `bOverwrite` is `False` (default), than the destination file will not be overwritten and `CopyTo` returns `bSuccess = False`.

Arguments:

- `sDestination`
/ Condition: required / Type: string /
 The path to destination file (either incl. file name or without file name)
- `bOverwrite`
/ Condition: optional / Type: bool / Default: False /
 - In case of the destination file already exists and `bOverwrite` is `True`, than the destination file will be overwritten.
 - In case of the destination file already exists and `bOverwrite` is `False` (default), than the destination file will not be overwritten and `CopyTo` returns `bSuccess = False`.

Returns:

- `bSuccess`
/ Type: bool /
 Indicates if the computation of the method was successful or not.
- `sResult`
/ Type: str /
 The result of the computation of the method.

4.2.8 Method: MoveTo

Moves the current file to `sDestination`, that can either be a path without file name or a path together with a file name.

Arguments:

- `sDestination`
/ Condition: required / Type: string /
 The path to destination file (either incl. file name or without file name)
- `bOverwrite`
/ Condition: optional / Type: bool / Default: False /
 - In case of the destination file already exists and `bOverwrite` is `True`, than the destination file will be overwritten.
 - In case of the destination file already exists and `bOverwrite` is `False` (default), than the destination file will not be overwritten and `MoveTo` returns `bSuccess = False`.

Returns:

- `bSuccess`
/ Type: bool /
 Indicates if the computation was successful or not
- `sResult`
/ Type: str /
 Contains details about what happens during computation

Chapter 5

CFolder.py

5.1 Function: `rm_dir_readonly`

Calls `os.chmod` in case of `shutil.rmtree` (within `Delete()`) throws an exception (making files writable).

5.2 Class: `CFolder`

The class `CFolder` provides a small set of folder functions with extended parametrization (like switches defining if a folder is allowed to be overwritten or not).

Most of the functions at least returns `bSuccess` and `sResult`.

- `bSuccess` is `True` in case of no error occurred.
- `bSuccess` is `False` in case of an error occurred.
- `bSuccess` is `None` in case of a very fatal error occurred (exceptions).
- `sResult` contains details about what happens during computation.

Every instance of `CFolder` handles one single folder only and forces exclusive access to this folder.

It is not possible to create an instance of this class with a folder that is already in use by another instance.

The constructor of `CFolder` requires the input parameter `sFolder`, that is the path and the name of a folder that is handled by the current class instance.

5.2.1 Method: `Delete`

Deletes the folder the current class instance contains.

Arguments:

- `bConfirmDelete`
/ Condition: optional / Type: bool / Default: True /
 Defines if it will be handled as error if the folder does not exist.
 If `True`: If the folder does not exist, the method indicates an error (`bSuccess = False`).
 If `False`: It doesn't matter if the folder exists or not.

Returns:

- `bSuccess`
/ Type: bool /
 Indicates if the computation of the method was successful or not.
- `sResult`
/ Type: str /
 The result of the computation of the method.

5.2.2 Method: Create

Creates the current folder sFolder.

Arguments:

- bOverwrite
/ Condition: optional / Type: bool / Default: False /
 - In case of the folder already exists and bOverwrite is True, than the folder will be deleted before creation.
 - In case of the folder already exists and bOverwrite is False (default), than the folder will not be touched.

In both cases the return value bSuccess is True - because the folder exists.
- bRecursive
/ Condition: optional / Type: bool / Default: False /
 - In case of bRecursive is True, than the complete destination path will be created (including all intermediate subfolders).
 - In case of bRecursive is False, than it is expected that the parent folder of the new folder already exists.

Returns:

- bSuccess
/ Type: bool /
Indicates if the computation of the method was successful or not.
- sResult
/ Type: str /
The result of the computation of the method.

5.2.3 Method: CopyTo

Copies the current folder to sDestination, that has to be a path to a folder **within** the source folder will be copied to (with it's original name),

In case of the destination folder already exists and bOverwrite is True, than the destination folder will be overwritten.

In case of the destination folder already exists and bOverwrite is False (default), than the destination folder will not be overwritten and CopyTo returns bSuccess = False.

Arguments:

- sDestination
/ Condition: required / Type: string /
The path to destination folder
- bOverwrite
/ Condition: optional / Type: bool / Default: False /
 - In case of the destination folder already exists and bOverwrite is True, than the destination folder will be overwritten.
 - In case of the destination folder already exists and bOverwrite is False (default), than the destination folder will not be overwritten and CopyTo returns bSuccess = False.

Returns:

- bSuccess
/ Type: bool /
Indicates if the computation of the method was successful or not.

- `sResult`
/ *Type*: str /
The result of the computation of the method.

Chapter 6

CString.py

The class `CString` contains some string computation methods like e.g. normalizing a path.

6.1 Method: NormalizePath

Normalizes local paths, paths to local network resources and internet addresses

Arguments:

- `sPath`
/ Condition: required */ Type:* str */*
 The path to be normalized. Paths can start with environment variables. Accepted are notations for both Windows (`%ENVVAR%`) and Linux (`${ENVVAR}`). Under Windows also the Linux notation will be resolved.
- `bWin`
/ Condition: optional */ Type:* bool */ Default:* False */*
 If `True` then returned path contains masked backslashes as separator, otherwise slashes
- `sReferencePathAbs`
/ Condition: optional */ Type:* str */ Default:* None */*
 In case of `sPath` is relative and `sReferencePathAbs` (expected to be absolute) is given, then the returned absolute path is a join of both input paths
- `bConsiderBlanks`
/ Condition: optional */ Type:* bool */ Default:* False */*
 If `True` then the returned path is encapsulated in quotes - in case of the path contains blanks
- `bExpandEnvVars`
/ Condition: optional */ Type:* bool */ Default:* True */*
 If `True` then in the returned path environment variables are resolved, otherwise not.
- `bMask`
/ Condition: optional */ Type:* bool */ Default:* True (requires `bWin=True`)*/*
 - If `bWin` is `True` and `bMask` is `True` then the returned path contains masked backslashes as separator.
 - If `bWin` is `True` and `bMask` is `False` then the returned path contains single backslashes only - this might be required for applications, that are not able to handle masked backslashes.
 - In case of `bWin` is `False` `bMask` has no effect.

Returns:

- `sPath`
/ Type: str */*
 The normalized path (is `None` in case of `sPath` is `None`)

6.2 Method: DetectParentPath

Computes the path to any parent folder inside a given path. Optionally DetectParentPath is able to search for files inside the identified parent folder.

Arguments:

- `sStartPath`
/ *Condition*: required / *Type*: str /
The path in which to search for a parent folder
- `sFolderName`
/ *Condition*: required / *Type*: str /
The name of the folder to search for within `sStartPath`. It is possible to provide more than one folder name separated by semicolon
- `sFileName`
/ *Condition*: optional / *Type*: str / *Default*: None /
The name of a file to search within the detected parent folder

Returns:

- `sDestPath`
/ *Type*: str /
Path and name of parent folder found inside `sStartPath`, None in case of `sFolderName` is not found inside `sStartPath`. In case of more than one parent folder is found `sDestPath` contains the first result and `listDestPaths` contains all results.
- `listDestPaths`
/ *Type*: list /
If `sFolderName` contains a single folder name this list contains only one element that is `sDestPath`. In case of `FolderName` contains a semicolon separated list of several folder names this list contains all found paths of the given folder names. `listDestPaths` is None (and not an empty list!) in case of `sFolderName` is not found inside `sStartPath`.
- `sDestFile`
/ *Type*: str /
Path and name of `sFileName`, in case of `sFileName` is given and found inside `listDestPaths`. In case of more than one file is found `sDestFile` contains the first result and `listDestFiles` contains all results. `sDestFile` is None in case of `sFileName` is None and also in case of `sFileName` is not found inside `listDestPaths` (and therefore also in case of `sFolderName` is not found inside `sStartPath`).
- `listDestFiles`
/ *Type*: list /
Contains all positions of `sFileName` found inside `listDestPaths`.
`listDestFiles` is None (and not an empty list!) in case of `sFileName` is None and also in case of `sFileName` is not found inside `listDestPaths` (and therefore also in case of `sFolderName` is not found inside `sStartPath`).
- `sDestPathParent`
/ *Type*: str /
The parent folder of `sDestPath`, None in case of `sFolderName` is not found inside `sStartPath` (`sDestPath` is None).

6.3 Method: StringFilter

This method provides a bunch of predefined filters that can be used singly or combined to come to a final conclusion if the string fulfils all criteria or not.

These filters can be e.g. used to select or exclude lines while reading from a text file. Or they can be used to select or exclude files or folders while walking through the file system.

The following filters are available:

bSkipBlankStrings

- Leading and trailing spaces are removed from the input string at the beginning
- In case of the result is an empty string and `bSkipBlankStrings` is `True`, the method immediately returns `False` and all other filters are ignored

sComment

- In case of the input string starts with the string `sComment`, the method immediately returns `False` and all other filters are ignored
- Leading blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- The idea behind this decision is: Ignore a string that is commented out

sStartsWith

- The criterion of this filter is fulfilled in case of the input string starts with the string `sStartsWith`
- Leading blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: `OR`)

sEndsWith

- The criterion of this filter is fulfilled in case of the input string ends with the string `sEndsWith`
- Trailing blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: `OR`)

sStartsWithNot

- The criterion of this filter is fulfilled in case of the input string does **not** start with the string `sStartsWithNot`
- Leading blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: `AND`)

sEndsWithNot

- The criterion of this filter is fulfilled in case of the input string does **not** end with the string `sEndsWithNot`
- Trailing blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: `AND`)

sContains

- The criterion of this filter is fulfilled in case of the input string contains the string `sContains` at any position

- Leading and trailing blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: OR)

sContainsNot

- The criterion of this filter is fulfilled in case of the input string does **not** contain the string `sContainsNot` at any position
- Leading and trailing blanks within the input string have no effect
- The decision also depends on `bCaseSensitive`
- More than one string can be provided (semicolon separated; logical join: AND)

sInclRegEx

- *Include* filter based on regular expressions (consider the syntax of regular expressions!)
- The criterion of this filter is fulfilled in case of the regular expression `sInclRegEx` matches the input string
- Leading and trailing blanks within the input string are considered
- `bCaseSensitive` has no effect
- A semicolon separated list of several regular expressions is **not** supported

sExclRegEx

- *Exclude* filter based on regular expressions (consider the syntax of regular expressions!)
- The criterion of this filter is fulfilled in case of the regular expression `sExclRegEx` does **not** match the input string
- Leading and trailing blanks within the input string are considered
- `bCaseSensitive` has no effect
- A semicolon separated list of several regular expressions is **not** supported

Further arguments:

- `sString`
/ *Condition*: required / *Type*: str /
The input string that has to be investigated.
- `bCaseSensitive`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, the standard filters work case sensitive, otherwise not.
- `bDebug`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If True, additional output is printed to console (e.g. the decision of every single filter), otherwise not.

Returns:

- `bAck`
/ *Type*: bool /
Final statement about the input string `sString` after filter computation

Further details together with code examples can be found within chapter **Description**, subsection **StringFilter**.

6.4 Method: FormatResult

Formats the result string `sResult` depending on `bSuccess`:

- `bSuccess` is `True` indicates *success*
- `bSuccess` is `False` indicates an *error*
- `bSuccess` is `None` indicates an *exception*

Additionally the name of the method that causes the result, can be provided (*optional*). This is useful for debugging.

Arguments:

- `sMethod`
/ *Condition*: optional / *Type*: str / *Default*: (empty string) /
Name of the method that causes the result.
- `bSuccess`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
/ *Condition*: optional / *Type*: str / *Default*: (empty string) /
The result of the computation of the method `sMethod`.

Returns:

- `sResult`
/ *Type*: str /
The formatted result string.

Chapter 7

CUtils.py

7.1 Function: PrettyPrint

Wrapper function to create and use a `CTypePrint` object. This wrapper function is responsible for printing out the content to console and to a file (depending on input parameter).

The content itself is prepared by the method `TypePrint` of class `CTypePrint`. This happens `PrettyPrint` internally.

The idea behind the `PrettyPrint` function is to resolve also the content of composite data types and provide for every parameter inside:

- the type
- the total number of elements inside (e.g. the number of keys inside a dictionary)
- the counter number of the current element
- the value

Arguments:

- `oData`
/ *Condition:* required / *Type:* (any Python data type) /
A variable of any Python data type.
- `hOutputFile`
/ *Condition:* optional / *Type:* file handle / *Default:* None /
If handle is not None the content is written to this file, otherwise not.
- `bToConsole`
/ *Condition:* optional / *Type:* bool / *Default:* True /
If True the content is written to console, otherwise not.
- `nIndent`
/ *Condition:* optional / *Type:* int / *Default:* 0 /
Sets the number of additional blanks at the beginning of every line of output (indentation).
- `sPrefix`
/ *Condition:* optional / *Type:* str / *Default:* None /
Sets a prefix string that is added at the beginning of every line of output.
- `bHexFormat`
/ *Condition:* optional / *Type:* bool / *Default:* False /
If True the output is printed in hexadecimal format (but valid for strings only).

Returns:

- `listOutLines` (*list*)
/ *Type*: list /
List of lines containing the prepared output

7.2 Class: CTypePrint

The class `CTypePrint` provides a method (`TypePrint`) to compute the following data:

- the type
- the total number of elements inside (e.g. the number of keys inside a dictionary)
- the counter number of the current element
- the value

of simple and composite data types.

The call of this method is encapsulated within the function `PrettyPrint` inside this module.

7.2.1 Method: TypePrint

The method `TypePrint` computes details about the input variable `oData`.

Arguments:

- `oData`
/ *Condition*: required / *Type*: any Python data type /
Python variable of any data type.
- `bHexFormat`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If `True` the output is provide in hexadecimal format.

Returns:

- `listOutLines`
/ *Type*: list /
List of lines containing the resolved content of `oData`.

7.3 Class: CUtils

The class `CUtils` contains useful methods.

7.3.1 Method: GetInstalledPackages

The method `GetInstalledPackages` computes a list of all installed Python packages. The list is returned as list of tuples containing the name and the version of the package.

It is also possible to let the method dump the list to a text file.

Arguments:

- `sOutputFile`
/ *Condition*: optional / *Type*: string / *Default*: None /
Path and name of a file to dump the package list to.

Returns:

- `listofTuplesPackages`
/ *Type*: list /
List of tuples containing the name and the version of the package.
- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method was successful or not.
- `sResult`
/ *Type*: str /
The result of the computation of the method.

7.3.2 Method: GetComponentName

Returns:

- `ComponentName`
/ *Type*: str /
Full name of the component including the version and the version date.

Chapter 8

Appendix

About PythonExtensionsCollection:

Author

Holger Queckenstedt (Holger.Queckenstedt@de.bosch.com)

Version

0.17.1 (22.06.2026)

Short Description

Additional Python functions

Homepage

[python-extensions-collection](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 9

History

Version 0.17.1 (22.06.2026)

Maintenance:

- Updated the package dependencies to add missing **dotdict** package.

Version 0.17.0 (28.05.2026)

Maintenance:

- Usage of **Setuptools** replaced by usage of **PIP/TOML/Setuptools**
- History converted from LaTeX format to reST format

Version 0.16.0 (29.01.2025)

Added:

- Method `CUtils.GetComponentName()`

Version 0.15.0 (30.05.2023)

Added:

- Method `CUtils.GetInstalledPackages()`

Version 0.14.0 (12.05.2023)

Removed:

- Computation of obsolete `dotdict` class from method `CUtils.CTypePrint.TypePrint`

Version 0.13.0 (03.04.2023)

Added:

- Ignore pattern file (in `CComparison`)

Version 0.12.0 (29.03.2023)

Added:

- Type `RobotFramework.TestsuitesManagement.Config.CConfig.dotdict` added to `CTypePrint`

Version 0.11.0 (15.11.2022)

CHAPTER 9. HISTORY

Maintenance:

- Converted parts of the documentation from reST format to LaTeX format (to improve the layout)

Version 0.10.0 (25.10.2022)*Added:*

- Module `CComparison` to compare two files (either based on original content or based on pattern)

Version 0.9.0 (27.07.2022)*Maintenance:*

- History reworked (requires **GenPackageDoc v. 0.26.0** at least)

Version 0.8.0 (28.06.2022)*Added:*

- Method: `CopyTo` (module `CFolder`)

Version 0.7.0 (10.06.2022)*Added:*

- Module `CFolder` added (with methods: `Create` and `Delete`)

Version 0.6.0 (02.06.2022)*Maintenance:*

- Code
- Documentation rebuild with **GenPackageDoc v. 0.16.0**

Version 0.5.0 (31.05.2022)*Maintenance:*

- Code
- Documentation rebuild with **GenPackageDoc v. 0.15.0**

Version 0.4.0 (24.05.2022)*Maintenance:*

- Code
- Documentation rebuild with **GenPackageDoc v. 0.13.0**

Version 0.3.0 (20.05.2022)*Changes:*

- Documentation tool chain switched to **GenPackageDoc**

Version 0.2.0 (02/2022)*Maintenance:*

- Code

CHAPTER 9. HISTORY

Version 0.1.0 (08/2021)

Initial version

[PythonExtensionsCollection in GitHub](#)

[PythonExtensionsCollection in PyPi](#)

PythonExtensionsCollection.pdf

Created at 07.09.2026 - 09:46:50

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.3 RobotframeworkExtensions

RobotframeworkExtensions

v. 0.12.0

Holger Queckenstedt

17.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Keywords	2
2.1.1	pretty_print	2
2.1.2	normalize_path	5
3	Collection.py	7
3.1	Class: Collection	7
3.1.1	Keyword: pretty_print	7
3.1.2	Keyword: normalize_path	8
3.1.3	Method: get_rf_parameters	8
3.1.4	Keyword: get_parameters	9
3.1.5	Method: log_rf_parameter	9
3.1.6	Keyword: log_parameter	10
3.1.7	Method: log_rf_metadata	11
3.1.8	Keyword: log_metadata	11
4	Appendix	12
5	History	13

Chapter 1

Introduction

The **robotframework-extensions-collection** extend the functionality of the Robot Framework by some useful keywords.

This covers for example string operations like normalizing a path and a pretty print method (especially for composite Python data types).

The sources of the **robotframework-extensions-collection** are available in [GitHub](#).

Informations about how to install the **robotframework-extensions-collection** can be found in the [README](#).

The **robotframework-extensions-collection** keywords are implemented in Python (as **PythonExtensionsCollection**) and the implementation can also be found in [GitHub](#).

Informations about how to install the **PythonExtensionsCollection** can be found in the [README](#).

Chapter 2

Description

2.1 Keywords

The `Collection` module of the **robotframework-extensions-collection** is the interface between the **PythonExtensionsCollection** and the RobotFramework AIO and contains the keyword definitions that can be imported in the following way:

```
Library    RobotframeworkExtensions.Collection    AS    rf.extensions
```

We recommend to use `AS` to shorten the long library name a little bit. That will make the robot code easier to read.

2.1.1 `pretty_print`

The `pretty_print` keyword logs the content of parameters of any Python data type. Simple data types are logged directly. Composite data types are resolved before.

The output contains for every parameter:

- the type
- the total number of elements inside (e.g. the number of keys inside a dictionary)
- the counter number of the current element
- the value

The trace level for output is `INFO`. The output is also returned as list of strings.

Example

The following RobotFramework AIO code defines - step by step - a parameter of composite data type (nested arrays and dictionaries) and prints the content of this with `pretty_print`:

```
set_test_variable    @{aItems1}    ${33}
...                  XYZ

set_test_variable    @{aItems}     A
...                  ${22}
...                  ${True}
...                  ${aItems1}

set_test_variable    &{dItems1}    A=${1}
...                  B=${2}

set_test_variable    &{dItems}     K1=value
...                  K2=${aItems}
...                  K3=${10}
...                  K4=${dItems1}

rf.extensions.pretty_print    ${dItems}
```

Result

```
[DOTDICT] (4/1) > {K1} [STR] : 'value'
[DOTDICT] (4/2) > {K2} [LIST] (4/1) > [STR] : 'A'
[DOTDICT] (4/2) > {K2} [LIST] (4/2) > [INT] : 22
[DOTDICT] (4/2) > {K2} [LIST] (4/3) > [BOOL] : True
[DOTDICT] (4/2) > {K2} [LIST] (4/4) > [LIST] (2/1) > [INT] : 33
[DOTDICT] (4/2) > {K2} [LIST] (4/4) > [LIST] (2/2) > [STR] : 'XYZ'
[DOTDICT] (4/3) > {K3} [INT] : 10
[DOTDICT] (4/4) > {K4} [DOTDICT] (2/1) > {A} [INT] : 1
[DOTDICT] (4/4) > {K4} [DOTDICT] (2/2) > {B} [INT] : 2
```

Every line of output has to be interpreted strictly from left to right.

For example the meaning of the fifth line of output

```
[DOTDICT] (4/2) > {K2} [LIST] (4/4) > [LIST] (2/1) > [INT] : 33
```

is:

- The type of input parameter `dItems` is `dotdict`
- The dictionary contains 4 keys
- The current line gives information about the second key of the dictionary
- The name of the second key is `K2`
- The value of the second key is of type `list`
- The list contains 4 elements
- The current line gives information about the fourth element of the list
- The fourth element of the list is of type `list`
- The list contains 2 elements
- The current line gives information about the first element of the list
- The first element of the list is of type `int` and has the value `33`

Types are encapsulated in square brackets, counter in round brackets and key names are encapsulated in curly brackets.

Prefix strings

Prefix strings (`sPrefix`) can be used to give the lines of output a meaning, or they are used just to print also the name of the pretty printed variable (`oData`).

Example

```
rf.extensions.pretty_print    ${dItems}    PrefixString
```

Result

```
PrefixString : [DOTDICT] (4/1) > {K1} [STR] : 'value'
PrefixString : [DOTDICT] (4/2) > {K2} [LIST] (4/1) > [STR] : 'A'
PrefixString : [DOTDICT] (4/2) > {K2} [LIST] (4/2) > [INT] : 22
PrefixString : [DOTDICT] (4/2) > {K2} [LIST] (4/3) > [BOOL] : True
PrefixString : [DOTDICT] (4/2) > {K2} [LIST] (4/4) > [LIST] (2/1) > [INT] : 33
PrefixString : [DOTDICT] (4/2) > {K2} [LIST] (4/4) > [LIST] (2/2) > [STR] : 'XYZ'
PrefixString : [DOTDICT] (4/3) > {K3} [INT] : 10
PrefixString : [DOTDICT] (4/4) > {K4} [DOTDICT] (2/1) > {A} [INT] : 1
PrefixString : [DOTDICT] (4/4) > {K4} [DOTDICT] (2/2) > {B} [INT] : 2
```

2.1.2 normalize_path

The `normalize_path` keyword normalizes local paths, paths to local network resources and internet addresses.

Background

It's not easy to handle paths - and especially the path separators - independent from the operating system.

Under Linux it is obvious that single slashes are used as separator within paths. Whereas the Windows explorer uses single backslashes. In both operating systems web addresses contains single slashes as separator when displayed in web browsers.

Using single backslashes within code - as content of string variables - is dangerous because the combination of a backslash and a letter can be interpreted as escape sequence - and this is maybe not the effect a user wants to have.

To avoid unwanted escape sequences backslashes have to be masked (by the usage of two of them: `"\\`"). But also this could not be the best solution because there are also applications (like the Windows explorer) that are not able to handle masked backslashes. They expect to get single backslashes within a path.

Preparing a path for best usage within code also includes collapsing redundant separators and up-level references. Python already provides functions to do this, but the outcome (path contains slashes or backslashes) depends on the operating system. And like already mentioned above also under Windows backslashes might not be the preferred choice.

It also has to be considered that redundant separators at the beginning of an address of a local network resource (like `\\server.com`) and or inside an internet address (like `https:\\server.com`) must **not** be collapsed! Unfortunately the Python function `normpath` does not consider this context.

To give the user full control about the format of a path, independent from the operating system and independent if it's a local path, a path to a local network resource or an internet address, the keyword `normalize_path` provides lot's of parameters to influence the result.

Example 1

Variable containing a path with:

- different types of path separators
- redundant path separators (*but backslashes have to be masked in the definition of the variable, this is **not** an unwanted redundancy*)
- up-level references

```
set_test_variable    ${sPath}    C:\\\\subfolder1\\\\.\\subfolder2\\\\\\\\\\\\\\.\\subfolder3↔
↔ \\\\
```

Printing the content of `sPath` shows how the path looks like when the masking of the backslashes is resolved:

```
C:\\subfolder1\\\\.\\subfolder2\\\\\\\\\\.\\subfolder3\\
```

Usage of the `normalize_path` keyword:

```
${sPath}    rf.extensions.normalize_path    ${sPath}
```

Result (content of `sPath`):

```
C:/subfolder3
```

In case we need the Windows version (with masked backslashes instead of slashes):

```
${sPath}    rf.extensions.normalize_path    ${sPath}    bWin=${True}
```

Result (content of `sPath`):

```
C:\\subfolder3
```

The masking of backslashes can be deactivated:

```
${sPath}    rf.extensions.normalize_path    ${sPath}    bWin=${True}    bMask=${False}
```

Result (content of `sPath`):

```
C:\subfolder3
```

Example 2

Variable containing a path of a local network resource (path starts with two masked backslashes):

```
set_test_variable    ${sPath}    \\anyserver.com\\part1\\part2\\part3\\part4
```

Result of normalization:

```
//anyserver.com/part1/part2/part3/part4
```

Example 3

Variable containing an internet address:

```
set_test_variable    ${sPath}    http:\\anyserver.com\\part1\\part2\\part3\\part4
```

Result of normalization:

```
http://anyserver.com/part1/part2/part3/part4
```

Chapter 3

Collection.py

The Collection module is the interface between the PythonExtensionsCollection and the Robot Framework. This library containing the keyword definitions, can be imported in the following way:

```
Library    RobotframeworkExtensions.Collection    AS    rf.extensions
```

3.1 Class: Collection

Module main class

3.1.1 Keyword: pretty_print

The pretty_print keyword logs the content of parameters of any Python data type (input: oData). Simple data types are logged directly. Composite data types are resolved before.

The output contains for every parameter:

- the type
- the total number of elements inside (e.g. the number of keys inside a dictionary)
- the counter number of the current element
- the value

The trace level for output is INFO.

The output is also returned as list of strings.

Arguments:

- oData
/ *Condition*: required / *Type*: any Python type /
Data to be pretty printed
- sPrefix
/ *Condition*: optional / *Type*: str / *Default*: None /
If not None, this prefix string is added to every output line.

Returns:

- listOutLines (*list*)
/ *Type*: list /
List of strings containing the resolved data structure of oData (same content as printed to console).

3.1.2 Keyword: `normalize_path`

The `normalize_path` keyword normalizes local paths, paths to local network resources and internet addresses

Arguments:

- `sPath`
/ Condition: required */ Type:* str */*
 The path to be normalized
- `bWin`
/ Condition: optional */ Type:* bool */ Default:* False */*
 If True then the returned path contains masked backslashes as separator, otherwise slashes
- `sReferencePathAbs`
/ Condition: optional */ Type:* str */ Default:* None */*
 In case of `sPath` is relative and `sReferencePathAbs` (expected to be absolute) is given, then the returned absolute path is a join of both input paths
- `bConsiderBlanks`
/ Condition: optional */ Type:* bool */ Default:* False */*
 If True then the returned path is encapsulated in quotes - in case of the path contains blanks
- `bExpandEnvVars`
/ Condition: optional */ Type:* bool */ Default:* True */*
 If True then in the returned path environment variables are resolved, otherwise not.
- `bMask`
/ Condition: optional */ Type:* bool */ Default:* True (requires `bWin=True`) */*
 If `bWin` is True and `bMask` is True then the returned path contains masked backslashes as separator.
 If `bWin` is True and `bMask` is False then the returned path contains single backslashes only - this might be required for applications, that are not able to handle masked backslashes.
 In case of `bWin` is False `bMask` has no effect.

Returns:

- `sPath`
/ Type: str */*
 The normalized path (is None in case of `sPath` is None)

3.1.3 Method: `get_rf_parameters`

the static method `get_rf_parameters` generates a dump of all Robot Framework parameters in the current scope (including the global parameters).

The output can be filtered (to limit the dumped parameters to the desired ones).

This method returns a dictionary containing all dumped parameters.

In this module the method `get_rf_parameters` is used by the keyword `get_parameters`. `get_rf_parameters` is made a separate static method to enable also other Python modules to use this method for their own needs.

All input filter parameters are explained in detail here: [PythonExtensionsCollection.pdf](#) (section 'String operations with CString', method 'StringFilter'). Notice: In the context of this method, the filter parameter of the **PythonExtensionsCollection** have been renamed to a decapitalized format without type prefix.

The actual names of all input filter parameters are: `casesensitive`, `startswith`, `endswith`, `startswithnotwith`, `endsnotwith`, `contains`, `containsnot`, `inclregex`, `exclregex`

Further input parameters are:

- `headline`
/ *Condition*: optional / *Type*: str / *Default*: None /
An additional headline logged before the parameter dump.
- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Flag to control the console output.

3.1.4 Keyword: `get_parameters`

This keyword generates a dump of all Robot Framework parameters in the current scope (including the global parameters).

The output can be filtered (to limit the dumped parameters to the desired ones).

This keyword returns a dictionary containing all dumped parameters.

All input filter parameters are explained in detail here: [PythonExtensionsCollection.pdf](#) (section 'String operations with CString', method 'StringFilter'). Notice: In the context of this keyword, the filter parameter of the **PythonExtensionsCollection** have been renamed to a decapitalized format without type prefix.

The actual names of all input filter parameters are: `casesensitive`, `startswith`, `endswith`, `startswithnotwith`, `endsnotwith`, `contains`, `containsnot`, `inclregex`, `exclregex`

Further input parameters are:

- `headline`
/ *Condition*: optional / *Type*: str / *Default*: None /
An additional headline logged before the parameter dump.
- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Flag to control the console output.

The implementation is based on the corresponding static method `get_rf_parameters` of this class.

3.1.5 Method: `log_rf_parameter`

The static method `log_rf_parameter` logs the value of a parameter in table format. The intention behind this is a better readability of log files.

- Simple data types like strings or integers are logged in a single table line.
- Lists are logged with a separate line for every list element.
- Dictionaries are logged with a separate line for every key.
- Lists and dictionaries are resolved at top level only, not recursively. Every parameter value is logged in string format.
- Every table line can be tagged with a prefix string (optional).
- Every table can get a headline (optional).

It is also possible to control the log level (default is "INFO") and if the table shall additionally be written to console or not (default is True).

Arguments:

- `parameter_value`
/ *Condition*: required / *Type*: all supported /
The value to be logged
- `prefix`
/ *Condition*: optional / *Type*: str / *Default*: None /
A prefix string added to the beginning of every line. This can be used to provide the name of the actual parameter whose value is to be logged (unfortunately this must be done manually because Python does not support this).
- `headline`
/ *Condition*: optional / *Type*: str / *Default*: None /
An additional headline logged before the table.
- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Flag to control the console output.

Returns:

(no return values)

3.1.6 Keyword: `log_parameter`

The `log_parameter` keyword logs the value of a parameter in table format. The intention behind this is a better readability of log files.

- Simple data types like strings or integers are logged in a single table line.
- Lists are logged with a separate line for every list element.
- Dictionaries are logged with a separate line for every key.
- Lists and dictionaries are resolved at top level only, not recursively. Every parameter value is logged in string format.
- Every table line can be tagged with a prefix string (optional).
- Every table can get a headline (optional).

It is also possible to control the log level (default is "INFO") and if the table shall additionally be written to console or not (default is True).

Arguments:

- `parameter_value`
/ *Condition*: required / *Type*: all supported /
The value to be logged
- `prefix`
/ *Condition*: optional / *Type*: str / *Default*: None /
A prefix string added to the beginning of every line. This can be used to provide the name of the actual parameter whose value is to be logged (unfortunately this must be done manually because Python does not support this).

- `headline`
/ *Condition*: optional / *Type*: str / *Default*: None /
An additional headline logged before the table.
- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Flag to control the console output.

Returns:

(no return values)

The implementation is based on the corresponding static method `log_rf_parameter` of this class.

3.1.7 Method: `log_rf_metadata`

The static method `log_rf_metadata` logs the value of Robot Framework metadata in table format. The intention behind this is a better readability of log files.

Arguments:

- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Flag to control the console output.

Returns:

(no return values)

3.1.8 Keyword: `log_metadata`

The `log_metadata` keyword logs the value of a metadata parameter in table format. The intention behind this is a better readability of log files.

Arguments:

- `level`
/ *Condition*: optional / *Type*: str / *Default*: "INFO" /
The log level of output.
- `console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Flag to control the console output.

Returns:

(no return values)

The implementation is based on the corresponding static method `log_rf_metadata` of this class.

Chapter 4

Appendix

About robotframework-extensions-collection:

Author

Holger Queckenstedt (Holger.Queckenstedt@de.bosch.com)

Version

0.12.0 (17.02.2026)

Short Description

Additional Robot Framework keywords

Homepage

[robotframework-extensions-collection](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 5

History

0.1.0	01/2022
<i>Initial version</i>	
0.2.0	03/2022
<i>Setup maintenance</i>	
0.3.0	05/2022
<i>Documentation tool chain switched to GenPackageDoc</i>	
0.4.0	24.05.2022
- Documentation rebuild with GenPackageDoc v. 0.13.0 - Code maintenance	
0.5.0	31.05.2022
<i>Adapted to GenPackageDoc v. 0.15.0</i>	
0.6.0	02.06.2022
- Documentation rebuild with GenPackageDoc v. 0.16.0 - Code maintenance	
0.7.0	28.06.2022
<i>PythonExtensionsCollection updated to version 0.8.0</i>	
0.8.0	27.07.2022
<i>History reworked (requires GenPackageDoc v. 0.26.0 at least)</i>	
0.9.0	29.03.2023
<i>Prefix parameter added to keyword pretty_print</i>	
0.10.0	06.04.2023
<i>Added logging of pretty_print output to console</i>	
0.11.0	16.02.2026
- Added keyword <code>get_parameters</code> to generate a dump of Robot Framework parameters in the current scope - Added keyword <code>log_parameter</code> to log the value of a parameter in table format, optionally with a headline - Added keyword <code>log_metadata</code> to log current metadata in table format	
0.12.0	17.02.2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	

CHAPTER 5. HISTORY

RobotframeworkExtensions.pdf

Created at 07.09.2026 - 09:46:53

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.4 JsonPreprocessor

JsonPreprocessor

v. 0.12.0

Mai Dinh Nam Son

02.02.2026

Contents

1	Introduction	1
2	Description	3
2.1	How to execute	3
2.2	User defined naming convention	4
2.3	VSCodium support	5
3	The JSONP format	6
3.1	Standard JSON format	6
3.2	Boolean and null values	8
3.3	Comments	9
3.4	Import of JSON files	10
3.5	Overwriting parameters	13
3.6	dotdict notation	22
3.7	Dynamic key names	24
3.8	Implicit creation of dictionaries	25
3.9	Python inline code	27
3.10	Special characters within key names	28
3.11	Byte sequences	29
4	CJsonPreprocessor.py	30
4.1	Class: CSyntaxType	30
4.2	Class: CNameMangling	30
4.3	Class: CPythonJSONDecoder	30
4.4	Class: CKeyChecker	30
4.5	Class: CTreeNode	30
4.5.1	Method: add_child	30
4.5.2	Method: get_path_to_root	30
4.6	Class: CTextProcessor	31
4.6.1	Method: load_and_remove_comments	31
4.6.2	Method: multiple_replace	31
4.6.3	Method: normalize_digits	31
4.7	Class: CJsonPreprocessor	32
4.7.1	Method: getVersion	32
4.7.2	Method: getVersionDate	32
4.7.3	Method: jsonLoad	32
4.7.4	Method: json_load	32
4.7.5	Method: jsonLoads	32

CONTENTS	CONTENTS
4.7.6 Method: json_loads	32
4.7.7 Method: jsonDump	33
4.7.8 Method: json_dump	33
5 Appendix	34
6 History	35
B	

Chapter 1

Introduction

JavaScript Object Notation (JSON) is a text-based format for storing any user defined data and can also be used for data interchange between different applications.

But this format has some limitations and the **JsonPreprocessor** has been introduced to fill the gaps.

The **JsonPreprocessor** extends the JSON format by the following features:

1. Parts of a JSON file can be commented out
2. A JSON file can import other JSON files (nested imports)
3. Parameter can be defined, referenced and overwritten (follow up definitions in configuration files overwrite previous definitions of the same parameter)
4. Also Python specific keywords like `True`, `False` and `None` can be used (additionally to the corresponding JSON keywords `true`, `false` and `null`)

The main goal of the **JsonPreprocessor** is to support huge sets of parameters for complex projects. And the features of the **JsonPreprocessor** support this complexity:

1. Like in usual programming languages code comments are useful to explain the meaning of the defined parameters.
2. Splitting all required parameters into several JSON files - that can import each other - enables to distinguish e.g. between local and global parameters or between specific and common parameters. Another advantage of a file split is: Smaller files with a more specific content are easier to maintain than a huge single file that contains all.
3. A possible use case for a file split would be to have a software containing several different components with each component requires an individual set of parameters - and therefore an own JSON file. Additionally all components also require a common set of parameters. In this case all common parameters can be defined within an own JSON file that is imported into all other JSON files containing the specific values. This procedure avoids redundancy in parameter definitions.
4. Parameters can be initialized in common JSON files and overwritten in specific JSON files that import the common ones.

But this has consequences: The new features cause some deviations from JSON standard.

These deviations harm the syntax highlighting of editors and also cause invalid findings of JSON format related static code checkers.

To avoid conflicts between the standard JSON format and the extended JSON format described here, the **JsonPreprocessor** uses the alternative file extension `.jsonp` for all JSON files of extended format.

CHAPTER 1. INTRODUCTION

References:

The **JsonPreprocessor** is hosted in PyPi (recommended for users) and in GitHub (recommended for developers):

- [JsonPreprocessor in PyPi](#)
- [JsonPreprocessor in GitHub](#)

Details about how to get the **JsonPreprocessor** can be found in the [README](#).

For the development environment **VSCodium** an extension is available to support the extended JSON format of the **JsonPreprocessor**: [vscode-jsonp](#)

Chapter 2

Description

2.1 How to execute

The **JsonPreprocessor** is implemented in Python3 and therefore requires a Python3 installation.

A basic Python script to use the **JsonPreprocessor** can look like this:

```
from JsonPreprocessor.CJsonPreprocessor import CJsonPreprocessor
import pprint

json_preprocessor = CJsonPreprocessor()
try:
    values = json_preprocessor.jsonLoad("./file.jsonp")
    pprint.pprint(values)
except Exception as reason:
    print(f'"{reason}"')
```

The main method of the **JsonPreprocessor** is: `jsonLoad`. Input is the path and the name of a JSON file. Output is a dictionary containing all values parsed from this JSON file.

In case of any errors while computing the JSON file, the **JsonPreprocessor** throws an exception. Therefore it is required to call the method `jsonLoad` inside a `try/except` block.

`pprint` is used in this example to give the output a better readability in console.

In chapter [The JSONP format](#) the format of JSON files used by the **JsonPreprocessor**, is described in detail. All discussed JSON files can be tested with the example script listed above.

2.2 User defined naming convention

By default, key names in JSON files can contain any characters (but cannot be empty). The JSONP format extends the JSON format with some features using square, curly and angle brackets as syntax elements: `[`, `]`, `{`, `}`, `<`, `>`, that will be described below in this document.

These brackets must not be used within key names!

It might be required to add further limitations to the set of allowed characters. For this purpose the **JsonPreprocessor** supports an optional user-defined convention for key names. This is enforced using a `regex` pattern passed to the constructor of the **JsonPreprocessor** class.

Assuming key names may only contain letters, digits and underscores, but must start with a letter, then the **JsonPreprocessor** has to be initialized in this way:

```
from JsonPreprocessor.CJsonPreprocessor import CJsonPreprocessor
import pprint
import regex

json_preprocessor = CJsonPreprocessor(keyPattern=r'^\p{L}[\p{L}\p{Nd}_]*$')
try:
    values = json_preprocessor.jsonLoad("./file.jsonp")
    pprint.pprint(values)
except Exception as reason:
    print(f'{reason}')
```

The regular expressions `\p{L}` and `\p{Nd}` are provided by the `regex` module:

- `\p{L}` matches all Unicode characters classified as **letter**
- `\p{Nd}` matches all Unicode characters classified as **decimal digit**

2.3 VSCodium support

In the introduction we mentioned that the JSON syntax extensions introduced by the **JsonPreprocessor**, harm the syntax highlighting of editors.

Either we give the JSON files the extension `.json`, then an editor expects a JSON file in standard syntax, or we change the extension to `.jsonp`, but in this case an editor usually does not know how to display a file of such type.

In case you use **VSCodium**, you can install a `jsonp` extension.

With this extension the VSCodium editor will be able to display `.jsonp` files properly.

Some Impressions:

```
{
  ..//.initialization
  .."project_values": {},
  ..//
  ..//.add.some.common.values
  ..${project_values}['common_project_param_1']:: "common.project.value.1",
  .. ${project_values}['common_project_param_2']:: "common.project.value.2",
  ..//
  ..//.import.feature.parameters
  .. "[import]":: "./featureA.jsonp",
  .. "[import]":: "./featureB.jsonp",
  .. "[import]":: "./featureC.jsonp"
}
```

```
//.a).standard.notation
"dict_1_key_2_value_2_standard":: ${params}[0]['dict_1_key_2'][1],
//.b).dotdict.notation
"dict_1_key_2_value_2_dotdict":: ${params.0.dict_1_key_2.1},
//--> 'The variable '${params}['0']['dict_1_key_2']['1']' is not available!'
//
//.c).standard.notation
"dict_2_A_key_2_value_2_standard":: ${params}[1]['dict_2_key_2']['dict_2_A_key_2'][1]
```

Chapter 3

The JSONP format

The JSONP format is computed by the Python application **JsonPreprocessor**. Additionally to this, it is also possible to use the JSONP format within tests of a test automation framework called **RobotFramework AIO** ([homepage](#)).

The component that within the **RobotFramework AIO** is responsible for the execution of the **JsonPreprocessor**, is called **TestsuitesManagement**. Because the **JsonPreprocessor** is a pure Python application, whereas the **RobotFramework AIO** is a framework with own syntax rules and conditions, in some cases the JSONP format deviates in both worlds. Where required you will find corresponding hints in this chapter. In all other cases the JSONP format is the same in **JsonPreprocessor** and **RobotFramework AIO**.

This chapter explains the format of JSONP files in detail. We concentrate here on the content of the JSONP files and the corresponding results, available in Python dictionary format.

3.1 Standard JSON format

The **JsonPreprocessor** supports JSON files with standard extension `.json` and standard content.

• JSON file:

```
{
  "param1" : "value1",
  "param2" : "value2"
}
```

Outcome:

```
{'param1': 'value1', 'param2': 'value2'}
```

A JSON file with extension `.jsonp` and same content will produce the same output.

We recommend to give every JSON file the extension `.jsonp` to have a strict separation between the standard and the extended JSON format.

The following example still contains standard JSON content, but with parameters of several different data types (simple and composite).

```
{
  "param_01" : "string",
  "param_02" : 123,
  "param_03" : 4.56,
  "param_04" : ["A", "B", "C"],
  "param_05" : {"A" : 1, "B" : 2, "C" : 3}
}
```

CHAPTER 3. THE JSONP FORMAT

3.1. STANDARD JSON FORMAT

This content produces the following output:

```
{'param_01': 'string',  
'param_02': 123,  
'param_03': 4.56,  
'param_04': ['A', 'B', 'C'],  
'param_05': {'A': 1, 'B': 2, 'C': 3}}
```

This output is of a certain dictionary type (named *dotdict*) that allows to access elements also with an object oriented dot notation (details about this format can be found in section [dotdict notation](#)).

3.2 Boolean and null values

JSON supports the boolean values `true` and `false`, and also the null value `null`.

In Python, the corresponding values differ: `True`, `False` and `None`.

Because the **JsonPreprocessor** is a Python application and therefore the returned content is required to be formatted Python compatible, the **JsonPreprocessor** does a conversion automatically.

Accepted in JSON files are both styles:

```
{
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

The output contains all keywords in Python style only:

```
{ 'param_06': True,
  'param_07': False,
  'param_08': None,
  'param_09': True,
  'param_10': False,
  'param_11': None}
```

3.3 Comments

Comments can be added to JSON files with `//`:

```
{
  // JSON keywords
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  // Python keywords
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

All lines starting with `//` are ignored by the **JsonPreprocessor**. The output of this example is the same as in the previous example.

Also block comments and inline comments are possible, realized by a pair of `/* */`:

```
{
  /*
  "param1" : 1,
  "param2" : "A",
  */

  "testlist" : ["A1", /*"B2", "C3",*/ "D4"]
}
```

Outcome:

```
{'testlist': ['A1', 'D4']}
```


3.4 Import of JSON files

We assume the following scenario:

A software component *A* requires a set of configuration parameters. A software component *B* that belongs to the same main software or to the same project, requires another set of configuration parameters. Additionally both components require a common set of parameters (with the same values).

The outcome is that at least we need two JSON configuration files:

1. A file `componentA.jsonp` containing all parameters required for component *A*
2. A file `componentB.jsonp` containing all parameters required for component *B*

But with this solution both JSON files would contain also the common set of parameters. This is unfavorable, because the corresponding values need to be maintained at two different positions.

Therefore we extend the list of JSON files by a file containing the common part only:

1. A file `common.jsonp` containing all parameters that are the same for component *A* and component *B*
2. A file `componentA.jsonp` containing remaining parameters (with specific values) required for component *A*
3. A file `componentB.jsonp` containing remaining parameters (with specific values) required for component *B*

Finally we use the import mechanism of the **JsonPreprocessor** to import the file `common.jsonp` in file `componentA.jsonp` and also in file `componentB.jsonp`.

This can be the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2"
}
```

Explanation:

JSON files are imported with the key `"[import]"`. The value of this key is the path and name of the JSON file to be imported.

A JSON file can contain more than one import. Imports can be nested: An imported JSON file can import further JSON files also.

Outcome:

The file `componentA.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_1': 'componentA value 1',
 'componentA_param_2': 'componentA value 2'}
```

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentB_param_1': 'componentB value 1',
 'componentB_param_2': 'componentB value 2'}
```

It can be seen that the returned dictionary contains both the parameters from the loaded JSON file and the parameters imported by the loaded JSON file.

Multiple imports of the same file

A JSONP file can be imported more than once anywhere in the set of JSONP configuration files. This mechanism can be used to define a common part for different subkeys.

Example:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

Outcome:

The content of `common.jsonp` is part of `componentA_param_1` and also part of `componentA_param_2`.

```
{'componentA_param_1': {'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_1_a': 'componentA_param_1_a value',
 'componentA_param_1_b': 'componentA_param_1_b value'},
 'componentA_param_2': {'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_2_a': 'componentA_param_2_a value',
 'componentA_param_2_b': 'componentA_param_2_b value'}}
```

Dynamic import paths

The values of `"[import]"` keys must be of type `str`. Therefore it's possible to use dollar operator expressions inside import paths.

An alternative version of the file `componentA.jsonp` from the example above, can look like this:

```
{
  "common_config_dir" : "./common_config",
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "${common_config_dir}/common.jsonp"↵
    ↵ ,
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "${common_config_dir}/common.jsonp"↵
    ↵ ,
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

The outcome is the same like in the previous example.

3.5 Overwriting parameters

We take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component *A*, a JSON file `componentB.jsonp` for component *B* and a JSON file `common.jsonp` for both components.

But now component *B* requires a different value of a common parameter: Within a JSON file we need to change the value of a parameter that is initialized within an imported file. That is possible.

This is now the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  // overwrite parameter initialized by imported file
  "common_param_2" : "common componentB value 2"
}
```

Explanation:

With

```
"common_param_2" : "common componentB value 2"
```

in `componentB.jsonp`, the initial definition

```
"common_param_2" : "common value 2"
```

in `common.jsonp` is overwritten.

Outcome:

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common componentB value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Important: *The value a parameter has finally, depends on the order of definitions, redefinitions and imports!*

In file `componentB.jsonp` we move the import of `common.jsonp` to the bottom:

```
{
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  "common_param_2" : "common componentB value 2"
  //
  // common parameters
  "[import]" : "./common.jsonp",
}
```

Now the imported file overwrites the value initialized in the importing file.

Outcome:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Up to now we considered simple data types only. In case we want to overwrite a parameter that is part of a composite data type, we need to extend the syntax. This is explained in the next examples.

Again we take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component A, a JSON file `componentB.jsonp` for component B and a JSON file `common.jsonp` for both components.

But now all values are part of composite data types like lists and dictionaries.

This is the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : ["common value 1.1", "common value 1.2"],
  "common_param_2" : {"common_key_2_1" : "common value 2.1" ,
                      "common_key_2_2" : "common value 2.2"}
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : ["componentA value 1.1", "componentA value 1.2"],
  "componentA_param_2" : {"componentA_key_2_1" : "componentA value 2.1" ,
                          "componentA_key_2_2" : "componentA value 2.2"}
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : ["componentB value 1.1", "componentB value 1.2"],
  "componentB_param_2" : {"componentB_key_2_1" : "componentB value 2.1" ,
                          "componentB_key_2_2" : "componentB value 2.2"}
}
```

Like in previous examples, the outcome is a merge of the imported JSON file and the importing JSON file, e.g. for `componentA.jsonp`:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentA_param_1': ['componentA value 1.1', 'componentA value 1.2'],
 'componentA_param_2': {'componentA_key_2_1': 'componentA value 2.1',
                        'componentA_key_2_2': 'componentA value 2.2'}}
```

Now the following questions need to be answered:

1. How to get the value of an already existing parameter?
2. How to get the value of a single element of a parameter of nested data type (list, dictionary)?
3. How to overwrite the value of a single element of a parameter of nested data type?
4. How to add an element to a parameter of nested data type?

We introduce another JSON file `componentB.2.jsonp` in which we import the JSON file `componentB.jsonp`. In this file we also add content to work with simple and composite data types to answer the questions above.

We introduce a new file `componentB.2.jsonp` that imports `componentB.jsonp` and creates new parameters based on already existing parameters:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  // some additional parameters of simple data type
  "string_val" : "ABC",
  "int_val"     : 123,
  "float_val"   : 4.56,
  "bool_val"    : true,
  "null_val"    : null,

  // access to existing parameters
  "string_val_b"      : ${string_val},
  "int_val_b"         : ${int_val},
  "float_val_b"       : ${float_val},
  "bool_val_b"        : ${bool_val},
  "null_val_b"        : ${null_val},
  "common_param_1_b"  : ${common_param_1},
  "componentB_param_2_b" : ${componentB_param_2}
}
```

Outcome:

```
{'bool_val': True,
 'bool_val_b': True,
 'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_1_b': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                       'componentB_key_2_2': 'componentB value 2.2'},
 'componentB_param_2_b': {'componentB_key_2_1': 'componentB value 2.1',
                          'componentB_key_2_2': 'componentB value 2.2'},
 'float_val': 4.56,
 'float_val_b': 4.56,
 'int_val': 123,
 'int_val_b': 123,
 'null_val': None,
 'null_val_b': None,
 'string_val': 'ABC',
 'string_val_b': 'ABC'}
```

The rules for accessing parameters are:

- Existing parameters are accessed by a dollar operator and a pair of curly brackets (`${...}`) with the parameter name inside.
- If the entire expression of the right-hand side of the colon is such a dollar operator expression, it is not required to encapsulate this expression in quotes (in opposite to what pure JSON would require).
- Without quotes, the dollar operator keeps the data type of the referenced parameter. If you use quotes, the value of the used parameter will be converted to type `str`. This implicit string conversion is limited to parameters of simple data types like integers and floats. Composite data types like lists and dictionaries cannot be used for that.

In more detail:

The dollar operator keeps the data type of the referenced parameter. In case of `int_val` is of type `int`, also `int_val_b` is of type `int`:

```
"int_val_b" : ${int_val},
```

It is not required to encapsulate dollar operator expressions at the right-hand side of the colon in quotes. But nevertheless, it is possible to use quotes. In case of:

```
"int_val_b" : "${int_val}",
```

the parameter `int_val_b` is of type `str`.

Further content can be added between the double quotes. This can be used to create composite strings:

```
"str_val" : "ABC",
"int_val" : 1,
"float_val" : 2.3,
"bool_val" : True,
"none_val" : None,
"list_val" : [1, 2, 3],
"dict_val" : {"A" : "B"},
"newparam1" : "prefix_${str_val}_suffix",
"newparam2" : "prefix_${int_val}_suffix",
"newparam3" : "prefix_${float_val}_suffix",
"newparam4" : "prefix_${bool_val}_suffix",
"newparam5" : "prefix_${none_val}_suffix"
```

Outcome:

```
{'bool_val': True,
 'dict_val': {'A': 'B'},
 'float_val': 2.3,
 'int_val': 1,
 'list_val': [1, 2, 3],
 'newparam1': 'prefix_ABC_suffix',
 'newparam2': 'prefix_1_suffix',
 'newparam3': 'prefix_2.3_suffix',
 'newparam4': 'prefix_True_suffix',
 'newparam5': 'prefix_None_suffix',
 'none_val': None,
 'str_val': 'ABC'}
```

Using composite data types inside strings is not supported:

```
"newparam6" : "prefix_${listval}_suffix"
```

or:

```
"newparam7" : "prefix_${dictval}_suffix"
```

Result for `"newparam6"`:

```
The substitution of parameter '${listval}' inside the string value 'prefix_${listval}←
↪ _suffix' is not supported! Composite data types like lists and dictionaries cannot ↪
↪ be substituted inside strings.
```


Value of a single element of a parameter of nested data type

To access an element of a list and a key of a dictionary, we change the content of file `componentB.2.jsonp` to:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  "list_element_0" : ${componentB_param_1}[0],
  "dict_key_2_2"   : ${common_param_2}['common_key_2_2']
}
```

Outcome:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'},
 'dict_key_2_2': 'common value 2.2',
 'list_element_0': 'componentB value 1.1'}
```

Overwrite the value of a single element of a parameter of nested data type

In the next example we overwrite the value of a list element and the value of a dictionary key.

Again we change the content of file `componentB.2.jsonp`:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${componentB_param_1}[0]      : "componentB value 1.1 (new)",
  ${common_param_2}['common_key_2_1'] : "common value 2.1 (new)"
}
```

The dollar operator syntax at the left-hand side of the colon is the same as previously used on the right-hand side. The entire expression at the left-hand side of the colon must *not* be encapsulated in quotes in this case.

Outcome:

The single elements of the list and the dictionary are updated, all other elements are unchanged.

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1 (new)',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1 (new)', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'}}
```

Add an element to a parameter of nested data type

Adding further elements to an already existing list is not possible in JSON! But it is possible to add keys to an already existing dictionary.

The following example extends the dictionary `common_param_2` by an additional key `common_key_2_3`:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${common_param_2}['common_key_2_3'] : "common value 2.3"
}
```

Outcome:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2',
                    'common_key_2_3': 'common value 2.3'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                       'componentB_key_2_2': 'componentB value 2.2'}}
```

Dictionary keys and indices as parameter

In all code examples above the indices of lists and the key names of dictionaries have been hard coded strings. It is also possible to use parameters:

```
{
  "index1"      : 0,
  "index2"      : 1,
  "key1"        : "keyA",
  "key2"        : "keyB",
  "testlist"    : ["A", "B"],
  "testdict"    : {"keyA" : "A", "keyB" : "B"},
  "tmp1"        : ${testlist}[${index1}],
  "tmp2"        : ${testdict}[${key1}],
  ${testlist}[${index1}] : ${testlist}[${index2}],
  ${testdict}[${key1}]   : ${testdict}[${key2}],
  ${testlist}[${index2}] : ${tmp1},
  ${testdict}[${key2}]   : ${tmp2}
}
```

Outcome:

```
{'index1': 0,
 'index2': 1,
 'key1': 'keyA',
 'key2': 'keyB',
 'testdict': {'keyA': 'B', 'keyB': 'A'},
 'testlist': ['B', 'A'],
 'tmp1': 'A',
 'tmp2': 'A'}
```

Meaning of single quotes in square brackets

Single quotes are used to convert the content inside to a string.

- In case of the parameter `param` is of type `str`, the expressions `[${param}]` and `['${param}']` have the same outcome: The content inside the square brackets is a string. The single quotes have no meaning in this case (because the parameter is already of type `str`).
- In case of the parameter `param` is of type integer, the quotes in `['${param}']` convert the integer value to a string. Without the quotes (`[${param}]`), the content inside the square brackets is an integer.

In the context of **JsonPreprocessor** JSON files, only strings and integers are expected to be inside square brackets (except the brackets are used to define a list). Other data types are not supported here.

Whether a string or an integer is expected, depends on the data type of the parameter, the square bracket expression belongs to. Dictionaries require a string (a key name), lists require an integer (an index). Deviations will cause an error.

Summarized the following combinations are valid (on both the left-hand side of the colon and the right-hand side of the colon):

```
${listparam} [ ${intparam} ]  
${listparam} [ 1 ]  
${dictparam} [ '${intparam}' ]  
${dictparam} [ ${stringparam} ]  
${dictparam} [ '${stringparam}' ]  
${dictparam} [ 'keyname' ]
```

Use of a common dictionary

The last example in this section covers the following use case:

- We have several JSON files, each for a certain purpose within a project (e.g. for every feature of this project a separate JSON file).
- They belong together and therefore they are all imported into a main JSON file that is the file that is handed over to the **JsonPreprocessor**.
- Every imported JSON file introduces a certain bunch of parameters. All parameters need to be a part of a common dictionary.
- Outcome is that finally only one single dictionary is used to access the parameters from all JSON files imported in the main JSON file.

These are the JSON files:

- `project.jsonp`

```
{
  // define some common values
  ${project_values}['common_project_param_1'] : "common project value 1",
  ${project_values}['common_project_param_2'] : "common project value 2",
  //
  // import feature parameters
  "[import]" : "../featureA.jsonp",
  "[import]" : "../featureB.jsonp",
  "[import]" : "../featureC.jsonp"
}
```

- `featureA.jsonp`

```
{
  // parameters required for feature A
  ${project_values}['featureA_params']['featureA_param_1'] : "featureA param 1 value",
  ${project_values}['featureA_params']['featureA_param_2'] : "featureA param 2 value"
}
```

- `featureB.jsonp`

```
{
  // parameters required for feature B
  ${project_values}['featureB_params']['featureB_param_1'] : "featureB param 1 value",
  ${project_values}['featureB_params']['featureB_param_2'] : "featureB param 2 value"
}
```

- `featureC.jsonp`

```
{
  // parameters required for feature C
  ${project_values}['featureC_params']['featureC_param_1'] : "featureC param 1 value",
  ${project_values}['featureC_params']['featureC_param_2'] : "featureC param 2 value"
}
```

It is not required to start the code listed above, with dictionary initializations like

```
"project_values" : {},
${project_values}['featureA_params'] : {},
${project_values}['featureB_params'] : {},
${project_values}['featureC_params'] : {},
```

These initializations are done implicitly by the **JsonPreprocessor**. Further details about the implicit creation of dictionaries can be found in section [Implicit creation of dictionaries](#).

It is for sure still possible to do the initialization of a dictionary explicitly with `{}`. But keep in mind: This deletes all already existing keys in this dictionary!

Outcome:

```
{'project_values': {'common_project_param_1': 'common project value 1',
                    'common_project_param_2': 'common project value 2',
                    'featureA_params': {'featureA_param_1': 'featureA param 1 value',
                                         'featureA_param_2': 'featureA param 2 value'},
                    'featureB_params': {'featureB_param_1': 'featureB param 1 value',
                                         'featureB_param_2': 'featureB param 2 value'},
                    'featureC_params': {'featureC_param_1': 'featureC param 1 value',
                                         'featureC_param_2': 'featureC param 2 value'}}
```

3.6 dotdict notation

Up to now we have accessed dictionary keys in this way (standard notation):

```
${dictionary}['key']['sub_key']
```

Additionally to this standard notation, the **JsonPreprocessor** supports the so called *dotdict* notation where keys are handled as attributes:

```
${dictionary.key.sub_key}
```

In standard notation keys are encapsulated in square brackets and all together is placed *outside* the curly brackets. In dotdict notation the dictionary name and the keys are separated by dots from each other. All together is placed *inside* the curly brackets.

In standard notation key names are allowed to contain dots:

```
${dictionary}['key']['sub.key']
```

In dotdict notation this would cause ambiguities:

```
${dictionary.key.sub.key}
```

Therefore it is not possible to implement in this way! In case you need to have dots inside key names, you must use the standard notation. We recommend to prefer underlines as separator - like done in the examples in this document.

Do you really need dots inside key names?

Please keep in mind: The dotdict notation is a reduced one. Because of parts are missing (e.g. the single quotes around key names), the outcome can be code that is really hard to capture.

In the following example we create a composite data structure and demonstate how to access single elements in both notations.

• JSON file:

```
{
  // composite data structure
  "params" : [{ "dict_1_key_1" : "dict_1_key_1 value",
                "dict_1_key_2" : ["dict_1_key_2 value 1", "dict_1_key_2 value 2"] },
              //
              { "dict_2_key_1" : "dict_2_key_1 value",
                "dict_2_key_2" : { "dict_2_A_key_1" : "dict_2_A_key_1 value",
                                   "dict_2_A_key_2" : ["dict_2_A_key_2 value 1", "↵
↵ dict_2_A_key_2 value 2"] } } } ],
  //
  // access to single elements of composite data structure
  //
  // a) standard notation
  "dict_1_key_2_value_2_standard" : ${params}[0]['dict_1_key_2'][1],
  // b) dotdict notation
  "dict_1_key_2_value_2_dotdict" : ${params.0.dict_1_key_2.1},
  //
  // c) standard notation
  "dict_2_A_key_2_value_2_standard" : ${params}[1]['dict_2_key_2']['dict_2_A_key_2'][1]
  // d) dotdict notation
  "dict_2_A_key_2_value_2_dotdict" : ${params.1.dict_2_key_2.dict_2_A_key_2.1}
}
```

Outcome:

In case of the composite data structure becomes more and more nested (and if also the key names contain numbers), understanding the expressions (like `${params.1.dict_2_key_2.dict_2_A_key_2.1}`) becomes more and more challenging!

```
{'dict_1_key_2_value_2_standard': 'dict_1_key_2 value 2',
 'dict_2_A_key_2_value_2_standard': 'dict_2_A_key_2 value 2',
 'params': [{ 'dict_1_key_1': 'dict_1_key_1 value',
               'dict_1_key_2': ['dict_1_key_2 value 1', 'dict_1_key_2 value 2']},
             { 'dict_2_key_1': 'dict_2_key_1 value',
               'dict_2_key_2': { 'dict_2_A_key_1': 'dict_2_A_key_1 value',
                                'dict_2_A_key_2': ['dict_2_A_key_2 value 1',
                                                    'dict_2_A_key_2 value 2']}}}]}
```

3.7 Dynamic key names

In section [Overwriting parameters](#) we mentioned the possibility to define the value of string parameters dynamically, e.g. in this way:

```
"str_val"    : "ABC",
"newparam1"  : "prefix_${str_val}_suffix",
```

The value of `newparam1` is defined by an expression that is encapsulated in quotes and contains - beneath hard coded parts - a dollar operator expression (that is the dynamic part).

The same is also possible on the left-hand side of the colon. In this case the name of a parameter is created dynamically.

Example:

```
"strval"    : "A",
"dictval"   : {"A_2" : 1},
${dictval}['${strval}_2'] : 2
```

In second line a new dictionary with key `A_2` is defined. In third line we overwrite the initial value of this key with another value. The name of this key is defined with the help of parameter `strval`.

Outcome:

```
{'dictval': {'A_2': 2}, 'strval': 'A'}
```

The same in dotdict notation:

```
"strval"    : "A",
"dictval"   : {"A_2" : 1},
${dictval}.${strval}_2 : 3
```

The precondition for using dynamic key names is that a key with the resulting name (here `A_2`) does exist already. Therefore this mechanism can be used to overwrite the value of existing keys, but cannot be used to create new keys!

This will not work (because of a key with name `A_2` does not yet exist):

```
"strval"    : "A",
"dictval"   : {"${strval}_2" : 1}
```

Outcome:

```
A substitution in key names is not allowed! Please update the key name "${strval}_2"
```

3.8 Implicit creation of dictionaries

Up to now we have discussed two different ways of creating nested dictionaries.

The first one is “on the fly”, like:

```
{
  "project_values" : { "keyA" : "keyA value",
                      "keyB" : { "keyB1" : "keyB1 value",
                                "keyB2" : { "keyB21" : "keyB21 value",
                                              "keyB22" : "keyB22 value" }}}
}
```

In case of it is required to split the definition into several files, we have to add keys (and also the initialization) line by line:

```
{
  "project_values" : {},
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB'] : {},
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2'] : {},
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

The result will be the same as in the previous example.

It can be seen now that this way of creating nested dictionaries is rather long-winded, because every initialization of a dictionary requires a separate line of code (at every level).

To shorten the code, the **JsonPreprocessor** supports an implicit creation of dictionaries.

This is the resulting code in standard notation:

```
{
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

And the same in dotdict notation (with precondition, that no key name contains a dot):

```
{
  ${project_values.keyA} : "keyA value",
  ${project_values.keyB.keyB1} : "keyB1 value",
  ${project_values.keyB.keyB2.keyB21} : "keyB21 value",
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

Caution:

We urgently recommend *not* to mixup both styles in one line of code. In case of keys contain a list and also numerical indices are involved, we recommend to prefer the standard notation.

Please be aware of: In case of a missing level in between an expression like

```
{
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

you will *not* get an error message! The entire data structure will be created implicitly. The impact is that this method is very susceptible to typing mistakes.

The implicit creation of data structures does not work with lists! In case you use a list index out of range, you will get a corresponding error message.

Key names

The implicit creation of data structures is only possible with *hard coded* key names. Parameters are not supported.

Example:

```
{
  "paramA" : "ABC",
  "subKey" : "ABC",
  ${testdict.subKey.subKey.paramA} : "DEF"
}
```

All sub key levels within the expression `${testdict.subKey.subKey.paramA}` are interpreted as hard coded strings, even in case of parameters with the same name do exist.

For example: The name of the implicitly created key at bottom level is `"paramA"`, and not the value `"ABC"` of the parameter with the same name (`"paramA"`).

Therefore the outcome is:

```
{'paramA': 'ABC', 'subKey': 'ABC', 'testdict': {'subKey': {'subKey': {'paramA': 'DEF'}}}}
```

Reference to existing keys

It is possible to use parameters to refer to *already existing* keys.

```
{
  // <data structure created implicitly>
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // <string parameter with name of an existing key>
  "keyName_3" : "subKey_3",

  // <parameter used to refer to an existing key>
  ${testdict.subKey_1.subKey_2.${keyName_3}} : "XYZ"
}
```

Outcome:

```
{'keyName_3': 'subKey_3',
 'testdict': {'subKey_1': {'subKey_2': {'subKey_3': 'XYZ'}}}}
```

Parameters cannot be used to create new keys.

```
{
  // <data structure created implicitly>
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // <string parameter with name of a not existing key>
  "keyName_4" : "subKey_4",

  // <usage of keyName\_4 is not possible here>
  ${testdict.subKey_1.subKey_2.subKey_3.${keyName_4}} : "XYZ"
}
```

Outcome is the following error:

```
"The implicit creation of data structures based on nested parameter is not supported ..."
```

The same error will happen in case of the standard notation is used:

```
{
  // <usage of keyName\_4 is not possible here>
  ${testdict}['subKey_1']['subKey_2']['subKey_3'][${keyName_4}] : "XYZ"
}
```

3.9 Python inline code

It might be required to have access to Python builtin functions, e.g. for combining multiple lists into a single list. In pure Python this can be realized in the following way:

```
A = [1, 2, 3]
B = [4, 5, 6]
C = A + B
print(C)
```

With result:

```
[1, 2, 3, 4, 5, 6]
```

In JSONP files, the implementation looks as follows:

```
"A" : [1,2,3],
"B" : [4,5,6],
"C" : <\textless{}\textless{}\>${A} + ${B}<\textgreater{}\textgreater{}\>
```

The expression `$(A) + $(B)` is handled as Python inline code. It is possible to use dollar operator expressions inside Python inline code to access any other parameter defined in JSONP files.

The syntax of Python inline code is:

$$\langle \text{less} \rangle \langle \text{less} \rangle \langle \text{emph}(\text{Python expression}) \rangle \langle \text{greater} \rangle \langle \text{greater} \rangle$$

Limitations

The Python inline code syntax cannot be used within strings. The following code is invalid:

```
"param" : "The value is <\textless{}\textless{}\>(\emph{Python expression})><\textgreater{}↵
↵ {} \textgreater{}>"
```

The usage of Python inline code is limited to the right-hand side of the colon (*key values*). This feature cannot be used to create key names.

Python inline code cannot be used immediately to select list elements or dictionary keys that shall be overwritten. The following code is invalid:

```
"param" : [1,2,3],
${param} < \textless{} \textless{} > 0 if True else 1 < \textgreater{} \textgreater{} : 4
```

This is a possible workaround:

```
"param" : [1,2,3],
"index" : <\textless{}\textless{}>0 if True else 1<\textgreater{}\textgreater{}>,
${param}[${index}] : 4
```

Further examples

Create a list of dictionary keys:

```
"dict"      : {"kA" : "vA", "kB" : "vB"},
"key_list"  : <\textless{}\textless{}>list($dict).keys())<\textgreater{}\textgreater{}>
```

Define an import path:

```
"path_1" : "../file_1.jsonp",
"path_2" : "../file_2.jsonp"
//
"[import]" : <\textless{}\textless{}\>${path_1} if True else ${path_2}<\textgreater{}\textgreater{}
    ↪ \textgreater{}\textgreater{}>
```

3.10 Special characters within key names

Basically, the **JsonPreprocessor** considers the JSON naming convention for key names. Within JSONP files allowed is what JSON allows, but with the following limitation: The JSONP format extends the JSON format with some features using square, curly and angle brackets as syntax elements: `[`, `]`, `{`, `}`, `<`, `>`.

These brackets must not be used within key names!

This JSONP code:

```
{
  "[ " : 1,
  "B" : ${[]}
}
```

causes:

```
Error: 'Invalid expression found: '${[]}' - The brackets mismatch!!!!'
```

All other special characters can be used immediately:

```
{
  "$" : 1,
  "B" : ${}$
}
```

Result:

```
DotDict({'$': 1, 'B': 1})
```

Strings are handled as raw strings. Therefore, masking has no effect:

```
{
  "\\$" : 1,
  "B" : ${\\$}
}
```

The backslashes are part of the key name:

```
DotDict({'\\$': 1, 'B': 1})
```

The masking requires two backslashes! A single backslash will cause a JSON syntax error!



Format deviation

In opposite to the **JsonPreprocessor**, the **TestsuitesManagement** of the **RobotFramework AIO** is more restrictive and does not allow any special characters. Key names are limited to letters and digits!

3.11 Byte sequences

The **JsonPreprocessor** supports the explicit embedding of binary values (in Hexadecimal Escape Notation).

```
"bytesequenc" : b'\x52\x6f\x62\x6f\x74\x46\x72\x61\x6d\x65\x77\x6f\x72\x6b\x20\x41\x49\↵  
↵ x4f'
```

The result is the resolved content:

```
DotDict({'bytesequenc': b'RobotFramework AIO'})
```

This feature requires the usage of single quotes: `b'...'`. Double quotes must not be used.

Chapter 4

CJsonPreprocessor.py

4.1 Class: CSyntaxType

4.2 Class: CNameMangling

4.3 Class: CPythonJSONDecoder

Extends the JSON syntax by the Python keywords `True`, `False` and `None`.

Arguments:

- `json.JSONDecoder`
/ *Type*: object /
Decoder object provided by `json.loads`

4.4 Class: CKeyChecker

CkeyChecker checks key names format based on a rule defined by user.

4.5 Class: CTreeNode

The CTreeNode class is a custom tree data structure that allows to create and manage hierarchical data.

4.5.1 Method: add_child

Add a child node to the current node.

Arguments:

- `value`
/ *Condition*: required / *Type*: str /
The value for the new child node.

Returns:

The new or existing child node.

4.5.2 Method: get_path_to_root

Retrieve the path from this node to the root.

4.6 Class: CTextProcessor

4.6.1 Method: load_and_remove_comments

Loads a given json file or json content and filters all C/C++ style comments.

Arguments:

- `jsonP`
/ *Condition*: required / *Type*: str /
Path of file to be processed or a JSONP content.
- `is_file`
/ *Condition*: required / *Type*: bool /
Indicates the jsonP is a path of file or a JSONP content, default value is True.

Returns:

- `sContentCleaned`
/ *Type*: str /
String version of JSON file after removing all comments.

4.6.2 Method: multiple_replace

Replaces multiple parts in a string.

Arguments:

- `input`
/ *Condition*: required / *Type*: str /

Returns:

- `output`
/ *Type*: str /

4.6.3 Method: normalize_digits

Convert/Replace all Unicode digits inside square brackets like [`<digits>`] to [`<ASCII digits>`].

Arguments:

- `input`
/ *Condition*: required / *Type*: str /
The string which need to find and convert Unicode digits to ASCII digits.

Returns:

- `result`
/ *Type*: str /
The string contains only ASCII digits within brackets.

Raises:

- `TypeError`: If input is not a string.

4.7 Class: CJsonPreprocessor

CJsonPreprocessor extends the JSON syntax by the following features:

- Allow c/c++-style comments within JSON files
- Allow to import JSON files into JSON files
- Allow to define and use parameters within JSON files
- Allow Python keywords `True`, `False` and `None`

4.7.1 Method: getVersion

Returns the version of JsonPreprocessor as string.

4.7.2 Method: getVersionDate

Returns the version date of JsonPreprocessor as string.

4.7.3 Method: jsonLoad

This is a wrapper for the `json_load()` function.

4.7.4 Method: json_load

This method is the entry point of JsonPreprocessor.

`json_load` loads the JSON file, preprocesses it and returns the preprocessed result as Python dictionary.

Arguments:

- `json_file`
/ *Condition*: required / *Type*: str /
Path and name of main JSON file. The path can be absolute or relative and is also allowed to contain environment variables.

Returns:

- `json_obj`
/ *Type*: dict /
Preprocessed JSON file(s) as Python dictionary

4.7.5 Method: jsonLoads

This is a wrapper for the `json_loads()` function.

4.7.6 Method: json_loads

`json_loads` loads the JSONP content, preprocesses it and returns the preprocessed result as Python dictionary.

Arguments:

- `jsonp_content`
/ *Condition*: required / *Type*: str /
The JSONP content.
- `reference_dir`
/ *Condition*: optional / *Type*: str / *Default*: None /
A reference path for loading imported files.

Returns:

- `json_obj`
/ *Type*: dict /
Preprocessed JSON content as Python dictionary

4.7.7 Method: jsonDump

This is a wrapper for the `json_dump()` function.

4.7.8 Method: json_dump

This method writes the content of a Python dictionary to a file in JSON format and returns a normalized path to this JSON file.

Arguments:

- `json_obj`
/ *Condition*: required / *Type*: dict /
- `out_file (string)`
/ *Condition*: required / *Type*: str /
Path and name of the JSON output file. The path can be absolute or relative and is also allowed to contain environment variables.

Returns:

- `out_file (string)`
/ *Type*: str /
Normalized path and name of the JSON output file.

Chapter 5

Appendix

About JsonPreprocessor:

Author

Mai Dinh Nam Son (son.maidinhnam@vn.bosch.com)

Version

0.12.0 (02.02.2026)

Short Description

Preprocessor for JSON files with extended syntax

Homepage

[python-jsonpreprocessor](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 6

History

0.1.0	01/2022
<i>Initial version</i>	
0.1.4	09/2022
<i>Documentation updated</i>	
0.2.3	05/2023
<i>dotdict format added</i>	
0.2.4	06/2023
<i>Maintenance of dotdict format and log output</i>	
0.3.0	09/2023
- Implicit creation of data structures - Dotdict feature bug fixing - Update nested parameters handling in key name and value - Nested parameter feature bug fixing - Nested parameters substitution and overwriting improvement - Jsonp file path computation improvement	
0.3.1	12/2023
- Add jsonDump method to write a file in JSON format - Improve nested parameter format - Improve error message log - Fix bugs of data structures implicitly - Improve index handling together with nested parameters	
0.3.3	01/2024
- Some bugs fixed in implicitly created data structures - Improved index handling together with nested parameters - Improved format of nested parameters; improved error messages - Added getVersion and getVersionDate methods to get current version and the date of the version	
0.4.0	03/2024
- Optimized regular expression patterns - Improved duplicated parameters handling - Added mechanism to prevent Python application freeze - Removed globals scope out of all exec method executions - Optimized errors handling while loading nested parameters - Fixed bugs	
0.5.0	04/2024

CHAPTER 6. HISTORY

<i>Extended debugging support. In case of JSON syntax errors, the JsonPreprocessor exception contains an extract of the JSON content nearby the position, where the error occurred.</i>	
0.6.0	05/2024
- <i>JsonPreprocessor</i> returns a <i>dotdict</i> - <i>Blocked dynamic key names</i> - <i>Improved error messages</i> - <i>Fixed bugs</i>	
0.6.1	05/2024
<i>Added pydotdict package to installation dependencies</i>	
0.7.0	06/2024
- <i>Added jsonLoads method that allows users to directly parse JSONP content from strings</i> - <i>Improved error messages</i> - <i>Fixed bugs</i>	
0.7.1	06/2024
<i>Maintained release workflow</i>	
0.8.0	08/2024
- <i>Implemented a naming convention check for key names within .jsonp files processed by the JsonPreprocessor</i> - <i>Fixed issues related to error handling deviation</i> - <i>Updated error messages log</i>	
0.8.1	10/2024
- <i>Fixed issues related to the naming convention check for key name</i> - <i>Improved implicit creation feature</i> - <i>Prevented side effects of token string in error messages log</i> - <i>Checked absolute path when overwriting parameter</i>	
0.8.2	10/2024
<i>Enhanced import JSON file feature which allows dynamic path of imported file</i>	
0.8.3	11/2024
<i>Fixed bugs and updated error messages related to dynamic imports</i>	
0.8.4	02/2025
<i>Fixed bugs, updated error messages, and improved composite data structure handling</i>	
0.9.0	03/2025
- <i>Added support of a user-defined convention for key names</i> - <i>Improved and aligned error messages</i> - <i>Fixed bugs and updated the Selftest according to the changes</i>	
0.9.1	05/2025
- <i>Enhanced cyclic import detection</i> - <i>Fixed bugs and updated error messages</i>	
0.9.2	06/2025
- <i>Improved invalid key name detection</i> - <i>Fixed bugs</i>	
0.9.3	07/2025

CHAPTER 6. HISTORY

- Allowed unicode digits to get a list element - Updated error messages	
0.10.0	07/2025
Added possibility to use Python inline code inside JSONP files	
0.10.1	08/2025
Fixed bugs in the Python inline code feature	
0.11.0	11/2025
Added support of explicit embedding of binary values	
0.11.1	12/2025
- Refactored codebase to comply with PEP 8 style guidelines - Fixed bugs	
0.11.2	01/2026
Stabilized the explicit embedding of binary values	
0.12.0	02.02.2026
Usage of Setuptools replaced by usage of PIP/TOML/Setuptools	

JsonPreprocessor.pdf

Created at 07.09.2026 - 09:46:55

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.5 RobotFramework_TestsuitesManagement

RobotFramework_TestsuitesManagement

v. 0.11.0

Mai Dinh Nam Son

27.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Meaning of "Test Suites Management"	2
2.2	Content of configuration files	3
2.2.1	Variant configuration file	3
2.2.2	Parameter configuration files	4
2.3	Access to configuration files	6
2.4	Activation of "Test Suites Management"	7
2.5	Variants selection	8
2.6	Local configuration	9
2.7	Priority of configuration parameters	10
2.8	Nested configuration files	11
2.9	Overwritten parameters	12
2.10	Naming convention	13
2.11	Version control interfaces	14
2.11.1	RobotFramework AIO interface for version control	14
2.11.2	Explicit version control of the RobotFramework AIO	14
2.11.3	Explicit version control of the TestsuitesManagement	15
2.11.4	User defined version control	15
2.11.5	Python interface for version control	17
3	The JSONP format	18
3.1	Standard JSON format	18
3.2	Boolean and null values	20
3.3	Comments	21
3.4	Import of JSON files	22
3.5	Overwriting parameters	25
3.6	dotdict notation	34
3.7	Dynamic key names	36
3.8	Implicit creation of dictionaries	37
3.9	Python inline code	39
3.10	Special characters within key names	40
3.11	Byte sequences	41
4	config.py	42
4.1	Method: load_config	42
4.2	Method: check_version	42

CONTENTS

CONTENTS

5	key_words.py	44
5.1	Class: CSetupKeywords	44
5.1.1	Keyword: testsuite_setup	44
5.1.2	Keyword: testsuite_teardown	44
5.1.3	Keyword: testcase_setup	44
5.1.4	Keyword: testcase_teardown	44
5.2	Class: CGeneralKeywords	44
5.2.1	Keyword: get_config	45
5.2.2	Keyword: load_json	45
5.2.3	Keyword: is_robotframework_aio	45
5.2.4	Keyword: get_package_context_file	45
5.2.5	Keyword: get_reference_version	45
5.2.6	Keyword: get_reference_version_date	45
5.2.7	Keyword: get_reference_app_name	45
5.2.8	Keyword: get_reference_installer_location	46
5.2.9	Keyword: get_tsm_version	46
5.2.10	Keyword: get_tsm_version_date	46
5.2.11	Keyword: get_tsm_app_name	46
5.2.12	Keyword: get_tsm_installer_location	46
5.2.13	Keyword: get_bundle_version	46
5.2.14	Keyword: get_bundle_version_date	46
5.2.15	Keyword: get_bundle_name	46
5.2.16	Keyword: get_bundle_installer_location	46
5.2.17	Keyword: check_version	46
5.2.18	Keyword: get_status_messages	46
6	on_failure_handle.py	47
7	event.py	48
8	scope_event.py	49
8.1	Class: ScopeEvent	49
8.2	Class: ScopeStart	49
8.3	Class: ScopeEnd	49
9	app_config.py	50
9.1	Function: print_bundle_version	50
9.2	Class: AppConfig	50
9.2.1	Method: is_robotframework_aio	50
9.2.2	Method: get_package_context_file	50
9.2.3	Method: get_reference_version	50
9.2.4	Method: get_reference_version_date	50
9.2.5	Method: get_reference_app_name	50
9.2.6	Method: get_reference_installer_location	50
9.2.7	Method: get_tsm_version	50
9.2.8	Method: get_tsm_version_date	51
9.2.9	Method: get_tsm_app_name	51

CONTENTS

CONTENTS

9.2.10	Method: <code>get_tsm_installer_location</code>	51
9.2.11	Method: <code>get_bundle_version</code>	51
9.2.12	Method: <code>get_bundle_version_date</code>	51
9.2.13	Method: <code>get_bundle_name</code>	51
9.2.14	Method: <code>get_bundle_installer_location</code>	51
10	<code>app_logger.py</code>	52
11	<code>lib_listener.py</code>	53
12	<code>new_struct.py</code>	54
13	<code>version.py</code>	55
13.1	Class: <code>enVersionCheckResult</code>	55
13.2	Class: <code>StatusMessages</code>	55
13.2.1	Method: <code>get_messages_dict</code>	55
13.2.2	Method: <code>get</code>	55
13.2.3	Method: <code>set</code>	55
13.3	Class: <code>CVersion</code>	55
13.3.1	Method: <code>verify_version</code>	55
13.3.2	Method: <code>verifyVersion</code>	56
13.3.3	Method: <code>check_version</code>	56
13.3.4	Method: <code>checkVersion</code>	56
13.3.5	Method: <code>get_last_error</code>	57
13.3.6	Method: <code>validate_min_version</code>	57
13.3.7	Method: <code>validate_max_version</code>	57
13.3.8	Method: <code>validate_sub_version</code>	57
13.3.9	Method: <code>tuple_version</code>	58
14	<code>__init__.py</code>	59
14.1	Class: <code>RobotFramework_TestsuitesManagement</code>	59
14.1.1	Method: <code>run_keyword</code>	59
14.1.2	Method: <code>get_keyword_tags</code>	59
14.1.3	Method: <code>get_keyword_documentation</code>	59
14.1.4	Method: <code>failure_occurred</code>	59
14.2	Class: <code>CConfigLevel</code>	59
14.3	Class: <code>CTestsuitesCfg</code>	59
15	Appendix	60
16	History	61

Chapter 1

Introduction

The **RobotFramework_TestsuitesManagement** enables users to define dynamic configuration values within separate configuration files in JSON format.

These configuration values are available during test execution - but under certain conditions that can be defined by the user (e.g. to realize a variant handling). This means: Not all parameter values are available during test execution - only the ones that belong to the current test scenario.

To realize this, the **RobotFramework_TestsuitesManagement** provides the following features:

1. Split all possible configuration values into several JSON configuration files, with every configuration file contains a specific set of values for configuration parameter
2. Use nested imports of JSON configuration files
3. Follow up definitions in configuration files overwrite previous definitions (of the same parameter)
4. Select between several criteria to let the Robot Framework use a certain JSON configuration file

How to install

The **RobotFramework_TestsuitesManagement** can be installed in two different ways: via PyPi (recommended for users) and via GitHub (recommended for developers).

Installation details can be found in the [README](#).

Further links

For self-study a tutorial is available containing lots of examples. Here you find the rendered [tutorial documentation](#).

For the development environment **VSCodium** an extension is available to support the features of the **RobotFramework_TestsuitesManagement**: [vscode-jsonp](#). This extension adapts e.g. the syntax highlighting of the editor.

Chapter 2

Description

2.1 Meaning of "Test Suites Management"

In the scope of the Robot Framework a test suite is either a single robot file containing one or more test cases, or a set of several robot files.

Usually all test cases of a test suite run under the same conditions - but these conditions may be different. For example the same test case is used to test several different variants of a system under test. Every variant requires individual values for certain configuration parameters.

Tests are carried out at several test benches. All test benches have different hardware configurations. Also the different test benches may require individual values for configuration parameters used in the tests.

Therefore the same tests have to run under different conditions!

The Robot Framework provides several places to define parameters: robot files, resource files, parameter files. But these parameters are fixed. Therefore we need a more dynamic way of accessing parameters. And we postulate the following: When switching between tests of several variants and test executions on several test benches, no changes shall be required within the test code.

The outcome is that another position has to be introduced to store values for variant and test bench specific parameters. And a possibility has to be provided to dynamically make either the one or the other set of values available during the execution of tests - depending on outer circumstances like "*which variant?*" and "*which test bench?*". Those dynamic configuration values are stored within separate configuration files in JSON format and the **RobotFramework_TestsuitesManagement** makes the values available globally during the test execution.

Two different kinds of JSON configuration files are involved:

1. *parameter configuration files*

These configuration files contain all parameter definitions (can be more than one configuration file in a project)

2. *variant configuration file*

This is a single configuration file containing the mapping between the several parameter configuration files and a name (usually the name of a variant). This name can be used in command line to select a certain parameter configuration file containing the values for this variant.

Background: It's easier simply to use a name for referencing a certain variant instead of having the need always to mention the path and name of a configuration file.

To realize a concrete test suites management for your project, you need to

- identify the parameters that are variant specific, depending on the number of variants in your project,
- identify the parameters that are test bench specific, depending on the number of test benches in your project,
- identify the parameters that are both: variant specific and test bench specific,
- identify the parameters that have the same value in all variants and test benches.

After this

- for every set of parameters (variant specific and bench specific) you have to introduce a certain parameter configuration file,
- in the variant configuration file you have to define for every variant a variant name together with the path to the corresponding parameter configuration file.

Basically all configuration files of the **RobotFramework_TestsuitesManagement** are implemented in JSON format. This format is extended by some useful features like code comments and imports (nested configuration files). This is explained in more detail in the following chapters. These features cause deviations from standard JSON format. To give applications like editors or syntax checkers a chance to handle these deviations (without invalid findings), all JSON configurations files of the **RobotFramework_TestsuitesManagement** have the extension `.jsonp`, instead of `.json`.

The content of the configuration files is described in the next section.

2.2 Content of configuration files

2.2.1 Variant configuration file

This file configures the access to all variant dependent `robot_config*.jsonp` files.

```
{
  "default": {
    "name": "robot_execution_config.jsonp",
    "path": "./config/"
  },
  "variant_1": {
    "name": "robot_config_variant_1.jsonp",
    "path": "./config/"
  },
  "variant_2": {
    "name": "robot_config_variant_2.jsonp",
    "path": "./config/"
  },
  "variant_3": {
    "name": "robot_config_variant_3.jsonp",
    "path": "./config/"
  }
}
```

The example above contains definitions for three variants with names:

`variant_1`, `variant_2` and `variant_3`. Additionally a variant named `default` is defined. This default configuration becomes active in case of no certain variant name is provided when the test suite is being executed.

The paths to the `robot_config*.jsonp` files could be absolute paths or relative to the position of the mapping file.

It is of course still possible to use the standard notation for relative paths:

```
"path": "./config/"
```

2.2.2 Parameter configuration files

In these configuration files all parameters are defined, that shall be available globally during test execution.

Some parameters are required. Optionally the user can add own ones. The following example shows the smallest version of a parameter configuration file containing only the most important parameters. This version is a default version and part of the **RobotFramework_TestsuitesManagement** installation.

```
{
  "WelcomeString" : "Hello... Robot Framework is running now!",
  "Maximum_version" : "1.99.0",
  "Minimum_version" : "0.6.0",
  "Project" : "RobotFramework Testsuites",
  "TargetName" : "Device_01"
}
```

`Project`, `WelcomeString` and `TargetName` are simple strings that can be used anyhow. `Maximum_version` and `Minimum_version` are part of a version control mechanism: In case of the version of the currently installed software is outside the range between `Minimum_version` and `Maximum_version`, the test execution stops with an error message.

What is the meaning of "currently installed software"?

- The first possibility is that the **RobotFramework_TestsuitesManagement** runs stand-alone, that means, it is not part of a larger bundle (like the RobotFramework AIO). The installation from PyPi or GitHub causes such a stand-alone installation. In this case the component version of the **RobotFramework_TestsuitesManagement** itself is used for a version control against `Minimum_version` and `Maximum_version`.
- The second possibility is that the **RobotFramework_TestsuitesManagement** runs as part of the RobotFramework AIO. In this case the version of the entire RobotFramework AIO is used for a version control instead.

The version control mechanism is optional. In case you do not need to have your tests under version control, you can set the versions to the value `null`.

```
"Maximum_version" : null,
"Minimum_version" : null,
```

As an alternative it is also possible to remove `Minimum_version` and `Maximum_version` completely.

In case you define only one single version number, only this version number is considered. The following combination makes sure, that the installed software at least is of version 0.6.0, but there is no upper version limit:

```
"Maximum_version" : null,
"Minimum_version" : "0.6.0",
```

Hint: The parameters are keys of an internal configuration dictionary. They have to be accessed in the following way:

```
Log    Maximum_version : ${CONFIG}[Maximum_version]
Log    Project : ${CONFIG}[Project]
```

The following example is an extended version of a configuration file containing also some user defined parameters.

```
{
  "WelcomeString" : "Hello... Robot Framework is running now!",
  "Maximum_version" : "1.99.0",
  "Minimum_version" : "0.6.0",
  "Project" : "RobotFramework Testsuites",
  "TargetName" : "Device_01"
  "params": {
    // global parameters
    "global" : {
      "param1" : "ABC",
      "param2" : 25
    }
  }
}
```

User defined parameters have to be placed inside `params:global`. The intermediate level `global` is introduced to enable further parameter scopes than `global` in future.

All user defined parameters have the scope `params:global` per default. Therefore they can be accessed directly:

```
Log    param1 : ${param1}
```

And another feature can be seen in the example above:

In the context of the **RobotFramework_TestsuitesManagement** the JSON format is an extended one. Deviating from JSON standard it is possible to comment out lines with starting them with a double slash `//`. This allows to add explanations about the meaning of the defined parameters already within the JSON file.

2.3 Access to configuration files

With an installed **RobotFramework_TestsuitesManagement** every test execution requires a configuration - that is the accessibility of a configuration file in JSON format. The **RobotFramework_TestsuitesManagement** provides four different possibilities - also called *level* - to realize such an access. These possibilities are sorted and the **RobotFramework_TestsuitesManagement** tries to access the configuration file in a certain order: Level 1 has the highest priority and level 4 has the lowest priority.

Level 1

Path and name of a parameter configuration file is provided in command line of the Robot Framework.

Level 2 (recommended)

The name of the variant is provided in command line of the Robot Framework.

This level requires that a variant configuration file is passed to the suite setup of the **RobotFramework_TestsuitesManagement**.

Level 2 includes the automated selection of a default variant (in case of no variant name is provided in command line). Also this default variant has to be defined within the variant configuration file.

Level 3

The **RobotFramework_TestsuitesManagement** searches for parameter configuration files within a folder `config` in current test suite folder. In case of such a folder exists and parameter configuration files are inside, they will be used.

Level 4 (unwanted, fallback solution only)

The **RobotFramework_TestsuitesManagement** uses the default configuration file that is part of the installation.

Summary

- With highest priority a parameter configuration file provided in command line, is considered - even in case of also other configuration files (level 2 - level 4) are available.
- If a parameter configuration file is not provided in command line, but a variant name, then the configuration belonging to this variant, is loaded - even in case of also other configuration files (level 3 - level 4) are available.
- If nothing is specified in command line, then the **RobotFramework_TestsuitesManagement** tries to find parameter configuration files within a `config` folder and take them if available - even in case of also the level 4 configuration file is available.
- In case of the user does not provide any information about parameter configuration files to use, the **RobotFramework_TestsuitesManagement** loads the default configuration from installation folder (fallback solution; level 4).

In this context two aspects are important to know for users:

1. *Which parameter configuration file is selected for the test execution?*
To answer this question the log file contains the path and the name of the selected parameter configuration file.
2. *For which reason is this parameter configuration file selected?*
To answer this question the log file also contains the level number. The level number indicates the reason.

With these log file entries the test execution is clearly understandable, traceable and scales for huge test suites.

Why is level 2 the recommended one?

Level 2 is the most flexible and extensible solution. Because the robot files contain a link to a variants configuration file, the possible sets of parameter values can already be taken out of the code.

The values selected by level 1, you only see in the log files, but not in the code, because the selection happens in command line only.

Level 3 has a rather strong binding between robot files and configuration files. If you start the test implementation based on level 3 and after this want to have a variant handling, then you have to switch from level 3 to level 2 - and this causes effort in implementation.

Whereas if you start with level 2 immediately and need to consider another set of configuration values for the same tests, then you only have to add another parameter configuration file and another entry in the variants configuration file, without changing any test implementation.

We strongly recommend not to mix up several different configuration levels in one project!

2.4 Activation of "Test Suites Management"

To activate the test suites management you have to import the **RobotFramework_TestsuitesManagement** library in the following way:

```
Library    RobotFramework_TestsuitesManagement    AS    tm
```

We recommend to use the **AS** option to shorten the robot code a little bit.

The next step is to call the **testsuite_setup** of the **RobotFramework_TestsuitesManagement** within the **Suite Setup** of your test:

```
Suite Setup    tm.testsuite_setup
```

As long as you

- do not provide a parameter configuration file in command line when executing the test suite (level 1),
- do not provide a variants configuration file as parameter of the **testsuite_setup** (level 2),
- do not have a **config** folder containing parameter configuration files in your test suites folder (level 3),

the **RobotFramework_TestsuitesManagement** falls back to the default configuration (level 4).

In case you want to realize a variant handling you have to provide the path and the name of a variants configuration file to the **testsuite_setup**:

```
Suite Setup    tm.testsuite_setup    .../config/exercise_variants.jsonp
```

The path to the variants configuration file can be absolute path or relative to the position of the robot file.

Another aspect is important: the **three dots**. The path to the **/config/exercise_variants.jsonp** file depends on the robot file location. A different number of **../** is required dependent on the directory depth of the robot file location.

Therefore we use here three dots to tell the **RobotFramework_TestsuitesManagement** to search from the robot file location up till the **/config/exercise_variants.jsonp** file is found:

```
./config/exercise_variants.jsonp
../config/exercise_variants.jsonp
../../config/exercise_variants.jsonp
../../../config/exercise_variants.jsonp
```

and so on.

Hint: You are free to move your test suites one or more level up or down in the file system, but using the three dots notation enables you to let the position of the **config** folder unchanged.

To ease the analysis of a test execution, the log file contains informations about the selected level and the path and the name of the used configuration file, for example:

```
Running with configuration level: 2
CfgFile Path: ./config/exercise_config.jsonp
```

Please consider: The **testsuite_setup** requires a variants configuration file (in the example above: **exercise_variants.jsonp**) - whereas the log file contains the resulting parameter configuration file (in the example above: **exercise_config.jsonp**), that is selected depending on the name of the variant provided in command line of the Robot Framework.

2.5 Variants selection

In a previous section the level concept for configuration files has been explained. This section contains corresponding code examples.

1. *Selection of a certain parameter configuration file in command line*

```
--variable config_file:"(path to parameter configuration file)"
```

2. *Selection of a certain variant per name in command line*

```
--variable variant:"(variant name)"
```

3. *Parameter configuration taken from `config` folder*

This `config` folder has to be placed in the same folder than the test suites.

Parameter configuration files within this folder are considered under two different conditions:

- The configuration file has the name `robot_config.jsonp`. That is a fix name predefined by the **Robot-Framework_TestsuitesManagement**.
- The configuration file has the same name than a robot file inside the test suites folder, e.g.:
 - Name of test suite file: `example.robot`
 - Path and name of corresponding parameter configuration file: `./config/example.jsonp`

With this rule it is possible to give every test suite in a certain folder an own individual configuration.

2.6 Local configuration

It might be required to execute tests on several different test benches with every test bench has it's own individual hardware that might require configuration parameter values that are test bench specific. This can be related to common configuration parameters and also to parameters that are variant specific. In the second case a configuration parameter is both variant specific *and* test bench specific.

The *local configuration* feature of the **RobotFramework_TestsuitesManagement** provides the possibility to define test bench specific configuration parameter values.

The meaning of *local* in this context is: placed on a certain test bench - and valid for this bench only.

Also this local configuration is based on configuration files in JSON format. These files are the last ones that are considered when the configuration is loaded. The outcome is that it is possible to define default values for test bench specific parameters in other configuration files - to be also test bench independent. And it is possible to use the local configuration to overwrite these default values with values that are specific for a certain test bench.

Important:

- Local configuration files are fragments only - and not a full configuration! Even so they need to follow the JSON syntax rules. This means, at least they have to start with an opening curly bracket and they have to end with a closing curly bracket.
- Local configuration files must not contain the mandatory top level parameters like the `WelcomeString` and others.

Using the local configuration feature is an option and the **RobotFramework_TestsuitesManagement** provides two ways to realize it:

1. *per command line*

Path and name of the local parameter configuration file is provided in command line of the Robot Framework with the following syntax:

```
--variable local_config:"(path to local configuration file)"
```

2. *per environment variable*

An environment variable named `ROBOT_LOCAL_CONFIG` exists and contains path and name of a local parameter configuration file.

The user has to create this environment variable!

This mechanism allows a user - without any command line extensions - automatically to refer on every test bench to an individual local configuration, simply by giving on every test bench this environment variable an individual value.

The command line has a higher priority than the environment variable. If both is available the local configuration is taken from command line.

Recommendation: *To avoid an accidental overwriting of local configuration files in version control systems we recommend to give those files names that are test bench specific.*

2.7 Priority of configuration parameters

In previous sections the level concept has been explained. This concept introduces four levels of priority that define, which of the possible sources of configuration parameters are processed. But there are other rules involved that influence the priority:

- The local configuration has higher priority than other parameter configurations
- The command line has higher priority than definitions within configuration files

Already in command line we have several possibilities to make settings:

- Set a parameter configuration file (with **RobotFramework_TestsuitesManagement** command line variable `config_file`, level 1)
- Set a variant name (with **RobotFramework_TestsuitesManagement** command line variable `variant`, level 2)
- Set a local configuration (with **RobotFramework_TestsuitesManagement** command line variable `local_config`)
- Set any other variables (directly with Robot Framework command line variable `--variable`)

And it is possible that in all four use cases the same parameters are used. Or in other words: It is possible to use the `--variable` mechanism to define a parameter that is also defined within a parameter configuration or within a local configuration - or in both together.

Finally this is the order of processing (with highest priority first):

1. Single command line variable (`--variable`)
2. Local configuration (`local_config`)
3. Variant specific configuration (`config_file` or `variant`)

Meaning:

1. Variant specific configuration is overwritten by local configuration
2. Local configuration is overwritten by single command line variable

What happens in case of a command line contains both a `config_file` and a `variant` ?

`config_file` is level 1 and `variant` is level 2. Level 1 has higher priority than level 2. Therefore `config_file` is the valid one. This does **not** mean that `config_file` overwrites `variant` ! In case of a certain level is identified (here: level 1), all other levels are ignored. The outcome is that - in this example - the `variant` has no meaning. Between different levels there is an *either or* relationship. And that is the reason for that it makes no sense to define both in command line, a `config_file` and a `variant`. The **RobotFramework_TestsuitesManagement** throws an error in this case.

But when additionally `--variable` is used to define a new value for a parameter that is already defined in one of the involved configuration files, then the configuration file value is overwritten by the command line value.

And even this is not all. The Robot Framework provides further possibilities to define parameters in command line, e.g. by `--variablefile`. `--variable` and `--variablefile` are Robot Framework mechanisms to define parameters, whereas `config_file` and `local_config` are corresponding **RobotFramework_TestsuitesManagement** mechanisms.

The rules behind all are: `--variable` overrules `--variablefile`. Robot Framework mechanisms overrule **RobotFramework_TestsuitesManagement** mechanisms.

To avoid the things becoming too much complicated, we urgently recommend not to mixup both mechanisms to define *different values for the same parameters* (but to overwrite only a single variable with `--variable` might be OK).

2.8 Nested configuration files

In case of a project requires more and more parameters, it makes sense to split the growing configuration file into smaller ones.

This means, at first we have to split all configuration parameters in

1. parameters that are specific for a certain variant,
2. common parameters that have the same value for all variants

Placing those common parameters in every single variant specific parameter configuration file would create a lot of redundancy. This would also complicate the maintenance.

The solution is to use the variant specific configuration files only for variant specific parameters and to put all common parameters in a separate configuration file. This common parameter file has to be imported in every variant specific parameter file.

The outcome is that still with the selection of a certain variant specific parameter file both types of parameters are available: the variant specific ones and the common ones.

This can be done in the following way:

For example we have the following variant specific configuration files:

```
config/config_variant1.jsonp
config/config_variant2.jsonp
```

Additionally we have a configuration file with common parameters:

```
config/config_common.jsonp
```

The import of `config_common.jsonp` into `config_variant1.jsonp` and into `config_variant2.jsonp` is possible in the following way:

```
"params" : {
  "global": {
    "[import]" : "./config_common.jsonp",
    "teststring" : "variant specific value"
  }
}
```

The key `[import]` indicates the import of another configuration file. The value of the key is the path and name of this file.

Imports can be nested. An imported configuration file is allowed to contain imports also.

The content of the importing file and the content of all imported files are merged. In case of duplicate parameter names follow up definitions overwrite previous definitions of the same parameter!

Important:

- All imported configuration files are fragments only - and not a full configuration! Even so they need to follow the JSON syntax rules. This means, at least they have to start with an opening curly bracket and they have to end with a closing curly bracket.
- Imported configuration files must not contain the mandatory top level parameters like the `WelcomeString` and others.

2.9 Overwritten parameters

Summarized the **RobotFramework_TestsuitesManagement** provides three different types of parameter configuration files to define parameters:

1. A full standard parameter configuration file containing at least the mandatory parameters and - as option - also user defined parameters
2. A parameter configuration file fragment that is imported in other configuration files by the `[import]` key
3. A local parameter configuration file that is also a fragment only, and accessed either by command line or environment variable

All types of configuration file can be used

1. to define new parameters
2. to overwrite already existing parameters

This possibility only belongs to user defined parameters with scope `params:global` !

Example:

1. *Define a new parameter:*

```
"params" : {
    "global": {
        "teststring" : "initial value"
    }
}
```

2. *Overwrite an already existing parameter:*

To overwrite a parameter is - after the initial definition - possible at any follow up position

- in the same configuration file or
- in other configuration files like the imported ones or
- in a local configuration file

With the following syntax:

```
${params}['global']['teststring'] : "new value"
```

The resulting value of a parameter at the end depends on the priority (computation order) described in previous sections of this description.

2.10 Naming convention

Keys that are defined within `params['global']` in JSONP configuration files become **Robot Framework** variables. Therefore, the key names must conform to **Robot Framework**'s naming conventions.

The naming convention is: Key names may only contain letters, digits and underscores, and must start with a letter. Internally this is realized by the following regex pattern: `^\p{L}[\p{L}\p{Nd}_]*$`.

Example

- Valid key names are: `"abcParam"`, `"abcParam01"`, `"abc_Param_01"`, ...
- Invalid key names are: `"+param01"`, `"param$01"`, `"abc#Param"`, `"01abc_Param"`, ...



Caution

The **Robot Framework** ignores underscores. Nevertheless, underscores are allowed inside variable names, but you must be aware of the consequences when using them! The following code example demonstrates the effects.

Example

```
"params" : {
    "global": {
        "ABC"      : "A",
        "ABC__"    : "B",
        "A__B__C"  : "C"
    }
}
```

With

```
Log    ABC: ${ABC}
Log    ABC__: ${ABC__}
Log    A__B__C: ${A__B__C}
```

the result is

```
INFO - ABC: C
INFO - ABC__: C
INFO - A__B__C: C
```



Impact

Three **Robot Framework** variables with different names are created, but all variables have the same value. Therefore, underscores must not be used to distinguish between variables!

2.11 Version control interfaces

The **RobotFramework AIO** supports a version control in two ways: *implicitly* and *explicitly*. An implicit version control is executed automatically at the beginning of every test suite (during `testsuite_setup`). A version expected by user and defined within the variant configuration file, is checked against the version of the installed **RobotFramework AIO**. In detail this is explained in 2.2.2.

Version control can also be executed explicitly by the user. This version control can refer to any versions (not only to the versions of already installed software such as **RobotFramework AIO** and **TestsuitesManagement**).

Additionally, it is possible to query the versions of **RobotFramework AIO** and **TestsuitesManagement** directly.

For this purpose, **TestsuitesManagement** provides two interfaces:

1. **RobotFramework AIO** interface
2. **Python** interface

2.11.1 RobotFramework AIO interface for version control

Like usual, the **TestsuitesManagement** library needs to be imported in this way:

```
Library    RobotFramework_TestsuitesManagement    AS    tm
```

2.11.2 Explicit version control of the RobotFramework AIO

The simplest implementation of a version control looks like this:

```

${result}=    tm.check_version    min_version=0.1.0    max_version=0.20.0
log          == check_version result: ${result}    console=yes

```

In this example, the specification is missing against which version (the `reference_version`) the `min_version` and the `max_version` should be checked. This automatically leads to the version of the installed software being taken as reference. There are two possibilities for this:

1. The **TestsuitesManagement** is installed together with the **RobotFramework AIO** (thus is part of the entire **RobotFramework AIO** bundle). In this case, the version of the installed **RobotFramework AIO** is the *reference* version.
2. The **TestsuitesManagement** is installed standalone (for example separately via PyPi). In this case, the version of the installed **TestsuitesManagement** is the *reference* version.

We assume now that the **RobotFramework AIO** is installed with version **0.14.2.26**. This fits to the expectation of minimum version **0.1.0** and maximum version **0.20.0** from the example above. Therefore the result is:

```

INFO - Version check passed (RobotFramework AIO (extended))
INFO - == check_version result: True

```

Exactly this is what already happened during the `testsuite_setup`. Therefore this version check will not provide any new insights. In the next example we check the version of the **TestsuitesManagement**.

2.11.3 Explicit version control of the TestsuitesManagement

At first we get the version of the **TestsuitesManagement**. After this we set this version as reference:

```
${tsm_version}=    tm.get_tsm_version
log      == tsm_version: ${tsm_version}    console=yes
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↔
               ↳ reference_version=${tsm_version}
log      == check_version result (TSM): ${result}    console=yes
```

It can be seen that the expectation of minimum version **0.1.0** and maximum version **0.2.0** does not fit to the current **TestsuitesManagement** version **0.9.0**. The version check and therefore also the test fails:

```
ERROR - Version check failed: The test execution requires the maximum version '0.2.0', ↔
               ↳ but the reference version '0.9.0' is younger
UNKNOWN - Execution will be aborted because of a failed version check
```

Whether the test result is **UNKNOWN** or **FAIL** depends on the type of the **RobotFramework AIO** installation. Details can be found here: robotframework-aio.org.

2.11.4 User defined version control

The keyword **check_version** can be used for any version checks:

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↔
               ↳ reference_version=1.2.3
```

Versions that are not relevant for a version check, can be set to **None**. In this example only the maximum version is checked against the reference:

```
${result}=    tm.check_version    min_version=${None}    max_version=0.2.0    ↔
               ↳ reference_version=1.2.3
```

If nothing is specified, the version check is skipped:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}    ↔
               ↳ reference_version=${None}
```

This is the same like:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}
```

Because of a version check is not executed, the reference version doesn't matter.

Result:

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to ↔
               ↳ None (RobotFramework AIO (extended))
```

The following formats are accepted for all version parameters:

- **major.minor.patch**
- **major.minor**
- **major**

Therefore, this is also possible:

```
${result}=    tm.check_version    min_version=0    max_version=2    reference_version=3
```

With result:

```
ERROR - Version check failed: The test execution requires the maximum version '2', but ↔
               ↳ the reference version '3' is younger
UNKNOWN - Execution will be aborted because of a failed version check
```

Every deviation cause a failed test:

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0.9    ↔
               ↪ reference_version=1.2.3
```

Result:

```
ERROR - Version format error: maximum version '0.2.0.9' contains too many parts (expected↔
       ↪ is: major.minor.patch)
UNKNOWN - Execution will be aborted because of a critical version check issue
```

It makes a difference, whether the reference version is defined or not!

- `reference_version` is not defined:

Depending on the installed software, the TestsuitesManagement decides what is used as reference (either the **RobotFramework AIO** or the **TestsuitesManagement**). This means, a name of the checked software is known. In this case the name is part of the logged output:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}
```

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to ↔
       ↪ None (RobotFramework AIO (extended))
```

- `reference_version` is defined:

In this case a corresponding application name that is assigned to this number, is not available. Therefore, the log does not contain a name.

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↔
               ↪ reference_version=1.2.3
```

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to ↔
       ↪ None
```

This is the same result as before, but without name (**RobotFramework AIO (extended)**).

2.11.5 Python interface for version control

The goal of this Python interface is to enable **RobotFramework AIO** users who work purely in Python, without using robot files, to verify which bundle version is installed. The bundle version is either the version of the entire **RobotFramework AIO** or, if installed standalone, the version of the **TestsuitesManagement**.

Example

```
from RobotFramework_TestsuitesManagement.Uutils.version import CVersion
oVersion = CVersion()
result, reason = oVersion.verify_version(min_version, max_version)
```

The parameters `min_version` and `max_version` define the expected range of the bundle version. This is verified against the version of the currently installed bundle. The format of the version string is: `\d\.\d\.\d`.

It is possible to set a version to `None` or to `""`. The impact is that this version number (*min* or *max*) is not part of the version check.

Two values are returned:

1. `result`
A boolean variable that is either `True` in case of the version check is passed or `False` in case of the version check fails.
2. `reason`
In case of the version check is passed, `reason` is `None`. In case the version check fails, `reason` contains a short token string indicating the reason for the failure.

The method `verify_version` does not return a full error message, but a small token string only. This enables users to formulate their own error messages.

These are the token strings together with their meanings:

- `without_version_check`
Version check not executed.
- `wrong_minmax`
Mismatch of *minimum version* and *maximum version*: The *minimum version* is younger than the *maximum version*.
- `conflict_min`
The test execution requires the *minimum version*, but the installed version is older.
- `conflict_max`
The test execution requires the *maximum version*, but the installed version is younger.
- `internal_error`
An internal error happened (any issue in JSON configuration file).

Chapter 3

The JSONP format

The JSONP format is computed by the Python application **JsonPreprocessor**. Additionally to this, it is also possible to use the JSONP format within tests of a test automation framework called **RobotFramework AIO** ([homepage](#)).

The component that within the **RobotFramework AIO** is responsible for the execution of the **JsonPreprocessor**, is called **TestsuitesManagement**. Because the **JsonPreprocessor** is a pure Python application, whereas the **RobotFramework AIO** is a framework with own syntax rules and conditions, in some cases the JSONP format deviates in both worlds. Where required you will find corresponding hints in this chapter. In all other cases the JSONP format is the same in **JsonPreprocessor** and **RobotFramework AIO**.

This chapter explains the format of JSONP files in detail. We concentrate here on the content of the JSONP files and the corresponding results, available in Python dictionary format.

3.1 Standard JSON format

The **JsonPreprocessor** supports JSON files with standard extension `.json` and standard content.

- JSON file:

```
{
  "param1" : "value1",
  "param2" : "value2"
}
```

Outcome:

```
{'param1': 'value1', 'param2': 'value2'}
```

A JSON file with extension `.jsonp` and same content will produce the same output.

We recommend to give every JSON file the extension `.jsonp` to have a strict separation between the standard and the extended JSON format.

The following example still contains standard JSON content, but with parameters of several different data types (simple and composite).

```
{
  "param_01" : "string",
  "param_02" : 123,
  "param_03" : 4.56,
  "param_04" : ["A", "B", "C"],
  "param_05" : {"A" : 1, "B" : 2, "C" : 3}
}
```

This content produces the following output:

```
{'param_01': 'string',  
'param_02': 123,  
'param_03': 4.56,  
'param_04': ['A', 'B', 'C'],  
'param_05': {'A': 1, 'B': 2, 'C': 3}}
```

This output is of a certain dictionary type (named *dotdict*) that allows to access elements also with an object oriented dot notation (details about this format can be found in section [dotdict notation](#)).

3.2 Boolean and null values

JSON supports the boolean values `true` and `false`, and also the null value `null`.

In Python, the corresponding values differ: `True`, `False` and `None`.

Because the **JsonPreprocessor** is a Python application and therefore the returned content is required to be formatted Python compatible, the **JsonPreprocessor** does a conversion automatically.

Accepted in JSON files are both styles:

```
{
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

The output contains all keywords in Python style only:

```
{ 'param_06': True,
  'param_07': False,
  'param_08': None,
  'param_09': True,
  'param_10': False,
  'param_11': None}
```

3.3 Comments

Comments can be added to JSON files with `//`:

```
{
  // JSON keywords
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  // Python keywords
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

All lines starting with `//` are ignored by the **JsonPreprocessor**. The output of this example is the same as in the previous example.

Also block comments and inline comments are possible, realized by a pair of `/* */`:

```
{
  /*
  "param1" : 1,
  "param2" : "A",
  */

  "testlist" : ["A1", /*"B2", "C3",*/ "D4"]
}
```

Outcome:

```
{'testlist': ['A1', 'D4']}
```

3.4 Import of JSON files

We assume the following scenario:

A software component *A* requires a set of configuration parameters. A software component *B* that belongs to the same main software or to the same project, requires another set of configuration parameters. Additionally both components require a common set of parameters (with the same values).

The outcome is that at least we need two JSON configuration files:

1. A file `componentA.jsonp` containing all parameters required for component *A*
2. A file `componentB.jsonp` containing all parameters required for component *B*

But with this solution both JSON files would contain also the common set of parameters. This is unfavorable, because the corresponding values need to be maintained at two different positions.

Therefore we extend the list of JSON files by a file containing the common part only:

1. A file `common.jsonp` containing all parameters that are the same for component *A* and component *B*
2. A file `componentA.jsonp` containing remaining parameters (with specific values) required for component *A*
3. A file `componentB.jsonp` containing remaining parameters (with specific values) required for component *B*

Finally we use the import mechanism of the **JsonPreprocessor** to import the file `common.jsonp` in file `componentA.jsonp` and also in file `componentB.jsonp`.

This can be the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "./common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "./common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2"
}
```

Explanation:

JSON files are imported with the key `"[import]"`. The value of this key is the path and name of the JSON file to be imported.

A JSON file can contain more than one import. Imports can be nested: An imported JSON file can import further JSON files also.

Outcome:

The file `componentA.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_1': 'componentA value 1',
 'componentA_param_2': 'componentA value 2'}
```

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentB_param_1': 'componentB value 1',
 'componentB_param_2': 'componentB value 2'}
```

It can be seen that the returned dictionary contains both the parameters from the loaded JSON file and the parameters imported by the loaded JSON file.

Multiple imports of the same file

A JSONP file can be imported more than once anywhere in the set of JSONP configuration files. This mechanism can be used to define a common part for different subkeys.

Example:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

Outcome:

The content of `common.jsonp` is part of `componentA_param_1` and also part of `componentA_param_2`.

```
{'componentA_param_1': {'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_1_a': 'componentA_param_1_a value',
 'componentA_param_1_b': 'componentA_param_1_b value'},
 'componentA_param_2': {'common_param_1': 'common value 1',
 'common_param_2': 'common value 2',
 'componentA_param_2_a': 'componentA_param_2_a value',
 'componentA_param_2_b': 'componentA_param_2_b value'}}
```

Dynamic import paths

The values of `"[import]"` keys must be of type `str`. Therefore it's possible to use dollar operator expressions inside import paths.

An alternative version of the file `componentA.jsonp` from the example above, can look like this:

```
{
  "common_config_dir" : "./common_config",
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "${common_config_dir}/common.jsonp"↵
    ↵ ,
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "${common_config_dir}/common.jsonp"↵
    ↵ ,
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

The outcome is the same like in the previous example.

3.5 Overwriting parameters

We take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component *A*, a JSON file `componentB.jsonp` for component *B* and a JSON file `common.jsonp` for both components.

But now component *B* requires a different value of a common parameter: Within a JSON file we need to change the value of a parameter that is initialized within an imported file. That is possible.

This is now the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  // overwrite parameter initialized by imported file
  "common_param_2" : "common componentB value 2"
}
```

Explanation:

With

```
"common_param_2" : "common componentB value 2"
```

in `componentB.jsonp`, the initial definition

```
"common_param_2" : "common value 2"
```

in `common.jsonp` is overwritten.

Outcome:

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common componentB value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Important: *The value a parameter has finally, depends on the order of definitions, redefinitions and imports!*

In file `componentB.jsonp` we move the import of `common.jsonp` to the bottom:

```
{
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  "common_param_2" : "common componentB value 2"
  //
  // common parameters
  "[import]" : "../common.jsonp",
}
```

Now the imported file overwrites the value initialized in the importing file.

Outcome:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Up to now we considered simple data types only. In case we want to overwrite a parameter that is part of a composite data type, we need to extend the syntax. This is explained in the next examples.

Again we take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component *A*, a JSON file `componentB.jsonp` for component *B* and a JSON file `common.jsonp` for both components.

But now all values are part of composite data types like lists and dictionaries.

This is the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : ["common value 1.1", "common value 1.2"],
  "common_param_2" : {"common_key_2_1" : "common value 2.1" ,
                      "common_key_2_2" : "common value 2.2"}
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : ["componentA value 1.1", "componentA value 1.2"],
  "componentA_param_2" : {"componentA_key_2_1" : "componentA value 2.1" ,
                          "componentA_key_2_2" : "componentA value 2.2"}
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : ["componentB value 1.1", "componentB value 1.2"],
  "componentB_param_2" : {"componentB_key_2_1" : "componentB value 2.1" ,
                          "componentB_key_2_2" : "componentB value 2.2"}
}
```

Like in previous examples, the outcome is a merge of the imported JSON file and the importing JSON file, e.g. for `componentA.jsonp`:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentA_param_1': ['componentA value 1.1', 'componentA value 1.2'],
 'componentA_param_2': {'componentA_key_2_1': 'componentA value 2.1',
                        'componentA_key_2_2': 'componentA value 2.2'}}
```

Now the following questions need to be answered:

1. How to get the value of an already existing parameter?
2. How to get the value of a single element of a parameter of nested data type (list, dictionary)?
3. How to overwrite the value of a single element of a parameter of nested data type?
4. How to add an element to a parameter of nested data type?

We introduce another JSON file `componentB.2.jsonp` in which we import the JSON file `componentB.jsonp`. In this file we also add content to work with simple and composite data types to answer the questions above.

We introduce a new file `componentB.2.jsonp` that imports `componentB.jsonp` and creates new parameters based on already existing parameters:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  // some additional parameters of simple data type
  "string_val" : "ABC",
  "int_val"    : 123,
  "float_val"  : 4.56,
  "bool_val"   : true,
  "null_val"   : null,

  // access to existing parameters
  "string_val_b"      : ${string_val},
  "int_val_b"         : ${int_val},
  "float_val_b"       : ${float_val},
  "bool_val_b"        : ${bool_val},
  "null_val_b"        : ${null_val},
  "common_param_1_b"  : ${common_param_1},
  "componentB_param_2_b" : ${componentB_param_2}
}
```

Outcome:

```
{'bool_val': True,
 'bool_val_b': True,
 'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_1_b': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'},
 'componentB_param_2_b': {'componentB_key_2_1': 'componentB value 2.1',
                           'componentB_key_2_2': 'componentB value 2.2'},
 'float_val': 4.56,
 'float_val_b': 4.56,
 'int_val': 123,
 'int_val_b': 123,
 'null_val': None,
 'null_val_b': None,
 'string_val': 'ABC',
 'string_val_b': 'ABC'}
```

The rules for accessing parameters are:

- Existing parameters are accessed by a dollar operator and a pair of curly brackets (`${...}`) with the parameter name inside.
- If the entire expression of the right-hand side of the colon is such a dollar operator expression, it is not required to encapsulate this expression in quotes (in opposite to what pure JSON would require).
- Without quotes, the dollar operator keeps the data type of the referenced parameter. If you use quotes, the value of the used parameter will be converted to type `str`. This implicit string conversion is limited to parameters of simple data types like integers and floats. Composite data types like lists and dictionaries cannot be used for that.

In more detail:

The dollar operator keeps the data type of the referenced parameter. In case of `int_val` is of type `int`, also `int_val_b` is of type `int`:

```
"int_val_b" : ${int_val},
```

It is not required to encapsulate dollar operator expressions at the right-hand side of the colon in quotes. But nevertheless, it is possible to use quotes. In case of:

```
"int_val_b" : "${int_val}",
```

the parameter `int_val_b` is of type `str`.

Further content can be added between the double quotes. This can be used to create composite strings:

```
"str_val" : "ABC",
"int_val" : 1,
"float_val" : 2.3,
"bool_val" : True,
"none_val" : None,
"list_val" : [1,2,3],
"dict_val" : {"A" : "B"},
"newparam1" : "prefix_${str_val}_suffix",
"newparam2" : "prefix_${int_val}_suffix",
"newparam3" : "prefix_${float_val}_suffix",
"newparam4" : "prefix_${bool_val}_suffix",
"newparam5" : "prefix_${none_val}_suffix"
```

Outcome:

```
{'bool_val': True,
 'dict_val': {'A': 'B'},
 'float_val': 2.3,
 'int_val': 1,
 'list_val': [1, 2, 3],
 'newparam1': 'prefix_ABC_suffix',
 'newparam2': 'prefix_1_suffix',
 'newparam3': 'prefix_2.3_suffix',
 'newparam4': 'prefix_True_suffix',
 'newparam5': 'prefix_None_suffix',
 'none_val': None,
 'str_val': 'ABC'}
```

Using composite data types inside strings is not supported:

```
"newparam6" : "prefix_${listval}_suffix"
```

or:

```
"newparam7" : "prefix_${dictval}_suffix"
```

Result for `"newparam6"`:

```
The substitution of parameter '${listval}' inside the string value 'prefix_${listval}<→
  ↳ _suffix' is not supported! Composite data types like lists and dictionaries cannot <→
  ↳ be substituted inside strings.
```

Value of a single element of a parameter of nested data type

To access an element of a list and a key of a dictionary, we change the content of file `componentB.2.jsonp` to:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  "list_element_0" : ${componentB_param_1}[0],
  "dict_key_2_2" : ${common_param_2}['common_key_2_2']
}
```

Outcome:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                       'componentB_key_2_2': 'componentB value 2.2'},
 'dict_key_2_2': 'common value 2.2',
 'list_element_0': 'componentB value 1.1'}
```

Overwrite the value of a single element of a parameter of nested data type

In the next example we overwrite the value of a list element and the value of a dictionary key.

Again we change the content of file `componentB.2.jsonp`:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${componentB_param_1}[0] : "componentB value 1.1 (new)",
  ${common_param_2}['common_key_2_1'] : "common value 2.1 (new)"
}
```

The dollar operator syntax at the left-hand side of the colon is the same as previously used on the right-hand side. The entire expression at the left-hand side of the colon must *not* be encapsulated in quotes in this case.

Outcome:

The single elements of the list and the dictionary are updated, all other elements are unchanged.

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1 (new)',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1 (new)', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                       'componentB_key_2_2': 'componentB value 2.2'}}
```

Add an element to a parameter of nested data type

Adding further elements to an already existing list is not possible in JSON! But it is possible to add keys to an already existing dictionary.

The following example extends the dictionary `common_param_2` by an additional key `common_key_2_3`:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${common_param_2}[ 'common_key_2_3' ] : "common value 2.3"
}
```

Outcome:

```
{ 'common_param_1': ['common value 1.1', 'common value 1.2'],
  'common_param_2': { 'common_key_2_1': 'common value 2.1',
                     'common_key_2_2': 'common value 2.2',
                     'common_key_2_3': 'common value 2.3'},
  'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
  'componentB_param_2': { 'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'}}
```

Dictionary keys and indices as parameter

In all code examples above the indices of lists and the key names of dictionaries have been hard coded strings. It is also possible to use parameters:

```
{
  "index1"          : 0,
  "index2"          : 1,
  "key1"             : "keyA",
  "key2"             : "keyB",
  "testlist"         : ["A", "B"],
  "testdict"         : { "keyA" : "A", "keyB" : "B"},
  "tmp1"             : ${testlist}[${index1}],
  "tmp2"             : ${testdict}[${key1}],
  ${testlist}[${index1}] : ${testlist}[${index2}],
  ${testdict}[${key1}]   : ${testdict}[${key2}],
  ${testlist}[${index2}] : ${tmp1},
  ${testdict}[${key2}]   : ${tmp2}
}
```

Outcome:

```
{ 'index1': 0,
  'index2': 1,
  'key1': 'keyA',
  'key2': 'keyB',
  'testdict': { 'keyA': 'B', 'keyB': 'A'},
  'testlist': ['B', 'A'],
  'tmp1': 'A',
  'tmp2': 'A'}
```

Meaning of single quotes in square brackets

Single quotes are used to convert the content inside to a string.

- In case of the parameter `param` is of type `str`, the expressions `[${param}]` and `['${param}']` have the same outcome: The content inside the square brackets is a string. The single quotes have no meaning in this case (because the parameter is already of type `str`).
- In case of the parameter `param` is of type integer, the quotes in `['${param}']` convert the integer value to a string. Without the quotes (`[${param}]`), the content inside the square brackets is an integer.

In the context of **JsonPreprocessor** JSON files, only strings and integers are expected to be inside square brackets (except the brackets are used to define a list). Other data types are not supported here.

Whether a string or an integer is expected, depends on the data type of the parameter, the square bracket expression belongs to. Dictionaries require a string (a key name), lists require an integer (an index). Deviations will cause an error.

Summarized the following combinations are valid (on both the left-hand side of the colon and the right-hand side of the colon):

```
${listparam} [ ${intparam} ]
${listparam} [ 1 ]
${dictparam} [ '${intparam}' ]
${dictparam} [ ${stringparam} ]
${dictparam} [ '${stringparam}' ]
${dictparam} [ 'keyname' ]
```

Use of a common dictionary

The last example in this section covers the following use case:

- We have several JSON files, each for a certain purpose within a project (e.g. for every feature of this project a separate JSON file).
- They belong together and therefore they are all imported into a main JSON file that is the file that is handed over to the **JsonPreprocessor**.
- Every imported JSON file introduces a certain bunch of parameters. All parameters need to be a part of a common dictionary.
- Outcome is that finally only one single dictionary is used to access the parameters from all JSON files imported in the main JSON file.

These are the JSON files:

- `project.jsonp`

```
{
  // define some common values
  ${project_values}['common_project_param_1'] : "common project value 1",
  ${project_values}['common_project_param_2'] : "common project value 2",
  //
  // import feature parameters
  "[import]" : "../featureA.jsonp",
  "[import]" : "../featureB.jsonp",
  "[import]" : "../featureC.jsonp"
}
```

- `featureA.jsonp`

```
{
  // parameters required for feature A
  ${project_values}['featureA_params']['featureA_param_1'] : "featureA param 1 value",
  ${project_values}['featureA_params']['featureA_param_2'] : "featureA param 2 value"
}
```

- `featureB.jsonp`

```
{
  // parameters required for feature B
  ${project_values}['featureB_params']['featureB_param_1'] : "featureB param 1 value",
  ${project_values}['featureB_params']['featureB_param_2'] : "featureB param 2 value"
}
```

- `featureC.jsonp`

```
{
  // parameters required for feature C
  ${project_values}['featureC_params']['featureC_param_1'] : "featureC param 1 value",
  ${project_values}['featureC_params']['featureC_param_2'] : "featureC param 2 value"
}
```

It is not required to start the code listed above, with dictionary initializations like

```
"project_values" : {},
${project_values}['featureA_params'] : {},
${project_values}['featureB_params'] : {},
${project_values}['featureC_params'] : {},
```

These initializations are done implicitly by the **JsonPreprocessor**. Further details about the implicit creation of dictionaries can be found in section [Implicit creation of dictionaries](#).

It is for sure still possible to do the initialization of a dictionary explicitly with `{}`. But keep in mind: This deletes all already existing keys in this dictionary!

Outcome:

```
{'project_values': {'common_project_param_1': 'common project value 1',
                    'common_project_param_2': 'common project value 2',
                    'featureA_params': {'featureA_param_1': 'featureA param 1 value',
                                         'featureA_param_2': 'featureA param 2 value'},
                    'featureB_params': {'featureB_param_1': 'featureB param 1 value',
                                         'featureB_param_2': 'featureB param 2 value'},
                    'featureC_params': {'featureC_param_1': 'featureC param 1 value',
                                         'featureC_param_2': 'featureC param 2 value'}}
```

3.6 dotdict notation

Up to now we have accessed dictionary keys in this way (standard notation):

```
${dictionary}['key']['sub_key']
```

Additionally to this standard notation, the **JsonPreprocessor** supports the so called *dotdict* notation where keys are handled as attributes:

```
${dictionary.key.sub_key}
```

In standard notation keys are encapsulated in square brackets and all together is placed *outside* the curly brackets. In dotdict notation the dictionary name and the keys are separated by dots from each other. All together is placed *inside* the curly brackets.

In standard notation key names are allowed to contain dots:

```
${dictionary}['key']['sub.key']
```

In dotdict notation this would cause ambiguities:

```
${dictionary.key.sub.key}
```

Therefore it is not possible to implement in this way! In case you need to have dots inside key names, you must use the standard notation. We recommend to prefer underlines as separator - like done in the examples in this document.

Do you really need dots inside key names?

Please keep in mind: The dotdict notation is a reduced one. Because of parts are missing (e.g. the single quotes around key names), the outcome can be code that is really hard to capture.

In the following example we create a composite data structure and demonstrate how to access single elements in both notations.

• JSON file:

```
{
  // composite data structure
  "params" : [{ "dict_1_key_1" : "dict_1_key_1 value",
                "dict_1_key_2" : ["dict_1_key_2 value 1", "dict_1_key_2 value 2"] },
              //
              { "dict_2_key_1" : "dict_2_key_1 value",
                "dict_2_key_2" : { "dict_2_A_key_1" : "dict_2_A_key_1 value",
                                   "dict_2_A_key_2" : ["dict_2_A_key_2 value 1", "↵
↵ dict_2_A_key_2 value 2"] } } } ],
  //
  // access to single elements of composite data structure
  //
  // a) standard notation
  "dict_1_key_2_value_2_standard" : ${params}[0]['dict_1_key_2'][1],
  // b) dotdict notation
  "dict_1_key_2_value_2_dotdict" : ${params.0.dict_1_key_2.1},
  //
  // c) standard notation
  "dict_2_A_key_2_value_2_standard" : ${params}[1]['dict_2_key_2']['dict_2_A_key_2'][1]
  // d) dotdict notation
  "dict_2_A_key_2_value_2_dotdict" : ${params.1.dict_2_key_2.dict_2_A_key_2.1}
}
```

Outcome:

In case of the composite data structure becomes more and more nested (and if also the key names contain numbers), understanding the expressions (like `${params.1.dict_2_key_2.dict_2_A_key_2.1}`) becomes more and more challenging!

```
{'dict_1_key_2_value_2_standard': 'dict_1_key_2 value 2',
 'dict_2_A_key_2_value_2_standard': 'dict_2_A_key_2 value 2',
 'params': [{ 'dict_1_key_1': 'dict_1_key_1 value',
               'dict_1_key_2': ['dict_1_key_2 value 1', 'dict_1_key_2 value 2']},
             { 'dict_2_key_1': 'dict_2_key_1 value',
               'dict_2_key_2': { 'dict_2_A_key_1': 'dict_2_A_key_1 value',
                                'dict_2_A_key_2': ['dict_2_A_key_2 value 1',
                                                    'dict_2_A_key_2 value 2']}}}]}
```

3.7 Dynamic key names

In section [Overwriting parameters](#) we mentioned the possibility to define the value of string parameters dynamically, e.g. in this way:

```
"str_val"    : "ABC",
"newparam1"  : "prefix_${str_val}_suffix",
```

The value of `newparam1` is defined by an expression that is encapsulated in quotes and contains - beneath hard coded parts - a dollar operator expression (that is the dynamic part).

The same is also possible on the left-hand side of the colon. In this case the name of a parameter is created dynamically.

Example:

```
"strval"    : "A",
"dictval"   : {"A_2" : 1},
${dictval}['${strval}_2'] : 2
```

In second line a new dictionary with key `A_2` is defined. In third line we overwrite the initial value of this key with another value. The name of this key is defined with the help of parameter `strval`.

Outcome:

```
{'dictval': {'A_2': 2}, 'strval': 'A'}
```

The same in dotdict notation:

```
"strval"    : "A",
"dictval"   : {"A_2" : 1},
${dictval}.${strval}_2 : 3
```

The precondition for using dynamic key names is that a key with the resulting name (here `A_2`) does exist already. Therefore this mechanism can be used to overwrite the value of existing keys, but cannot be used to create new keys!

This will not work (because of a key with name `A_2` does not yet exist):

```
"strval"    : "A",
"dictval"   : {"${strval}_2" : 1}
```

Outcome:

```
A substitution in key names is not allowed! Please update the key name "${strval}_2"
```

3.8 Implicit creation of dictionaries

Up to now we have discussed two different ways of creating nested dictionaries.

The first one is “on the fly”, like:

```
{
  "project_values" : {"keyA" : "keyA value",
                     "keyB" : {"keyB1" : "keyB1 value",
                              "keyB2" : {"keyB21" : "keyB21 value",
                                         "keyB22" : "keyB22 value"}}}
}
```

In case of it is required to split the definition into several files, we have to add keys (and also the initialization) line by line:

```
{
  "project_values" : {},
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB'] : {},
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2'] : {},
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

The result will be the same as in the previous example.

It can be seen now that this way of creating nested dictionaries is rather long-winded, because every initialization of a dictionary requires a separate line of code (at every level).

To shorten the code, the **JsonPreprocessor** supports an implicit creation of dictionaries.

This is the resulting code in standard notation:

```
{
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

And the same in dotdict notation (with precondition, that no key name contains a dot):

```
{
  ${project_values.keyA} : "keyA value",
  ${project_values.keyB.keyB1} : "keyB1 value",
  ${project_values.keyB.keyB2.keyB21} : "keyB21 value",
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

Caution:

We urgently recommend *not* to mixup both styles in one line of code. In case of keys contain a list and also numerical indices are involved, we recommend to prefer the standard notation.

Please be aware of: In case of a missing level in between an expression like

```
{
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

you will *not* get an error message! The entire data structure will be created implicitly. The impact is that this method is very susceptible to typing mistakes.

The implicit creation of data structures does not work with lists! In case you use a list index out of range, you will get a corresponding error message.

Key names

The implicit creation of data structures is only possible with *hard coded* key names. Parameters are not supported.

Example:

```
{
  "paramA" : "ABC",
  "subKey" : "ABC",
  ${testdict.subKey.subKey.paramA} : "DEF"
}
```

All sub key levels within the expression `${testdict.subKey.subKey.paramA}` are interpreted as hard coded strings, even in case of parameters with the same name do exist.

For example: The name of the implicitly created key at bottom level is `"paramA"`, and not the value `"ABC"` of the parameter with the same name (`"paramA"`).

Therefore the outcome is:

```
{'paramA': 'ABC', 'subKey': 'ABC', 'testdict': {'subKey': {'subKey': {'paramA': 'DEF'}}}}
```

Reference to existing keys

It is possible to use parameters to refer to *already existing* keys.

```
{
  // <data structure created implicitly>
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // <string parameter with name of an existing key>
  "keyName_3" : "subKey_3",

  // <parameter used to refer to an existing key>
  ${testdict.subKey_1.subKey_2.${keyName_3}} : "XYZ"
}
```

Outcome:

```
{'keyName_3': 'subKey_3',
 'testdict': {'subKey_1': {'subKey_2': {'subKey_3': 'XYZ'}}}}
```

Parameters cannot be used to create new keys.

```
{
  // <data structure created implicitly>
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // <string parameter with name of a not existing key>
  "keyName_4" : "subKey_4",

  // <usage of keyName\_4 is not possible here>
  ${testdict.subKey_1.subKey_2.subKey_3.${keyName_4}} : "XYZ"
}
```

Outcome is the following error:

```
"The implicit creation of data structures based on nested parameter is not supported ..."
```

The same error will happen in case of the standard notation is used:

```
{
  // <usage of keyName\_4 is not possible here>
  ${testdict}['subKey_1']['subKey_2']['subKey_3'][${keyName_4}] : "XYZ"
}
```

3.9 Python inline code

It might be required to have access to Python builtin functions, e.g. for combining multiple lists into a single list. In pure Python this can be realized in the following way:

```
A = [1,2,3]
B = [4,5,6]
C = A + B
print(C)
```

With result:

```
[1, 2, 3, 4, 5, 6]
```

In JSONP files, the implementation looks as follows:

```
"A" : [1,2,3],
"B" : [4,5,6],
"C" : <\textless{}\textless{}>${A} + ${B}<\textgreater{}\textgreater{}>
```

The expression `${A} + ${B}` is handled as Python inline code. It is possible to use dollar operator expressions inside Python inline code to access any other parameter defined in JSONP files.

The syntax of Python inline code is:

```
<\textless{}\textless{}><(\emph{Python expression})><\textgreater{}\textgreater{}>
```

Limitations

The Python inline code syntax cannot be used within strings. The following code is invalid:

```
"param" : "The value is <\textless{}\textless{}><(\emph{Python expression})><\textgreater{}↵
↵ {}>\textgreater{}>"
```

The usage of Python inline code is limited to the right-hand side of the colon (*key values*). This feature cannot be used to create key names.

Python inline code cannot be used immediately to select list elements or dictionary keys that shall be overwritten. The following code is invalid:

```
"param" : [1,2,3],
${param} [<\textless{}\textless{}>0 if True else 1<\textgreater{}\textgreater{}>] : 4
```

This is a possible workaround:

```
"param" : [1,2,3],
"index" : <\textless{}\textless{}>0 if True else 1<\textgreater{}\textgreater{}>,
${param}[${index}] : 4
```

Further examples

Create a list of dictionary keys:

```
"dict"      : {"kA" : "vA", "kB" : "vB"},
"key_list"  : <\textless{}\textless{}>list(${dict}.keys())<\textgreater{}\textgreater{}>
```

Define an import path:

```
"path_1" : "./file_1.jsonp",
"path_2" : "./file_2.jsonp"
//
"[import]" : <\textless{}\textless{}>${path_1} if True else ${path_2}<\textgreater{}↵
↵ \textgreater{}>
```

3.10 Special characters within key names

Basically, the **JsonPreprocessor** considers the JSON naming convention for key names. Within JSONP files allowed is what JSON allows, but with the following limitation: The JSONP format extends the JSON format with some features using square, curly and angle brackets as syntax elements: `[`, `]`, `{`, `}`, `<`, `>`.

These brackets must not be used within key names!

This JSONP code:

```
{
  "[ " : 1,
  "B" : ${[]}
}
```

causes:

```
Error: 'Invalid expression found: '${[]}' - The brackets mismatch!!!!'
```

All other special characters can be used immediately:

```
{
  "$" : 1,
  "B" : ${$}
}
```

Result:

```
DotDict({'$': 1, 'B': 1})
```

Strings are handled as raw strings. Therefore, masking has no effect:

```
{
  "\\$" : 1,
  "B" : ${\\$}
}
```

The backslashes are part of the key name:

```
DotDict({'\\$': 1, 'B': 1})
```

The masking requires two backslashes! A single backslash will cause a JSON syntax error!



Format deviation

In opposite to the **JsonPreprocessor**, the **TestsuitesManagement** of the **RobotFramework AIO** is more restrictive and does not allow any special characters. Key names are limited to letters and digits!

3.11 Byte sequences

The **TestsuitesManagement** configuration (in JSONP format) supports the explicit embedding of binary values (in Hexadecimal Escape Notation).

```
"bytesequence" : b'\x52\x6f\x62\x6f\x74\x46\x72\x61\x6d\x65\x77\x6f\x72\x6b\x20\x41\x49\↵  
↵ x4f'
```

The result is the resolved content:

```
log    bytesequence: ${bytesequence}
```

```
INFO - bytesequence: RobotFramework AIO
```

This feature requires the usage of single quotes: `b'...'`. Double quotes must not be used.

Chapter 4

config.py

Defines the properties of configuration and holds the identified config files.

The loading configuration method is divided into 4 levels: level1 has the highest priority, level4 has the lowest priority.

Level1: Defined in command line

Level2: Read from content of JSON config file

```

{
  "default": {
    "name": "robot_config.jsonp",
    "path": "./config/"
  },
  "variant_0": {
    "name": "robot_config.jsonp",
    "path": "./config/"
  },
  "variant_1": {
    "name": "robot_config_variant_1.jsonp",
    "path": "./config/"
  },
  ...
  ...
}
```

According to the ConfigName, RobotFramework_TestsuitesManagement will choose the corresponding config file. `./config/` indicates the relative path to json config file.

Level3: Read in testsuite folder: `/config/robot_config.jsonp`

Level4: Read from RobotFramework AIO installation folder:

`/RobotFramework/defaultconfig/robot_config.jsonp`

4.1 Method: load_config

This load_config method uses to load configuration's parameters from json files.

Arguments:

- No input parameter is required

Returns:

- No return variable

4.2 Method: check_version

This method validates the current package version with maximum and minimum version.

In case the current version is not between min and max version, then the execution of testsuite is terminated with "unknown" state

Chapter 5

key__words.py

5.1 Class: CSetupKeywords

This class defines the keywords for the setup and the teardown of testcases and testsuites.

5.1.1 Keyword: testsuite__setup

This keyword loads the RobotFramework AIO configuration, checks the version of the RobotFramework AIO and logs out the basic information about the test execution.

Arguments:

- `testsuite_cfg_file`
/ *Condition*: required / *Type*: string /
`testsuite_cfg_file=''` and variable `config_file` is not set RobotFramework AIO will check for configuration level 3, and level 4.
`testsuite_cfg_file` is set with a `<json_config_file_path>` and variable `config_file` is not set RobotFramework AIO will load configuration level 2.

Returns:

- No return variable

5.1.2 Keyword: testsuite__teardown

This keyword writes information about the testsuite result to the log files.

5.1.3 Keyword: testcase__setup

This keyword writes the number of counted tests to the log files.

5.1.4 Keyword: testcase__teardown

This keyword writes information about the testcase result to the log files.

5.2 Class: CGeneralKeywords

Class to define general keywords, that have nothing to do with the setups and teardowns of suites and tests.

5.2.1 Keyword: get_config

This `get_config` defines the `Get Config` keyword gets the current config object of RobotFramework AIO.

Arguments:

- No parameter is required

Returns:

- `config_obj.config_params`
/ *Type*: json /

5.2.2 Keyword: load_json

Loads a json file and returns a json object.

Arguments:

- `json_file`
/ *Condition*: required / *Type*: string /
The path of Json configuration file.
- `level`
/ *Condition*: required / *Type*: int /
Level = 1 -> loads the content of `json_file`.
level != 1 -> loads the json file which is set with variant (likes loading config level2)

Returns:

- `json_data`
/ *Type*: json /

5.2.3 Keyword: is_robotframework_aio

Returns * `True`: RobotFramework AIO is installed * `False`: RobotFramework AIO is not installed (= standalone installation of TestsuitesManagement)

5.2.4 Keyword: get_package_context_file

Returns path and name of package context file

5.2.5 Keyword: get_reference_version

Returns the version number used as reference for version checks. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.6 Keyword: get_reference_version_date

Returns the version date belonging to the reference. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.7 Keyword: get_reference_app_name

Returns the name of the reference application. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.8 Keyword: get_reference_installer_location

Returns the location of the reference installer. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.9 Keyword: get_tsm_version

Returns the version of the TestsuitesManagement.

5.2.10 Keyword: get_tsm_version_date

Returns the version date of the TestsuitesManagement.

5.2.11 Keyword: get_tsm_app_name

Returns the application name of the TestsuitesManagement.

5.2.12 Keyword: get_tsm_installer_location

Returns the location of the TestsuitesManagement installer.

5.2.13 Keyword: get_bundle_version

Returns the version of the entire RobotFramework AIO bundle.

5.2.14 Keyword: get_bundle_version_date

Returns the version date of the entire RobotFramework AIO bundle.

5.2.15 Keyword: get_bundle_name

Returns the name of the entire RobotFramework AIO bundle.

5.2.16 Keyword: get_bundle_installer_location

Returns the location of the installer of the entire RobotFramework AIO bundle.

5.2.17 Keyword: check_version

This keyword executes a version check, min_version and max_version are checked against the reference_version.

5.2.18 Keyword: get_status_messages

Returns the StatusMessages object.

Chapter 6

on_failure_handle.py

Chapter 7

event.py

Chapter 8

scope_event.py

8.1 Class: ScopeEvent

8.2 Class: ScopeStart

8.3 Class: ScopeEnd

Chapter 9

app_config.py

9.1 Function: print_bundle_version

Prints the version of the entire RobotFramework AIO bundle.

9.2 Class: AppConfig

Configuration class that contains all the information about the application.

9.2.1 Method: is_robotframework_aio

Returns * True: RobotFramework AIO is installed * False: RobotFramework AIO is not installed (= standalone installation of TestsuitesManagement)

9.2.2 Method: get_package_context_file

Returns path and name of package context file

9.2.3 Method: get_reference_version

Returns the version number used as reference for version checks. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

9.2.4 Method: get_reference_version_date

Returns the version date belonging to the reference. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

9.2.5 Method: get_reference_app_name

Returns the name of the reference application. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

9.2.6 Method: get_reference_installer_location

Returns the location of the reference installer. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

9.2.7 Method: get_tsm_version

Returns the version of the TestsuitesManagement.

9.2.8 Method: get_tsm_version_date

Returns the version date of the TestsuitesManagement.

9.2.9 Method: get_tsm_app_name

Returns the application name of the TestsuitesManagement.

9.2.10 Method: get_tsm_installer_location

Returns the location of the TestsuitesManagement installer.

9.2.11 Method: get_bundle_version

Returns the version of the entire RobotFramework AIO bundle.

9.2.12 Method: get_bundle_version_date

Returns the version date of the entire RobotFramework AIO bundle.

9.2.13 Method: get_bundle_name

Returns the name of the entire RobotFramework AIO bundle.

9.2.14 Method: get_bundle_installer_location

Returns the location of the installer of the entire RobotFramework AIO bundle.

Chapter 10

app_logger.py

Creates a completely isolated, application-specific logger.
Returns the created logger

Chapter 11

lib_listener.py

This `LibListener` class defines the hook methods.

- `_start_suite` hooks to every starting testsuite of robot run.
- `_end_suite` hooks to every ending testsuite of robot run.
- `_start_test` hooks to every starting test case of robot run.
- `_end_test` hooks to every ending test case of robot run.

Chapter 12

`new_struct.py`

This `CStruct` class creates the given attributes dynamically at runtime.

Chapter 13

version.py

13.1 Class: `enVersionCheckResult`

Defines different states that identify the result of the version check

13.2 Class: `StatusMessages`

Dictionary wrapper for status messages of version check. Needs to use the same keys like defined in `enVersionCheckResult`

13.2.1 Method: `get_messages_dict`

Returns the entire messages dictionary.

Background: Robot Framework requires a concrete dictionary, and not a class object containing a dictionary

13.2.2 Method: `get`

Get a status message by key.

13.2.3 Method: `set`

Set a status message.

13.3 Class: `CVersion`

Validate a user-defined version against a reference version.

The reference can be:

- the version of the `TestsuitesManagement` (in case of a stand-alone installation)
- the version of the `RobotFramework AIO` (`TestsuitesManagement` is part of a bundle)

13.3.1 Method: `verify_version`

This method executes a version check at low level (a token string is returned only).

The `min_version` and the `max_version` are checked against the `reference_version`.

If the `reference_version` is `None` (= not defined by user), the internally defined `bundle_version` (either `RobotFramework AIO` or `TestsuitesManagement`) will be used as reference instead.

Arguments:

- `min_version`
/ *Condition*: optional / *Type*: str /
- `max_version`
/ *Condition*: optional / *Type*: str /
- `reference_version`
/ *Condition*: optional / *Type*: str /

Returns:

- `result`
/ *Type*: str /
A token string indicating the result of the version check.

13.3.2 Method: verifyVersion

This is a wrapper for the `verify_version()` function.

13.3.3 Method: check_version

This method executes the version check at high level.

The `min_version` and the `max_version` are checked against the `reference_version`.

If the `reference_version` is `None` (= not defined by user), the internally defined `bundle_version` (either RobotFramework AIO or TestsuitesManagement) will be used as reference instead.

The execution includes error messages and exception handling (in opposite to the low level method `verify_version`). Impact is that this method influences the execution of the application that calls it.

Arguments:

- `min_version`
/ *Condition*: optional / *Type*: str /
- `max_version`
/ *Condition*: optional / *Type*: str /
- `reference_version`
/ *Condition*: optional / *Type*: str /
- `ext_logger`
/ *Condition*: optional / *Type*: object /
- `status_messages`
/ *Condition*: optional / *Type*: dict /

Returns:

- `True`
/ *Type*: boolean /
Executed version check passed or check not executed.
- `False`
/ *Type*: boolean /
Executed version check failed.

13.3.4 Method: checkVersion

This is a wrapper for the `check_version()` function.

13.3.5 Method: get_last_error

Returns the most recently occurred error (during execution of low level method `verify_version`)

13.3.6 Method: validate_min_version

Static method to validate the required minimum version against the reference version.

Arguments:

- `tuple_ref_version`
/ *Condition*: required / *Type*: tuple /
The version used as reference
- `tuple_min_version`
/ *Condition*: required / *Type*: tuple /
The required minimum version

Returns:

- True or False

13.3.7 Method: validate_max_version

Static method to validate the required maximum version against the reference version.

Arguments:

- `tuple_ref_version`
/ *Condition*: required / *Type*: tuple /
The version used as reference
- `tuple_max_version`
/ *Condition*: required / *Type*: tuple /
The required maximum version

Returns:

- True or False

13.3.8 Method: validate_sub_version

Static method to validate the format of the provided sub-version and parse it into a sub-tuple for version comparison.

Arguments:

- `version`
/ *Condition*: required / *Type*: str /
A part of the entire version string (either the major version, the minor version or the patch version)

Returns:

- `list_sub_version`
/ *Type*: tuple /

13.3.9 Method: tuple_version

Static method to convert a version string to a tuple of the format: (major, minor, patch)

In case minor/patch version is missing, it is set to 0. E.g: "1" is transformed to "1.0.0" and "1.1" is transformed to "1.1.0"

This method also supports version strings containing additional tags indicating certain types of versions:

- Alpha version: 'a'
- Beta version: 'b'
- Release candidate: 'rc'

Examples: "1.2rc3", "1.2.1b1"

Arguments:

- `version`
/ *Condition*: required / *Type*: str /
The version string to be converted

Returns:

- `list_version`
/ *Type*: tuple /
A tuple which contains the (major, minor, patch) version.

Chapter 14

`__init__.py`

14.1 Class: `RobotFramework_TestsuitesManagement`

The **`RobotFramework_TestsuitesManagement`** enables users to define dynamic configuration values within separate configuration files in JSONP format.

Details about how to install can be found in the [README.rst](#)

14.1.1 Method: `run_keyword`

This is a wrapper method for Robot Framework's keyword execution mechanism.

14.1.2 Method: `get_keyword_tags`

This method retrieves the keyword tags using the parent (`DynamicCore`) implementation.

14.1.3 Method: `get_keyword_documentation`

This method delegates the request for keyword documentation to Robot Framework's dynamic library core (`DynamicCore`).

14.1.4 Method: `failure_occurred`

This method is executed when `RobotFramework_TestsuitesManagement` keyword fails. By default, executes the registered run-on-failure keyword. `RobotFramework_TestsuitesManagement` can overwrite this hook method in case provides custom functionality instead.

Arguments:

- None

Returns:

- None

14.2 Class: `CConfigLevel`

14.3 Class: `CTestsuitesCfg`

Chapter 15

Appendix

About RobotFramework_TestsuitesManagement:

Author

Mai Dinh Nam Son (son.maidinhnam@vn.bosch.com)

Version

0.11.0 (27.02.2026)

Short Description

Functionality to manage Robot Framework testsuites

Homepage

[robotframework-testsuitesmanagement](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 16

History

0.1.0	06/2022
<i>Initial version</i>	
0.2.2	07/2022
<i>Created documentation and updated message logs</i>	
0.3.0	07/2022
<i>Added local configuration feature; documentation rework</i>	
0.4.0	03/2023
<i>Maintenance of log output; maintenance of JSON schema validation of configuration file</i>	
0.5.0	03/2023
<i>Added JSON Dotdict feature</i>	
0.5.1	04/2023
<i>Added JSON self-test component</i>	
0.6.0	04/2023
- Reworked the method for loading configuration level 3 - Improved version check method	
0.7.0	05/2023
- Reworked the method for loading configuration level 4 - Improved error message logs - Added a method to check invalid local configuration file	
0.7.1	05/2023
Introduce package context which allows RobotFramework_TestsuitesManagement work in many contexts: - stand-alone - as part of RobotFramework AIO package	
0.7.3	09/2023
<i>New RobotFramework 6.1 adaptation; Maintenance of log output</i>	
0.7.4	12/2023
- Improve dotdict feature - Update logging - Code maintenance	
0.7.5	03/2024

CHAPTER 16. HISTORY

<i>Simplified local configuration loading</i>	
0.7.6	04/2024
- Added naming convention check for parameter names. - Updated logging message.	
0.7.7	04/2024
<i>Extended debugging support. In case of JSON syntax errors in configuration files, the Robot Framework log files contain an extract of the JSON content nearby the position, where the error occurred.</i>	
0.7.8	05/2024
<i>Improved error messages in case of issues while loading the test configuration.</i>	
0.7.9	06/2024
- Updated JSON schema validation. - Resolved redundant code.	
0.7.10	08/2024
- Changed naming convention check behavior of TestsuitesManagement to align with Robot Framework core. - Code maintenance. - Updated documentation. - Improved error message log.	
0.7.11	10/2024
- Fixed missing metadata information in the log while running configuration level 1. - Code improvement. - Improved Maximum and Minimum version check mechanism. - Updated an information for UNKNOWN state.	
0.7.12	2/2025
<i>Fixed bugs and updated error messages of naming convention check feature.</i>	
0.8.0	3/2025
- Added a naming convention check for Robot Framework variables which are set in JSONP configuration files. - Extended logging of test results. - Replaced <i>re</i> package by <i>regex</i> package.	
0.8.1	5/2025
<i>regex module added to the package requirements.</i>	
0.8.2	9/2025
<i>Added the possibility to trigger a version check of the current package (by method <code>versionCheck</code>).</i>	
0.9.0	11/2025
- Rework of version check mechanism. - Added keywords and methods to access several version numbers separately.	
0.9.1	01/2026
- Stabilized the version check mechanism. - Updated code following PEP 8 rules. - Fixed bugs.	
0.10.0	02/2026

CHAPTER 16. HISTORY

<i>Restored the console script robfwaio_version.</i>	
0.11.0	02/2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools.</i>	

10.6 QConnectBase

QConnectBase

v. 1.4.0

Nguyen Huynh Tri Cuong

11.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	User interface	2
2.1.1	Keyword overview	2
2.1.2	Library import	2
2.1.3	Usecases	3
2.2	Verify examples	6
2.2.1	Verify a single message (1)	6
2.2.2	Verify a single message (2)	6
2.2.3	Verify a block of messages (1)	7
2.2.4	Verify a block of messages (2)	8
2.2.5	Verify a block of messages (3)	9
2.2.6	Verify a block of messages (4)	10
2.3	=== REWORKED ===	10
2.4	Connection types	11
2.5	Usage	11
2.5.1	connect	11
2.5.2	disconnect	12
2.5.3	send command	12
2.5.4	transfer file	13
2.5.5	transfer item	13
2.5.6	verify	13
2.6	Example	17
2.7	Auxiliary Utilities	19
2.7.1	Introduction to RMQSignal	19
2.7.2	Prerequisites	19
2.7.3	Library Keywords and Usage	20
2.7.4	Example of Inter-Process Communication	22
2.8	Contribution Guidelines	22
2.9	Configure Git and correct EOL handling	23
2.10	Sourcecode Documentation	23
2.11	Feedback	23
2.12	About	23
2.12.1	Maintainers	23
2.12.2	Contributors	23
2.13	License	23

CONTENTS

CONTENTS

3	<code>__init__.py</code>	25
4	<code>connection_base.py</code>	26
4.1	Class: <code>BrokenConnError</code>	26
4.2	Class: <code>EndOfBlockNotFound</code>	26
4.3	Class: <code>NoFilteredMsgFound</code>	26
4.4	Class: <code>ConnectionBase</code>	26
4.4.1	Method: <code>is_supported_platform</code>	26
4.4.2	Method: <code>is_precondition_pass</code>	26
4.4.3	Method: <code>get_connection_type</code>	26
4.4.4	Method: <code>error_instruction</code>	27
4.4.5	Method: <code>quit</code>	27
4.4.6	Method: <code>connect</code>	27
4.4.7	Method: <code>disconnect</code>	27
4.4.8	Method: <code>send_obj</code>	28
4.4.9	Method: <code>read_obj</code>	28
4.4.10	Method: <code>wait_4_trace</code>	28
4.4.11	Method: <code>wait_4_trace_continuously</code>	29
4.4.12	Method: <code>create_and_activate_trace_queue</code>	29
4.4.13	Method: <code>deactivate_and_delete_trace_queue</code>	30
4.4.14	Method: <code>activate_trace_queue</code>	30
4.4.15	Method: <code>deactivate_trace_queue</code>	31
4.4.16	Method: <code>check_timeout</code>	31
4.4.17	Method: <code>pre_msg_check</code>	31
4.4.18	Method: <code>post_msg_check</code>	31
5	<code>connection_manager.py</code>	33
5.1	Class: <code>InputParam</code>	33
5.2	Class: <code>ConnectParam</code>	33
5.3	Class: <code>SendCommandParam</code>	33
5.4	Class: <code>VerifyParam</code>	33
5.5	Class: <code>ConnectionManager</code>	33
5.5.1	Method: <code>import_modules_from_paths</code>	33
5.5.2	Method: <code>quit</code>	33
5.5.3	Method: <code>add_connection</code>	33
5.5.4	Method: <code>remove_connection</code>	34
5.5.5	Method: <code>get_connection_by_name</code>	34
5.5.6	Keyword: <code>disconnect</code>	34
5.5.7	Keyword: <code>connect</code>	35
5.5.8	Keyword: <code>send_command</code>	35
5.5.9	Keyword: <code>transfer_file</code>	36
5.5.10	Keyword: <code>transfer_item</code>	36
5.5.11	Keyword: <code>execute_script</code>	37
5.5.12	Keyword: <code>set_default_verify_timeout</code>	37
5.5.13	Keyword: <code>set_default_emergency_timeout</code>	37
5.5.14	Keyword: <code>verify</code>	38
5.6	Class: <code>TestOption</code>	39

CONTENTS

CONTENTS

6	constants.py	40
6.1	Class: SocketType	40
6.2	Class: String	40
7	grpc_client.py	41
7.1	Class: GrpcClientConfig	41
7.2	Class: GrpcClient	41
7.2.1	Method: connect	41
7.2.2	Method: disconnect	42
7.2.3	Method: close	42
7.2.4	Method: quit	42
7.2.5	Method: wait_4_trace	42
8	rabbitmq_client.py	44
8.1	Class: RabbitmqClientConfig	44
8.2	Class: RabbitmqClient	44
8.2.1	Method: connect	44
8.2.2	Method: close	44
8.2.3	Method: quit	44
8.3	Class: RMQSignal	44
8.3.1	Method: send_signal	44
8.3.2	Method: unset_signal_receiver_name	45
8.3.3	Method: set_signal_receiver_name	45
8.3.4	Method: consume_channel	45
8.3.5	Method: wait_for_signal	46
8.3.6	Method: wait_for_signals	46
9	qlogger.py	47
9.1	Class: ColorFormatter	47
9.1.1	Method: format	47
9.2	Class: QFileHandler	47
9.2.1	Method: get_log_path	47
9.2.2	Method: get_config_supported	48
9.3	Class: QDefaultFileHandler	48
9.3.1	Method: get_log_path	48
9.3.2	Method: get_config_supported	48
9.4	Class: QConsoleHandler	48
9.4.1	Method: get_config_supported	49
9.5	Class: QLogger	49
9.5.1	Method: get_logger	49
9.5.2	Method: set_handler	49
10	serial_base.py	50
10.1	Class: SerialConfig	50
10.2	Class: SerialSocket	50
10.2.1	Method: connect	50
10.2.2	Method: disconnect	50

CONTENTS

CONTENTS

10.2.3	Method: quit	50
10.3	Class: SerialClient	50
10.3.1	Method: connect	51
11	raw_tcp.py	52
11.1	Class: RawTCPBase	52
11.2	Class: RawTCPServer	52
11.3	Class: RawTCPClient	52
12	ssh_client.py	53
12.1	Class: AuthenticationType	53
12.2	Class: SSHConfig	53
12.3	Class: SSHClient	53
12.3.1	Method: connect	53
12.3.2	Method: transfer_file	53
12.3.3	Method: transfer_item	54
12.3.4	Method: close	54
12.3.5	Method: quit	54
13	tcp_base.py	55
13.1	Class: TCPConfig	55
13.2	Class: TCPBase	55
13.2.1	Method: close	55
13.2.2	Method: quit	55
13.2.3	Method: connect	55
13.2.4	Method: disconnect	55
13.3	Class: TCPBaseServer	56
13.3.1	Method: accept_connection	56
13.4	Class: TCPBaseClient	56
14	utils.py	57
14.1	Function: has_capturing_groups	57
14.2	Function: validate_regex_pattern	57
14.3	Class: Singleton	57
14.4	Class: DictToClass	57
14.5	Class: Utils	57
14.5.1	Method: get_all_descendant_classes	57
14.5.2	Method: get_all_sub_classes	58
14.5.3	Method: caller_name	58
14.5.4	Method: load_library	58
14.5.5	Method: is_ascii_or_unicode	58
14.6	Class: Job	59
14.7	Class: ResultType	59
14.8	Class: ResponseMessage	59
14.8.1	Method: get_json	59
14.8.2	Method: get_data	59
15	Appendix	60

<u>CONTENTS</u>	<u>CONTENTS</u>
16 History	61
E	

Chapter 1

Introduction

The **QConnectBase** package provides **Robot Framework** keywords for interacting with devices, servers, or services through various communication protocols such as TCP/IP, SSH, or Serial. It allows users to establish multiple connections, send commands, and easily verify responses.

Key features

- Support for multiple simultaneous connections.
- Built-in keywords for sending and receiving data.
- Protocol abstraction for TCP/IP, SSH, and Serial.
- Flexible connection management and session handling.
- Extensible architecture for custom protocols or behaviors.

Developers can extend the **QConnectBase** code to support additional connection types or custom behaviors, ensuring the library remains flexible and adaptable to diverse automation needs.

The sources of the **QConnectBase** are available in [GitHub](#).

Information about how to install **QConnectBase** can be found in the [README](#).

Chapter 2

Description

2.1 User interface

2.1.1 Keyword overview

QConnectBase contains the following keywords:

- `connect`
Establishes a connection to a specified host.
- `disconnect`
Terminates the connection to a host.
- `send_command`
Sends a command to the connected host.
- `verify`
Verifies the answer received from the connected host. Optionally sends a command (usually to trigger the answer).
- `transfer_item`
Sends an item to the connected host. Receives items from the connected host also.
- `transfer_file`
Deprecated: Use `transfer_item` instead!

2.1.2 Library import

The keywords can be accessed with the following library import:

```
Library    QConnectBase.ConnectionManager    AS    conn_manager
```

We recommend to shorten the *QConnectBase* library name by using the `AS` statement.

2.1.3 Usecases

QConnectBase supports various types of connections. This results in different `connect` parameters. The following examples refer to the connection type `TCPIPClient`.

The `connect` parameters for the connection type `TCPIPClient` are defined as follows:

```
*** Variables ***

${HOST}=    localhost
${PORT}=    ${1234}

&{TCPIPClientParams}    conn_type=TCPIPClient
...                    address=${HOST}
...                    port=${PORT}
...                    logfile=./tcp_ip_incoming.log
```

The logfile parameter is optional. It contains the path and the name of a file in which **QConnectBase** logs all incoming messages.

————— ★ —————

Connect to a host

Required are a name of the connection together with the connection parameters.

```
conn_manager.connect    conn_name=my_test_connection
...                    conn_conf=${TCPIPClientParams}
```

In case of errors, **QConnectBase** terminates the test execution. If you want to keep full control over the test flow, you'll need to use the Robot Framework BuiltIn keyword `run_keyword_and_ignore_error` (*the same for all other keywords also*).

```
${status}    ${result}=    run_keyword_and_ignore_error    conn_manager.connect    ↵
    ↪    conn_name=my_test_connection
...
    ↪    conn_conf=${TCPIPClientParams}    ...    ↵
```

`status` can be one of `PASS`, `FAIL`. `result` is either `None` or contains an error message.

A full description of the keyword interface can be found [here](#).

————— ★ —————

Terminate a connection

```
conn_manager.disconnect    conn_name=my_test_connection
```

The connection must be established before, using `connect`.

A full description of the keyword interface can be found [here](#).

————— ★ —————

Send a command

Send a command to the connected host.

```
conn_manager.send_command    conn_name=my_test_connection    <\textless{}the command to ↵
    ↪    send\textgreater{}>
```

The connection must be established before, using `connect`.

A full description of the keyword interface can be found [here](#).

————— ★ —————

Verify an answer

Verify an answer received from connected host. Optionally send a command (usually to trigger the answer).

`verify` distinguishes between the verification of a single incoming message and a block of several incoming messages. The verification occurs by checking whether an incoming message or a block of messages matches one or more than one pattern. `verify` patterns are regular expressions. Several different patterns are supported. Some allow matching groups, while others do not.

This results in the following possibilities:

1. Verify a single incoming message.
 - (a) The pattern does not contain matching groups:
In case of a match, `verify` returns `PASS` (but not the entire message), otherwise `FAIL`.
 - (b) The pattern contain one or more than one matching group:
In case of a match, `verify` returns a list of substrings captured from the incoming message, otherwise `None`.
2. Verify a block of several incoming messages.
In case of a block is verified, the pattern is interpreted as *multiline* pattern by the regex engine. This multiline pattern is applied to all messages of the block **in one single step**. This means: No iteration of a list of messages happens!
 - (a) The pattern does not contain matching groups:
In case of a match, `verify` returns `PASS` (but not the entire block of messages), otherwise `FAIL`.
 - (b) The pattern contain one or more than one matching group:
In case of a match, `verify` returns a list of substrings captured from the entire block of messages, otherwise `None`.

The `verify` keyword has the following prototype:

```
verify    conn_name    <\textless{} the name of the connection \textgreater{}>
          search_pattern <\textless{} the regular expression used to verify the message(s<
↪ ) \textgreater{}>
          timeout       <\textless{} timeout while waiting for messages within a match\↪
↪ _try \textgreater{}>
          match_try     <\textless{} number of attempts to match the search\_pattern \↪
↪ \textgreater{}>
          fetch_block   <\textless{} whether to wait for a single message or for a block↪
↪ \textgreater{}>
          eob_pattern   <\textless{} pattern to define the end of a block \textgreater{}>↪
↪ >
          filter_pattern <\textless{} pattern to make a preselection of block messages \↪
↪ \textgreater{}>
          send_cmd      <\textless{} command sent to the connected host \textgreater{}>
          kwargs        <\textless{} additional parameter \textgreater{}>
```

The boolean parameter `fetch_block` is the main switch to distinguish between waiting for a single message or for a block of messages. The default is `False` (= wait for a single message only).

The regex parameter `search_pattern`

- decides if the received message is the desired one or not (if `fetch_block` is `False`),
- decides if all received messages of a block are the desired ones or not (if `fetch_block` is `True`).

`verify` waits within `timeout` `match_try` times. Every single `match_try` causes `verify` to send `send_cmd` again (if defined).

Block detection

A block is a list of incoming messages and is used if a certain pattern (the `search_pattern`) is not expected to match a single message only but several messages instead. Therefore, the `search_pattern` is a multiline pattern. A block of messages has a begin and an end.

- The block detection starts with the execution of the `verify` keyword (= *moment in time*)
- The block detection ends when an incoming message matches a pattern named `eob_pattern` (= *event driven*).

Like the `search_pattern`, the `eob_pattern` is a regular expression - but must not contain matching groups. It's a simple yes/no decision. The `eob_pattern` does not extract any content.

All messages belonging to a block, are stored within an internal message queue intermediately. After the block is detected (match of `eob_pattern`), the list of messages within this queue is converted to a single string with hard coded line ending characters (`\r\n`).

```
"message1\r\nmessage2\r\nmessage3\r\nmessage4"
```

And exactly this is finally the string on which the `search_pattern` is applied.

Preselection of incoming messages

Up to now, we have assumed that during block detection, every incoming message is temporarily stored in the message queue. But that doesn't necessarily have to be the case. If many of the incoming messages are irrelevant for later content analysis, it is possible to place a filter in front that only allows messages with the desired content to pass through.

This filter is called `filter_pattern`. Optionally this filter can be used to make a preselection of messages. If the `filter_pattern` is not defined, all incoming messages will be accepted and stored in the message queue.

Like the `eob_pattern`, the `filter_pattern` is a regular expression - and must also not contain matching groups. It's a simple yes/no decision. The `filter_pattern` does not extract any content.

Consider: If you use backslashes within regular expressions defined in Robot Framework code, you have to mask the backslashes: `\\` !

A full description of the keyword interface can be found [here](#).

★

2.2 Verify examples

Because the `verify` keyword provides a lot of parameters, this section contains some examples that demonstrate the usage.

2.2.1 Verify a single message (1)

Verify a single message without matching groups.

Use case:

- Outgoing message: `TEST-MESSAGE`
- Received message: `TEST-MESSAGE ACK`
- `fetch_block` is `False` (*default*)
- No matching groups within the `search_pattern`
- `search_pattern` matches the received message

Code:

```
{result}=    conn_manager.verify    conn_name=TEST-CONNECTION
            ...                    search_pattern=TEST-MESSAGE\\sACK
            ...                    send_cmd=TEST-MESSAGE
```

Consider: In Robot Framework code, backslashes need to be masked: `\\`!

Result:

Because the `search_pattern` does not contain matching groups, the result is `None`.

```
INFO - Search pattern 'TEST-MESSAGE\\sACK' matched (try 1) on connection 'TEST-CONNECTION'
INFO - ${result} = None
```

★

2.2.2 Verify a single message (2)

Verify a single message with matching groups.

Use case:

- Outgoing message: `TEST-MESSAGE-1`
- Received message: `TEST-MESSAGE-1 ACK`
- `fetch_block` is `False` (*default*)
- `search_pattern` contains matching groups
- `search_pattern` matches the received message

Code:

```
conn_manager.verify    conn_name=TEST-CONNECTION
...                    search_pattern=TEST-MESSAGE-(\\d)\\s(\\w+)
...                    send_cmd=TEST-MESSAGE-1
```

Result:

Because `search_pattern` contains matching groups, the result is a list containing the captured substrings from incoming message.

```
INFO - Search pattern 'TEST-MESSAGE-(\d)\s(\w+)' matched (try 1) on connection '↔
↪ TEST-CONNECTION'
INFO - ${result} = ['1', 'ACK']
```

The result list can be iterated as follows:

```
FOR    ${element}    IN    @{result}
log    element: '${element}'
END
```

★

2.2.3 Verify a block of messages (1)

Verify a block of messages without filtering and without `search_pattern`.

Use case:

- Outgoing message: `FETCHBLOCK-1`
- Received list of messages:

```
transmission started
transmission point 1 passed
any noise message
transmission point 2 passed
any noise message
transmission point 3 passed
any noise message
transmission ended with code '0'
```

- `fetch_block` is `True`
- No `filter_pattern` defined
- No `search_pattern` defined

Code:

```
${result}=    conn_manager.verify    conn_name=TEST-CONNECTION
...           ...                    eob_pattern=transmission\\sended
...           ...                    fetch_block=${True}
...           ...                    send_cmd=FETCHBLOCK-1
...           ...                    timeout=10
```

In this example it requires eight seconds to receive all messages that belongs to the block. Therefore, it is required to define a `timeout` also (here: 10 seconds). The default value (1 second) would cause a verify timeout.

Result:

- Because the `filter_pattern` is not defined, all incoming messages are stored intermediately in the internal message queue.
- Because the `search_pattern` is not defined, the default value ('`.*`') becomes active. Therefore, we will have a match in every case.

```
INFO - Search pattern '.*' matched (try 1) on connection 'TEST-CONNECTION'
INFO - ${result} = None
```

★

2.2.4 Verify a block of messages (2)

Verify a block of messages without filtering - but with `search_pattern` without matching groups.

Use case:

- Outgoing message: `FETCHBLOCK-1`
- Received list of messages:

```
transmission started
transmission point 1 passed
any noise message
transmission point 2 passed
any noise message
transmission point 3 passed
any noise message
transmission ended with code '0'
```

- `fetch_block` is `True`
- No `filter_pattern` defined
- No matching groups within the `search_pattern`
- `search_pattern` matches the received block of messages

Code:

```
${result}=    conn_manager.verify    conn_name=TEST-CONNECTION
...           eob_pattern=transmission\\sended
...           search_pattern=transmission
...           fetch_block=${True}
...           send_cmd=FETCHBLOCK-1
...           timeout=10
```

Result:

- Because the `filter_pattern` is not defined, all incoming messages are stored intermediately in the internal message queue.
- Because the `search_pattern` does not contain matching groups, the result is `None`.

```
INFO - Search pattern 'transmission' matched (try 1) on connection 'TEST-CONNECTION'
INFO - ${result} = None
```

★

2.2.5 Verify a block of messages (3)

Verify a block of messages without filtering - but with `search_pattern` containing matching groups.

Use case:

- Outgoing message: `FETCHBLOCK-1`
- Received list of messages:

```
transmission started
transmission point 1 passed
any noise message
transmission point 2 passed
any noise message
transmission point 3 passed
any noise message
transmission ended with code '0'
```

- `fetch_block` is `True`
- No `filter_pattern` defined
- `search_pattern` contains matching groups
- `search_pattern` matches the received block of messages

Code:

```
$(result)=      conn_manager.verify      conn_name=TEST-CONNECTION
...              ...                      eob_pattern=transmission\\sended
...              ...                      search_pattern=^transmission\\sstarted\\r\\n(.*)\\r↵
↵ \\ntransmission\\sended with\\scode\\s'(\d+)'
...              ...                      fetch_block=${True}
...              ...                      send_cmd=FETCHBLOCK-1
...              ...                      timeout=10
```

In this example, the `search_pattern` is used to capture two parts of the content:

- the entire message content between `transmission started` and `transmission ended`,
- the exit code at end of the last message.

Result:

- Because the `filter_pattern` is not defined, all incoming messages are stored intermediately in the internal message queue.
- Because the `search_pattern` contains matching groups, the result is a list of all captured message parts.

```
INFO - Search pattern '^transmission\sstarted\r\n(.*)\r\ntransmission\sended\swith\scode↵
↵ s'(\d+)'' matched (try 1) on connection 'TEST-CONNECTION'
INFO - ${result} = ['transmission point 1 passed\r\nany noise message\r\ntransmission ↵
↵ point 2 passed\r\nany noise message\r\ntransmission point 3 passed\r\nany noise ↵
↵ message', '0']
```

Remember: The `search_pattern` is a multiline pattern! Therefore, the first element of the result list is a single string that is a concatenation of all affected messages from the message queue (with hard coded line endings).

★

2.2.6 Verify a block of messages (4)

Verify a block of messages with `search_pattern` containing matching groups and with `filter_pattern` defined.

Use case:

- Outgoing message: `FETCHBLOCK-1`
- Received list of messages:

```
transmission started
transmission point 1 passed
any noise message
transmission point 2 passed
any noise message
transmission point 3 passed
any noise message
transmission ended with code '0'
```

- `fetch_block` is `True`
- `filter_pattern` defined
- `search_pattern` contains matching groups
- `search_pattern` matches the received block of messages

Code:

```
$(result)=      conn_manager.verify      conn_name=TEST-CONNECTION
...              eob_pattern=transmission\\sended
...              filter_pattern=transmission
...              search_pattern=^transmission\\sstarted\\r\\n(.*)\\r↵
↵ \\ntransmission\\sended with\\scode\\s'(\d+)'
...              fetch_block=${True}
...              send_cmd=FETCHBLOCK-1
...              timeout=10
```

In this example, the `search_pattern` is used to capture two parts of the content:

- the entire message content between `transmission started` and `transmission ended`,
- the exit code at end of the last message.

But we only want to have the `transmission point` messages within our results. We are not interested in all `noise` messages. Therefore, we use the `filter_pattern` to collect the `transmission` messages only.

Result:

- Because the `filter_pattern` is defined, the result only contains messages that matches this filter.
- Because the `search_pattern` contains matching groups, the result is a list of all captured message parts.

```
INFO - Search pattern '^transmission\sstarted\r\n(.*)\r\ntransmission\sended\swith\scode↵
↵ s'(\d+)' matched (try 1) on connection 'TEST-CONNECTION'
INFO - ${result} = ['transmission point 1 passed\r\ntransmission point 2 passed\r↵
↵ ntransmission point 3 passed', '0']
```

Now the result only contains the `transmission point` messages together with the exit code.

Remember: The `search_pattern` is a multiline pattern! Therefore, the first element of the result list is a single string that is a concatenation of all affected messages from the message queue (with hard coded line endings).

2.3 === REWORKED ===

2.4 Connection types

TODO

2.5 Usage

QConnectBase Library support following keywords for testing connection in RobotFramework.

2.5.1 connect

Use for establishing a connection.

Syntax:

```
connect conn_name=[conn_name]    conn_conf=[conn_conf]
```

Notice

Only `conn_name` and `conn_conf` are now required.



Legacy syntax with 4 arguments is still supported for backward compatibility

```
connect  conn_name=[conn_name]  conn_type=[conn_type]  conn_conf=<↔>
↔ [conn_conf]    conn_mode=[conn_mode]
```

Arguments:

- **conn_name**: Name of the connection.
- **conn_conf**: A dictionary containing configurations for the connection. It must include **conn_type**, and optionally **conn_mode** and other connection-specific fields (depending on type). This replaces the need to pass **conn_type** and **conn_mode** as separate arguments. Each **conn_type** requires a specific structure in **conn_conf**.

Below are examples for each supported type:

- **TCPIPClient**: Create a Raw TCPIP connection to TCP Server.

```
{
  "conn_type": "TCPIPClient",
  "address": [server host], # Optional. Default value is "↵
↵ localhost".
  "port": [server port]      # Optional. Default value is 1234.
  "logfile": [Log file path. Possible values: 'nonlog', '↵
↵ console', [user define path] ]
}
```

- **SSHClient**: Create a client connection to a SSH server.

```
{
  "conn_type": "SSHClient",
  "address" : [server host], # Optional. Default value is "↵
↵ localhost".
  "port" : [server host],    # Optional. Default value is 22.
  "username" : [username],   # Optional. Default value is "↵
↵ root".
  "password" : [password],   # Optional. Default value is "".
  "authentication" : "password" | "keyfile" | "passwordkeyfile"↵
↵ , # Optional. Default value is "".
  "key_filename" : [filename or list of filenames], # Optional.↵
↵ Default value is null.
  "logfile": [Log file path. Possible values: 'nonlog', '↵
↵ console', [user define path] ]
}
```

- **SerialClient**: Create a client connection via Serial Port.

```
{
  "conn_type": "SerialClient",
  "port" : [comport or null],
  "baudrate" : [Baud rate such as 9600 or 115200 etc.],
  "bytesize" : [Number of data bits. Possible values: 5, 6, 7, ↵
↵ 8],
  "stopbits" : [Number of stop bits. Possible values: 1, 1.5, 2↵
↵ ],
  "parity" : [Enable parity checking. Possible values: 'N', 'E'↵
↵ , 'O', 'M', 'S'],
  "rtscts" : [Enable hardware (RTS/CTS) flow control.],
  "xonxoff" : [Enable software flow control.],
  "logfile": [Log file path. Possible values: 'nonlog', '↵
↵ console', [user define path] ]
}
```

2.5.2 disconnect

Use for disconnect a connection by name.

Syntax:

```
disconnect conn_name
```

Arguments:

conn_name: Name of the connection.

2.5.3 send command

Use for sending a command to the other side of connection.

Syntax:

```
send    command    conn_name=[conn_name]    command=[command]    [argument
name]=[argument value]
```

Arguments:

conn_name: Name of the connection.
command: Command to be sent.

2.5.4 transfer file

Use for transferring file from local to remote and vice versa. (Only available for SSHClient connection type)

Syntax:

```
transfer file conn_name=[conn_name]    src=[source]    dest=[destination]
type=[transfer type]
```

Arguments:

conn_name: Name of the connection.
src: Your source file's path.
dest: The destination path on the remote side.
type: 'put' to send from local to remote, 'get' to bring it back.

2.5.5 transfer item

Use for transferring item from local to remote and vice versa. (Only available for SSHClient connection type)

Syntax:

```
transfer item conn_name=[conn_name]    src=[source]    dest=[destination]
type=[transfer type]
```

Arguments:

conn_name: Name of the connection.
src: Your source item's path.
dest: The destination path on the remote side.
type: 'put' to send from local to remote, 'get' to bring it back.

2.5.6 verify

The keyword `verify` verifies a single message (or several messages) received from the connected server. Several pattern are available to filter the messages. Optionally, a message can be sent.

Prototype:

```
verify(conn_name:      str,
       search_pattern: str=".*",
       timeout:         int=1,
       match_try:       int=1,
       fetch_block:     bool=False,
       eob_pattern:     str=".*",
       filter_pattern:  str=".*",
       send_cmd:        str="",
       kwargs:          dict) -> dict
```

Arguments:

- **conn_name:**
Name of the connection.

- **send_cmd:**

Command to be sent to the other side of connection and waiting for response.

Default value: empty string `''`.

- **search_pattern:**

Expectation expressed as a **regular expression pattern** (more robust than a plain string comparison). It will match:

- a single line by default (`fetch_block` not used)
- multiple lines if `fetch_block` is enabled

Default value: `.*`, which means "match any character (`.`) repeated zero or more times (`*`)".

In practice, this will always match the response, regardless of its content, unless you specify a stricter pattern.

- **timeout:**

Timeout (in seconds) for each `match_try` (if used) while waiting for the response to match the pattern.

Default value: `5`.

- **match_try:**

Number of times to try matching the pattern.

Default value: `1`.

- **fetch_block:**

If `True`, every response line will be put into a block until a line match `eob_pattern` pattern.

Default value: `False`.

- **eob_pattern:**

Applicable only when `fetch_block` is `True`.

Regular expression for matching the endline when using `fetch_block`.

Default value: `.*`.

- **filter_pattern:**

Applicable only when `fetch_block` is `True`.

Regular expression for filtering every line to put into the block of response when using `fetch_block`.

Default value: `.*`.

Return value:

A list of captured string from search.

Example

```
${result} = verify conn_name=SSH_Connection
...               search_pattern=(?<\textless{}>=\s).*([0-9]).*(command).$
...               send_cmd=echo This is the 1st test command.
```

- `${result}[0]` will be `1st`, i.e. the first *captured string* defined in the pattern `([0-9].*)`.
- `${result}[1]` will be `command`, i.e. the second *captured string* defined in the pattern `(command)`.

The `${result}` can be iterated in `FOR` loop as below example:

```
FOR    ${item}    IN    &{result}
Log To Console    Captured string: ${item}
END
```

The following diagrams illustrate the internal flow of the `verify` keyword in `QConnectBase`, including handling of `fetch_block`, `filter_pattern`, `eob_pattern`, `timeout`, and `match_try`.

The sequence diagram shows how messages are read from the connection in a separate thread, and how the `verify` keyword processes them depending on the keyword arguments.

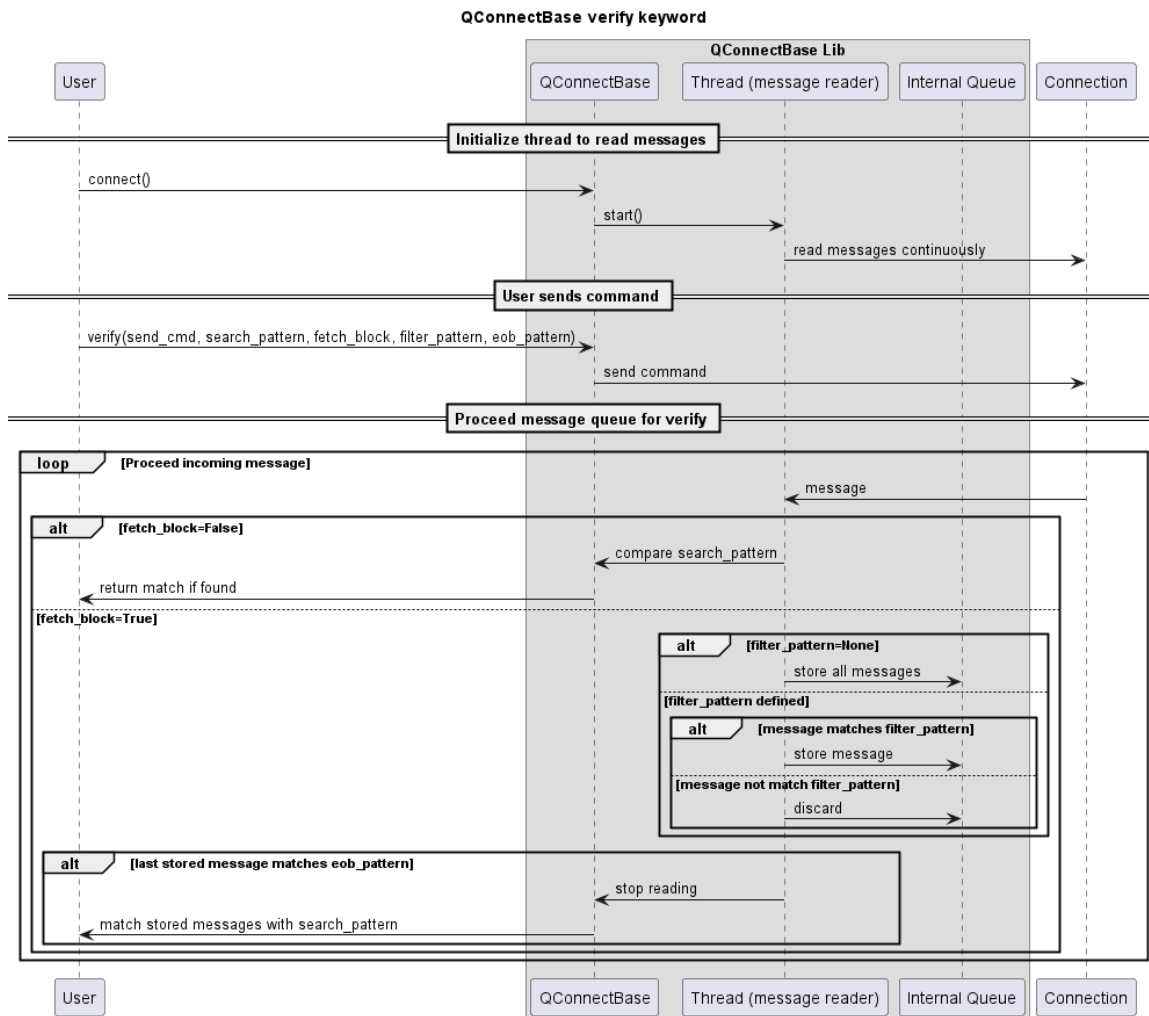


Figure 2.1: Sequence to process messages

The activity diagram illustrates the decision flow of the `verify` keyword. It shows how messages are read, optionally filtered, collected into a block, and checked against `search_pattern`. Each `match_try` has its own timeout, and the keyword stops immediately if a match is found.

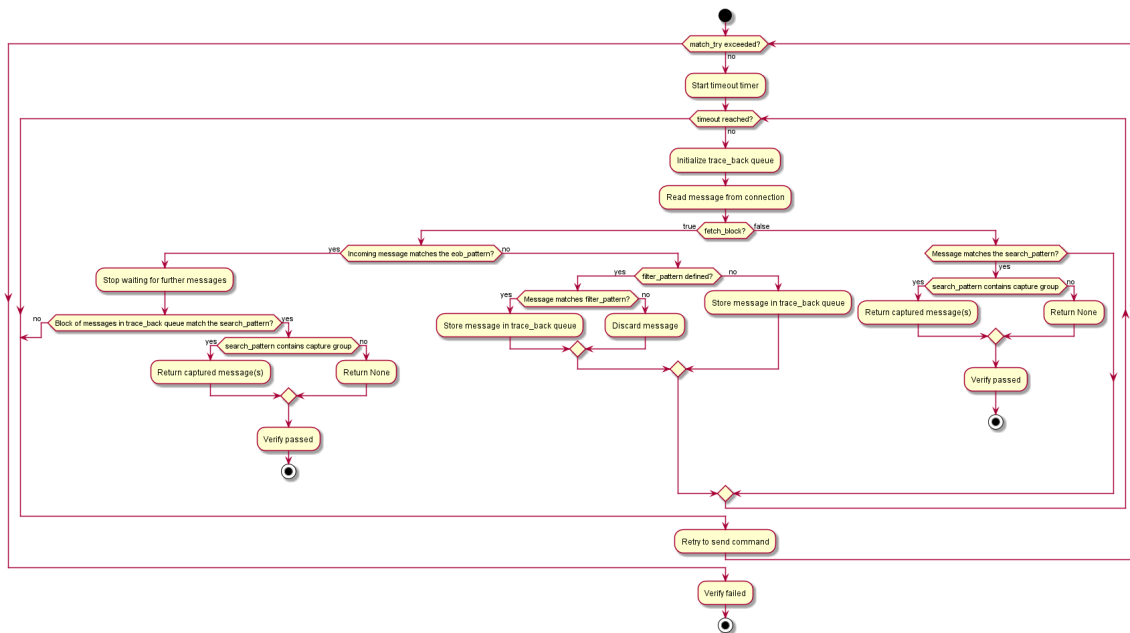


Figure 2.2: Verify workflow

2.6 Example

```

*** Settings ***
Documentation      Suite description
Library           QConnectBase.ConnectionManager

*** Test Cases ***
Test SSH Connection
    # Create config for connection.
    ${config_string}=    concatenate
    ...    {
    ...    "address": "127.0.0.1",
    ...    "port": 8022,
    ...    "username": "root",
    ...    "password": "",
    ...    "authentication": "password",
    ...    "key_filename": null
    ...    }
    log to console          \nConnecting with configurations:\n${config_string}
    ${config}=              evaluate          json.loads('"'${config_string}'"')    json

    # Connect to the target with above configurations.
    connect                 conn_name=test_ssh
    ...                     conn_type=SSHClient
    ...                     conn_conf=${config}

    # Send command 'cd ..' and 'ls' then wait for the response '.' pattern.
    send command            conn_name=test_ssh
    ...                     command=cd ..

    ${res}=                Verify          conn_name=${CONNECTION_NAME}
    ...                     send_cmd=echo ~~START~~;ls -la;echo ~~END$?~~\r\n
    ...                     search_pattern=\~\~START\~\~\r\n([\s\S]*)\~\~END(\\d+)\~\~↵
    ↵ \~
    ...                     fetch_block=True
    ...                     eob_pattern=^\~\~END(\\d*)\~\~

    Log To Console          ${res}[1]

    # Copy a local file to the SFTP server
    transfer file           conn_name=test_ssh
    ...                     src=${src_put}
    ...                     dest=${dest_put}
    ...                     type=put

    # Copy a remote item from the SFTP server to the local host
    transfer item           conn_name=test_ssh
    ...                     src=${src_get}
    ...                     dest=${dest_get}
    ...                     type=get

    # Disconnect
    disconnect              test_ssh

```

Listing 2.1: Robot code example

Explanation:

In the example above, we will establish an SSH connection to the remote host 127.0.0.1:8002 and then navigate back one directory level to list all files/folders within that directory.

To achieve this, we utilize the following steps:

1. First, we use the `send command` keyword, which is designed to send commands to the server and immediately return without waiting for a response. In this case, we send the command "cd .." to the remote host.
2. Next, we employ the `verify` keyword, specifically designed for verifying the response from the server after sending a command. The command sent is specified by the `send_cmd` argument (here, it is the "ls -la" command).

CHAPTER 2. DESCRIPTION

2.6. EXAMPLE

We use a regular expression defined by the `search_pattern` argument to verify the response and extract the list of files/folders after executing the `"ls -la"` command.

However, by default, if no additional arguments are provided, only the first line of the response after running the command will be captured. To address this limitation, we introduce two additional arguments: `fetch_block` and `eob_pattern`.

When `fetch_block` is set to `"True"`, the `Verify` keyword will capture all lines returned from the server until it encounters a line that matches the pattern defined by `eob_pattern` (End of Block pattern). This extended functionality ensures that we can effectively capture multi-line returns from the server.

To handle variable-length output, the `send_cmd` parameter encapsulates the command between `"START"` and `"END$?"`. By setting `fetch_block` to `"True"` and specifying the `eob_pattern`, we can accurately capture the entire list returned by the server between `"START"` and `"END$?"`.

Example: Verifying service startup logs with `fetch_block`

Use case: This use case demonstrates how to use `verify` with `fetch_block` and `eob_pattern` to monitor the startup of a Linux systemd service. When restarting a service, the log usually contains multiple lines: some indicate that the service is starting, some show internal information, and finally, one line confirms that the service has been started successfully. By using `fetch_block`, all these lines can be collected into a block until the end-of-block pattern (`eob_pattern`) is reached.

Command under test:

```
systemctl restart cron; journalctl -u cron --since "now"
```

Sample log output:

```
Sep 12 15:25:01 myhost systemd[1]: Stopping Regular background program processing daemon↵
↵ ...
Sep 12 15:25:01 myhost systemd[1]: Starting Regular background program processing daemon↵
↵ ...
Sep 12 15:25:01 myhost cron[5678]: (CRON) INFO (pidfile fd = 3)
Sep 12 15:25:01 myhost cron[5678]: (CRON) INFO (Running @reboot jobs)
Sep 12 15:25:01 myhost systemd[1]: Started Regular background program processing daemon.
```

Robot Framework usage:

```
${result} = verify    conn_name=SSH_Connection
...                  fetch_block=True
...                  search_pattern=( [^\r\n]*Starting Regular background program ↵
↵ processing daemon.*)
...                  eob_pattern=.*Started Regular background program processing daemon↵
↵ .*
...                  send_cmd=systemctl restart cron; journalctl -u cron --since "now"
```

Returned block:

```
Sep 12 15:25:01 myhost systemd[1]: Starting Regular background program processing daemon↵
↵ ...
Sep 12 15:25:01 myhost cron[5678]: (CRON) INFO (pidfile fd = 3)
Sep 12 15:25:01 myhost cron[5678]: (CRON) INFO (Running @reboot jobs)
Sep 12 15:25:01 myhost systemd[1]: Started Regular background program processing daemon.
```

Notes:

- `search_pattern` defines the entry point (*Starting...*).
- `eob_pattern` defines the stopping point (*Started...*).
- If the service fails to start (no *Started...* line appears), the keyword will eventually timeout, which makes this a convenient way to verify service startup behavior in automated tests.

Example: Filtering service startup logs with `fetch_block` and `filter_pattern`

Use case: When a Linux device boots, it sends a large block of startup logs through the connection (e.g., COM port). We only care about one particular service ("systemd-networkd"). By using `fetch_block` together with

`filter_pattern`, we can extract only the relevant messages. Finally, `eob_pattern` is used to stop reading logs as soon as the service reports a successful startup.

Important Note:



`filter_pattern` filters the incoming lines, keeping only those relevant to the service of interest. `eob_pattern` is applied **after filtering**, meaning it must match one of the filtered lines. If it does not, the keyword will not stop until timeout occurs. This ensures the block stops at the correct line related to the service and avoids including unrelated logs.

```
#{result} = verify conn_name=COM_Connection
...               fetch_block=True
...               search_pattern=(.*)
...               filter_pattern=.*systemd-networkd.*
...               eob_pattern=.*eth0: Gained carrier.*
...               send_cmd=dmesg --follow
```

Connection log (sample):

```
[ 1.234567] systemd[1]: Starting udev Kernel Device Manager...
[ 1.456789] systemd[1]: Started udev Kernel Device Manager.
[ 2.123456] systemd[1]: Starting Network Service...
[ 2.223344] systemd-networkd[120]: eth0: link is up
[ 2.223355] systemd-networkd[120]: eth0: Gained carrier
[ 2.345678] systemd[1]: Started Network Service.
[ 2.456789] systemd[1]: Starting Login Service...
[ 2.567890] systemd[1]: Started Login Service.
```

Returned block after applying `filter_pattern` and stopping at `eob_pattern`:

```
[ 2.223344] systemd-networkd[120]: eth0: link is up
[ 2.223355] systemd-networkd[120]: eth0: Gained carrier
```

Explanation:

- `fetch_block=True` collects all filtered lines until `eob_pattern` is matched.
- `filter_pattern=.*systemd-networkd.*` ensures only logs from the relevant service are included.
- `eob_pattern=.*eth0: Gained carrier.*` stops collection when the filtered line matches the pattern.
- Lines from other services (e.g., udev, Login Service) are automatically excluded from the block.

2.7 Auxiliary Utilities

2.7.1 Introduction to RMQSignal

The *RMQSignal* class serves as an extension to the core functionalities, providing support for Inter-Process Communication (IPC) based on the RabbitMQ infrastructure. This utility module is designed to facilitate the exchange of signals accompanied by payloads between various processes, enhancing the overall communication mechanism.

2.7.2 Prerequisites

For Windows:

To use this library, installing RabbitMQ is required. For RabbitMQ installation on Windows, Earlang needs to be installed beforehand.

For user convenience, we have created a .bat file to install all necessary components. Users simply need to run the file `QConnectBase/tools/setup_rabbitmq.bat` to install the entire infrastructure required for these Auxiliary Utilities.

For Ubuntu:

1. Update the system. First, ensure your system is up to date by running the command:


```
sudo apt update && sudo apt upgrade -y
```

2. Add the required repositories. Add the official RabbitMQ signing key and repository by running the following commands:

```
sudo apt install curl gnupg -y
curl -fsSL https://packages.rabbitmq.com/gpg | sudo apt-key add -
sudo add-apt-repository 'deb https://dl.bintray.com/rabbitmq/debian focal main'
```

3. Install RabbitMQ Server. Now, proceed with installing RabbitMQ:

```
sudo apt update && sudo apt install rabbitmq-server -y
```

4. Enable and start the RabbitMQ Service. After installation, enable and start the RabbitMQ service:

```
sudo systemctl enable rabbitmq-server
sudo systemctl start rabbitmq-server
```

2.7.3 Library Keywords and Usage

The keywords provided by this library offer a range of functionalities designed to streamline various tasks and processes. Below are some of the key functionalities along with their usage:

Send Signal Keyword

Usage: Used to send a signal to other processes.

Syntax: Send Signal [signal_name] [payloads] [Receiver]

Arguments:

- `signal_name` (string): Defines the name of the signal.
- `payloads`: Specifies the payloads of the signal.
- `Receiver` (optional, default is None): Specifies the receiver. If defined, the signal will be sent only to that specified receiver. If not defined, it sends a broadcast to all receivers.

Wait For Signal Keyword

Usage: Used to wait for a specific signal within a timeout.

Syntax: Wait For Signal [signal_name] [timeout]

Arguments:

- `signal_name` (string): Defines the name of the signal to wait for.
- `timeout` (optional, default is 10): Specifies the timeout in seconds to wait for the specific signal.

Return: Returns the payloads of the received signal, if received within the timeout.

Wait For Signals Keyword

Usage: Used to wait for multiple specific signals within a timeout.

Syntax: Wait For Signals [signal_names] [timeout]

Arguments:

- `signal_names` (list of strings): Defines the list of signal names to wait for.
- `timeout` (optional, default is 10): Specifies the timeout in seconds to wait for the full set of specific signals.

Return: Returns a list of payloads of the signals in the same order if the full set of signals is received within the timeout.

Set Signal Receiver Name Keyword

Usage: Used to set a specific name for the Signal Receiver, which defines a unique name for the current process. Other processes can send signals directly to the current process using this receiver name.

Syntax: Set Signal Receiver Name [receiver] [force]

Arguments:

- `receiver` (string): Specifies the name to set for the current process signal receiver.
- `force` (optional, default is True): Determines whether to force-create the signal receiver. If `force` is set to False, an exception will be raised if any process has already defined this name as a signal receiver.

Unset Signal Receiver Name Keyword

Usage: Used to unset the current Signal Receiver name of the current process.

Syntax: Unset Signal Receiver Name

2.7.4 Example of Inter-Process Communication

Content of ProcessA.robot file:

```
*** Settings ***
Library           QConnectBase.message.RMQSignal

*** Test Cases ***
Test signal
    Log To Console    Process A start
    Send Signal       HelloFromProcA    Hello, I'm A            receiverB
    Sleep             1s
    Send Signal       Hello2FromProcA    Hello 2nd time, I'm A    receiverB
    ${res}=           Wait For Signal    HelloFromProcB          ${20}
    Log To Console    Get resp signal from ProcB: ${res}
```

Content of ProcessB.robot file:

```
*** Settings ***
Library           QConnectBase.message.RMQSignal

*** Test Cases ***
Receiving Multiple Signals
    Log To Console    message
    Set Signal Receiver Name    receiverB
    ${list_of_signal} =    Create List    HelloFromProcA    Hello2FromProcAfull
    ${res}=               Wait For Signals    ${list_of_signal}    ${15}
    Sleep                 2s
    Log To Console        Get signal from ProcA: ${res}
    Send Signal           HelloFromProcB    Hello A, I'm B. Got your signal:${res}
```

Explanation: In the above test cases, two separate Robot Framework test files, ProcessA.robot, and ProcessB.robot are used for inter-process communication.

In ProcessB.robot, the test case 'Receiving Multiple Signals' starts by setting up a signal receiver 'receiverB'. It then waits for a set of signals (HelloFromProcA and Hello2FromProcA) from ProcessA. Once both signals are received within 15 seconds, it logs to console the received signals' payloads and waits for 2 seconds before sending a response signal 'HelloFromProcB' to ProcessA.

In ProcessA.robot, the test case 'Sending Multiple Signals' sends HelloFromProcA signal to 'receiverB' in ProcessB. After a 1-second pause, it sends another signal 'Hello2FromProcA' to 'receiverB'. Then, it waits for a response, expecting signal 'HelloFromProcB' from ProcessB within 20 seconds. Upon receiving signal, it logs the received payloads to console.

2.8 Contribution Guidelines

QConnectBaseLibrary is designed for ease of making an extension library. By that way you can take advantage of the QConnectBaseLibrary's infrastructure for handling your own connection protocol. For creating an extension library for QConnectBaseLibrary, please following below steps.

1. Create a library package which have the prefix name is **robotframework-qconnect-*[your specific name]***.
2. Your hadling connection class should be derived from **QConnectBase.connection_base.ConnectionBase** class.
3. In your *Connection Class*, override below attributes and methods:
 - **_CONNECTION_TYPE**: name of your connection type. It will be the input of the `conn_type` argument when using **connect** keyword. Depend on the type name, the library will detemine the correct connection handling class.
 - **__init__(self, _mode, config)**: in this constructor method, you should:
 - Prepare resource for your connection.
 - Initialize receiver thread by calling `self._init_thread_receiver(cls._socket_instance, mode="")` method.
 - Configure and initialize the lowlevel receiver thread (if it's necessary) as below

```

self._llrecv_thrd_obj = None
self._llrecv_thrd_term = threading.Event()
self._init_thrd_llrecv(cls._socket_instance)

```

- Incase you use the lowlevel receiver thread for receiving data byte by byte. You should implement the **thrd_llrecv_from_connection_interface()** method. This method is a mediate layer which will receive the data from connection at the very beginning, do some process then put them in a queue for the **receiver thread** above getting later.
- Create the queue for this connection (use Queue.Queue).
- **connect()**: implement the way you use to make your own connection protocol.
- **_read()**: implement the way to receive data from connection.
- **_write()**: implement the way to send data via connection.
- **disconnect()**: implement the way you use to disconnect your own connection protocol.
- **quit()**: implement the way you use to quit connection and clean resource.

2.9 Configure Git and correct EOL handling

Here you can find the references for [Dealing with line endings](#).

Every time you press return on your keyboard you're actually inserting an invisible character called a line ending. Historically, different operating systems have handled line endings differently. When you view changes in a file, Git handles line endings in its own way. Since you're collaborating on projects with Git and GitHub, Git might produce unexpected results if, for example, you're working on a Windows machine, and your collaborator has made a change in OS X.

To avoid problems in your diffs, you can configure Git to properly handle line endings. If you are storing the .gitattributes file directly inside of your repository, than you can assure that all EOL are manged by git correctly as defined.

2.10 Sourcecode Documentation

For investigating sourcecode, please refer to [QConnectBase library documentation](#)

2.11 Feedback

If you have any problem when using the library or think there is a better solution for any part of the library, I'd love to know it, as this will all help me to improve the library. Please don't hesitate to contact me.

Do share your valuable opinion, I appreciate your honest feedback!

2.12 About

2.12.1 Maintainers

[Nguyen Huynh Tri Cuong](#)

2.12.2 Contributors

[Nguyen Huynh Tri Cuong](#)

[Thomas Pollerspöck](#)

2.13 License

Copyright 2020-2023 Robert Bosch GmbH

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

*CHAPTER 2. DESCRIPTION**2.13. LICENSE*

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

CHAPTER 3. `__INIT__.PY`

Chapter 3

`__init__.py`

Class to manage all connections.

Chapter 4

connection_base.py

4.1 Class: BrokenConnError

4.2 Class: EndOfBlockNotFound

4.3 Class: NoFilteredMsgFound

4.4 Class: ConnectionBase

Base class for all connection classes.

4.4.1 Method: is_supported_platform

Check if current platform is supported.

Returns:

/ Type: bool /
True if platform is supported.
False if platform is not supported.

4.4.2 Method: is_precondition_pass

Check for precondition.

Returns:

/ Type: bool /
True if passing the precondition.
False if failing the precondition.

4.4.3 Method: get_connection_type

Get the connection type.

Returns:

/ Type: str /
The connection type.

4.4.4 Method: error_instruction

Get the error instruction.

Returns:

/ Type: str /
Error instruction string.

4.4.5 Method: quit

» This method MUST be overridden in derived class «

Abstract method for quitting the connection.

Arguments:

- `is_disconnect_all`
/ Condition: optional / Type: bool /
Determine if it's necessary to disconnect all connections.

Returns:

(no returns)

4.4.6 Method: connect

» This method MUST be overridden in derived class «

Abstract method for quitting the connection.

Arguments:

- `device`
/ Condition: required / Type: str /
Device name.
- `files`
/ Condition: optional / Type: list /
Trace file list if using dlt connection.
- `test_connection`
/ Condition: optional / Type: bool /
Determine if it's necessary for testing the connection.

Returns:

(no returns)

4.4.7 Method: disconnect

» This method MUST be overridden in derived class «

Abstract method for disconnecting connection.

Arguments:

- `device`
/ Condition: required / Type: str /
Device's name.

Returns:

(no returns)

4.4.8 Method: send_obj

Wrapper method to send message to a tcp connection.

Arguments:

- `send_cmd`
/ *Condition*: required / *Type*: str /
Data to be sent.
- `cr`
/ *Condition*: optional / *Type*: str /
Determine if it's necessary to add newline character at the end of command.

Returns:

(no returns)

4.4.9 Method: read_obj

Wrapper method to get the response from connection.

Returns:

- `msg`
/ *Type*: str /
Responded message.

4.4.10 Method: wait_4_trace

Suspend the control flow until a Trace message is received which matches to a specified regular expression.

Arguments:

- `search_obj`
/ *Condition*: required / *Type*: str /
Regular expression all received trace messages are compare to. Can be passed either as a string or a regular expression object. Refer to Python documentation for module 're'.
- `use_fetch_block`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Determine if 'fetch block' feature is used.
- `end_of_block_pattern`
/ *Condition*: optional / *Type*: str / *Default*: None /
The end of block pattern.
- `filter_pattern`
/ *Condition*: optional / *Type*: str / *Default*: '.*' /
Pattern to filter message line by line.
- `timeout`
/ *Condition*: optional / *Type*: float / *Default*: 0 /
Timeout parameter specified as a floating point number in the unit 'seconds'.
- `fct_args`
/ *Condition*: optional / *Type*: Tuple / *Default*: None /
List of function arguments passed to be sent.

Returns:

- `match`

/ *Type*: `re.Match` /

If no trace message matched to the specified regular expression and a timeout occurred, return `None`.

If a trace message has matched to the specified regular expression, a match object is returned as the result. The complete trace message can be accessed by the `'string'` attribute of the match object. For access to groups within the regular expression, use the `group()` method. For more information, refer to Python documentation for module `'re'`.

4.4.11 Method: `wait_4_trace_continuously`

Getting trace log continuously without creating a new trace queue.

Arguments:

- `trace_queue`

/ *Condition*: required / *Type*: `Queue` /

Queue to store the traces.

- `timeout`

/ *Condition*: optional / *Type*: `int` / *Default*: 0 /

Timeout for waiting a matched log.

- `fct_args`

/ *Condition*: optional / *Type*: `Tuple` / *Default*: `None` /

Arguments to be sent to connection.

Returns:

- `None`

/ *Type*: `None` /

If no trace message matched to the specified regular expression and a timeout occurred.

- `match`

/ *Type*: `re.Match` /

If a trace message has matched to the specified regular expression, a match object is returned as the result. The complete trace message can be accessed by the `'string'` attribute of the match object. For access to groups within the regular expression, use the `group()` method. For more information, refer to Python documentation for module `'re'`.

4.4.12 Method: `create_and_activate_trace_queue`

Create Queue and assign it to `_trace_queue` object and activate the queue with the search element.

Arguments:

- `search_element`

/ *Condition*: required / *Type*: `str` /

Regular expression all received trace messages are compare to.

Can be passed either as a string or a regular expression object. Refer to Python documentation for module `'re'`.

- `use_fetch_block`

/ *Condition*: optional / *Type*: `bool` / *Default*: `False` /

Determine if 'fetch block' feature is used.

- `end_of_block_pattern`

/ *Condition*: optional / *Type*: `str` / *Default*: `None` /

The end of block pattern.

- `regex_line_filter_pattern`
/ *Condition*: optional / *Type*: `re.Pattern` / *Default*: `None` /
Regular expression object to filter message line by line.

Returns:

- `trq_handle, trace_queue`
/ *Type*: tuple /
The handle and search object

4.4.13 Method: deactivate_and_delete_trace_queue

Deactivate trace queue and delete.

Arguments:

- `trq_handle`
/ *Condition*: required / *Type*: `int` /
Trace queue handle.
- `trace_queue`
/ *Condition*: required / *Type*: `Queue` /
Trace queue object.

Returns:

(no returns)

4.4.14 Method: activate_trace_queue

Activates a trace message filter specified as a regular expression. All matching trace messages are put in the specified queue object.

Arguments:

- `search_obj`
/ *Condition*: required / *Type*: `str` /
Regular expression all received trace messages are compare to. Can be passed either as a string or a regular expression object. Refer to Python documentation for module 're'.
- `trace_queue`
/ *Condition*: required / *Type*: `Queue` /
A queue object all trace message which matches the regular expression are put in. The using application must assure, that the queue is emptied or deleted.
- `use_fetch_block`
/ *Condition*: optional / *Type*: `bool` / *Default*: `False` /
Determine if 'fetch block' feature is used.
- `end_of_block_pattern`
/ *Condition*: optional / *Type*: `str` / *Default*: `None` /
The end of block pattern.
- `line_filter_pattern`
/ *Condition*: optional / *Type*: `re.Pattern` / *Default*: `None` /
Regular expression object to filter message line by line.

Returns:

- `handle_id`
/ *Type*: `int` /
Handle to deactivate the message filter.

4.4.15 Method: deactivate_trace_queue

Deactivates a trace message filter previously activated by `ActivateTraceQ()` method.

Arguments:

- `handle`
/ *Condition*: required / *Type*: int /
Integer object returned by `ActivateTraceQ()` method.

Returns:

- `is_success`
/ *Type*: bool /

System Message

WARNING/2 in <string>, line 391

Bullet list ends without a blank line; unexpected unindent. backrefs:

- . `False` : No trace message filter active with the specified handle (i.e. handle is not in use).
- . `True` : Trace message filter successfully deleted.

4.4.16 Method: check_timeout

» This method will be override in derived class «

Check if responded message come in `cls._RESPOND_TIMEOUT` or we will raise a timeout event.

Arguments:

- `timeout`
/ *Condition*: required / *Type*: int /
Timeout in seconds.

Returns:

(no returns)

4.4.17 Method: pre_msg_check

» This method will be override in derived class «

Pre-checking message when receiving it from connection.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Received message to be checked.

Returns:

(no returns)

4.4.18 Method: post_msg_check

» This method should be overridden in the derived class «

Post-checking message when receiving it from connection.

Arguments:

- msg
/ *Condition*: required / *Type*: str /
Received message to be checked.

Returns:
(no returns)

Chapter 5

connection_manager.py

5.1 Class: InputParam

5.2 Class: ConnectParam

Class for storing parameters for connect action.

5.3 Class: SendCommandParam

Class for storing parameters for send command action.

5.4 Class: VerifyParam

Class for storing parameters for verify action.

5.5 Class: ConnectionManager

Class to manage all connections.

5.5.1 Method: import_modules_from_paths

Import all modules from given paths.

Arguments:

- paths
/ *Condition*: required / *Type*: list /
List of paths to import modules from.

5.5.2 Method: quit

Quit connection manager.

Returns:

(no returns)

5.5.3 Method: add_connection

Add a connection to managed dictionary.

Arguments:

- `name`
/ *Condition*: required / *Type*: str /
Connection's name.
- `conn`
/ *Condition*: required / *Type*: socket.socket /
Connection object.

Returns:*(no returns)***5.5.4 Method: remove_connection**

Remove a connection by name.

Arguments:

- `conn_name`
/ *Condition*: required / *Type*: str /
Connection's name.

Returns:*(no returns)***5.5.5 Method: get_connection_by_name**

Get an exist connection by name.

Arguments:

- `conn_name`
/ *Condition*: required / *Type*: str /
Connection's name.
- `raise_exception`
/ *Condition*: optional / *Type*: bool /
If True, raise exception when connection is not found.

Returns:

- `conn`
/ *Type*: socket.socket /
Connection object.

5.5.6 Keyword: disconnect

Keyword for disconnecting a connection by name.

Arguments:

- `conn_name`
/ *Condition*: required / *Type*: str /
Connection's name.

Returns:*(no returns)*

5.5.7 Keyword: connect

Making a connection.

Arguments:

- `conn_name`
/ Condition: required */ Type:* str */*
 Name of connection. It can be specified in `conn_conf` dictionary.
- `conn_conf`
/ Condition: optional */ Type:* dictionary */ Default:* None */*
 A dictionary containing configurations for the connection.
 It must include `conn_type`, and optionally `conn_mode` and other connection-specific fields (depending on type).
 Example `conn_conf` for `TCPIPClient`:

```
{
    "conn_type": "TCPIPClient",
    "address": [server host], # Optional. Default value is "localhost".
    "port": [server port] # Optional. Default value is 1234.
}
```
- `conn_type` (deprecated)
/ Condition: optional */ Type:* str */ Default:* 'TCPIPClient' */*
 Type of connection. It can be specified in `conn_conf` dictionary.
 Supported connection types:
 - `TCPIPClient`: Create a Raw TCP/IP connection to TCP Server.
 - `SSHClient`: Create a client connection to a SSH server.
 - `SerialClient`: Create a client connection via Serial Port.
 In addition to the connection types listed above, other types are also available through classes inheriting from `QConnectBase`.
- `conn_mode` (deprecated)
/ Condition: optional */ Type:* str */ Default:* " */*
 Connection mode. It can be specified in `conn_conf` dictionary.

Returns:

(no returns)

5.5.8 Keyword: send_command

Send command to a connection.

Arguments:

- `conn_name`
/ Condition: required */ Type:* str */*
 Name of connection.
- `command`
/ Condition: required */ Type:* str */*
 Command to be sent.

- `kwargs`

/ *Condition*: optional / *Type*: dict / *Default*: {} /

The optional arguments depend on the connection type used in the 'connect' keyword.

Returns:

(no returns)

5.5.9 Keyword: transfer_file

DEPRECATED!! Use keyword transfer_item instead. Transfer file from local to remote and vice versa.

Arguments:

- `conn_name`

/ *Condition*: required / *Type*: str /

Name of connection.

- `src`

/ *Condition*: required / *Type*: str /

Source file path.

- `dest`

/ *Condition*: required / *Type*: str /

Destination file path.

- `type`

/ *Condition*: required / *Type*: str /

Transfer file type.

'get' - Copy a remote file from the SFTP server to the local host.

'put' - Copy a local file to the SFTP server.

Returns:

(no returns)

5.5.10 Keyword: transfer_item

Transfer item from local to remote and vice versa.

Arguments:

- `conn_name`

/ *Condition*: required / *Type*: str /

Name of connection.

- `src`

/ *Condition*: required / *Type*: str /

Source item path.

- `dest`

/ *Condition*: required / *Type*: str /

Destination item path.

- `type`

/ *Condition*: required / *Type*: str /

Transfer item type.

'get' - Copy a remote item from the SFTP server to the local host.

'put' - Copy a local item to the SFTP server.

Returns:

(no returns)

5.5.11 Keyword: `execute_script`

Executes a script file by sending commands to a device through the provided connection.

Arguments:

- `conn_name`
/ *Condition*: required / *Type*: str /
Name of connection.
- `script_path`
/ *Condition*: required / *Type*: str /
Script file path.

Returns:

(no returns)

5.5.12 Keyword: `set_default_verify_timeout`

Set the default verify timeout value for the connection.

Supports flexible input formats such as:

- Duration with units (e.g. '1h 10s', '2m30s', '500ms')
- HH:MM:SS format (e.g. '01:00:10' for 1 hour, 0 minutes, 10 seconds)
- Plain numeric values (e.g. '42') interpreted as seconds

Arguments:

- `time_str`
/ *Condition*: required / *Type*: str or float or int /

A string representing the duration. Units supported include:

- h for hours
- m for minutes (or ms for milliseconds)
- s for seconds
- ms for milliseconds

If no unit is specified, the value is interpreted as seconds.

5.5.13 Keyword: `set_default_emergency_timeout`

Set the default emergency timeout value for the connection.

Supports flexible input formats such as: - Duration with units (e.g. '1h 10s', '2m30s', '500ms') - HH:MM:SS format (e.g. '01:00:10' for 1 hour, 0 minutes, 10 seconds) - Plain numeric values (e.g. '42') interpreted as seconds

Arguments:

- `time_out`
/ *Condition*: required / *Type*: str or float or int /

A string representing the duration. Units supported include:

- h for hours
- m for minutes (or ms for milliseconds)
- s for seconds
- ms for milliseconds

If no unit is specified, the value is interpreted as seconds.

5.5.14 Keyword: verify

Verify a pattern from connection response after sending a command.

Arguments:

- `conn_name`
/ Condition: required */ Type:* str */*
 Name of connection.
- `search_pattern`
/ Condition: optional */ Type:* str */ Default:* .* */*
 Expectation expressed as a **regular expression pattern** (more robust than a plain string comparison).
 It will match:
 - a single line by default (`fetch_block` not used)
 - multiple lines if `fetch_block` is enabled
 Default value: .*, which means "match any character (.) repeated zero or more times (*)".
 In practice, this will always match the response, regardless of its content, unless you specify a stricter pattern.
- `timeout`
/ Condition: optional */ Type:* float */ Default:* 5 */*
 Timeout parameter specified as a floating point number in the unit 'seconds'.
- `match_try`
/ Condition: optional */ Type:* int */ Default:* 1 */*
 Number of time for trying to match the pattern.
- `fetch_block`
/ Condition: optional */ Type:* bool */ Default:* False */*
 Determine if 'fetch block' feature is used.
 If True, every single line of received message will be put into a block until a line match `eob_pattern` pattern.
- `eob_pattern`
/ Condition: optional */ Type:* str */ Default:* None */*
 Applicable only when `fetch_block` is True.
 Regular expression for matching the endline when using `fetch_block`.
- `filter_pattern`
/ Condition: optional */ Type:* str */ Default:* '.*' */*
 Applicable only when `fetch_block` is True.
 Regular expression for filtering every line to put into the block of response when using `fetch_block`.
- `send_cmd`
/ Condition: optional */ Type:* str */ Default:* " */*
 Command to be sent.
- `kwargs`
/ Condition: optional */ Type:* Dict */ Default:* None */*
 The optional arguments depend on the connection type used in the 'connect' keyword.
 Supported options:

Connection Type	Argument	Explanation
Winapp	element_def	Definition for detecting GUI item: <i>Type</i> : str / <i>Default</i> : "

Returns:

- res
/ *Type*: list /
List of captured string from search_pattern
For example, if search_pattern is (?<=\s).*([0-9].*)..*(command).\$, and the response from connection is This is the 1st test command., then the returned list will be ['1st', 'command'].
Thus:
 - `{res}[0]` will be **1st**, i.e. the first *captured string* defined in the pattern `([0-9].*)`.
 - `{res}[1]` will be **command**, i.e. the second *captured string* defined in the pattern `(command)`.

5.6 Class: TestOption

Chapter 6

constants.py

6.1 Class: `SocketType`

6.2 Class: `String`

Chapter 7

grpc_client.py

7.1 Class: GrpcClientConfig

Class to store the configuration for gRPC client connection.

Either `target` (direct host:port) or both `service_name` and `consul_addr` must be provided. When both are given, `target` wins.

7.2 Class: GrpcClient

gRPC client connection class.

Robot Framework usage:

```
*** Settings ***
Library      QConnectBase.connection_manager.ConnectionManager

*** Test Cases ***
Greet
    Connect    grpc_hello    conn_conf={'conn_type': 'GrpcClient',
    ...                                              'service_name': 'hello',
    ...                                              'consul_addr': 'http://localhost:8500',
    ...                                              'full_service_name': 'hello.v1.HelloService'}
    ${res}=    Verify        grpc_hello
    ...        send_cmd={'method': "Greet", "args": {"name": "World"}}
    Should Be Equal    ${res}[message]    Hello, World!
    [Teardown]    Disconnect    grpc_hello
```

The `send_cmd` is a JSON document describing the invocation. Two shapes are accepted:

1. `{"method": "<MethodName>", "args": {...}}` — uses the `full_service_name` from the connection config.
2. `{"service": "<pkg.Service>", "method": "<MethodName>", "args": {...}}` — overrides the configured service per call (useful for sending into a multi-service binary).

The response is captured as a Python dict (the gRPC response message converted via `google.protobuf.json_format.MessageToJson`) and returned to the test author by the standard `Verify` keyword. The `search_pattern` argument of `Verify` is ignored for this connection type (gRPC returns structured data, not a text stream); use Robot's built-in `Should Be Equal`, `Dictionary Should Contain`, `Get From Dictionary`, etc. against the returned dict instead.

7.2.1 Method: connect

Open the gRPC channel. Resolves via Consul if `target` is empty.

Returns:

(no returns)

7.2.2 Method: disconnect

Disconnect the gRPC channel.

Arguments:

- `device`
/ *Condition*: optional / *Type*: str /
Unused; kept for QConnectBase contract.

Returns:

(no returns)

7.2.3 Method: close

Close gRPC connection.

Returns:

(no returns)

7.2.4 Method: quit

Quit and stop receiver thread.

Arguments:

- `is_disconnect_all`
/ *Condition*: optional / *Type*: bool /
Forwarded to the base class; kept for the contract.

Returns:

(no returns)

7.2.5 Method: wait_4_trace

gRPC override of :meth:'ConnectionBase.wait_4_trace'.

System Message

ERROR/3 in <string>, line 102
Unknown interpreted text role "meth".

Unlike text-stream connections (TCP, Serial, SSH), gRPC responses are **structured objects**, not lines of text. Searching them line-by-line with a regex is meaningless. This override therefore:

- **Ignores** `search_obj` (forced internally to `'.*'`) so the receiver always matches.
- **Ignores** `use_fetch_block` / `end_of_block_pattern` / `filter_pattern` for the same reason.
- Sends the request via `send_obj` (which calls `_send` → `GrpcReflectClient.call_unary` and queues the response).
- Waits up to `timeout` seconds (or the connection's configured `timeout_seconds` when `timeout` is 0) for one response.
- **Returns the response as a Python dict**, which `ConnectionManager.verify` then surfaces to the test author unchanged. The test asserts content with Robot Framework's built-ins (`Should Be Equal`, `Dictionary Should Contain`, `Get From Dictionary`, etc.) instead of regex on a text stream.

Arguments:

- `search_obj`
/ Condition: optional / Type: str / Default: None /
 Ignored. Kept in the signature so the standard `Verify` keyword call shape stays compatible.
- `timeout`
/ Condition: optional / Type: float / Default: 0 /
 Maximum time to wait for the gRPC response, in seconds. 0 means fall back to the connection's configured `timeout_seconds`.
- `use_fetch_block`
/ Condition: optional / Type: bool / Default: False /
 Ignored.
- `end_of_block_pattern`
/ Condition: optional / Type: str / Default: None /
 Ignored.
- `filter_pattern`
/ Condition: optional / Type: str / Default: '' /*
 Ignored.
- `fct_args`
/ Condition: optional / Type: dict /
 Forwarded to `send_obj` (typically contains `send_cmd`).

Returns:

- `response`
/ Type: dict or None /
 The response message converted to a dict. None if no response arrived within `timeout`. When the gRPC call itself failed, the dict has `{"_grpc_error": True, "type": ..., "message": ...}`.

Chapter 8

rabbitmq_client.py

8.1 Class: RabbitmqClientConfig

Class to store the configuration for SSH connection.

8.2 Class: RabbitmqClient

Rabbitmq client connection class.

8.2.1 Method: connect

Implementation for creating a rabbitmq connection.

Returns:

(no returns)

8.2.2 Method: close

Close rabbitmq connection.

Returns:

(no returns)

8.2.3 Method: quit

Quit and stop receiver thread.

Returns:

(no returns)

8.3 Class: RMQSignal

RMQSignal class.

8.3.1 Method: send_signal

Send sinal to other processes.

Arguments:

- signal_name
/ Condition: required / Type: str /
Signal to be sent.

- `payload`
/ *Condition*: required / *Type*: str /
Payloads for signal.
- `receiver`
/ *Condition*: optional / *Type*: str / *Default*: None /
Specific the signal receiver to send signal to.

Returns:*(no returns)***8.3.2 Method: unset_signal_receiver_name**

Unset signal receiver.

Returns:*(no returns)***8.3.3 Method: set_signal_receiver_name**

Set the signal receiver to be received signal.

Arguments:

- `receiver`
/ *Condition*: optional / *Type*: str / *Default*: " /
Name a signal receiver to receive signal.
- `force`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Force create the signal receiver (delete the exist signal receiver with the same name).

Returns:*(no returns)***8.3.4 Method: consume_channel**

Consume the message from specific queue.

Arguments:

- `exchange`
/ *Condition*: required / *Type*: str /
Name of the exchange.
- `queue_name`
/ *Condition*: required / *Type*: str /
Name of the queue.
- `routing_key`
/ *Condition*: required / *Type*: str /
Routing key string.
- `stop_event`
/ *Condition*: required / *Type*: Event /
Event to notify stopping consuming.

- `signal_name`
/ *Condition*: required / *Type*: str or list /
Name of the signal to be wait for.
- `messages`
/ *Condition*: required / *Type*: list or dict /
Storage for received messages.
- `queue_delete`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Determine if we should delete the queue at the end.

Returns:*(no returns)***8.3.5 Method: wait_for_signal**

Wait for specific signal in timeout.

Arguments:

- `signal_name`
/ *Condition*: optional / *Type*: str /
Name of the signal to wait for.
- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 10 /
Timeout for waitting the signal. Default is 10 seconds.

Returns:

/ *Type*: str /
Payloads of the watting signal if received.

8.3.6 Method: wait_for_signals

Wait for multiple specific signals in timeout.

Arguments:

- `signal_name`
/ *Condition*: optional / *Type*: list /
List of the signals to wait for.
- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 10 /
Timeout for waitting the signal. Default is 10 seconds.

Returns:

/ *Type*: str /
List of payloads of the watting signals if received.

Chapter 9

qlogger.py

9.1 Class: ColorFormatter

Custom formatter class for setting log color.

9.1.1 Method: format

Set the color format for the log.

Arguments:

- `record`
/ *Condition*: required / *Type*: str /
Log record.

Returns:

/ *Type*: logging.Formatter /
Log with color formatter.

9.2 Class: QFileHandler

Handler class for user defined file in config.

9.2.1 Method: get_log_path

Get the log file path for this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: str /
Log file path.

9.2.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

9.3 Class: QDefaultFileHandler

Handler class for default log file path.

9.3.1 Method: get_log_path

Get the log file path for this handler.

Arguments:

- `logger_name`
/ *Condition*: required / *Type*: str /
Name of the logger.

Returns:

/ *Type*: str /
Log file path.

9.3.2 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

9.4 Class: QConsoleHandler

Handler class for console log.

9.4.1 Method: get_config_supported

Check if the connection config is supported by this handler.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

/ *Type*: bool /
True if the config is supported.
False if the config is not supported.

9.5 Class: QLogger

Logger class for QConnect Libraries.

9.5.1 Method: get_logger

Get the logger object.

Arguments:

- `logger_name`
/ *Condition*: required / *Type*: str /
Name of the logger.

Returns:

- `logger`
/ *Type*: Logger /
Logger object. .

9.5.2 Method: set_handler

Set handler for logger.

Arguments:

- `config`
/ *Condition*: required / *Type*: DictToClass /
Connection configurations.

Returns:

- `handler_ins`
/ *Type*: logging.handler /
None if no handler is set.
Handler object.

Chapter 10

serial_base.py

10.1 Class: SerialConfig

Class to store the configuration for Serial connection.

10.2 Class: SerialSocket

Class for handling serial connection.

10.2.1 Method: connect

Connect to serial port.

Returns:

(no returns)

10.2.2 Method: disconnect

Disconnect serial port.

Arguments:

- `_device`
/ *Condition*: required / *Type*: str /
Unused

Returns:

(no returns)

10.2.3 Method: quit

Quit serial connection.

Returns:

(no returns)

10.3 Class: SerialClient

Serial client class.

10.3.1 Method: connect

Connect to the Serial port.

Returns:

(no returns)

Chapter 11

raw_tcp.py

11.1 Class: RawTCPBase

Base class for a raw tcp connection.

11.2 Class: RawTCPServer

Class for a raw tcp connection server.

11.3 Class: RawTCPClient

Class for a raw tcp connection client.

Chapter 12

ssh__client.py

12.1 Class: AuthenticationType

12.2 Class: SSHConfig

Class to store the configuration for SSH connection.

12.3 Class: SSHClient

SSH client connection class.

12.3.1 Method: connect

Implementation for creating a SSH connection.

Returns:

(no returns)

12.3.2 Method: transfer_file

DEPRECATED!! Use keyword transfer_item instead. Transfer file from local to remote and vice versa.

Arguments:

- src
/ Condition: required / Type: str /
Source file path.
- dest
/ Condition: required / Type: str /
Destination file path.
- transfer_type
/ Condition: required / Type: str /
Transfer file type.

'get' - Copy a remote file from the SFTP server to the local host
'put' - Copy a local file to the SFTP server

Returns:

(no returns)

12.3.3 Method: transfer_item

Transfer item from local to remote and vice versa.

Arguments:

- `src`
/ *Condition*: required / *Type*: str /
Source item path.
- `dest`
/ *Condition*: required / *Type*: str /
Destination item path.
- `transfer_type`
/ *Condition*: required / *Type*: str /
Transfer item type.

 'get' - Copy a remote item from the SFTP server to the local host
 'put' - Copy a local item to the SFTP server

Returns:

(no returns)

12.3.4 Method: close

Close SSH connection.

Returns:

(no returns)

12.3.5 Method: quit

Quit and stop receiver thread.

Returns:

(no returns)

Chapter 13

tcp_base.py

13.1 Class: TCPConfig

Class to store configurations for TCP connection.

13.2 Class: TCPBase

Base class for a tcp connection.

13.2.1 Method: close

Close connection.

Returns:

(no returns)

13.2.2 Method: quit

Quit connection.

Arguments:

- `is_disconnect_all`
/ *Condition*: required / *Type*: bool /
Determine if it's necessary for disconnect all connection.

Returns:

(no returns)

13.2.3 Method: connect

» Should be override in derived class.

Establish the connection.

Returns:

(no returns)

13.2.4 Method: disconnect

» Should be override in derived class.

Disconnect the connection.

Returns:

(no returns)

13.3 Class: TCPBaseServer

Base class for TCP server.

13.3.1 Method: `accept__connection`

Wrapper method for handling accept action of TCP Server.

Returns:

(no returns)

13.4 Class: TCPBaseClient

Base class for TCP client.

Chapter 14

utils.py

14.1 Function: has_capturing_groups

Checks whether a pattern contains capturing groups

14.2 Function: validate_regex_pattern

Checks whether the given pattern is valid

14.3 Class: Singleton

Class to implement Singleton Design Pattern. This class is used to derive the TTFisClientReal as only a single instance of this class is allowed.

Disabled pyLint Messages: R0903: Too few public methods (%s/%s)

System Message

ERROR/3 in <string>, line 19
Unexpected indentation. backrefs:

Used when class has too few public methods, so be sure it's really worth it.

This base class implements the Singleton Design Pattern required for the TTFisClientReal. Adding further methods does not make sense.

14.4 Class: DictToClass

Class for converting dictionary to class object.

14.5 Class: Utils

Class to implement utilities for supporting development.

14.5.1 Method: get_all_descendant_classes

Get all descendant classes of a class

Arguments: cls: Input class for finding descendants.

Returns:

/ *Type*: list /

Array of descendant classes.

14.5.2 Method: get_all_sub_classes

Get all children classes of a class

Arguments:

- `cls`
/ *Condition*: required / *Type*: class /
Input class for finding children.

Returns:

/ *Type*: list /
Array of children classes.

14.5.3 Method: caller_name

Get a name of a caller in the format module.class.method

Arguments:

- `skip`
/ *Condition*: required / *Type*: int /

Specifies how many levels of stack to skip while getting caller name. `skip=1` means "who calls me", `skip=2` "who calls my caller" etc.

Returns:

/ *Type*: str /
An empty string is returned if skipped levels exceed stack height

14.5.4 Method: load_library

Load native library depend on the calling convention.

Arguments:

- `path`
/ *Condition*: required / *Type*: str /
Library path.
- `is_stdcall`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Determine if the library's calling convention is stdcall or cdecl.

Returns:

Loaded library object.

14.5.5 Method: is_ascii_or_unicode

Check if the string is ascii or unicode

Arguments: `str_check`: string for checking codecs: encoding type list

Returns:

/ *Type*: bool /
True : if checked string is ascii or unicode
False : if checked string is not ascii or unicode

14.6 Class: Job

14.7 Class: ResultType

Result Types.

14.8 Class: ResponseMessage

Response message class

14.8.1 Method: get_json

Convert response message to json

Returns:

Response message in json format

14.8.2 Method: get_data

Get string data result

Returns:

String result

Chapter 15

Appendix

About robotframework-qconnect-base:

Author

Nguyen Huynh Tri Cuong (cuong.nguyenhuynhtri@vn.bosch.com)

Version

1.4.0 (11.02.2026)

Short Description

Robot Framework library for TCP, SSH, serial connection

Homepage

[robotframework-qconnect-base](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

≥ 3.11

License

Apache-2.0

Chapter 16

History

1.1.0	07/2022
<i>Initial version</i>	
1.1.5	06/2025
<i>Modified timeout handling including <code>verify_timeout</code> and <code>emergency_timeout</code>.</i>	
<i>Added support for time string parsing and new keywords: <code>set_default_verify_timeout</code>, <code>set_default_emergency_timeout</code>.</i>	
<i>Made <code>search_pattern</code> an optional parameter.</i>	
<i>Improved error handling when disconnecting non-existent connections.</i>	
<i>Fixed issues causing infinite wait on SSH disconnect and other minor bugs.</i>	
<i>Ensured compatibility with Python 3.11.</i>	
<i>Updated log format and improved documentation.</i>	
<i>Applied self-test maintenance and improved connection configuration handling.</i>	
1.1.6	07/2025
<i>Added support item transfer for SSH connection</i>	
1.1.7	08/2025
- Enhanced the input parameter of keyword <code>connect</code>. - Improved logging message and levels. - Improved singleton implementation. - Updated documentation to give more detail about parameters of <code>verify keyword</code>.	
1.1.8	09/2025
- Modified the response of <code>verify keyword</code> to return list of captured strings. - Updated to raise <code>UNKNOWN</code> instead of <code>ERROR</code> when providing invalid <code>connection_type</code>. - Fixed issue that <code>Broken Connection</code> is not detected on Linux.	
1.1.9	10/2025
<i>Improved the logging messages</i>	
1.1.10	10/2025
<i>Changed logic of <code>verify keyword</code> to check the <code>eob_pattern</code> before applying <code>filter_pattern</code></i>	
1.2.0	11/2025

CHAPTER 16. HISTORY

- Updated the default value of the <code>eob_pattern</code> parameter in the <code>verify</code> keyword to <code>None</code>. - Improved the <code>verify</code> keyword to validate all input regex patterns. - Updated the <code>connect</code> keyword to make <code>conn_name</code> a required parameter.	
1.2.1	11/2025
Fixed exception chaining to avoid 'During handling of the above exception' warning in debug log file.	
1.2.2	11/2025
- Improved parameter <code>logfile</code>: error handling, datatype validation and acceptable values. - Fixed issue with logger did not write any log messages when using <code>console</code>. - Aligned behavior of writing logfile: overwrite the existing one. - Enforced a minimum timeout of <code>0.001s</code> for the <code>verify</code> keyword	
1.2.3	12/2025
- Fixed issue with SSH connection that shell is not ready when sending command. - Enhanced the error message of <code>verify</code> keyword for the all failure cases.	
1.3.0	01/2026
- Added verification for optional parameters in the <code>verify</code> keyword due to connection type.	
1.4.0	02/2026
- Usage of <code>Setuptools</code> replaced by usage of <code>PIP/TOML/Setuptools</code>	

QConnectBase.pdf

Created at 07.09.2026 - 09:47:03

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.7 QConnectWinapp

QConnectWinapp

v. 1.1.0

Nguyen Huynh Tri Cuong

24.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Getting Started	2
2.2	Usage	2
2.2.1	Configurations for Winapp connection type	2
2.2.2	Using the Automation Inspector Tool to define a GUI control	2
2.3	Example	6
2.4	Contribution Guidelines	7
2.5	Configure Git and correct EOL handling	8
2.6	Sourcecode Documentation	8
2.7	Feedback	9
2.8	About	9
2.8.1	Maintainers	9
2.8.2	Contributors	9
2.8.3	3rd Party Licenses	9
2.8.4	Used Encryption	9
2.8.5	License	9
3	element_handler.py	12
3.1	Method: get_supported_level	12
3.2	Method: get_attribute	12
3.3	Method: divert_action	12
4	winapp_client.py	13
4.1	Class: CustomWebDriver	13
4.1.1	Method: start_session	13
4.2	Class: WinappConfig	13
4.3	Class: WinAppClient	13
4.3.1	Method: connect	13
4.3.2	Method: perform_action	13
4.3.3	Method: send_obj	14
4.3.4	Method: wait_4_trace	14
4.3.5	Method: disconnect	15
4.3.6	Method: quit	15
5	Appendix	16
6	History	17

Chapter 1

Introduction

QConnectWinapp is an extension for the **Robot Framework** keyword library **QConnectBase** and designed to simplify and automate Windows application GUI testing. **QConnectWinapp** is built to provide seamless and efficient GUI testing experiences for Windows applications.

QConnectWinapp uses the powerful backend of **WinAppDriver** to drive the testing of your Windows applications, ensuring reliable and accurate results.

With **QConnectWinapp**, you can easily and quickly automate the testing of your Windows applications, saving time and effort while ensuring that your applications are of the highest quality. Whether you're a seasoned tester or just starting out, **QConnectWinapp** offers a range of features and tools to make GUI testing simple and efficient.

References:

- [QConnectBase](#)
- [QConnectWinapp](#)
- [WinAppDriver](#)

Chapter 2

Description

2.1 Getting Started

Details about the installation and the prerequisites can be found in the [README](#).

2.2 Usage

QConnectWinapp is a backend extension for the QConnectBase library that adds support for testing WinApp UI. From the user's perspective, this means they now have an additional connection type for WinApp testing when using the **connect** keyword in the QConnectBase library. With QConnectWinapp, users can now easily automate tests for Windows desktop applications, and take advantage of its integration with WinAppDriver to interact with UI elements and validate their behavior.

Please refer to [QConnectBase](#) for more information on how to use the **connect** keyword. In this section, we will focus on the **Winapp** connection type and how to configure it.

2.2.1 Configurations for Winapp connection type

QConnectBaseLibrary has already supported below connection types:

- **TCPIPClient**: Create a Raw TCPIP connection to TCP Server.
- **SSHClient**: Create a client connection to a SSH server.
- **SerialClient**: Create a client connection via Serial Port.
- **DLT**: Create a client connection to Diagnostic Log and Trace(DLT) Module.

QConnectWinapp add one more connection type for Winapp UI testing call **Winapp**

Below is the description of the configuration string for Winapp connection type:

```
{
  "host": [host ip],      # Optional. Default value is "localhost".
  "port": [listening port] # Optional. Default value is 4723.
  "caps": [Desired Capabilities string in JSON format], # E.g. { "app": "C:/Program ↵
↵ Files/BOSCH/CMST/CMST.exe"}
  "logfile": [Log file path. Possible values: 'nonlog', 'console', [user define path]]
}
```

2.2.2 Using the Automation Inspector Tool to define a GUI control

The Automation Inspector tool provides a convenient way to interact with controls of an Application Under Test (AUT) and create resource files for testing. Follow the steps below to effectively utilize the tool:

Accessing the Tool

The Automation Inspector tool can be found at the following location: C:/Program Files/RobotFramework/python39/Lib/site-packages/QConnectWinapp/tools/bin/release.

Table 2.1: List of Capabilities

Capability	Description
app	The package full name or the executable file's full path of the application to automate (e.g. Microsoft.WindowsCalculator_8wekyb3d8bbwe!App).
deviceName	Mostly optional, but the name of your device
automationName	The name of the automation engine to use (e.g. windows).

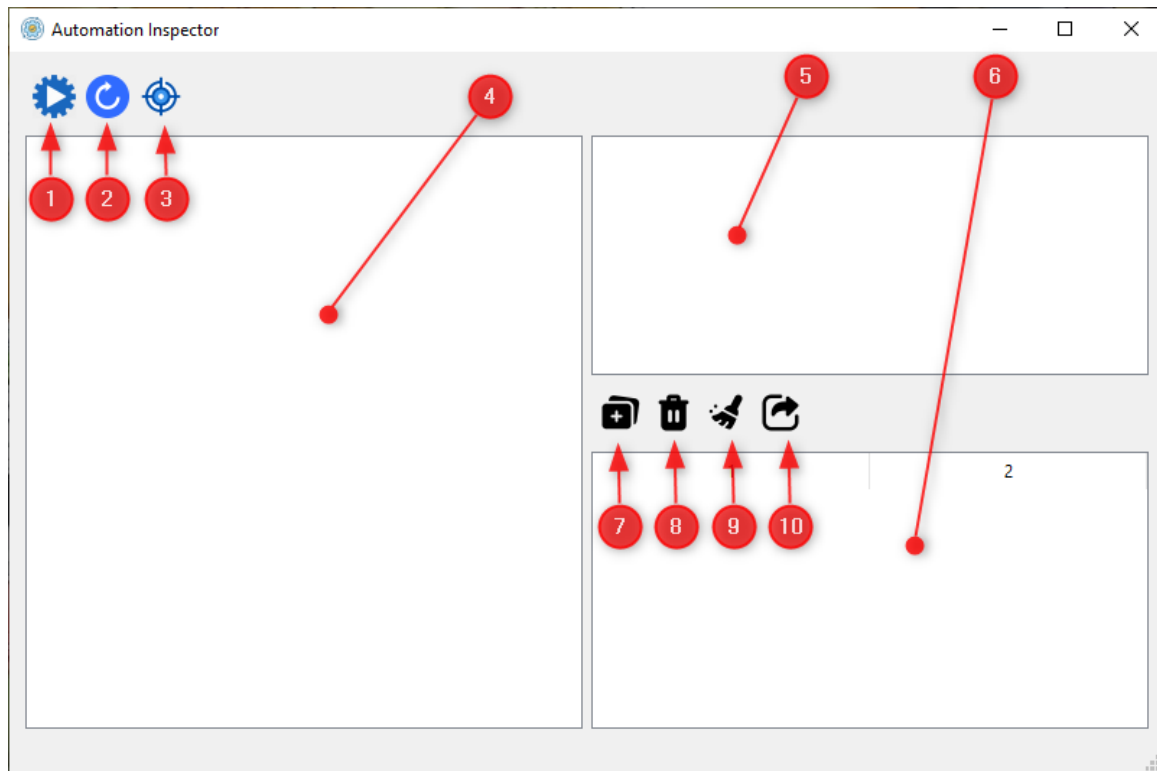


Figure 2.1: Automation Inspector tool.

Tool Overview

Upon launching the Automation Inspector tool, you will be presented with a graphical user interface that offers a range of functionalities. The main features (please refer to the Figure 2.1) of the tool are described below:

1. **Exec AUT button:** Press this button to start an Application Under Test (AUT).
2. **Refresh button:** Press this button to refresh the information of the AUT.
3. **Inspect controls button:** After pressing this button, users can hover their mouse over controls on the AUT. The tool will automatically highlight and display the attributes of the currently highlighted control.
4. **Elements treeview area:** This area displays the list of controls of the AUT in a hierarchical tree structure. Clicking on a node in the tree displays its attributes in the Properties table (5).
5. **Properties table:** This area displays the attributes of the node selected in the Elements treeview area (4) or highlighted using the Inspect controls feature (3). Users can check checkboxes to select attributes for defining the control to be tested and add them to the Definitions table (6) using the Add definition button (7).
6. **Definitions table:** This table contains the definitions of controls to be tested. Users can select attributes in the Properties table (5) and add them to this table using the Add definition button (7). Users can also check checkboxes to select definitions for exporting to a .resource file using the Export button (10).

7. **Add definition button:** Press this button to add the selected attributes from the Properties table (5) as definitions for a control in the Definitions table (6).
8. **Remove Definition button:** Press this button to remove the selected definition from the Definitions table (6).
9. **Clear Definitions button:** Press this button to remove all definitions from the Definitions table (6).
10. **Export button:** Export all selected definitions from the Definitions table (6) into a .resource file.

Creating a Resource File

To create a resource file using the Automation Inspector tool, follow these steps:

1. Launch the Automation Inspector tool from C:/Program Files/RobotFramework/python39/Lib/site-packages/QConnectWinapp/tools/bin/release.
2. Start the Application Under Test (AUT) by clicking the **Exec AUT** button. After clicking, a dialog will open, allowing the user to select the path to the executable file for the AUT. In the following example, I will input the path for the Calculator application.

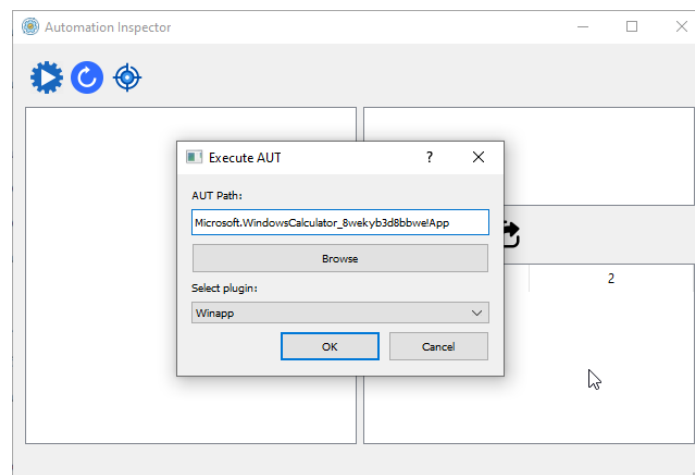


Figure 2.2: Select the AUT path to execute

3. After selecting the AUT executable, click the **OK** button to start the AUT. The **Elements treeview** will be populated, showing a hierarchical tree of controls present in the AUT.
4. To view the properties of a control, you can either:
 - Click on a node in the **Elements treeview** to display its properties in the **Properties table**.
 - Select the **Inspect controls** button and hover the mouse over a control in the AUT. The corresponding control in the **Elements treeview** will be selected and its properties will be shown in the **Properties table**.
5. Select the desired attributes for control definitions by checking the checkboxes in the **Properties table**.

CHAPTER 2. DESCRIPTION

2.2. USAGE

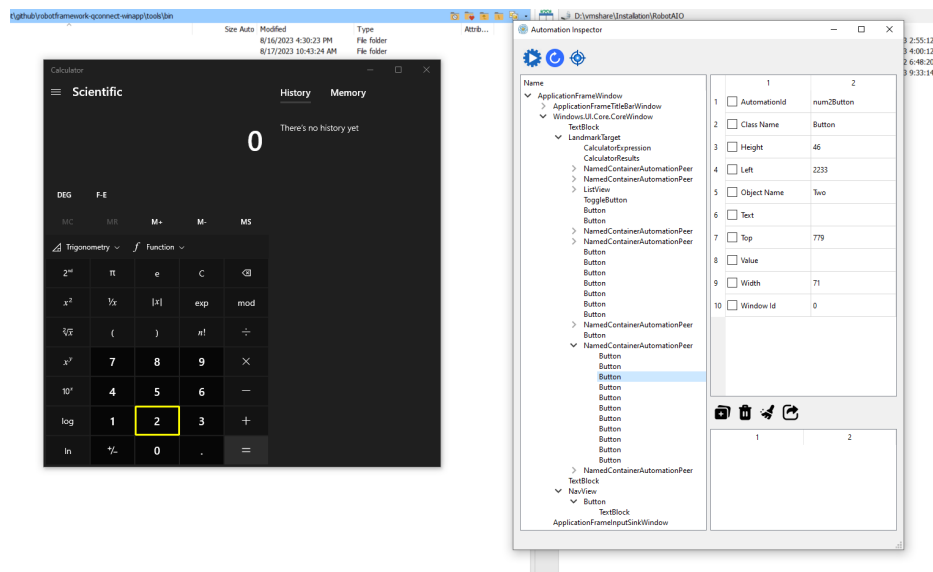


Figure 2.3: View control properties

- Click the **Add definition** button to add the selected attributes to the **Definitions** table. After clicking the button, a dialog will appear for the user to declare a name corresponding with the definition. In the following example, I will fill the name as 'btn2Num', and the definition will be '"AutomationId":"num2Button"', which means that the 'btn2Num' is a control with the AutomationId property value of 'num2Button'.
- Repeat the process to define multiple controls and their attributes.

CHAPTER 2. DESCRIPTION

2.3. EXAMPLE

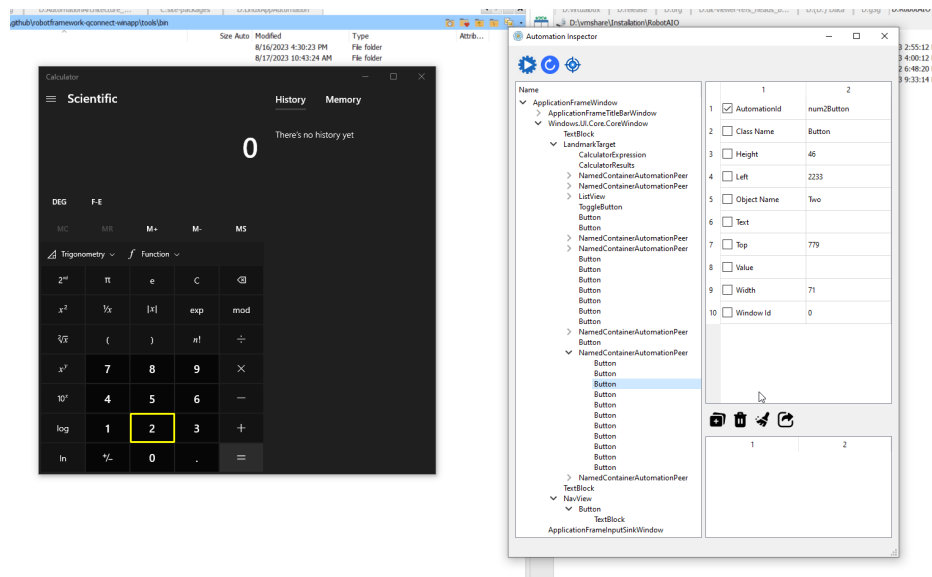


Figure 2.4: Select property for definition

8. Use the **Export** button to export selected definitions to a .resource file.

By following these steps, you can effectively utilize the Automation Inspector tool to interact with controls, define attributes, and create resource files for testing purposes.

2.3 Example

In this example, I provide a test scenario that allows adding on the Calculator app as follows: the script will open the Calculator application on Windows, then press the '8' button, press the '+' button, press the '9' button, and then press the '=' button. Then, verify if the result textbox displays the result '17'.

```
*** Settings ***
Documentation    Suite description
Library         QConnectBase.ConnectionManager
Resource        QConnectWinapp/GUIAction.resource

*** Variables ***
${CONNECTION_NAME}    TEST_CONN

&{Number 8}          AutomationId=num8Button
&{Number 9}          AutomationId=num9Button
&{Equal}              AutomationId=equalButton
&{Result}             AutomationId=CalculatorResults
&{Plus}               AutomationId=plusButton

*** Test Cases ***
Test Adding
    ${config_string}=    concatenate
    ...    {\n
    ...    "host":    "localhost",\n
    ...    "port":    4723,\n
    ...    "caps":\n
    ...    {\n
    ...    "app":    "Microsoft.WindowsCalculator_8wekyb3d8bbwe!App"\n
    ...    }\n
    ...    }\n
    log to console    \nConnecting with below configurations:\n${config_string}
```

```

${config}=          evaluate          json.loads('"'${config_string}"')          json

connect              conn_name=${CONNECTION_NAME}
...                  conn_type=Winapp
...                  conn_conf=${config}

send command         conn_name=${CONNECTION_NAME}
...                  element_def=${Number 8}
...                  command=${Action.click}

Sleep                1s

send command         conn_name=${CONNECTION_NAME}
...                  element_def=${Plus}
...                  command=${Action.click}

Sleep                1s

send command         conn_name=${CONNECTION_NAME}
...                  element_def=${Number 9}
...                  command=${Action.click}

Sleep                1s

send command         conn_name=${CONNECTION_NAME}
...                  element_def=${Equal}
...                  command=${Action.click}

${res}=              verify           conn_name=${CONNECTION_NAME}
...                  element_def=${Result}
...                  search_pattern=17
...                  send_cmd=${Action.get_text}
...                  timeout=20

log to console       \nCalculation result: ${res}[0]

*** Keyword ***
Close Connection
disconnect           ${CONNECTION_NAME}

```

Explanation:

- `&{Number 8} AutomationId=num8Button` : This line is used to identify the 'Number 8' button as the control on the application with an Accessible Name (AutomationId) of 'num8Button'.
- `"app": "Microsoft.WindowsCalculator_8wekyb3d8bbwe!App"` : This line specifies that the AUT is the application with the package name 'Microsoft.WindowsCalculator_8wekyb3d8bbwe!App'. You can also use the full path to the exe file of the AUT.
- `send command conn_name=${CONNECTION_NAME}...command=${Action.click}` : This line means to click on the Number 8 button on the AUT.
- `${Action.click}` has been defined in the resource file `QConnect Winapp/GUIAction.resource`.
Currently supported in the GUIAction.resource:

2.4 Contribution Guidelines

QConnectBaseLibrary is designed for ease of making an extension library. By that way you can take advantage of the QConnectBaseLibrary's infrastructure for handling your own connection protocol. For creating an extension library for QConnectBaseLibrary, please following below steps.

1. Create a library package which have the prefix name is **robotframework-qconnect-*[your specific name]***.

Action	Description
<code>\${Action.click}</code>	Clicks on a GUI element
<code>\${Action.get_text}</code>	Gets the text of a GUI element
<code>\${Action.get_visible}</code>	Get the visibility of a GUI element
<code>\${Action.get_enable}</code>	Check if a GUI element is enabled

Table 2.2: Actions supported in `GUIAction.resource`

2. Your handling connection class should be derived from `QConnectionLibrary.connection_base.ConnectionBase` class.
3. In your *Connection Class*, override below attributes and methods:
 - **`_CONNECTION_TYPE`**: name of your connection type. It will be the input of the `conn_type` argument when using **`connect`** keyword. Depend on the type name, the library will determine the correct connection handling class.
 - **`__init__(self, _mode, config)`**: in this constructor method, you should:
 - Prepare resource for your connection.
 - Initialize receiver thread by calling **`self._init_thread_receiver(cls._socket_instance, mode="")`** method.
 - Configure and initialize the lowlevel receiver thread (if it's necessary) as below


```
self._llrecv_thrd_obj = None
self._llrecv_thrd_term = threading.Event()
self._init_thrd_llrecv(cls._socket_instance)
```
 - In case you use the lowlevel receiver thread. You should implement the **`thrd_llrecv_from_connection_inter`** method. This method is a mediate layer which will receive the data from connection at the very beginning, do some process then put them in a queue for the **`receiver thread`** above getting later.
 - Create the queue for this connection (use `Queue.Queue`).
 - **`connect()`**: implement the way you use to make your own connection protocol.
 - **`_read()`**: implement the way to receive data from connection.
 - **`_write()`**: implement the way to send data via connection.
 - **`disconnect()`**: implement the way you use to disconnect your own connection protocol.
 - **`quit()`**: implement the way you use to quit connection and clean resource.

2.5 Configure Git and correct EOL handling

Here you can find the references for [Dealing with line endings](#).

Every time you press return on your keyboard you're actually inserting an invisible character called a line ending. Historically, different operating systems have handled line endings differently. When you view changes in a file, Git handles line endings in its own way. Since you're collaborating on projects with Git and GitHub, Git might produce unexpected results if, for example, you're working on a Windows machine, and your collaborator has made a change in OS X.

To avoid problems in your diffs, you can configure Git to properly handle line endings. If you are storing the `.gitattributes` file directly inside of your repository, then you can assure that all EOL are managed by git correctly as defined.

2.6 Sourcecode Documentation

For investigating sourcecode, please refer to [QConnectWinapp Documentation](#)

2.7 Feedback

If you have any problem when using the library or think there is a better solution for any part of the library, I'd love to know it, as this will all help me to improve the library. Connect with me at cuong.nguyenhuynhtri@vn.bosch.com.

Do share your valuable opinion, I appreciate your honest feedback!

2.8 About

2.8.1 Maintainers

[Nguyen Huynh Tri Cuong](#)

2.8.2 Contributors

[Nguyen Huynh Tri Cuong](#)

[Thomas Pollerspoeck](#)

2.8.3 3rd Party Licenses

You must mention all 3rd party licenses (e.g. OSS) licenses used by your project here. Example:

Name	License	Type
Apache Felix.	Apache 2.0 License.	Dependency

2.8.4 Used Encryption

Declaration of the usage of any encryption (see BIOS Repository Policy §4.a).

2.8.5 License



Copyright (c) 2009, 2018 Robert Bosch GmbH and its subsidiaries. This program and the accompanying materials are made available under the terms of the Bosch Internal Open Source License v4 which accompanies this distribution, and is available at <http://bios.intranet.bosch.com/bioslv4.txt>

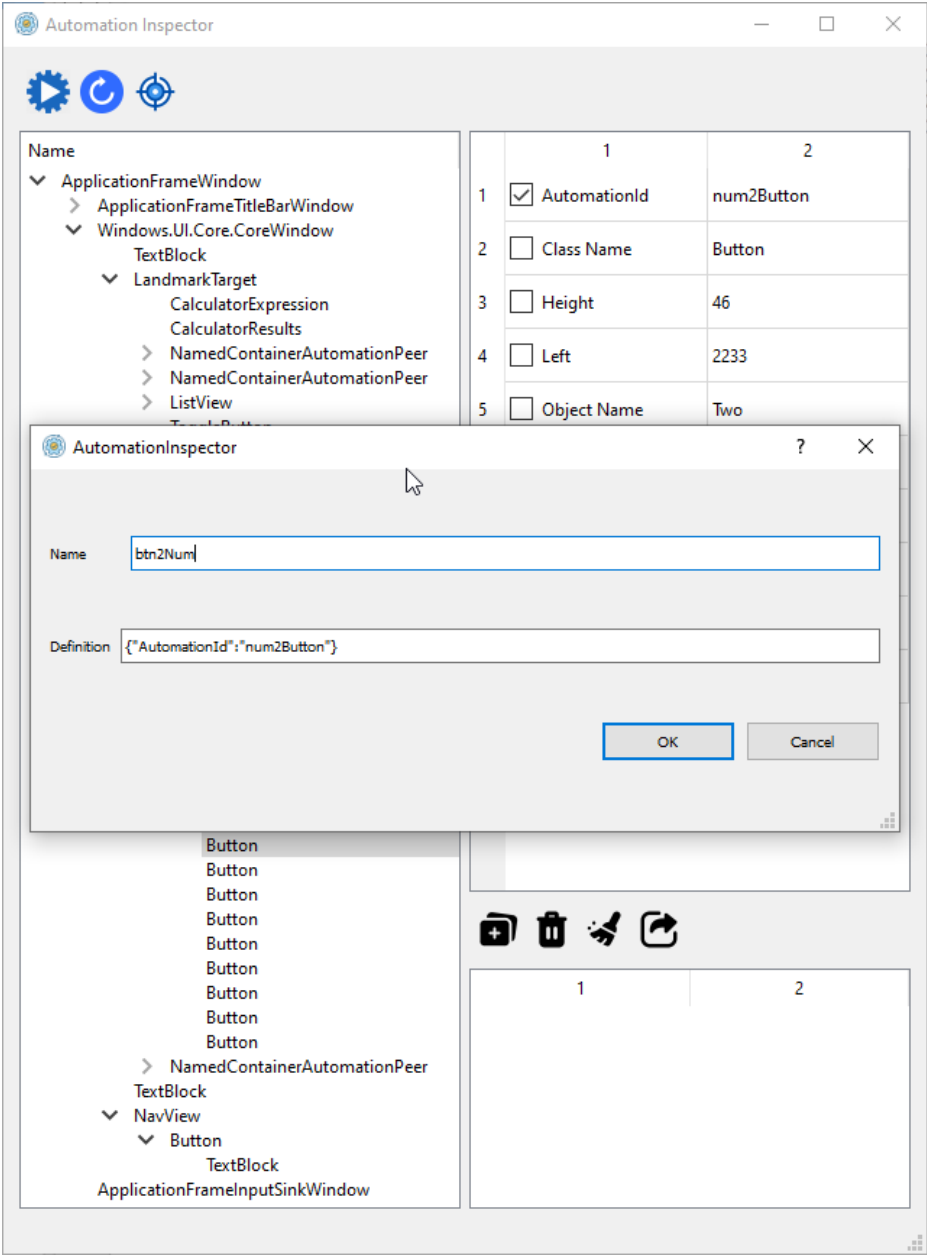


Figure 2.5: Declare a test automation control

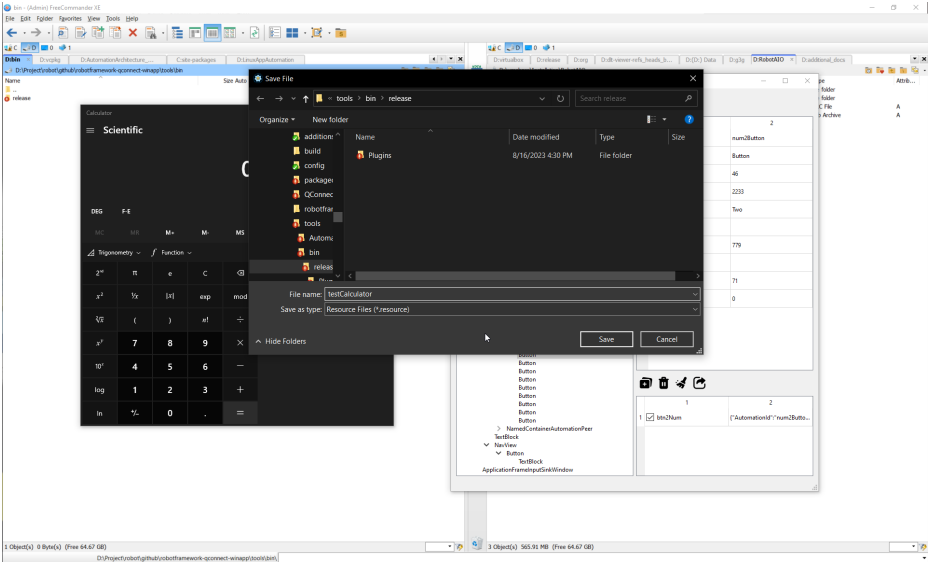


Figure 2.6: Export to resource file

Chapter 3

element_handler.py

3.1 Method: get_supported_level

Get the supported level of this handler for a specific element object.

Arguments:

- `ele_obj`
/ *Condition*: required / *Type*: WebElement /
Winapp GUI element object.

Returns:

/ *Type*: int /
Supported level of this action handler for the element object.

3.2 Method: get_attribute

Get element object's property.

Arguments:

- `attr`
/ *Condition*: required / *Type*: str /
Property's name to be got value.

Returns:

/ *Type*: str /
Property's value.

3.3 Method: divert_action

Divert action string to the corresponding method, execute the method and return value.

Arguments:

- `action`
/ *Condition*: required / *Type*: str /
Action string to be diverted.

Returns:

/ *Type*: str /
Corresponding method's return.

Chapter 4

winapp_client.py

4.1 Class: CustomWebDriver

Customer WebDriver class for Winapp.

4.1.1 Method: start_session

Creates a new session with the desired capabilities.

Override for Winapp

Arguments:

- `capabilities`
/ *Condition*: required / *Type*: Union /
Read <https://github.com/appium/appium/blob/master/docs/en/writing-running-appium/caps.md> for more details.
- `browser_profile`
/ *Condition*: optional / *Type*: str / *Default*: None /
Browser profile

4.2 Class: WinappConfig

Class to store the configuration for SSH connection.

4.3 Class: WinAppClient

Winapp client connection class.

4.3.1 Method: connect

Connect to WinappDriver which is listening on configured port.

Returns:

(no returns)

4.3.2 Method: perform_action

Perform an action on user defined element.

Arguments:

- `obj_defined_dict`
/ *Condition*: required / *Type*: dict /
User's definition for a GUI element.
- `cmd`
/ *Condition*: required / *Type*: str /
Action to be perform on GUI element.
- `time_wait`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Timeout to find a GUI element based on user's definitions.

Returns:

/ *Type*: WebElement /
GUI element.

4.3.3 Method: send_obj

Action to be send to a GUI element.

Arguments:

- `obj`
/ *Condition*: required / *Type*: str /
Data to be sent.
- `element_def`
/ *Condition*: required / *Type*: dict /
User's definition for a GUI element.
- `args`
/ *Condition*: optional / *Type*: tuple /
Optional arguments.

Returns:

(no returns)

4.3.4 Method: wait_4_trace

Perform an action on GUI element and wait to receive a return which matches to a specified regular expression.

Arguments:

- `search_obj`
/ *Condition*: required / *Type*: str /
Regular expression all received trace messages are compare to. Can be passed either as a string or a regular expression object. Refer to Python documentation for module 're'.
- `use_fetch_block`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Determine if 'fetch block' feature is used.
- `end_of_block_pattern`
/ *Condition*: optional / *Type*: str / *Default*: '.*' /
The end of block pattern.

- `filter_pattern`
/ Condition: optional / Type: str / Default: '.' /
 Pattern to filter message line by line.
- `timeout`
/ Condition: optional / Type: int / Default: 0 /
 Timeout parameter specified as a floating point number in the unit 'seconds'.
- `element_def`
/ Condition: required / Type: dict /
 User's definition for a GUI element.
- `args`
/ Condition: optional / Type: tuple /
 Optional arguments.

Returns:

- `match`
/ Type: re.Match /
 If no return value matched to the specified regular expression and a timeout occurred, return None.
 If a return value has matched to the specified regular expression, a match object is returned as the result. The complete trace message can be accessed by the 'string' attribute of the match object. For access to groups within the regular expression, use the `group()` method. For more information, refer to Python documentation for module 're'.

4.3.5 Method: disconnect

Abstract method for disconnecting connection.

Arguments:

- `_device`
/ Condition: required / Type: str /
 Unused.

Returns:

(no returns)

4.3.6 Method: quit

Quiting the connection.

Returns:

(no returns)

Chapter 5

Appendix

About robotframework-qconnect-winapp:

Author

Nguyen Huynh Tri Cuong (cuong.nguyenhuynhtri@vn.bosch.com)

Version

1.1.0 (24.02.2026)

Short Description

Extension for Robot Framework QConnectBase library to test Winapp GUIs

Homepage

[robotframework-qconnect-winapp](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

>=3.11

License

Apache-2.0

Chapter 6

History

1.0.0	07/2022
Initial version	
1.1.0	24.02.2026
Usage of Setuptools replaced by usage of PIP/TOML/Setuptools	

10.8 RobotFramework_DBus

RobotFramework_DBus

v. 0.2.0

Nguyen Huynh Tri Cuong

23.02.2026

CONTENTS

CONTENTS

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Key Features	1
2	Description	2
2.1	Prerequisites	2
2.2	Getting Started	2
2.3	Usage	2
2.3.1	connect	2
2.3.2	disconnect	3
2.3.3	set signal received handler	3
2.3.4	unset signal received handler	4
2.3.5	register signal	4
2.3.6	call dbus method	4
2.3.7	wait for signal	5
2.4	Remote testing	5
2.5	Example	5
2.5.1	Example 1 - Synchronized Dbus Signal Waiting	6
2.5.2	Example 2 - Managing Multiple Dbus Signals with 'Register Signal' Keyword	6
2.5.3	Example 3 - Non-blocking Dbus Signal Handling with 'Set Signal Received Handler'	7
2.5.4	Example 4 - Invoking Dbus Methods: Using 'Call Dbus Method' Keyword	8
2.5.5	Example 5 - Testing a Remote Dbus Service: Setup and Steps	8
2.6	Configure Git and correct EOL handling	9
2.7	Feedback	9
2.8	About	9
2.8.1	Maintainers	9
2.8.2	Contributors	9
2.8.3	3rd Party Licenses	9
2.8.4	Used Encryption	9
2.8.5	License	9
3	__init__.py	11
4	priority_queue.py	12
5	register_keyword.py	13
5.1	Method: get_kw_name	13

CONTENTS

CONTENTS

5.2	Method: <code>callback_func</code>	13
6	<code>scheduled_job.py</code>	14
6.1	Method: <code>stop</code>	14
6.2	Method: <code>run</code>	14
7	<code>thread_safe_dict.py</code>	15
7.1	Method: <code>clear</code>	15
7.2	Method: <code>pop</code>	15
7.3	Method: <code>popitem</code>	15
7.4	Method: <code>update</code>	15
8	<code>utils.py</code>	17
8.1	Class: <code>Singleton</code>	17
8.2	Class: <code>DictToClass</code>	17
8.3	Class: <code>Utils</code>	17
8.3.1	Method: <code>make_unique_token</code>	17
8.3.2	Method: <code>get_all_descendant_classes</code>	17
8.3.3	Method: <code>get_all_sub_classes</code>	18
8.3.4	Method: <code>caller_name</code>	18
8.3.5	Method: <code>load_library</code>	18
8.3.6	Method: <code>is_ascii_or_unicode</code>	19
9	<code>dbus_client_agent.py</code>	20
9.1	Function: <code>run_agent</code>	20
9.2	Class: <code>DBusClientExecutor</code>	20
9.2.1	Method: <code>connect</code>	20
9.2.2	Method: <code>disconnect</code>	20
9.2.3	Method: <code>quit</code>	20
9.2.4	Method: <code>get_monitoring_signal_payloads</code>	21
9.2.5	Method: <code>add_signal_to_captured_dict</code>	21
9.2.6	Method: <code>register_monitored_signal</code>	21
9.2.7	Method: <code>wait_for_signal</code>	21
9.2.8	Method: <code>call_dbus_method</code>	22
9.3	Class: <code>DBusClientAgent</code>	22
9.3.1	Method: <code>get_session_token</code>	22
9.3.2	Method: <code>initialize_dbus_client</code>	22
9.3.3	Method: <code>connect</code>	23
9.3.4	Method: <code>disconnect</code>	23
9.3.5	Method: <code>quit</code>	23
9.3.6	Method: <code>get_monitoring_signal_payloads</code>	23
9.3.7	Method: <code>register_monitored_signal</code>	24
9.3.8	Method: <code>wait_for_signal</code>	24
9.3.9	Method: <code>call_dbus_method</code>	24
10	<code>dbus_client.py</code>	26
10.1	Method: <code>connect</code>	26
10.2	Method: <code>disconnect</code>	26

CONTENTS

CONTENTS

10.3	Method: quit	26
10.4	Method: set_signal_received_handler	26
10.5	Method: unset_signal_received_handler	27
10.6	Method: add_signal_to_captured_dict	27
10.7	Method: register_monitored_signal	27
10.8	Method: wait_for_signal	28
10.9	Method: call_dbus_method	28
10.10	Method: call_dbus_method_with_keyword_args	28
11	dbus_client_remote.py	29
11.1	Method: connect	29
11.2	Method: disconnect	29
11.3	Method: quit	29
11.4	Method: do_signal_check	29
11.5	Method: set_signal_received_handler	30
11.6	Method: unset_signal_received_handler	30
11.7	Method: register_monitored_signal	30
11.8	Method: wait_for_signal	30
11.9	Method: call_dbus_method	31
12	dbus_manager.py	32
12.1	Method: quit	32
12.2	Method: add_connection	32
12.3	Method: remove_connection	32
12.4	Method: get_connection_by_name	33
12.5	Keyword: disconnect	33
12.6	Keyword: connect	33
12.7	Keyword: set_signal_received_handler	34
12.8	Keyword: unset_signal_received_handler	34
12.9	Keyword: register_signal	35
12.10	Keyword: call_dbus_method	35
12.11	Keyword: wait_for_signal	35
13	Appendix	37
14	History	38

Chapter 1

Introduction

1.1 Introduction

RobotFramework_DBus is a Python extension library for Robot Framework. It is designed to support the automation testing of Dbus services. The library provides a set of keywords that enable testing of Dbus methods and signals using both synchronous and asynchronous mechanisms.

Built upon the Python *dasbus* library, **RobotFramework_DBus** simplifies the process of interacting with Dbus services and facilitates efficient testing of their functionality. It offers a comprehensive set of features to streamline the testing process and ensure the reliability of Dbus-based applications.

1.2 Key Features

- Integration with Robot Framework: **RobotFramework_DBus** seamlessly integrates with Robot Framework, allowing you to leverage its powerful test automation capabilities for Dbus services.
- Dbus Service Automation: With **RobotFramework_DBus**, you can easily automate the testing of Dbus services. It provides a range of keywords specifically designed for this purpose.
- Testing Dbus Methods and Signals: The library enables you to test both Dbus methods and signals. You can validate the behavior of Dbus methods and monitor the emission of signals during the testing process.
- Synchronous and Asynchronous Testing: **RobotFramework_DBus** supports both synchronous and asynchronous testing approaches. You can choose the appropriate mechanism based on your testing requirements.

Whether you are testing a single Dbus service or a complex system of interconnected services, **RobotFramework_DBus** offers a reliable and efficient solution for Dbus automation testing. It empowers you to ensure the quality and robustness of your Dbus-based applications.

Chapter 2

Description

2.1 Prerequisites

Before using **RobotFramework_DBus**, make sure you have the following libraries installed:

- **pycairo**: This library provides Python bindings for the Cairo graphics library. It is used for rendering graphics and creating visual elements in your applications.
- **PyGObject**: PyGObject is a Python package that provides bindings for the GObject library. It allows you to use GObject-based libraries, such as GTK+, in Python applications.
- **dbus**: **RobotFramework_DBus** is built upon the *dbus* library, which is a Pythonic D-Bus library. Make sure you have *dbus* installed before using **RobotFramework_DBus**.
- **pyinstaller**: PyInstaller is a tool used to package Python applications into standalone executables. If you plan to distribute your application as an executable, you will need to have *pyinstaller* installed.

Ensure that the above libraries are installed and properly configured in your Python environment before using **robotframework-dbus**. This will ensure the smooth functioning and compatibility of the library with your system.

2.2 Getting Started

You can checkout all [robotframework-dbus](#) sourcecode from the [GitHub](#).

After checking out the source completely, you can install by running below command inside **robotframework-dbus** directory.

```
python setup.py install
```

2.3 Usage

RobotFramework_DBus Library support following keywords for testing connection in RobotFramework.

2.3.1 connect

Use for establishing a connection.

Syntax:

```
connect      conn_name=[connection name]
...          namespace=[namespace]
...          object_path=[object path]
...          mode=[test mode]
...          host=[remote host]
...          port=[remote port]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

namespace: The namespace of the Dbus service. This identifies the specific service or group of services. It is used to differentiate between different service instances. The namespace should be a string that uniquely identifies the service.
E.g. namespace=org.example.HelloWorld

object_path: The object path of the Dbus service. This identifies the specific object within the service that the action will be performed on. The object path should be a string that follows the Dbus object path naming convention. It typically consists of a hierarchical structure separated by slashes (/).
E.g. object_path=/org/example/HelloWorld

mode: The mode of testing the Dbus service. Possible values are 'local' or 'remote'. 'local' indicates testing on the current system, while 'remote' indicates testing on a remote system. Default is 'local'.

host: The IP address or hostname of the remote system where the Dbus agent is running. This parameter is applicable only if 'mode' is set to 'remote'. Default is 'localhost'.

2.3.2 disconnect

Use for disconnecting from the Dbus service by connection name.

Syntax:

```
disconnect          conn_name
```

Arguments:

conn_name: The name or identifier of the connection instance to disconnect from. This parameter is optional and can be used to specify a specific connection to disconnect. If the connection name is 'ALL', all connections will be disconnected.

2.3.3 set signal received handler

Use to set a signal received handler for a specific Dbus connection and signal.

Syntax:

```
set signal received handler    conn name=[conn name]
...                           signal=[signal name]
...                           handler=[keyword handles signal emitted event]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

signal: The name of the Dbus signal to be set emitted handler.

handler: The robotframework keyword to handle the received signal. The handler should accept the necessary parameters based on the signal being handled.

2.3.4 unset signal received handler

Use to unset a signal received handler for a specific signal.

Syntax:

```
unset signal received handler    conn name=[conn name]
...                             signal=[signal name]
...                             handler=[keyword handles signal emitted event]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

signal: The name of the Dbus signal to be unset emitted handler.

handler: The robotframework keyword which is handling the signal emitted event.

2.3.5 register signal

Use to register a Dbus signal or signals to be monitored for a specific connection.

Syntax:

```
register signal                  conn name=[conn name]
...                             signal=[signal name]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

signal: The name of the Dbus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

2.3.6 call dbus method

Use to call a Dbus method with the specified method name and input arguments.

Syntax:

```
call dbus method                [conn name]    [method name]    [args]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

method_name: The name of the Dbus method to be called.

args: Input arguments to be passed to the method.

Return value:

Return from called method.

2.3.7 wait for signal

Use to wait for a specific DBus signal to be received within a specified timeout period.

Syntax:

```
wait for signal      conn name=[conn name]
...                  signal=[signal name]
...                  timeout=[timeout]
```

Arguments:

conn_name: The name or identifier of the connection instance used to interact with the DBus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used. Default is 'default_conn'.

signal: The name of the DBus signal to wait for.

timeout: The maximum time (in seconds) to wait for the signal.

Return value:

The signal payloads.

2.4 Remote testing

After installing the library, you can find the DBus Client Agent at `/opt/rfwaio/python39/install/bin/`.

For remote testing, follow these steps:

1. Start the DBus Agent on the remote system by the following command:

```
dbus_client_agent [-h] [--host HOST] [--port PORT]
```

The DBus Client Agent supports the following command-line arguments:

- host (str, optional) The host where the agent is running. Default is 0.0.0.0.
- port (int, optional) The port where the agent is listening. Default is 2507.

2. On the host test PC, using the `connect` keyword with the `remote` mode and specify the correct host using the `host` parameter.
3. Use the other keywords in the same way as local testing.

2.5 Example

Kindly be advised that all the examples presented within this session are designed to interact with the DBus service `sample server.py` residing in the `atest/` directory.

2.5.1 Example 1 - Synchronized DBus Signal Waiting

Scenario: In this example, I will use the `wait for signal` keyword from the `RobotFramework_DBus` library to wait for a DBus signal to be emitted using the synchronized mechanism. This means that the keyword following `wait for signal` will only be executed after the signal is emitted, and the `wait for signal` keyword will return a value.

```
*** Settings ***
Library      RobotFramework_DBus.DBusManager

*** Test Cases ***
Hello World
    connect    conn_name=test_dbus
    ...        namespace=org.example.HelloWorld
    ...        mode=local

    ${ret}=    Wait For Signal    conn_name=test_dbus
    ...        signal=YellowMessage
    ...        timeout=10

    Log To Console    ${ret}

    Disconnect    test_dbus
```

Explanation: In the aforementioned example, we establish a connection to the DBus service `org.example.HelloWorld` using the `'connect'` keyword and name this connection `'test_dbus'`. Subsequently, the `'Wait for signal'` keyword will wait for the DBus service to emit the `YellowMessage` signal within a 10-second timeframe and return the payloads of this signal immediately upon its emission within the specified timeframe (failing if the timeout is exceeded). The content of the payloads will be printed to the console.

2.5.2 Example 2 - Managing Multiple DBus Signals with 'Register Signal' Keyword

Scenario: In Example 1, if we include an additional `Wait for signal` keyword to wait for the `GreenMessage` signal to be emitted after waiting for `YellowMessage`, there is a possibility of missing the occurrence of `GreenMessage` if it is emitted before `YellowMessage`. In such a scenario, we can utilize the `Register Signal` keyword to register `GreenMessage` in the watchlist. This ensures continuous monitoring of the signal starting immediately after executing this keyword and prevents the occurrence of missing the emitted `GreenMessage` event.

```
*** Settings ***
Library      RobotFramework_DBus.DBusManager

*** Test Cases ***
Hello World
    connect    conn_name=test_dbus
    ...        namespace=org.example.HelloWorld
    ...        mode=local

    Register Signal    conn_name=test_dbus    signal=GreenMessage

    ${ret}=    Wait For Signal    conn_name=test_dbus
    ...        signal=YellowMessage
    ...        timeout=10
    Log To Console    ${ret}

    ${ret}=    Wait For Signal    conn_name=test_dbus
    ...        signal=GreenMessage
    ...        timeout=10
    Log To Console    ${ret}

    ${ret}=    Call Dbus Method    test_dbus    Hello    World
    Disconnect    test_dbus
```

Explanation:

In the above example, by using the `Register Signal` keyword, we have registered the `GreenMessage` signal in the watchlist. If the `GreenMessage` event is emitted during the `Wait for signal YellowMessage` timeframe, it will be

recorded. Then, when we execute the `Wait for signal` keyword for **GreenMessage**, it will immediately return because the **GreenMessage** event has already occurred.

2.5.3 Example 3 - Non-blocking DBus Signal Handling with 'Set Signal Received Handler'

Scenario: In the two examples above, the testcase is blocked at the `Wait for Signal` keyword until the signal event is emitted. In the next example, we will explore the usage of the `Set Signal Received Handler` keyword. This is a non-blocking keyword that allows us to register a user-defined keyword, such as a callback function. The registered keyword will be called when the emitted signal event occurs. The testcase will continue with the next keyword immediately after the `Set Signal Received Handler` keyword, without waiting.

```
*** Settings ***
Library      RobotFramework_DBus.DBusManager
Library      test/dbus/client.py

*** Test Cases ***
Hello World
    connect    conn_name=test_dbus
    ...        namespace=org.example.HelloWorld
    ...        mode=local

    Set Signal Received Handler    conn_name=test_dbus
    ...                            signal=RedMessage
    ...                            handler=On Received Red Signal

    Log To Console    The test is continuing...

    Sleep    10s

    Disconnect    test_dbus

*** Keyword ***
On Received Red Signal
    [Arguments]    ${arg1}=default 1
    log to console    Client received red signal. Payload: ${arg1}
    Unset Signal Received Handler    conn_name=test_dbus    signal=RedMessage    handler=↔
    ↔ On Received Red Signal
```

Explanation:

In the above example, by using the `Set Signal Received Handler` keyword, we have registered the user-defined keyword `On Received Red Signal` as a callback function. When the **RedMessage** is emitted, this keyword will be executed. During the time when the **RedMessage** has not been emitted yet, the testcase will continue running the subsequent keywords as usual. It is important to note that the user-defined keyword should have an argument to handle the payload of the signal. If there is no longer a need to invoke the callback function, we should unregister the signal handler using the `Unset Signal Received Handler` keyword.

2.5.4 Example 4 - Invoking DBus Methods: Using 'Call Dbus Method' Keyword

Scenario:

In the next example, we will explore how to use the `Call Dbus Method` keyword to invoke a method in the DBus service and retrieve the return value from this method.

```
*** Settings ***
Library      RobotFramework_DBus.DBusManager
Library      test/dbus/client.py

*** Test Cases ***
Hello World
    connect    conn_name=test_dbus
    ...        namespace=org.example.HelloWorld
    ...        mode=local

    ${ret}=    Call Dbus Method    test_dbus    Hello    World
    Log To Console    ${ret}

    Disconnect    test_dbus
```

2.5.5 Example 5 - Testing a Remote DBus Service: Setup and Steps

Scenario: In the next example, we will explore how to test a DBus service on a different PC. To be able to test on a remote system, we need to perform two steps:

1. On the System Under Test (SUT), run the following command:

```
dbus_client_agent 0.0.0.0 2507
```

The DBus Client Agent supports the following command-line arguments:

- host (str, optional) The host where the agent is running. Default is 0.0.0.0.
- port (int, optional) The port where the agent is listening. Default is 2507.

2. On the test PC, make slight modifications to the keyword's connect parameters as follows:

```
*** Settings ***
Library      RobotFramework_DBus.DBusManager
Library      test/dbus/client.py

*** Test Cases ***
Hello World
    connect    conn_name=test_dbus
    ...        namespace=org.example.HelloWorld
    ...        mode=remote
    ...        host=172.17.0.2
    ...        port=2507

    Set Signal Received Handler    conn_name=test_dbus
    ...                            signal=RedMessage
    ...                            handler=On Received Red Signal

    Log To Console    The test is continuing...

    Sleep    10s

    Disconnect    test_dbus

*** Keyword ***
On Received Red Signal
    [Arguments]    ${arg1}=default 1
    log to console    Client received red signal. Payload: ${arg1}
    Unset Signal Received Handler    conn_name=test_dbus    signal=RedMessage    handler=↔
    ↔ On Received Red Signal
```

Explanation:

In the above example, we set the value of the mode parameter for the `connect` keyword to `remote`, and add two parameters: `host` and `port`. The `host` parameter is set to the IP address of the System Under Test (SUT), which is 172.17.0.2, and the `port` parameter is set to 2507, which is the port that the agent is listening on.

2.6 Configure Git and correct EOL handling

Here you can find the references for [Dealing with line endings](#).

Every time you press return on your keyboard you're actually inserting an invisible character called a line ending. Historically, different operating systems have handled line endings differently. When you view changes in a file, Git handles line endings in its own way. Since you're collaborating on projects with Git and GitHub, Git might produce unexpected results if, for example, you're working on a Windows machine, and your collaborator has made a change in OS X.

To avoid problems in your diffs, you can configure Git to properly handle line endings. If you are storing the `.gitattributes` file directly inside of your repository, than you can assure that all EOL are manged by git correctly as defined.

2.7 Feedback

If you have any problem when using the library or think there is a better solution for any part of the library, I'd love to know it, as this will all help me to improve the library. Connect with me at cuong.nguyenhuyhtri@vn.bosch.com.

Do share your valuable opinion, I appreciate your honest feedback!

2.8 About

2.8.1 Maintainers

[Nguyen Huynh Tri Cuong](#)

2.8.2 Contributors

[Nguyen Huynh Tri Cuong](#)

[Thomas Pollerspoeck](#)

2.8.3 3rd Party Licenses

You must mention all 3rd party licenses (e.g. OSS) licenses used by your project here. Example:

Name	License	Type
Apache Felix.	Apache 2.0 License.	Dependency

2.8.4 Used Encryption

No encryption is used in this project.

2.8.5 License

This project is licensed under the Apache License, Version 2.0.

Copyright 2020-2026 Robert Bosch GmbH

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

CHAPTER 3. `__INIT__.PY`

Chapter 3

`__init__.py`

Class to manage all dbus communications.

Chapter 4

priority_queue.py

Chapter 5

register_keyword.py

A class that provides a keyword as a callback function for a DBus signal received.

5.1 Method: get_kw_name

Get the handle keyword name.

Returns:

/ Type: str /

The handle keyword name.

5.2 Method: callback_func

Constructor for RegisterKeyword class.

Arguments:

- `observer`

/ Condition: optional / Type: tuple / Default: None /

Input arguments to be passed to the callback method.

Returns:

(no returns)

Chapter 6

scheduled_job.py

A threaded job that executes a function at a specified interval.

6.1 Method: stop

Stop the execution of the job.

Returns:

(no returns)

6.2 Method: run

Start the job execution loop.

Returns:

(no returns)

Chapter 7

thread_safe_dict.py

This class provides a dictionary with thread-safe operations by utilizing locks for synchronized access.

7.1 Method: clear

Remove all key-value pairs from the dictionary.

Returns:

(no returns)

7.2 Method: pop

Remove and return the value associated with a given key.

Arguments:

- key
/ *Condition*: required / *Type*: Any /
The key of dictionary to be pop.

Returns:

- - / *Type*: Any /
Value of the given key.

7.3 Method: popitem

Remove and return an arbitrary key-value pair from the dictionary.

Returns:

(no returns)

7.4 Method: update

Update the dictionary with key-value pairs from another dictionary.

Arguments:

- dict
/ *Condition*: optional / *Type*: dict / *Default*: None /
Another dictionary-like object to update from.

Returns:
(no returns)

Chapter 8

utils.py

8.1 Class: Singleton

Class to implement Singleton Design Pattern. This class is used to derive the DBusManager as only a single instance of this class is allowed.

8.2 Class: DictToClass

Class for converting dictionary to class object.

8.3 Class: Utils

Class to implement utilities for supporting development.

8.3.1 Method: make_unique_token

Generates a unique session token of specified length.

The *make_unique_token* function generates a unique session token of the specified length. The session token can be used to identify and associate a session with a specific client.

The session token is a string value that is guaranteed to be unique for each invocation of this function. It can be used as a secure identifier to track and manage client sessions within the DBusAgent.

Arguments:

- `length`
/ *Condition*: optional / *Type*: int / *Default*: 16 /
The length of the session token. Defaults to 16.

Returns:

- `token`
/ *Type*: str /
A unique token.

8.3.2 Method: get_all_descendant_classes

Get all descendant classes of a class

Arguments:

- `cls`
/ *Condition*: required / *Type*: class /
Input class for finding children.

Returns:

/ *Type*: list /
 Array of descendant classes.

8.3.3 Method: get_all_sub_classes

Get all children classes of a class

Arguments:

- `cls`
 / *Condition*: required / *Type*: class /
 Input class for finding children.

Returns:

/ *Type*: list /
 Array of children classes.

8.3.4 Method: caller_name

Get a name of a caller in the format module.class.method

Arguments:

- `skip`
 / *Condition*: required / *Type*: int /

Specifies how many levels of stack to skip while getting caller name. `skip=1` means "who calls me", `skip=2` "who calls my caller" etc.

Returns:

/ *Type*: str /
 An empty string is returned if skipped levels exceed stack height

8.3.5 Method: load_library

Load native library depend on the calling convention.

Arguments:

- `path`
 / *Condition*: required / *Type*: str /
 Library path.
- `is_stdcall`
 / *Condition*: optional / *Type*: bool / *Default*: True /
 Determine if the library's calling convention is stdcall or cdecl.

Returns:

Loaded library object.

8.3.6 Method: is_ascii_or_unicode

Check if the string is ascii or unicode

Arguments: str_check: string for checking codecs: encoding type list

Returns:

/ *Type*: bool /

True : if checked string is ascii or unicode

False : if checked string is not ascii or unicode

Chapter 9

dbus_client_agent.py

9.1 Function: run_agent

Run the DBus Agent with the specified configuration.

Description:

The *run_agent* function starts the DBus Agent with the provided configuration. It parses the command-line arguments, such as the host and port options, and then starts the agent on the specified host and port.

The agent listens for incoming requests from clients and manages client sessions. It provides a mechanism to execute requests on corresponding DBus services through the assigned Executor instances.

Command-line Arguments:

--host (str, optional): The host where the agent is running. Default is '0.0.0.0'.

—port (int, optional): The port where the agent is listening. Default is 2507.

9.2 Class: DBusClientExecutor

The DBusClientExecutor class represents an executor responsible for handling client requests on specific DBus services. It receives requests from the DBusAgent and executes them on the corresponding DBus service.

9.2.1 Method: connect

Create a proxy object to DBus object.

Returns:

(no returns)

9.2.2 Method: disconnect

Disconnect the DBus proxy from the remote object.

Returns:

(no returns)

9.2.3 Method: quit

Quit the DBus client.

Returns:

(no returns)

9.2.4 Method: get_monitoring_signal_payloads

Get the payloads of a specific signal.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal to get payloads.

Returns:

- `payloads`
/ *Type*: str /
The signal's payloads.

9.2.5 Method: add_signal_to_captured_dict

Add a signal and its payloads to the captured dictionary when the signal be emitted.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal(s) which has been raised.
- `loop`
/ *Condition*: optional / *Type*: EventLoop / *Default*: None /
The Event loop which is running to wait for the raised signal.
- `payloads`
/ *Condition*: optional / *Type*: Any / *Default*: "" /
The payloads of the raised signal.

Returns:

(no returns)

9.2.6 Method: register_monitored_signal

Register a DBus signal or signals to be monitored for a specific connection.

Arguments:

- `signal`
/ *Condition*: optional / *Type*: str / *Default*: "" /
The name of the DBus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

Returns:

(no returns)

9.2.7 Method: wait_for_signal

Wait for a specific DBus signal to be received within a specified timeout period.

Arguments:

- `wait_signal`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus signal to wait for.
- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
The maximum time (in seconds) to wait for the signal.

Returns:

- `payloads`
/ *Type*: str /
The signal payloads.

9.2.8 Method: call_dbus_method

Call a DBus method with the specified method name and input arguments.

Arguments:

- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

- `mtd_ret`
/ *Type*: Any /
Return from called method.

9.3 Class: DBusClientAgent

The `DBusClientAgent` class acts as a mediator between clients and the corresponding DBus services they request. It manages client connections, session tokens, and assigns the appropriate `DBusClientExecutor` for each client session.

9.3.1 Method: get_session_token

Generates a unique session token for a client and returns it.

Returns:

- `token`
/ *Type*: str /
The random and unique token.

9.3.2 Method: initialize_dbus_client

Initializes an `DBusClientExecutor` instance for a specific client.

An `DBusClientExecutor` is an object responsible for executing requests from clients on corresponding DBus services. By initializing a separate `DBusClientExecutor` for each client, the `DBusClientAgent` ensures that requests from different clients are handled independently.

The client session token is a unique identifier that can be used to associate the DBusClientExecutor with the specific client. This allows the DBusClientAgent to route incoming requests to the correct DBusClientExecutor based on the client session token.

Arguments:

- `session`
/ Condition: required / Type: str /
 The client's session token.
- `namespace`
/ Condition: optional / Type: str / Default: "" /
 The namespace of the DBus service. This identifies the specific service or group of services. It is used to differentiate between different service instances. The namespace should be a string that uniquely identifies the service.
- `object_path`
/ Condition: optional / Type: str / Default: None /
 The object path of the DBus service. This identifies the specific object within the service that the action will be performed on. The object path should be a string that follows the DBus object path naming convention. It typically consists of a hierarchical structure separated by slashes (/).

Returns:

(no returns)

9.3.3 Method: connect

Create a proxy object to DBus service.

Returns:

(no returns)

9.3.4 Method: disconnect

Disconnect the DBus proxy from the remote object.

Returns:

(no returns)

9.3.5 Method: quit

Quit the DBus client.

Returns:

(no returns)

9.3.6 Method: get_monitoring_signal_payloads

Get the payloads of a specific signal.

Arguments:

- `session`
/ Condition: required / Type: str /
 The client's session token.
- `signal`
/ Condition: required / Type: str /
 The name of the DBus signal to get payloads.

Returns:

- `payloads`
/ *Type*: str /
The signal's payloads.

9.3.7 Method: register_monitored_signal

Register a DBus signal or signals to be monitored for a specific connection.

Arguments:

- `session`
/ *Condition*: required / *Type*: str /
The client's session token.
- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

Returns:

(no returns)

9.3.8 Method: wait_for_signal

Wait for a specific DBus signal to be received within a specified timeout period.

Arguments:

- `session`
/ *Condition*: required / *Type*: str /
The client's session token.
- `wait_signal`
/ *Condition*: optional / *Type*: str / *Default*: "" /
The name of the DBus signal to wait for.
- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
The maximum time (in seconds) to wait for the signal.

Returns:

- `payloads`
/ *Type*: str /
The signal payloads.

9.3.9 Method: call_dbus_method

Call a DBus method with the specified method name and input arguments.

Arguments:

- `session`
/ *Condition*: required / *Type*: str /
The client's session token.

- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

/ *Type*: Any /
Return from called method.

Chapter 10

dbus_client.py

A client class for interacting with a specific DBus service.

10.1 Method: connect

Create a proxy object to DBus object.

Returns:

(no returns)

10.2 Method: disconnect

Disconnect the DBus proxy from the remote object.

Returns:

(no returns)

10.3 Method: quit

Quit the DBus client.

Returns:

(no returns)

10.4 Method: set_signal_received_handler

Set a signal received handler for a specific signal.

Arguments:

- signal
/ *Condition*: required / *Type*: str /
The name of the DBus signal to handle.
- handler
/ *Condition*: required / *Type*: str /
The keyword to handle the received signal. The handler should accept the necessary parameters based on the signal being handled.

Returns:

(no returns)

10.5 Method: unset_signal_received_handler

Unset a signal received handler for a specific signal.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal to handle.
- `handle_keyword`
/ *Condition*: optional / *Type*: str / *Type*: None /
The keyword which is handling for signal emitted event.

Returns:

(no returns)

10.6 Method: add_signal_to_captured_dict

Add a signal and its payloads to the captured dictionary when the signal be emitted.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal(s) which has been raised.
- `loop`
/ *Condition*: optional / *Type*: EventLoop / *Default*: None /
The Event loop which is running to wait for the raised signal.
- `payloads`
/ *Condition*: optional / *Type*: Any / *Default*: "" /
The payloads of the raised signal.

Returns:

(no returns)

10.7 Method: register_monitored_signal

Register a DBus signal or signals to be monitored for a specific connection.

Arguments:

- `signal`
/ *Condition*: optional / *Type*: str / *Default*: "" /
The name of the DBus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

Returns:

(no returns)

10.8 Method: wait_for_signal

Wait for a specific Dbus signal to be received within a specified timeout period.

Arguments:

- `wait_signal`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the Dbus signal to wait for.
- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
The maximum time (in seconds) to wait for the signal.

Returns:

- `payloads`
/ *Type*: str /
The signal payloads.

10.9 Method: call_dbus_method

Call a Dbus method with the specified method name and input arguments.

Arguments:

- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the Dbus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

- / *Type*: Any /
Return from called method.

10.10 Method: call_dbus_method_with_keyword_args

Call a Dbus method with the specified method name and input arguments.

Arguments:

- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the Dbus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

- `ret_obj`
/ *Type*: Any /
Connection object.

Chapter 11

dbus_client_remote.py

A client class for interacting with a specific DBus service on a remote machine.

11.1 Method: connect

Create a proxy object to DBus object.

Returns:

(no returns)

11.2 Method: disconnect

Disconnect the DBus proxy from the remote object.

Returns:

(no returns)

11.3 Method: quit

Quit the DBus client.

Returns:

(no returns)

11.4 Method: do_signal_check

Checking if the signal was emitted.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal to check.
- `call_back_func`
/ *Condition*: required / *Type*: callable /
The function to be callback when receiving the signal.

Returns:

(no returns)

11.5 Method: set_signal_received_handler

Set a signal received handler for a specific signal.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal to handle.
- `handler`
/ *Condition*: required / *Type*: str /
The keyword to handle the received signal. The handler should accept the necessary parameters based on the signal being handled.

Returns:

(no returns)

11.6 Method: unset_signal_received_handler

Unset a signal received handler for a specific signal.

Arguments:

- `signal`
/ *Condition*: required / *Type*: str /
The name of the DBus signal to handle.

Returns:

(no returns)

11.7 Method: register_monitored_signal

Register a DBus signal or signals to be monitored for a specific connection.

Arguments:

- `signal`
/ *Condition*: optional / *Type*: str / *Default*: " " /
The name of the DBus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

Returns:

(no returns)

11.8 Method: wait_for_signal

Wait for a specific DBus signal to be received within a specified timeout period.

Arguments:

- `wait_signal`
/ *Condition*: optional / *Type*: str / *Default*: " " /
The name of the DBus signal to wait for.

- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
The maximum time (in seconds) to wait for the signal.

Returns:

- `payloads`
/ *Type*: str /
The signal payloads.

11.9 Method: call_dbus_method

Call a Dbus method with the specified method name and input arguments.

Arguments:

- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the Dbus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

- `ret_obj`
/ *Type*: Any /
Connection object.

Chapter 12

dbus__manager.py

Class to manage all DBus connections.

12.1 Method: quit

Quit connection manager.

Returns:

(no returns)

12.2 Method: add_connection

Add a connection to managed dictionary.

Arguments:

- name
/ *Condition*: required / *Type*: str /
Connection's name.
- conn
/ *Condition*: required / *Type*: DBusClient /
Connection object.

Returns:

(no returns)

12.3 Method: remove_connection

Remove a connection by name.

Arguments:

- connection_name
/ *Condition*: required / *Type*: str /
Connection's name.

Returns:

(no returns)

12.4 Method: get_connection_by_name

Get an exist connection by name.

Arguments:

- `connection_name`
/ *Condition*: required / *Type*: str /
Connection's name.

Returns:

- `conn`
/ *Type*: socket.socket /
Connection object.

12.5 Keyword: disconnect

Keyword for disconnecting a connection by name.

Arguments:

- `connection_name`
/ *Condition*: required / *Type*: str /
Connection's name.

Returns:

(no returns)

12.6 Keyword: connect

Keyword used to establish a Dbus connection.

Arguments:

- `conn_name`
/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /
The name or identifier of the connection instance used to interact with the Dbus service. This parameter is optional and can be used to uniquely identify a specific connection when multiple connections are established. If not provided, a default connection will be used.
- `namespace`
/ *Condition*: optional / *Type*: str / *Default*: "" /
The namespace of the Dbus service. This identifies the specific service or group of services. It is used to differentiate between different service instances. The namespace should be a string that uniquely identifies the service.
- `object_path`
/ *Condition*: optional / *Type*: str / *Default*: None /
The object path of the Dbus service. This identifies the specific object within the service that the action will be performed on. The object path should be a string that follows the Dbus object path naming convention. It typically consists of a hierarchical structure separated by slashes (/).
- `mode`
/ *Condition*: optional / *Type*: str / *Default*: 'local' /
The mode of testing the Dbus service. Possible values are 'local' or 'remote'. 'local' indicates testing on the current system, while 'remote' indicates testing on a remote system.

- `host`

/ *Condition*: optional / *Type*: str / *Default*: 'localhost' /

The IP address or hostname of the remote system where the Dbus agent is running.

This parameter is applicable only if *mode* is set to 'remote'.

- `port`

/ *Condition*: optional / *Type*: int / *Default*: 2507 /

The port number on which the Dbus agent is listening on the remote system.

This parameter is applicable only if *mode* is set to 'remote'.

Returns:

(no returns)

12.7 Keyword: set_signal_received_handler

Keyword used to set a signal received handler for a specific Dbus connection and signal.

Arguments:

- `conn_name`

/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /

The name of the Dbus connection.

- `signal`

/ *Condition*: optional / *Type*: str / *Default*: " /

The name of the Dbus signal to handle.

- `handler`

/ *Condition*: optional / *Type*: str / *Default*: None /

The keyword to handle the received signal. The handler should accept the necessary parameters based on the signal being handled.

Returns:

(no returns)

12.8 Keyword: unset_signal_received_handler

Keyword used to set a signal received handler for a specific Dbus connection and signal.

Arguments:

- `conn_name`

/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /

The name of the Dbus connection.

- `signal`

/ *Condition*: optional / *Type*: str / *Default*: " /

The name of the Dbus signal to handle.

- `handler`

/ *Condition*: optional / *Type*: str / *Default*: None /

The robotframework keyword which is handling the signal emitted event.

Returns:

(no returns)

12.9 Keyword: register_signal

Keyword used to register a DBus signal or signals to be monitored for a specific connection.

Arguments:

- `conn_name`
/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /
The name of the DBus connection.
- `signal`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus signal(s) to register. It can be a single signal name as a string, or multiple signal names joined by ','. For example: "signal1,signal2,signal3".

Returns:

(no returns)

12.10 Keyword: call_dbus_method

Keyword used to call a DBus method with the specified method name and input arguments.

Arguments:

- `conn_name`
/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /
The name of the DBus connection.
- `method_name`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus method to be called.
- `args`
/ *Condition*: optional / *Type*: tuple / *Default*: None /
Input arguments to be passed to the method.

Returns:

- `ret_obj`
/ *Type*: Any /
Return from called method.

12.11 Keyword: wait_for_signal

Keyword used to wait for a specific DBus signal to be received within a specified timeout period.

Arguments:

- `conn_name`
/ *Condition*: optional / *Type*: str / *Default*: 'default_conn' /
The name of the DBus connection.
- `signal`
/ *Condition*: optional / *Type*: str / *Default*: " /
The name of the DBus signal to wait for.

- `timeout`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
The maximum time (in seconds) to wait for the signal.

Returns:

- `payloads`
/ *Type*: str /
The signal payloads.

Chapter 13

Appendix

About robotframework-dbus:

Author

Nguyen Huynh Tri Cuong (cuong.nguyenhuynhtri@vn.bosch.com)

Version

0.2.0 (23.02.2026)

Short Description

Robot Framework library for automated testing of DBus services

Homepage

[robotframework-dbus](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

≥ 3.11

License

Apache-2.0

Chapter 14

History

0.1.0	05/2023
Initial version	
0.2.0	23.02.2026
Usage of Setuptools replaced by usage of PIP/TOML/Setuptools	

10.9 PrometheusInterface

PrometheusInterface

v. 0.9.0

Holger Queckenstedt

23.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Test setup	2
2.2	Installations	2
2.3	Configuration	3
2.4	Library import	3
2.5	Support of Prometheus metric types	4
2.5.1	Counters and gauges	4
3	prometheus_interface.py	5
3.1	Method: convert_to_int_or_float	5
3.2	Keyword: get_version	5
3.3	Keyword: who_am_i	5
3.4	Keyword: where_am_i	5
3.5	Keyword: get_port_number	5
3.6	Keyword: add_info	5
3.7	Keyword: set_info	6
3.8	Keyword: add_counter	6
3.9	Keyword: inc_counter	7
3.10	Keyword: add_gauge	7
3.11	Keyword: set_gauge	8
3.12	Keyword: inc_gauge	8
3.13	Keyword: dec_gauge	9
3.14	Keyword: add_summary	9
3.15	Keyword: observe_summary	10
3.16	Keyword: add_histogram	10
3.17	Keyword: observe_histogram	11
4	Appendix	12
5	History	13

Chapter 1

Introduction

The interface library **robotframework-prometheus** provides **Robot Framework** keywords to communicate with the monitoring system **Prometheus**. With the help of this interface it is possible to use **Prometheus** to monitor data provided by **Robot Framework** tests.

T.B.C.

Chapter 2

Description

2.1 Test setup

How many tests are passed? How many tests are failed? On which test benches do these tests run? Did any resets occur during test execution? How about the temporary unavailability of required test external components?

To monitor all these informations, a test system setup is necessary consisting of at least the following components:

1. A test framework that executes the test and provides the data to be monitored (here: the **Robot Framework**). This includes the possibility that more than one test framework runs in parallel.
2. A component that collects and stores data from all executed test frameworks (here: the monitoring system **Prometheus**).
3. A component that visualizes all data that have been collected (here: **Grafana**).

This is not the only way to establish such a test system, but in this document we concentrate on **Robot Framework**, **Prometheus** and **Grafana**.

To be able to collect values, **Prometheus** requires an http based counterpart. And the **Robot Framework** must be enabled to support this counterpart. In pure Python this is realized by a **Prometheus Python client library**. On **Robot Framework** level this is done by this component **robotframework-prometheus** that is a mapping between the interface of the **Prometheus Python client library** and **Robot Framework** keywords.

Or in other words: **robotframework-prometheus** uses the **Prometheus Python client library** to provide values to **Prometheus** to enable the data visualization in **Grafana**.

The **Prometheus Python client library** is part of the installation dependencies of **robotframework-prometheus**. **Prometheus** and **Grafana** are components that have to be downloaded, installed and configured separately.

2.2 Installations

1. Prometheus

To install **Prometheus** please visit the [homepage](#). This homepage also contains a *getting started* section containing useful hints about how to configure the application.

Further informations can also be found [here](#).

2. Grafana

The advantage of using **Grafana** for data visualization is to have a *ready to use* interface to **Prometheus** available. Other possible solutions are not in focus here.

To install **Grafana** please visit the [homepage](#).

How to create a Prometheus data source in Grafana, is described [here](#).

3. Prometheus Python client library

This library belongs to the installation dependencies of **robotframework-prometheus**. A separate installation is not required. In case you want to learn more about this client library please visit the following web pages: [\[1\]](#), [\[2\]](#), [\[3\]](#).

2.3 Configuration

Prometheus can be configured with a configuration file in YAML format. This includes the used port numbers. The example files in the **robotframework-prometheus** repository use the port numbers 8000, 8001 and 8002. Therefore the configuration file of **Prometheus** requires the following entry:

```
- targets: ["localhost:8000", "localhost:8001", "localhost:8002"]
```

2.4 Library import

After installation (see [README](#)), the interface library can be found within the `site-packages` that is the usual place for installed Python modules. It is recommended to introduce an environment variable to store the `site-packages` path, e.g. `ROBOTPYTHONSITEPACKAGESPATH`.

Now within robot files the interface library can be imported in the following way:

```
*** Settings ***
Library    ${ROBOTPYTHONSITEPACKAGESPATH}/PrometheusInterface/prometheus_interface.py
```

If nothing else is specified, the default port 8000 is used. It can be required to run several **Robot Framework** instances in parallel. In this case every instance needs it's own port number. Within the **Prometheus** YAML file we already have three port numbers defined (like described above).

Let's assume now we want to execute testsuite *A* parallel to testsuite *B*. Then we can assign port number 8001 to testsuite *A*:

```
*** Settings ***
Library    ${ROBOTPYTHONSITEPACKAGESPATH}/PrometheusInterface/prometheus_interface.py  ↔
↪ port_number=${8001}
```

and we assign port number 8002 to testsuite *B*:

```
*** Settings ***
Library    ${ROBOTPYTHONSITEPACKAGESPATH}/PrometheusInterface/prometheus_interface.py  ↔
↪ port_number=${8002}
```

Every testsuite needs it's own robot or resoure file in which this import happens!

To support a better readability of the test code we recommend to import the interface library with a certain name:

```
*** Settings ***
Library    ${ROBOTPYTHONSITEPACKAGESPATH}/PrometheusInterface/prometheus_interface.py  ↔
↪ port_number=${8001}    AS    rf.prometheus_interface
```

With `rf` is the abbreviation of **Robot Framework**.

2.5 Support of Prometheus metric types

2.5.1 Counters and gauges

The difference between a counter and a gauge is: A counter can be incremented only, whereas a gauge can be incremented, decremented and set to a certain value explicitly. Both counters and gauges have to be added before.

A counter is added in this way:

```
rf.prometheus_interface.add_counter    name=(name of counter)    description=(↔
↪ description of counter)    labels=(label names)
```

`name` and `description` are required, `labels` is optional.

Example:

We want to count *passed*, *failed* and *unknown* tests. A possible definition of counter can look like this:

```
rf.prometheus_interface.add_counter    name=num_passed    description=number of passed ↔
↪ tests    labels=room;testbench;testname
rf.prometheus_interface.add_counter    name=num_failed    description=number of failed ↔
↪ tests    labels=room;testbench;testname
rf.prometheus_interface.add_counter    name=num_unknown    description=number of unknown ↔
↪ tests    labels=room;testbench;testname
```

In this example we assume that several different tests are executed in several rooms and on several testbenches. It is not necessary to add a separate counter for every test on every testbench in every room. Only one counter (counting *passed* tests) is required, but with several labels. Every label will be a separate series of measurements of the corresponding counter. Every label is also a filter criteria in **Grafana** when configuring metrics.

During the lifetime of a testsuite a counter can be added only once. Therefore we suggest to place the adding into an `__init__.robot` file.

A counter can be incremented (by default value 1) in this way:

```
rf.prometheus_interface.inc_counter    name=(name)    labels=(label values)
```

A counter can also be incremented by a user defined value:

```
rf.prometheus_interface.inc_counter    name=(name)    value=(user defined increment)    ↔
↪ labels=(label values)
```

In `add_counter`, `labels` is a semicolon separated list of label *names*. In `inc_counter`, `labels` is a semicolon separated list of label *values*. Label names and label values must fit together in `add_counter` and `inc_counter`.

Example:

```
rf.prometheus_interface.inc_counter    name=num_passed    labels=Room_1;Testbench 2;↪
↪ Suite-A-Test-01
```

The same with gauges. A gauge is added in this way:

```
rf.prometheus_interface.add_gauge    name=(name of gauge)    description=(description of↪
↪ gauge)    labels=(label names)
```

`name` and `description` are required, `labels` is optional.

A gauge can e.g. be set to a certain value

```
rf.prometheus_interface.set_gauge    name=(name of gauge)    value=(value of gauge)    ↔
↪ labels=(label values)
```

As with counters, `labels` is a semicolon separated list of names or values.

Chapter 3

prometheus__interface.py

The class 'prometheus_interface' provides to communicate with the monitoring system Prometheus. For this purpose the 'Prometheus Python client library' is used.

3.1 Method: convert__to__int__or__float

Little helper to convert a string value to an integer or a float

3.2 Keyword: get__version

Returns the version of this interface library

3.3 Keyword: who__am__i

Returns the full name of this interface library

3.4 Keyword: where__am__i

Returns path to this interface library

3.5 Keyword: get__port__number

Returns the port number assigned to this instance of the library

3.6 Keyword: add__info

This keyword adds a new info. The content of an existing info can be defined with `set_info`.

Arguments:

- name
The name of the new info
/ *Condition*: required / *Type*: str /
- description
The description of the new info
/ *Condition*: required / *Type*: str /

- `labels`

A semicolon separated list of label names assigned to the new info

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.7 Keyword: set_info

This keyword defines the content of an info. The info has to be added with 'add_info' before.

Arguments:

- `name`

The name of the info

/ *Condition*: required / *Type*: str /

- `info`

The info itself (every info is a key-value information).

/ *Condition*: required / *Type*: dict /

- `labels`

A semicolon separated list of labels assigned to the info. The order of labels must fit to the order of label names like defined in `add_info`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.8 Keyword: add_counter

This keyword adds a new counter. The values of existing counters can be changed with `inc_counter`.

Arguments:

- `name`

The name of the new counter

/ *Condition*: required / *Type*: str /

- `description`

The description of the new counter

/ *Condition*: required / *Type*: str /

- labels

A semicolon separated list of label names assigned to the new counter

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- success

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- result

/ *Type*: str /

The result of the computation of the keyword

3.9 Keyword: inc_counter

This keyword increments a counter. The counter has to be added with 'add_counter' before.

Arguments:

- name

The name of the counter

/ *Condition*: required / *Type*: str /

- value

The value of increment. If not given, the value of the counter is incremented by value 1.

/ *Condition*: optional / *Type*: int / *Default*: None /

- labels

A semicolon separated list of labels assigned to the counter. The order of labels must fit to the order of label names like defined in add_counter.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- success

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- result

/ *Type*: str /

The result of the computation of the keyword

3.10 Keyword: add_gauge

This keyword adds a new gauge. The values of existing gauges can be changed with set_gauge, inc_gauge and dec_gauge.

Arguments:

- name

The name of the new gauge

/ *Condition*: required / *Type*: str /

- description

The description of the new gauge

/ *Condition*: required / *Type*: str /

- `labels`

A semicolon separated list of label names assigned to the new gauge

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.11 Keyword: `set_gauge`

This keyword sets the value for a gauge. The gauge has to be added with '`add_gauge`' before.

Arguments:

- `name`

The name of the gauge

/ *Condition*: required / *Type*: str /

- `value`

The new value of the gauge.

/ *Condition*: optional / *Type*: int / *Default*: None /

- `labels`

A semicolon separated list of labels assigned to the gauge. The order of labels must fit to the order of label names like defined in `add_gauge`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.12 Keyword: `inc_gauge`

This keyword increments a gauge. The gauge has to be added with '`add_gauge`' before.

Arguments:

- `name`

The name of the gauge

/ *Condition*: required / *Type*: str /

- `value`

The value of increment. If not given, the value of the gauge is incremented by value 1.

/ *Condition*: optional / *Type*: int / *Default*: None /

- labels

A semicolon separated list of labels assigned to the gauge. The order of labels must fit to the order of label names like defined in `add_gauge`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- success

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- result

/ *Type*: str /

The result of the computation of the keyword

3.13 Keyword: dec_gauge

This keyword decrements a gauge. The gauge has to be added with 'add_gauge' before.

Arguments:

- name

The name of the gauge

/ *Condition*: required / *Type*: str /

- value

The value of decrement. If not given, the value of the gauge is decremented by value 1.

/ *Condition*: optional / *Type*: int / *Default*: None /

- labels

A semicolon separated list of labels assigned to the gauge. The order of labels must fit to the order of label names like defined in `add_gauge`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- success

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- result

/ *Type*: str /

The result of the computation of the keyword

3.14 Keyword: add_summary

This keyword adds a new summary. The values of existing summaries can be set with `observe_summary``.

Arguments:

- name

The name of the new summary

/ *Condition*: required / *Type*: str /

- description

The description of the new summary

/ *Condition*: required / *Type*: str /

- `labels`

A semicolon separated list of label names assigned to the new summary

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.15 Keyword: `observe_summary`

This keyword observes a summary. The summary has to be added with '`add_summary`' before.

Arguments:

- `name`

The name of the summary

/ *Condition*: required / *Type*: str /

- `value`

The value assigned to the summary.

/ *Condition*: required / *Type*: int or float /

- `labels`

A semicolon separated list of labels assigned to the summary. The order of labels must fit to the order of label names like defined in `add_summary`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.16 Keyword: `add_histogram`

This keyword adds a new histogram. The values of existing histograms can be set with `observe_histogram``.

Arguments:

- `name`

The name of the new histogram

/ *Condition*: required / *Type*: str /

- `description`

The description of the new histogram

/ *Condition*: required / *Type*: str /

- `labels`

A semicolon separated list of label names assigned to the new histogram

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

3.17 Keyword: `observe_histogram`

This keyword observes a histogram. The histogram has to be added with '`add_histogram`' before.

Arguments:

- `name`

The name of the histogram

/ *Condition*: required / *Type*: str /

- `value`

The value assigned to the histogram.

/ *Condition*: required / *Type*: int or float /

- `labels`

A semicolon separated list of labels assigned to the histogram. The order of labels must fit to the order of label names like defined in `add_histogram`.

/ *Condition*: optional / *Type*: str / *Default*: None /

Returns:

- `success`

/ *Type*: bool /

Indicates if the computation of the keyword was successful or not

- `result`

/ *Type*: str /

The result of the computation of the keyword

Chapter 4

Appendix

About robotframework-prometheus:

Author

Holger Queckenstedt (Holger.Queckenstedt@de.bosch.com)

Version

0.9.0 (23.02.2026)

Short Description

Robot Framework keywords to communicate with the monitoring system Prometheus

Homepage

[robotframework-prometheus](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 5

History

0.1.0	05/2024
Initial version	
0.2.0	05/2024
Added metric type 'Gauge'; code maintenance	
0.3.0	05/2024
Added keywords 'inc_gauge' and 'dec_gauge'	
0.4.0	05/2024
Added interface description	
0.5.0	05/2024
Adapted handling of library version and date	
0.6.0	06/2024
Added metric type 'Info'	
0.6.1	06/2024
- Added prometheus-client as dependency package - Maintained package's repository and workflow files	
0.7.0	10/2024
Added metric types 'Summary' and 'Histogram'	
0.8.0	10/2024
"blank within key name" fix	
0.9.0	23.02.2026
Usage of Setuptools replaced by usage of PIP/TOML/Setuptools	

10.10 RobotFramework_UDS

RobotFramework_UDS

v. 0.2.0

Mai Minh Tri

23.02.2026

Contents

1	Introduction	1
2	Description	2
2.1	Overview	2
2.2	UDS Connector (DoIP)	2
2.3	Configuration	2
2.4	Supported UDS Services	2
2.5	Enhancements Usability with ODXTools Integration	2
2.6	Multiple Connections	4
2.7	Examples	5
3	DiagnosticServices.py	11
3.1	Class: DiagnosticServices	11
3.1.1	Method: convert_sub_param	11
3.1.2	Method: convert_request_data_type	11
3.1.3	Method: get_diag_service_by_name	12
3.1.4	Method: get_encoded_request_message	12
3.1.5	Method: get_decode_response_message	12
3.1.6	Method: get_full_positive_response_data	13
3.1.7	Method: get_did_codec	13
3.2	Class: PDXCodec	13
3.3	Class: ServiceID	13
4	UDSKeywords.py	14
4.1	Class: UDSDeviceManager	14
4.2	Class: UDSDevice	14
4.3	Class: UDSKeywords	14
4.3.1	Keyword: connect_uds_connector	14
4.3.2	Keyword: create_uds_connector	14
4.3.3	Keyword: load_pdx	15
4.3.4	Keyword: create_config	16
4.3.5	Keyword: get_uds_device_config_by_keys	17
4.3.6	Keyword: update_uds_device_config_by_keys	17
4.3.7	Keyword: set_config	18
4.3.8	Keyword: connect	18
4.3.9	Keyword: disconnect	18
4.3.10	Keyword: access_timing_parameter	18
4.3.11	Keyword: clear_diagnostic_information	19

CONTENTS

CONTENTS

4.3.12	Keyword: communication_control	19
4.3.13	Keyword: control_dtc_setting	20
4.3.14	Keyword: diagnostic_session_control	20
4.3.15	Keyword: dynamically_define_did	21
4.3.16	Keyword: ecu_reset	21
4.3.17	Keyword: io_control	21
4.3.18	Keyword: link_control	22
4.3.19	Keyword: read_data_by_identifier	23
4.3.20	Keyword: read_dtc_information	23
4.3.21	Keyword: read_memory_by_address	24
4.3.22	Keyword: request_download	24
4.3.23	Keyword: request_transfer_exit	24
4.3.24	Keyword: request_upload	25
4.3.25	Keyword: routine_control	25
4.3.26	Method: security_access	25
4.3.27	Keyword: tester_present	26
4.3.28	Keyword: transfer_data	26
4.3.29	Keyword: write_data_by_identifier	26
4.3.30	Keyword: write_memory_by_address	27
4.3.31	Keyword: request_file_transfer	27
4.3.32	Keyword: authentication	28
4.3.33	Keyword: routine_control_by_name	29
4.3.34	Keyword: read_data_by_name	29
4.3.35	Keyword: get_encoded_request_message	30
4.3.36	Keyword: get_decoded_positive_response_message	30
4.3.37	Keyword: write_data_by_name	30
4.3.38	Keyword: io_control_by_name	31
4.3.39	Keyword: send_uds_request_by_name	31
4.3.40	Keyword: reconnect	32
5	__init__.py	33
6	Appendix	34
7	History	35

Chapter 1

Introduction

The library **RobotFramework-UDS** provides a set of **Robot Framework** keywords for sending UDS (Unified Diagnostic Services) requests and interpreting responses from automotive electronic control units (ECUs).

Whether you're testing diagnostic sessions, reading data, or controlling routines on an ECU, the UDS Library simplifies these tasks by offering specific keywords like `DiagnosticSessionControl`, `ReadDataByIdentifier`, and `RoutineControl`.

These keywords are designed to handle the complexity of UDS communication, enabling you to write efficient and reliable automated tests.

Moreover, you can now refer to UDS services by their readable names rather than hexadecimal IDs e.g `ReadDataByName`, `RoutineControlByName`. It helps to make your tests more intuitive and easier to maintain.

Chapter 2

Description

2.1 Overview

The **RobotFramework-UDS** is designed to interface with automotive ECUs using the UDS protocol over the DoIP (Diagnostic over IP) transport layer. This library abstracts the complexities of UDS communication, allowing users to focus on writing high-level test cases that validate specific diagnostic services and responses.

2.2 UDS Connector (DoIP)

Currently, the library supports the **DoIP** (Diagnostic over IP) transport layer, which is commonly used in modern vehicles for diagnostic communication. DoIP allows for faster data transfer rates and easier integration with network-based systems compared to traditional CAN-based diagnostics.

2.3 Configuration

In order to connect and send/receive message properly using the **RobotFramework-UDS** certain configurations must be set up:

- **DoIP Configuration:** The library requires the IP address and port of the ECU or the gateway through which the ECU is accessed.
- **Data Identifiers and Codec:** Define the Data Identifiers (DIDs) and corresponding codecs in the library's configuration. This enables correct encoding and decoding of data between the test cases and the ECU.
- **Session Management:** Some UDS services may require the ECU to be in a specific diagnostic session (e.g., extended diagnostics). The library should be configured to manage these session transitions seamlessly.

2.4 Supported UDS Services

The **RobotFramework-UDS** library supports almost UDS service as defined in [ISO 14229](#), providing comprehensive coverage for ECU diagnostics.

For detailed information on specific services and how to use them, please refer to the next section.

2.5 Enhancements Usability with ODXTools Integration

The **RobotFramework-UDS** library comes with [odxtools](#) fully integrated, allowing you to use readable service names instead of dealing with hex IDs.

You can now specify service names directly in your test cases, making them more readable and user-friendly.

The **RobotFramework-UDS** also supports automatic data type conversion for request parameters, enabling users to provide input values that are seamlessly converted into the correct data types required for UDS requests.

The **RobotFramework-UDS** library supports features such as Routine Control By Name, Read Data By Name, and Write Data By Name, enabling users to send requests to UDS using the service's name instead of its identifier.

The **RobotFramework-UDS** provides a versatile "Send UDS Request By Name" keyword that can send and receive UDS messages without requiring the command type specification (e.g., Read, Write, RoutineControl). This allows users to send requests without needing to know the specific UDS command type associated with a service, as the keyword automatically detects and handles it.

Examples

Send UDS Request By Name

```
Test user can send UDS request without needing to specify the command type
Log    Test user can send UDS request without needing to specify the command type

Log    readCPUClockFrequencies_Read
${service_name_list}=    Create List    readCPUClockFrequencies_Read
${responses}=    Send UDS Request By Name    ${service_name_list}
Log    ${responses}    console=True

FOR    ${request_id}    IN    @{responses.keys()}
    Log    Key: ${request_id}, Value: ${responses["${request_id}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_id}"]}
    FOR    ${item}    IN    @{response.keys()}
        Log    ${item} : ${response["${item}"]}    console=True
    END
END

Log    CAM1PowerSupply_Set
${param_dict_input_output_control}=    Create Dictionary    mode=passiv
${response}=    Send UDS Request By Name    CAM1PowerSupply_Set    ${←
↪ param_dict_input_output_control}

Log    ${response}    console=True
FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END

Log    RealTimeClock_Write
${param_dict_write_data_by_name}=    Create Dictionary    Day=26    Month=September    ←
↪ Year=2024    Hour=10    Second=45    Minute=0
${res}=    Send UDS Request By Name    RealTimeClock_Write    ${←
↪ param_dict_write_data_by_name}
Log    ${res}    console=True

Log    Routine Control By Name service: StartIperfServer_Start

${param_dict_routine_control_by_name}=    Create Dictionary    port=5101    argument=←
↪ -i 0.5 -B 192.168.1.
${response}=    Send UDS Request By Name    StartIperfServer_Start    ${←
↪ param_dict_routine_control_by_name}

Log    ${response}    console=True
FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END
```

Routine Control By Name

```
Test user can use Routine Control By Name service on ECU
Log    Routine Control By Name service: StartIperfServer_Start

${param_dict}=    Create Dictionary    port=5101    argument=-i 0.5 -B 192.168.1.
${response}=    Routine Control By Name    StartIperfServer_Start    ${param_dict}

Log    ${response}    console=True
FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END
```

Read Data By Name

```

Test user can use Read Data By Name service on ECU
Log    Use Read Data By Name service

Log    Use Read Data By Identifier - 25392

${list_identifiers}=    Create List    0x6330
${res}=    Read Data By Identifier    ${list_identifiers}
Log    ${res}    console=True
Log    ${res[0x6330]}    console=True

Log    readCPUClockFrequencies_Read

${service_name_list}=    Create List    readCPUClockFrequencies_Read
${responses}=    Read Data By Name    ${service_name_list}
Log    ${responses}    console=True

FOR    ${request_did}    IN    @({responses.keys()})
    Log    Key: ${request_did}, Value: ${responses["${request_did}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_did}"]}
    FOR    ${item}    IN    @({response.keys()})
        Log    ${item} : ${response["${item}"]}    console=True
    END
END

Test Read List Services
${list_read_services}=    Create List    TestManager_SWVersion_Read
...                        CTSSWVersion_Read
...                        CPU_Load_Read

FOR    ${service_name}    IN    @({list_read_services})
    Log    Read Data of ${service_name}    console=True
    ${list_service}=    Create List    ${service_name}
    ${res}=    Read Data By Name    ${list_service}
    Log    ${res}    console=True
END

```

Write Data By Name

```

Test user can use Write Data By Name service on ECU
Log    Write Data By Name service

Log    RealTimeClock_Write
${PARAM_DICT}=    Create Dictionary    Day=26    Month=September    Year=2024    Hour↔
↔ =10    Second=45    Minute=0
${res}=    Write Data By Name    RealTimeClock_Write    ${PARAM_DICT}
Log    ${res}    console=True

${DICT}=    Create Dictionary    ipAddress=155
${res}=    Write Data By Name    CTS_IPAddress_Write    ${DICT}
Log    ${res}    console=True

Log    Using service did instead service's name
${res}=    Write Data By Identifier    25382    ${PARAM_DICT}
Log    ${res}    console=True

```

2.6 Multiple Connections

The **RobotFramework-UDS** can be extended to manage multiple connections simultaneously. This is beneficial when working with complex vehicle systems or simultaneously testing multiple ECUs.

2.7 Examples

Single Connection

```

*** Settings ***
Library      RobotFramework_TestsuitesManagement    AS    testsuites
Library      RobotFramework_UDS

*** Variables ***
${SUT_IP_ADDRESS}=      SUT_IP_ADDRESS
${SUT_LOGICAL_ADDRESS}= SUT_LOGICAL_ADDRESS
${TB_IP_ADDRESS}=       TB_IP_ADDRESS
${TB_LOGICAL_ADDRESS}=  TB_LOGICAL_ADDRESS
${ACTIVATION_TYPE}=     0
${DEVICE_NAME_1}=       UDS Connector 1
${FILE_1}=              path/file.pdx
${VARIANT_1}=           variant

*** Test Cases ***
Test user can connect single UDS connection
    Log      Test user can connect single UDS connection
    Log      If no device_name is provided, it will default to 'default'

    Create UDS Connector    ecu_ip_address= ${SUT_IP_ADDRESS_1}
    ...                    ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
    ...                    client_ip_address= ${TB_IP_ADDRESS_1}
    ...                    client_logical_address= ${TB_LOGICAL_ADDRESS_1}
    ...                    activation_type= ${ACTIVATION_TYPE_1}

    Connect UDS Connector
    Open UDS Connection
    Load PDX    ${FILE_1}    ${VARIANT_1}
    ${service_name_list}=    Create List    readCPUClockFrequencies_Read
    Read Data By Name    ${service_name_list}
    Close UDS Connection

```

Multiple Connections

```

*** Variables ***
${SUT_IP_ADDRESS_1}=      SUT_IP_ADDRESS_1
${SUT_LOGICAL_ADDRESS_1}= SUT_LOGICAL_ADDRESS_1
${TB_IP_ADDRESS_1}=       TB_IP_ADDRESS_1
${TB_LOGICAL_ADDRESS_1}=  TB_LOGICAL_ADDRESS_1
${ACTIVATION_TYPE_1}=     0
${DEVICE_NAME_1}=         UDS Connector 1
${FILE_1}=                path/file_1.pdx
${VARIANT_1}=             VARIANT_1

${SUT_IP_ADDRESS_2}=      SUT_IP_ADDRESS_2
${SUT_LOGICAL_ADDRESS_2}= SUT_LOGICAL_ADDRESS_2
${TB_IP_ADDRESS_2}=       TB_IP_ADDRESS_2
${TB_LOGICAL_ADDRESS_2}=  TB_LOGICAL_ADDRESS_2
${ACTIVATION_TYPE_2}=     0
${DEVICE_NAME_2}=         UDS Connector 2
${FILE_2}=                path/file_2.pdx
${VARIANT_2}=             VARIANT_2

${ERROR_STR}=    NegativeResponseException: ReadDataByIdentifier service execution ↩
                 ↩ returned a negative response IncorrectMessageLengthOrInvalidFormat (0x13)

*** Test Cases ***
Test user can connect multiple UDS connection
    Log      Test user can connect multiple UDS connection
    Log      Connect to device 1
    Create UDS Connector    device_name= ${DEVICE_NAME_1}
    ...                    ecu_ip_address= ${SUT_IP_ADDRESS_1}
    ...                    ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
    ...                    client_ip_address= ${TB_IP_ADDRESS_1}

```

CHAPTER 2. DESCRIPTION

2.7. EXAMPLES

```

...          client_logical_address= ${TB_LOGICAL_ADDRESS_1}
...          activation_type= ${ACTIVATION_TYPE_1}
Connect UDS Connector    device_name= ${DEVICE_NAME_1}

Open UDS Connection    device_name= ${DEVICE_NAME_1}
Load PDX    ${FILE_1}    ${VARIANT_1}    device_name= ${DEVICE_NAME_1}
${service_name_list_1}=    Create List    readCPUClockFrequencies_Read
Read Data By Name    ${service_name_list_1}    device_name= ${DEVICE_NAME_1}

Log    Connect to device 2
Create UDS Connector    device_name= ${DEVICE_NAME_2}
...          ecu_ip_address= ${SUT_IP_ADDRESS_2}
...          ecu_logical_address= ${SUT_LOGICAL_ADDRESS_2}
...          client_ip_address= ${TB_IP_ADDRESS_2}
...          client_logical_address= ${TB_LOGICAL_ADDRESS_2}
...          activation_type= ${ACTIVATION_TYPE_2}
Connect UDS Connector    device_name= ${DEVICE_NAME_2}

Open UDS Connection    device_name= ${DEVICE_NAME_2}
Load PDX    ${FILE_2}    ${VARIANT_2}    device_name= ${DEVICE_NAME_2}
${service_name_list_2}=    Create List    CPULoad_Read
Log    Expected device 2 cannot send readCPUClockFrequencies_Read service like device↵
↵ 1
Run Keyword And Expect Error    ${ERROR_STR}    Read Data By Name    ${↵
↵ service_name_list_1}    device_name= ${DEVICE_NAME_2}

Read Data By Name    ${service_name_list_2}    device_name= ${DEVICE_NAME_2}

Test user can connect multiple UDS connection but connect to the same ECU
Log    Test user can connect multiple UDS connection
Log    Connect to device 1
Create UDS Connector    device_name= ${DEVICE_NAME_1}
...          ecu_ip_address= ${SUT_IP_ADDRESS_1}
...          ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
...          client_ip_address= ${TB_IP_ADDRESS_1}
...          client_logical_address= ${TB_LOGICAL_ADDRESS_1}
...          activation_type= ${ACTIVATION_TYPE_1}
Connect UDS Connector    device_name= ${DEVICE_NAME_1}

Log    Open uds connection
Open UDS Connection    device_name= ${DEVICE_NAME_1}
Load PDX    ${FILE_1}    ${VARIANT_1}    device_name= ${DEVICE_NAME_1}
${service_name_list_1}=    Create List    readCPUClockFrequencies_Read
Read Data By Name    ${service_name_list_1}    device_name= ${DEVICE_NAME_1}

Log    Connect to device 2 but same IP as device 1
Log    The expected test case result in an error
Run Keyword And Expect Error    TimeoutError: ECU failed to respond in time    Create↵
↵ UDS Connector    device_name= ${DEVICE_NAME_2}
...          ↵
↵          ecu_ip_address= ${SUT_IP_ADDRESS_1}
...          ↵
↵          ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
...          ↵
↵          client_ip_address= ${TB_IP_ADDRESS_1}
...          ↵
↵          client_logical_address= ${TB_LOGICAL_ADDRESS_1}
...          ↵
↵          activation_type= ${ACTIVATION_TYPE_1}

Test users can reconnect to the closed ECU
Log    Connect to device 2
Create UDS Connector    device_name= ${DEVICE_NAME_2}
...          ecu_ip_address= ${SUT_IP_ADDRESS_2}
...          ecu_logical_address= ${SUT_LOGICAL_ADDRESS_2}
...          client_ip_address= ${TB_IP_ADDRESS_2}
...          client_logical_address= ${TB_LOGICAL_ADDRESS_2}
...          activation_type= ${ACTIVATION_TYPE_1}

```


CHAPTER 2. DESCRIPTION

2.7. EXAMPLES

```

Connect UDS Connector    device_name= ${DEVICE_NAME_2}

Open UDS Connection      device_name= ${DEVICE_NAME_2}
Load PDX    ${FILE_2}    ${VARIANT_2}    device_name= ${DEVICE_NAME_2}
${service_name_list_2}=    Create List    CPULoad_Read
Read Data By Name    ${service_name_list_2}    device_name= ${DEVICE_NAME_2}
Close UDS Connection    device_name= ${DEVICE_NAME_2}

Log    Connect to device 1
Create UDS Connector    device_name= ${DEVICE_NAME_1}
...    ecu_ip_address= ${SUT_IP_ADDRESS_1}
...    ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
...    client_ip_address= ${TB_IP_ADDRESS_1}
...    client_logical_address= ${TB_LOGICAL_ADDRESS_1}
...    activation_type= ${ACTIVATION_TYPE_1}
Connect UDS Connector    device_name= ${DEVICE_NAME_1}

Open UDS Connection      device_name= ${DEVICE_NAME_1}
Load PDX    ${FILE_1}    ${VARIANT_1}    device_name= ${DEVICE_NAME_1}
${service_name_list_1}=    Create List    readCPUClockFrequencies_Read
Read Data By Name    ${service_name_list_1}    device_name= ${DEVICE_NAME_1}
Close UDS Connection    device_name= ${DEVICE_NAME_1}

Log    Re-open uds connection device 2
Open UDS Connection      device_name= ${DEVICE_NAME_2}
Read Data By Name    ${service_name_list_2}    device_name= ${DEVICE_NAME_2}

Log    Expected device 2 cannot send readCPUClockFrequencies_Read service like device↵
↵ 1
Run Keyword And Expect Error    ${ERROR_STR}    Read Data By Name    ${↵
↵ service_name_list_1}    device_name= ${DEVICE_NAME_2}
Close UDS Connection    device_name= ${DEVICE_NAME_2}

```

Routine Control By Name

```

Test user can use Routine Control By Name service on ECU
Log    Routine Control By Name service: StartIperfServer_Start

${param_dict}=    Create Dictionary    port=5101    argument=-i 0.5 -B 192.168.1.
${response}=    Routine Control By Name    StartIperfServer_Start    ${param_dict}

Log    ${response}    console=True
FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END

Test user can use Routine Control By Name service if service have sub-service on ECU
Log    Routine Control By Name service: Routine_Control_Start, sub-service: ↵
↵ StartIperfServer_Start

${param_dict}=    Create Dictionary    port=5101    argument=-i 0.5 -B 192.168.1.
${response}=    Routine Control By Name    Routine_Control_Start    ${param_dict}    ↵
↵ sub_service=StartIperfServer

Log    ${response}    console=True
FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END

```

Input Output Control By Name

```

Test user can use Input Output Control By Name service on ECU
Log    Input Output Control By Name service: CAM1PowerSupply_Set

${param_dict}=    Create Dictionary    mode=passiv
${response}=    Input Output Control By Name    CAM1PowerSupply_Set    ${param_dict}

```

```

Log    ${response}    console=True
FOR    ${item}    IN    @({response.keys()})
    Log    ${item} : ${response["${item}"]}    console=True
END

```

Test user can use Input Output Control By Name service if the service has sub-service on ECU

```

Log    Input Output Control By Name service: IOControl_Control, sub_service:
internalFan_PWM
${param_dict}=    Create Dictionary    pwm=10
${response}=    Input Output Control By Name    IOControl_Control    ${param_dict}
sub_service=internalFan_PWM

```

```

Log    ${response}    console=True
FOR    ${item}    IN    @({response.keys()})
    Log    ${item} : ${response["${item}"]}    console=True
END

```

Read Data By Name

Test user can use Read Data By Name service on ECU

```

Log    Use Read Data By Name service

```

```

Log    Use Read Data By Identifier - 25392

```

```

${list_identifiers}=    Create List    0x6330
${res}=    Read Data By Identifier    ${list_identifiers}
Log    ${res}    console=True
Log    ${res[0x6330]}    console=True

```

```

Log    readCPUClockFrequencies_Read

```

```

${service_name_list}=    Create List    readCPUClockFrequencies_Read
${responses}=    Read Data By Name    ${service_name_list}
Log    ${responses}    console=True

```

```

FOR    ${request_id}    IN    @({responses.keys()})
    Log    Key: ${request_id}, Value: ${responses["${request_id}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_id}"]}
    FOR    ${item}    IN    @({response.keys()})
        Log    ${item} : ${response["${item}"]}    console=True
    END
END

```

Test Read List Services

```

${list_read_services}=    Create List    TestManager_SWVersion_Read
...                        CTSSWVersion_Read
...                        CPULoad_Read

```

```

FOR    ${service_name}    IN    @({list_read_services})
    Log    Read Data of ${service_name}    console=True
    ${list_service}=    Create List    ${service_name}
    ${res}=    Read Data By Name    ${list_service}
    Log    ${res}    console=True
END

```

Test user can use Read Data By Name service if the services have sub-service on ECU

```

${list_read}=    Create List    Identification_Read
${sub_service}=    Create List    CTSSWVersion
${dict_sub_service}=    Create Dictionary    Identification_Read=${sub_service}

${responses}=    Read Data By Name    ${list_read}    parameters=${dict_sub_service}

```

```

Log    ${responses}    console=True

```

```

FOR    ${request_id}    IN    @({responses.keys()})
    Log    Key: ${request_id}, Value: ${responses["${request_id}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_id}"]}

```

```

FOR    ${item}    IN    @{response.keys()}
    Log    ${item} : ${response["${item}"]}    console=True
END
END

Append To List    ${list_read}    readCPUClockFrequency_Read    ↵
↵ Temperature_Sensor_17_Read

${responses}=    Read Data By Name    ${list_read}    parameters=${dict_sub_service}

Log    ${responses}    console=True

FOR    ${request_did}    IN    @{responses.keys()}
    Log    Key: ${request_did}, Value: ${responses["${request_did}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_did}"]}
    FOR    ${item}    IN    @{response.keys()}
        Log    ${item} : ${response["${item}"]}    console=True
    END
END

${read}=    Create List    readCPUClockFrequency_Read    Identification_Read
${s_service}=    Create List    CTSSWVersion
${dict_sub_service}=    Create Dictionary    Identification_Read=${s_service}
${responses}=    Read Data By Name    ${read}    parameters=${dict_sub_service}

Log    ${responses}    console=True

FOR    ${request_did}    IN    @{responses.keys()}
    Log    Key: ${request_did}, Value: ${responses["${request_did}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_did}"]}
    FOR    ${item}    IN    @{response.keys()}
        Log    ${item} : ${response["${item}"]}    console=True
    END
END
END

```

Write Data By Name

```

Test user can use Write Data By Name service on ECU
Log    Write Data By Name service

Log    RealTimeClock_Write
${PARAM_DICT}=    Create Dictionary    Day=26    Month=September    Year=2024    Hour↵
↵ =10    Second=45    Minute=0
${res}=    Write Data By Name    RealTimeClock_Write    ${PARAM_DICT}
Log    ${res}    console=True

${DICT}=    Create Dictionary    ipAddress=155
${res}=    Write Data By Name    CTS_IPAddress_Write    ${DICT}
Log    ${res}    console=True

Log    Using service did instead service's name
${res}=    Write Data By Identifier    25382    ${PARAM_DICT}
Log    ${res}    console=True

```

Get Encoded Request Message

```

Test Get Encoded Request Message - Simple request parameters
Log    Test Get Encoded Request Message - mainCPUStressTest_Start
${service_name}=    Set Variable    mainCPUStressTest_Start
${param_dict}=    Create Dictionary    cores=5    load=50
${res}=    Get Encoded Request Message    ${service_name}    ${param_dict}
Log    ${res}    console=True
Log    ${res.hex()}    console=True

Test Get Encoded Request Message - Complex request parameters
Load PDX    C:/Users/MAR3HC/Desktop/UDS/robotframework-uds/test/pdx/XTS_MPCI_MaaS↵
↵ .23.45.pdx    XTS_MPCI_MaaS

```

```

Log      Test Get Encoded Request Message - CAN_MasterSlaveEnduranceRun_Start
${service_name}=    Set Variable    CAN_MasterSlaveEnduranceRun_Start
${res}=    Get Encoded Request Message    ${service_name}    ${canConfig}
Log      ${res}    console=True
Log      ${res.hex()}    console=True

```

Get Decoded Request Message

```

Test Get Decoded Response Message
Log      Test Get Decoded Response Message
${service_name}=    Set Variable    StartIperfServer_Start
${response_str}=    Set Variable    \x012#\x00\x00\x18\x05H
${response_byte}=    Convert To Bytes    ${response_str}
${res}=    Get Decoded Response Message    ${service_name}    ${response_byte}
Log      ${res}    console=True

```

Send UDS Request By Name

```

Test user can send UDS request without needing to specify the command type
Log      Test user can send UDS request without needing to specify the command type

Log      readCPUClockFrequencies_Read
${service_name_list}=    Create List    readCPUClockFrequencies_Read
${responses}=    Send UDS Request By Name    ${service_name_list}
Log      ${responses}    console=True

FOR      ${request_did}    IN    @{responses.keys()}
    Log      Key: ${request_did}, Value: ${responses["${request_did}"]}    console=True
    ${response}=    Set Variable    ${responses["${request_did}"]}
    FOR      ${item}    IN    @{response.keys()}
        Log      ${item} : ${response["${item}"]}    console=True
    END
END

Log      CAM1PowerSupply_Set
${param_dict_input_output_control}=    Create Dictionary    mode=passiv
${response}=    Send UDS Request By Name    CAM1PowerSupply_Set    ${↔}
↪ param_dict_input_output_control

Log      ${response}    console=True
FOR      ${item}    IN    @{response.keys()}
    Log      ${item} : ${response["${item}"]}    console=True
END

Log      RealTimeClock_Write
${param_dict_write_data_by_name}=    Create Dictionary    Day=26    Month=September    ↔
↪ Year=2024    Hour=10    Second=45    Minute=0
${res}=    Send UDS Request By Name    RealTimeClock_Write    ${↔}
↪ param_dict_write_data_by_name
Log      ${res}    console=True

Log      Routine Control By Name service: StartIperfServer_Start

${param_dict_routine_control_by_name}=    Create Dictionary    port=5101    argument=↔
↪ -i 0.5 -B 192.168.1.
${response}=    Send UDS Request By Name    StartIperfServer_Start    ${↔}
↪ param_dict_routine_control_by_name

Log      ${response}    console=True
FOR      ${item}    IN    @{response.keys()}
    Log      ${item} : ${response["${item}"]}    console=True
END

```

Chapter 3

DiagnosticServices.py

3.1 Class: DiagnosticServices

3.1.1 Method: convert_sub_param

Recursive convert sub parameters in given request to correct data type

Arguments:

- `odx_param`
/ *Condition*: required / *Type*: object /
The ODX parameters.
- `req_sub_param`
/ *Condition*: required / *Type*: dict /
The dictionary of request parameter.

Returns:

- `req_sub_param`
/ *Type*: dict /
The dictionary of request parameters with the correct data types.

3.1.2 Method: convert_request_data_type

Convert given request parameters (dictionary) to correct data type

Arguments:

- `service`
/ *Condition*: required / *Type*: object /
The diagnostic service.
- `parameter_dict`
/ *Condition*: required / *Type*: dict /
The dictionary of request parameter.

Returns:

- `parameter_dict`
/ *Type*: dict /
The dictionary of request parameters with the correct data types.

3.1.3 Method: get_diag_service_by_name

Retrieve the list of diagnostic services from a PDX file using a specified list of service names.

Arguments:

- `service_name_list`
/ *Condition*: required / *Type*: list /
The list of service names

Returns:

- `diag_service_list`
/ *Type*: list /
The list of diagnostic services from a PDX file.

3.1.4 Method: get_encoded_request_message

Retrieve the encode request message from parameters dictionary.

Arguments:

- `service_name`
/ *Condition*: required / *Type*: str /
The service's names
- `parameter_dict`
/ *Condition*: required / *Type*: dict /
The dictionary of request parameter.

Returns:

- `encode_message`
/ *Type*: bytes /
The encoded message.

3.1.5 Method: get_decode_response_message

Retrieve the encode request message from parameters dictionary.

Arguments:

- `service_name`
/ *Condition*: required / *Type*: str /
The service's names
- `raw_message`
/ *Condition*: required / *Type*: bytes /
The raw message from the response.

Returns:

- `decode_message`
/ *Type*: bytes /
The decoded message.

3.1.6 Method: `get_full_positive_response_data`

Retrieve the complete byte data from the response, as the UDS removes the service ID.

Arguments:

- `service_name`
/ *Condition*: required / *Type*: str /
The service's names
- `data`
/ *Condition*: required / *Type*: bytes /
The raw message from the response.

Returns:

- `positive_response_data`
/ *Type*: bytes /
The complete byte data from the response.

3.1.7 Method: `get_did_codec`

Retrieves a dictionary of DID codecs for a given diagnostic service ID.

Arguments:

- `service_id`
/ *Condition*: required / *Type*: int /
The service's did

Returns:

- `did_codec`
/ *Type*: dict /
A dictionary where the keys are DIDs

3.2 Class: PDXCodec

3.3 Class: ServiceID

Chapter 4

UDSKeywords.py

4.1 Class: UDSDeviceManager

4.2 Class: UDSDevice

4.3 Class: UDSKeywords

4.3.1 Keyword: connect_uds_connector

Connects a UDS connector for the specified device.

Arguments:

- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" /
Name of the device to connect to. If the device does not exist, a *ValueError* will be raised.
- `config`
/ *Condition*: optional / *Type*: dict / *Default*: `default_client_config` /
Configuration settings for the UDS client, applied if the device is not already available.
- `close_connection`
/ *Condition*: optional / *Type*: bool / *Default*: False /
Indicates whether to close the connection automatically when done.

Raises:

- `ValueError`
Raised if the specified device does not exist, suggesting the use of "Create UDS Connector" to create a new device.

Returns:

- `None`
No return value. The function initializes or updates the UDS connector for the specified device if not already available.

4.3.2 Keyword: create_uds_connector

Establishes a connection with an ECU.

Arguments:

- `communication_name`
Type: str / *Condition*: required /
 Specifies the type of communication to establish.
- `ecu_ip_address`
Type: str / *Condition*: required /
 The IP address of the ECU for establishing the connection. Should be a valid IPv4 (e.g., "192.168.1.1") or IPv6 address (e.g., "2001:db8::").
- `ecu_logical_address`
Type: any / *Condition*: required /
 The logical address of the ECU.
- `tcp_port`
Type: int / *Condition*: optional / *Default*: **TCP_DATA_UNSECURED** /
 TCP port used for unsecured data communication.
- `udp_port`
Type: int / *Condition*: optional / *Default*: **UDP_DISCOVERY** /
 UDP port used for ECU discovery.
- `activation_type`
Type: RoutingActivationRequest.ActivationType / *Condition*: optional / *Default*: ActivationTypeDefault /
 Specifies the activation type, which can be the default (ActivationTypeDefault) or a value based on application-specific settings.
- `protocol_version`
Type: int / *Condition*: optional / *Default*: 0x02 /
 The version of the protocol used for the connection.
- `client_logical_address`
Type: int / *Condition*: optional / *Default*: None /
 The logical address this DoIP client will use to identify itself. Per specification, this should be within the range 0x0E00 to 0x0FFF.
- `client_ip_address`
Type: str / *Condition*: optional / *Default*: None /
 If specified, binds to this IP as the source for UDP and TCP communication. Can be an IPv4 or IPv6 address, matching the type of `ecu_ip_address`.
- `use_secure`
Type: Union[bool, ssl.SSLContext] / *Condition*: optional / *Default*: False /
 Enables TLS if set to True. Uses a default SSL context by default; can be set to a preconfigured SSL context for more control. If enabled, consider changing `tcp_port` to 3496.
- `auto_reconnect_tcp`
Type: bool / *Condition*: optional / *Default*: False /
 Enables automatic reconnection of TCP sockets if closed by the peer.

4.3.3 Keyword: load_pdx

Load PDX file

Arguments:

- `pdx_file`
Type: str /
 PDX file path
- `variant`
Type: str /

4.3.4 Keyword: create_config

Creates a configuration for the UDS connector.

Arguments:

- `exception_on_negative_response : bool`
When set to True, raises a `NegativeResponseException` if the server responds with a negative response. If False, the Response's *positive* property will be set to False.
- `exception_on_invalid_response : bool`
When set to True, raises an `InvalidResponseException` if `interpret_response` encounters an invalid response. If False, the Response's *valid* property will be set to False.
- `exception_on_unexpected_response : bool`
When set to True, raises an `UnexpectedResponseException` if the server returns an unexpected response, such as an unmatched subfunction echo. If False, the Response's *unexpected* property will be set to True.
- `security_algo : Callable[[int, bytes, Any], bytes]`
Security algorithm function for the SecurityAccess service.
Signature: *security_algo(level, seed, params) -> bytes*
 - `level : int` — The requested security level.
 - `seed : bytes` — The seed provided by the server.
 - `params : Any` — Parameters provided by *security_algo_params*.
- `security_algo_params : object or dict`
Parameters passed to the security algorithm specified in `security_algo`.
- `data_identifiers : dict[int, Union[str, DidCodec]]`
A dictionary mapping data identifiers to a codec (string or `DidCodec`) for encoding/decoding values in services like `ReadDataByIdentifier`, `WriteDataByIdentifier`, etc.
- `input_output : dict[int, Union[str, DidCodec, dict]]`
Dictionary mapping IO data identifiers to a codec for `InputOutputControlByIdentifier` service. Supports composite codecs with sub-dictionaries specifying bitmasks.
- `tolerate_zero_padding : bool`
When True, ignores trailing zeros in response data to prevent `InvalidResponseException` if the protocol uses zero-padding.
- `ignore_all_zero_dtc : bool` For `ReadDTCInformation` service, skips DTCs with an ID of 0x000000, useful if the protocol uses zero-padding. See online documentation for further details.
- `server_address_format : int`
Specifies the `MemoryLocation` address format to use when not explicitly provided.
- `server_memorysize_format : int`
Specifies the `MemoryLocation` memory size format to use when not explicitly provided.
- `extended_data_size : dict[int, int]`
Specifies DTC extended data record sizes.

Example:

```
{
    0x123456: 45, # DTC 0x123456 has an extended data size of 45 bytes.
    0x123457: 23 # DTC 0x123457 has an extended data size of 23 bytes.
}
```

- `dtc_snapshot_did_size : int`
Number of bytes for encoding data identifiers in `ReadDTCInformation` (default: 2).

- `standard_version : int`
UDS standard version, valid values are 2006, 2013, or 2020 (default: 2020).
- `request_timeout : float`
Maximum wait time (in seconds) for a response after sending a request. Defaults to 5 seconds. Set to None to wait indefinitely.
- `p2_timeout : float`
Maximum wait time (in seconds) for a first response after sending a request, per ISO 14229-2:2013 (default: 1 second).
- `p2_star_timeout : float`
Maximum wait time (in seconds) after receiving a requestCorrectlyReceived-ResponsePending (0x78) response from the server (default: 5 seconds).
- `use_server_timing : bool`
When True, uses P2 and P2* timing values provided by the server for sessions with 2013 or later standards. Defaults to True.

4.3.5 Keyword: `get_uds_device_config_by_keys`

Retrieves the UDS device configuration for the specified keys.

Arguments:

- `keys`
/ *Condition*: optional / *Type*: list / A list of configuration keys to retrieve from the UDS device. If no keys are provided, the full configuration will be returned.
- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" / The name of the device for which the configuration should be retrieved.

Returns:

- `configs`
/ *Type*: dict / A dictionary containing the configuration values for the specified keys.

4.3.6 Keyword: `update_uds_device_config_by_keys`

Updates the UDS device configuration by multiple keys.

Arguments:

- `keys_dict`
/ *Condition*: required / *Type*: dict / A dictionary where keys are configuration keys and values are the new values to set for those keys.
- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" / The name of the device for which the configuration should be updated. If the device does not exist, a *ValueError* will be raised.

Returns:

- No return value. The method updates the configuration of the specified UDS device with the provided key-value pairs.

4.3.7 Keyword: set_config

This method sets the UDS config.

Arguments:

- `config`
/ *Type*: Configuration /
The configuration to set for the UDS client.
- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" / The name of the device for which the configuration should be set. If the device does not exist, a *ValueError* will be raised.

Returns:

- No return value. The method updates the configuration of the specified UDS device with the provided configuration.

4.3.8 Keyword: connect

Opens a UDS connection.

Arguments:

- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" / The name of the device for which the configuration should be set. If the device does not exist, a *ValueError* will be raised.

Returns:

- No return value. The method opens the UDS connection for the specified device.

4.3.9 Keyword: disconnect

Closes a UDS connection.

Arguments:

- No specific arguments for this method.

4.3.10 Keyword: access_timing_parameter

Sends a generic request for AccessTimingParameter service.

Arguments:

- `access_type`
/ *Condition*: required / *Type*: int /
The service subfunction:
 - readExtendedTimingParameterSet = 1
 - setTimingParametersToDefaultValues = 2
 - readCurrentlyActiveTimingParameters = 3
 - setTimingParametersToGivenValues = 4
- `timing_param_record`
/ *Condition*: optional / *Type*: bytes /
The parameters data. Specific to each ECU.

Returns:

- `response`
/ *Type*: `Response` /
The response from the `AccessTimingParameter` service request.

4.3.11 Keyword: `clear_diagnostic_information`

Requests the server to clear its active Diagnostic Trouble Codes.

Arguments:

- `group`
/ *Type*: `int` /
The group of DTCs to clear. It may refer to Powertrain DTCs, Chassis DTCs, etc. Values are defined by the ECU manufacturer except for two specific values:
 - `0x000000` : Emissions-related systems
 - `0xFFFFFFFF` : All DTCs
- `memory_selection`
/ *Condition*: optional / *Type*: `int` /
MemorySelection byte (0-0xFF). This value is user-defined and introduced in the 2020 version of ISO-14229-1. Only added to the request payload when different from `None`. Default: `None`.

Returns:

- `response`
/ *Type*: `Response` /
The response from the server after attempting to clear the active Diagnostic Trouble Codes.

4.3.12 Keyword: `communication_control`

Switches the transmission or reception of certain messages on/off with `CommunicationControl` service.

Arguments:

- `control_type`
/ *Condition*: required / *Type*: `int` /
The action to request such as enabling or disabling some messages. This value can also be ECU manufacturer-specific:
 - `enableRxAndTx = 0`
 - `enableRxAndDisableTx = 1`
 - `disableRxAndEnableTx = 2`
 - `disableRxAndTx = 3`
 - `enableRxAndDisableTxWithEnhancedAddressInformation = 4`
 - `enableRxAndTxWithEnhancedAddressInformation = 5`
- `communication_type`
/ *Condition*: required / *Type*: `CommunicationType<CommunicationType>`, `bytes`, `int` /
Indicates what section of the network and the type of message that should be affected by the command. Refer to `CommunicationType<CommunicationType>` for more details. If an *integer* or *bytes* is given, the value will be decoded to create the required `CommunicationType<CommunicationType>` object.

- `node_id`
/ Condition: optional / Type: int /
DTC memory identifier (`nodeIdentificationNumber`). This value is user-defined and introduced in the 2013 version of ISO-14229-1. Possible only when control type is `enableRxAndDisableTxWithEnhancedAddressInformation` or `enableRxAndTxWithEnhancedAddressInformation`. Only added to the request payload when different from `None`. Default: `None`.

Returns:

- `response`
/ Type: Response /
The response from the `CommunicationControl` service request.

4.3.13 Keyword: `control_dtc_setting`

Controls some settings related to the Diagnostic Trouble Codes by sending a `ControlDTCSetting` service request. It can enable/disable some DTCs or perform some ECU-specific configuration.

Arguments:

- `setting_type`
/ Condition: required / Type: int /
Allowed values are from 0 to 0x7F:
 - `on = 1`
 - `off = 2`
 - `vehicleManufacturerSpecific = (0x40, 0x5F)` # For logging purposes only.
 - `systemSupplierSpecific = (0x60, 0x7E)` # For logging purposes only.
- `data`
/ Condition: optional / Type: bytes /
Optional additional data sent with the request called *DTCSettingControlOptionRecord*.

Returns:

- `response`
/ Type: Response /
The response from the `ControlDTCSetting` service request.

4.3.14 Keyword: `diagnostic_session_control`

Requests the server to change the diagnostic session with a `DiagnosticSessionControl` service request.

Arguments:

- `newsession`
/ Condition: required / Type: int /
The session to try to switch:
 - `defaultSession = 1`
 - `programmingSession = 2`
 - `extendedDiagnosticSession = 3`
 - `safetySystemDiagnosticSession = 4`

Returns:

- `response`
/ Type: Response /
The response from the `DiagnosticSessionControl` service request.

4.3.15 Keyword: dynamically_define_did

Defines a dynamically defined DID.

Arguments:

- `did`
/ Type: int /
 The data identifier to define.
- `did_definition`
/ Type: DynamicDidDefinition<DynamicDidDefinition> or MemoryLocation<MemoryLocation> /
 The definition of the DID. Can be defined by source DID or memory address. If a `MemoryLocation<MemoryLocation>` object is given, the definition will automatically be by memory address.

Returns:

- `response`
/ Type: Response /
 The response from the request to define the dynamically defined DID.

4.3.16 Keyword: ecu_reset

Requests the server to execute a reset sequence through the ECUReset service.

Arguments:

- `reset_type`
/ Condition: required / Type: int /
 The type of reset to perform:
 - `hardReset` = 1
 - `keyOffOnReset` = 2
 - `softReset` = 3
 - `enableRapidPowerShutDown` = 4
 - `disableRapidPowerShutDown` = 5

Returns:

- `response`
/ Type: Response /
 The response from the ECUReset service request.

4.3.17 Keyword: io_control

Substitutes the value of an input signal or overrides the state of an output by sending an `InputOutputControlByIdentifier` service request.

Arguments:

- `did`
/ Condition: required / Type: int /
 Data identifier to represent the IO.
- `control_param`
/ Condition: optional / Type: int /
 Control parameters:

- returnControlToECU = 0
- resetToDefault = 1
- freezeCurrentState = 2
- shortTermAdjustment = 3

- values

/ *Condition*: optional / *Type*: list, dict, IOValues<IOValues> /

Optional values to send to the server. This parameter will be given to DidCodec<DidCodec>.encode() method. It can be:

- A list for positional arguments
- A dict for named arguments
- An instance of IOValues<IOValues> for mixed arguments

- masks

/ *Condition*: optional / *Type*: list, dict, IOMask<IOMask>, bool /

Optional mask record for composite values. The mask definition must be included in config['input_output']. It can be:

- A list naming the bit mask to set
- A dict with the mask name as a key and a boolean setting or clearing the mask as the value
- An instance of IOMask<IOMask>
- A boolean value to set all masks to the same value.

Returns:

- response

/ *Type*: dict /

The decoded response data.

4.3.18 Keyword: link_control

Controls the communication baudrate by sending a LinkControl service request.

Arguments:

- control_type

/ *Condition*: required / *Type*: int /

Allowed values are from 0 to 0xFF:

- verifyBaudrateTransitionWithFixedBaudrate = 1
- verifyBaudrateTransitionWithSpecificBaudrate = 2
- transitionBaudrate = 3

- baudrate

/ *Condition*: required / *Type*: Baudrate<Baudrate> /

Required baudrate value when control_type is either verifyBaudrateTransitionWithFixedBaudrate (1) or verifyBaudrateTransitionWithSpecificBaudrate (2).

Returns:

- response

/ *Type*: Response /

The response from the LinkControl service request.

4.3.19 Keyword: read_data_by_identifier

Requests a value associated with a data identifier (DID) through the ReadDataByIdentifier service.

Arguments:

- `data_id_list`
/ *Type*: int | list[int] /
The list of DIDs to be read.

Returns:

- `response`
/ *Type*: Response /
The response from the ReadDataByIdentifier service request.

4.3.20 Keyword: read_dtc_information

Performs a ReadDiagnosticInformation service request.

Arguments:

- `subfunction`
/ *Condition*: required / *Type*: int /
The subfunction for the ReadDiagnosticInformation service.
- `status_mask`
/ *Condition*: optional / *Type*: int /
Status mask to filter the diagnostic information.
- `severity_mask`
/ *Condition*: optional / *Type*: int /
Severity mask to filter the diagnostic information.
- `dtc`
/ *Condition*: optional / *Type*: int | Dtc /
The Diagnostic Trouble Code to query. Can be an integer or a Dtc object.
- `snapshot_record_number`
/ *Condition*: optional / *Type*: int /
Snapshot record number to specify the snapshot to read.
- `extended_data_record_number`
/ *Condition*: optional / *Type*: int /
Extended data record number to specify the extended data to read.
- `extended_data_size`
/ *Condition*: optional / *Type*: int /
Size of the extended data to read.
- `memory_selection`
/ *Condition*: optional / *Type*: int /
Memory selection to specify the memory to be accessed.

Returns:

- `response`
/ *Type*: Response /
The response from the ReadDiagnosticInformation service request.

4.3.21 Keyword: read_memory_by_address

Reads a block of memory from the server by sending a ReadMemoryByAddress service request.

Arguments:

- `memory_location`
/ *Condition*: required / *Type*: MemoryLocation<MemoryLocation> /
The address and the size of the memory block to read.

Returns:

- `response`
/ *Type*: Response /
The response from the ReadMemoryByAddress service request.

4.3.22 Keyword: request_download

Informs the server that the client wants to initiate a download from the client to the server by sending a RequestDownload service request.

Arguments:

- `memory_location`
/ *Condition*: required / *Type*: MemoryLocation<MemoryLocation> /
The address and size of the memory block to be written.
- `dfi`
/ *Condition*: optional / *Type*: DataFormatIdentifier<DataFormatIdentifier> /
Optional defining the compression and encryption scheme of the data. If not specified, the default value of 00 will be used, specifying no encryption and no compression.

Returns:

- `response`
/ *Type*: Response /
The response from the RequestDownload service request.

4.3.23 Keyword: request_transfer_exit

Informs the server that the client wants to stop the data transfer by sending a RequestTransferExit service request.

Arguments:

- `data`
/ *Condition*: optional / *Type*: bytes /
Optional additional data to send to the server.

Returns:

- `response`
/ *Type*: Response /
The response from the RequestTransferExit service request.

4.3.24 Keyword: request_upload

Informs the server that the client wants to initiate an upload from the server to the client by sending a RequestUpload service request.

Arguments:

- `memory_location`
/ *Condition*: required / *Type*: `MemoryLocation<MemoryLocation>` /
The address and size of the memory block to be written.
- `dfi`
/ *Condition*: optional / *Type*: `DataFormatIdentifier<DataFormatIdentifier>` /
Optional defining the compression and encryption scheme of the data. If not specified, the default value of 00 will be used, specifying no encryption and no compression.

Returns:

- `response`
/ *Type*: `Response` /
The response from the RequestUpload service request.

4.3.25 Keyword: routine_control

Sends a generic request for the RoutineControl service.

Arguments:

- `routine_id`
/ *Condition*: required / *Type*: `int` /
The 16-bit numerical ID of the routine.
- `control_type`
/ *Condition*: required / *Type*: `int` /
The service subfunction. Valid values are:
 - `startRoutine` = 1
 - `stopRoutine` = 2
 - `requestRoutineResults` = 3
- `data`
/ *Condition*: optional / *Type*: `bytes` /
Optional additional data to give to the server.

Returns:

- `response`
/ *Type*: `Response` /
The response from the RoutineControl service request.

4.3.26 Method: security_access

Successively calls `request_seed` and `send_key` to unlock a security level with the SecurityAccess service. The key computation is done by calling `config['security_algo']`.

Arguments:

- `level`
/ *Condition*: required / *Type*: int /
The level to unlock. Can be the odd or even variant of it.
- `seed_params`
/ *Condition*: optional / *Type*: bytes /
Optional data to attach to the RequestSeed request (securityAccessDataRecord).

Returns:

- `response`
/ *Type*: Response /
The response from the SecurityAccess service request.

4.3.27 Keyword: tester_present

Sends a TesterPresent request to keep the session active.

Arguments:

- No specific arguments for this method.

Returns:

- `response`
/ *Type*: Response /
The response from the TesterPresent request.

4.3.28 Keyword: transfer_data

Transfers a block of data to/from the client to/from the server by sending a TransferData service request and returning the server response.

Arguments:

- `sequence_number`
/ *Condition*: required / *Type*: int /
Corresponds to an 8-bit counter that should increment for each new block transferred. Allowed values are from 0 to 0xFF.
- `data`
/ *Condition*: optional / *Type*: bytes /
Optional additional data to send to the server.

Returns:

- `response`
/ *Type*: Response /
The response from the TransferData service request.

4.3.29 Keyword: write_data_by_identifier

Requests to write a value associated with a data identifier (DID) through the WriteDataByIdentifier service.

Arguments:

- `did`
/ *Condition*: required / *Type*: int /
The DID to write its value.
- `value`
/ *Condition*: required / *Type*: dict /
Value given to the `DidCodec.encode` method. The payload returned by the codec will be sent to the server.

Returns:

- `response`
/ *Type*: Response /
The response from the `WriteDataByIdentifier` service request.

4.3.30 Keyword: write_memory_by_address

Writes a block of memory in the server by sending a `WriteMemoryByAddress` service request.

Arguments:

- `memory_location`
/ *Condition*: required / *Type*: `MemoryLocation<MemoryLocation>` /
The address and the size of the memory block to write.
- `data`
/ *Condition*: required / *Type*: bytes /
The data to write into memory.

Returns:

- `response`
/ *Type*: Response /
The response from the `WriteMemoryByAddress` service request.

4.3.31 Keyword: request_file_transfer

Sends a `RequestFileTransfer` request **Arguments:**

- `moop`
/ *Condition*: required / *Type*: int /
Mode of operation:
 - `AddFile` = 1
 - `DeleteFile` = 2
 - `ReplaceFile` = 3
 - `ReadFile` = 4
 - `ReadDir` = 5
 - `ResumeFile` = 6
- `path`
/ *Condition*: required / *Type*: str /
The path of the file or directory.
- `dfi`
/ *Condition*: optional / *Type*: `DataFormatIdentifier` /
`DataFormatIdentifier` defining the compression and encryption scheme of the data. Defaults to no compression and no encryption. Use for:

- AddFile = 1
- ReplaceFile = 3
- ReadFile = 4
- ResumeFile = 6

- `filesize`

/ *Condition*: optional / *Type*: int | Filesize /

The filesize of the file to write. If *Filesize*, uncompressed and compressed sizes will be encoded as needed. Use for:

- AddFile = 1
- ReplaceFile = 3
- ResumeFile = 6

Returns:

- `response`

/ *Type*: Response /

The response from the file operation.

4.3.32 Keyword: authentication

Sends an Authentication request introduced in 2020 version of ISO-14229-1. **Arguments:**

- `authentication_task`

/ *Condition*: required / *Type*: int /

The authentication task (subfunction) to use:

- deAuthenticate = 0
- verifyCertificateUnidirectional = 1
- verifyCertificateBidirectional = 2
- proofOfOwnership = 3
- transmitCertificate = 4
- requestChallengeForAuthentication = 5
- verifyProofOfOwnershipUnidirectional = 6
- verifyProofOfOwnershipBidirectional = 7
- authenticationConfiguration = 8

- `communication_configuration`

/ *Condition*: optional / *Type*: int /

Configuration about security in future diagnostic communication (vehicle manufacturer specific). Allowed values are from 0 to 255.

- `certificate_client`

/ *Condition*: optional / *Type*: bytes /

The certificate to verify.

- `challenge_client`

/ *Condition*: optional / *Type*: bytes /

Client challenge containing vehicle manufacturer-specific data or a random number.

- `algorithm_indicator`

/ *Condition*: optional / *Type*: bytes /

Algorithm used in Proof of Ownership (POWN). This is a 16-byte value containing the BER-encoded OID of the algorithm.

- `certificate_evaluation_id`
/ *Condition*: optional / *Type*: int /
Unique ID for evaluating the transmitted certificate. Allowed values are from 0 to 0xFFFF.
- `certificate_data`
/ *Condition*: optional / *Type*: bytes /
Certificate data for verification.
- `proof_of_ownership_client`
/ *Condition*: optional / *Type*: bytes /
Proof of Ownership of the challenge to be verified by the server.
- `ephemeral_public_key_client`
/ *Condition*: optional / *Type*: bytes /
Client's ephemeral public key for Diffie-Hellman key agreement.
- `additional_parameter`
/ *Condition*: optional / *Type*: bytes /
Additional parameter provided if required by the server.

Returns:

- `response`
/ *Type*: Response /
The server's response to the authentication request.

4.3.33 Keyword: routine__control__by__name

Sends a request for the RoutineControl service by routine name.

Arguments:

- param `routine_name` (required): Name of the routine
– type `routine_name`: str
- param `data` (optional): Optional additional data to give to the server
– type `data`: bytes

Returns:

- `response`
/ *Type*: Response /
The server's response to the RoutineControl request.

4.3.34 Keyword: read__data__by__name

Get diagnostic service list by a list of service names.

Arguments:

- param `service_name_list`: List of service names * type `service_name_list`: list[str]
- param `parameters`: Parameter list * type `parameters`: list[]

Returns:

- `response` / *Type*: Response / The server's response containing the diagnostic service list.

4.3.35 Keyword: get_encoded_request_message

Get diagnostic service encoded request (bytes value).

Arguments:

- param service_name: Diagnostic service's name * type service_name: string
- param parameters_dict: Parameter dictionary * type parameters_dict: dict

Returns:

- encoded_message / *Type*: bytes / The encoded message in bytes value.

4.3.36 Keyword: get_decoded_positive_response_message

Get diagnostic service decoded positive response message.

Arguments:

- param service_name: Diagnostic service's name * type service_name: string
- param response_data: Bytes data from the response * type parameters_dict: bytes
- param device_name: Name of the device * type device_name: string

Returns:

- decode_message / *Type*: dict / The decode message in dictionary.

4.3.37 Keyword: write_data_by_name

Requests to write a value associated with a name of service through the WriteDataByName service.

Arguments:

- service_name
/ *Condition*: required / *Type*: str /
The name to write its value.
- value
/ *Condition*: required / *Type*: dict /
Value given to the DidCodec.encode method. The payload returned by the codec will be sent to the server.
- sub_services / *Condition*: optional / *Type*: str /
A dictionary representing the sub-services of the main service, formatted as: { 'Name of main service': ['Name of sub-service'] }

Returns:

- response
/ *Type*: Response /
The response from the WriteDataByIdentifier service request.

4.3.38 Keyword: io_control_by_name

Sends a request for the IOControl service by name of input output control service.

Arguments:

- `io_control_name`
/ *Condition*: required / *Type*: str /
Name of the input output control service
- `value`
/ *Condition*: optional / *Type*: dict /
Optional additional data to give to the server
- `masks`
/ *Condition*: optional / *Type*: list, dict, IOMask<IOMask>, bool /
Optional mask record for composite values. The mask definition must be included in `config['input_output']`. It can be:
 - A list naming the bit mask to set
 - A dict with the mask name as a key and a boolean setting or clearing the mask as the value
 - An instance of IOMask<IOMask>
 - A boolean value to set all masks to the same value.

Returns:

- `response`
/ *Type*: dict /
The decoded response data.

4.3.39 Keyword: send_uds_request_by_name

Sends a UDS request by the name of the specified diagnostic service.

Arguments:

- `service_name`
/ *Condition*: optional / *Type*: str / *Default*: None /
Name of the diagnostic service to request.
- `device_name`
/ *Condition*: optional / *Type*: str / *Default*: "default" /
Name of the device to which the UDS request will be sent.
- `kwargs`
/ *Condition*: optional / *Type*: dict /
Additional parameters specific to certain services. Possible values include:
 - `reset_type`: (int) Reset type for ECU reset services.
 - `parameters`: (dict) Parameters for WRITE_DATA_BY_IDENTIFIER, INPUT_OUTPUT_CONTROL_BY_IDENTIFIER and ROUTINE_CONTROL services.
 - `mask`: (any) Mask value for the INPUT_OUTPUT_CONTROL_BY_IDENTIFIER service.
 - `groups`: (int) Group identifiers for the CLEAR_DIAGNOSTIC_INFORMATION service (default: 0xFFFF).
 - `memory_selection`: (any) Memory selection for the CLEAR_DIAGNOSTIC_INFORMATION service.
 - `control_type`: (int) Control type for COMMUNICATION_CONTROL service.
 - `communication_type`: (int) Communication type for COMMUNICATION_CONTROL service.

- node_id: (any) Node ID for COMMUNICATION_CONTROL service.
- setting_type: (int) Setting type for CONTROL_DTC_SETTING service.
- data: (any) Data for CONTROL_DTC_SETTING and TRANSFER_EXIT services.
- session_type: (int) Session type for DIAGNOSTIC_SESSION_CONTROL service.
- subfunction: (int) Subfunction for READ_DTC_INFORMATION service.
- status_mask: (any) Status mask for READ_DTC_INFORMATION service.
- severity_mask: (any) Severity mask for READ_DTC_INFORMATION service.
- dtc: (any) Diagnostic Trouble Code (DTC) for READ_DTC_INFORMATION service.
- snapshot_record_number: (any) Snapshot record number for READ_DTC_INFORMATION service.
- extended_data_record_number: (any) Extended data record number for READ_DTC_INFORMATION service.
- extended_data_size: (any) Extended data size for READ_DTC_INFORMATION service.
- memory_location: (any) Memory location for READ_MEMORY_BY_ADDRESS, REQUEST_DOWNLOAD, REQUEST_UPLOAD, and WRITE_MEMORY_BY_ADDRESS services.
- dfi: (any) Data Format Identifier (DFI) for REQUEST_DOWNLOAD and REQUEST_UPLOAD services.
- level: (any) Security level for SECURITY_ACCESS service.
- seed_params: (any) Seed parameters for SECURITY_ACCESS service.
- sequence_number: (int) Sequence number for TRANSFER_DATA service.
- data: (any) Data for TRANSFER_DATA and TRANSFER_EXIT services.

Returns:

- response
/ *Type*: Response /
The decoded response data from the service request.

4.3.40 Keyword: reconnect

Attempts to re-establish the connection.

Arguments:

- close_delay
/ *Condition*: optional / *Type*: float / *Default*: 2 second /
Time to wait between closing and re-opening socket
- device_name
/ *Condition*: optional / *Type*: str / *Default*: "default" /
Name of the device to which the UDS request will be sent.

Chapter 5

`__init__.py`

RobotFramework_UDS is a Robot Framework library aimed to provide UDP client to handle request/response.

Chapter 6

Appendix

About RobotFramework-UDS:

Author

Mai Minh Tri (tri.maiMinh@vn.bosch.com)

Version

0.2.0 (23.02.2026)

Short Description

Robot Framework keywords for UDS (Unified Diagnostic Services) communication

Homepage

[robotframework-uds](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 7

History

0.1.0	09/2024
<i>Initial version</i>	
0.1.4	09/2024
<i>- Update Write Data By Name keyword to return response with given service name instead of did.</i>	
0.1.5	10/2024
<i>- Update Read Data By Name keyword to return response with given service name instead of did.</i> <i>- Update Get Encoded Request Message keyword to support convert string to proper data type for request parameters.</i> <i>- Update Get Decoded Response Message keyword to support decoded response data.</i>	
0.1.6	10/2024
<i>Temporary solution for encoding byte field param with dynamic length</i>	
0.1.7	10/2024
<i>Improve the encoding function for byte data</i>	
0.1.8	10/2024
<i>- Remove temporary solution for encoding byte field param with dynamic length</i> <i>- Update package requirements to use odxtools version greater than 8.2.1</i>	
0.1.9	10/2024
<i>Add Input Output Control By Name keyword</i>	
0.1.10	10/2024
<i>Add Send UDS Request By Name keyword</i>	
0.1.11	02/2025
<i>Fix the type error that occurs when user utilize string parameters in the Create UDS Connector keyword</i>	
0.1.12	02/2025
<i>Update Read/Write data by identifier keywords can be used without the need for PDX file</i>	
0.1.13	03/2025
<i>Support new pdx format using TABLE-KEY</i>	
0.1.14	03/2025

CHAPTER 7. HISTORY

<i>Add the Reconnect keyword to enable users to attempt re-establishing the connection</i>	
0.1.15	05/2025
<i>Align Read data by name keyword's response to contain short name of service</i>	
0.1.16	08/2025
<i>Added keywords for managing UDS device configurations:</i> - Get UDS Device Config By Keys - Update UDS Device Config By Keys	
0.2.0	23.02.2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	

RobotFramework_ UDS.pdf*Created at 07.09.2026 - 09:47:14**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.11 RobotLog2RQM

RobotLog2RQM

v. 1.8.1

Tran Duy Ngoan

09.06.2026

CONTENTS

CONTENTS

Contents

1	Introduction	1
2	Description	2
2.1	Get Robot Framework XML result	2
2.2	Tool features	2
2.2.1	Usage	3
2.2.2	Basic import feature	3
2.2.3	Verify the given arguments	4
2.2.4	Import multiple *.xml result files	4
2.2.5	Create missing Test Case on RQM	4
2.2.6	Update existing Test Case on RQM	5
2.2.7	Test suite feature	5
2.2.8	Project with Configuration Management (CM)	5
2.2.9	Naming convention of generated resources	6
2.3	Robot Framework Test Case Information on RQM:	7
2.4	Additional Tool: RQMTool	9
2.4.1	Purpose	9
2.4.2	Usage	9
2.4.3	Examples	9
2.4.4	Notes	10
3	CRQM.py	11
3.1	Function: get_xml_tree	11
3.2	Class: Identifier	11
3.3	Class: CRQMClient	11
3.3.1	Method: login	12
3.3.2	Method: verifyProjectName	12
3.3.3	Method: disconnect	12
3.3.4	Method: config	12
3.3.5	Method: userURL	13
3.3.6	Method: integrationURL	13
3.3.7	Method: webIDfromResponse	14
3.3.8	Method: webIDfromGeneratedID	14
3.3.9	Method: getResourceByID	15
3.3.10	Method: getAllByResource	15
3.3.11	Method: getAllBuildRecords	15
3.3.12	Method: getAllConfigurations	15

CONTENTS

CONTENTS

3.3.13	Method: <code>getAllTeamAreas</code>	16
3.3.14	Method: <code>getTestArtifactsFromResource</code>	16
3.3.15	Method: <code>addTeamAreaNode</code>	16
3.3.16	Method: <code>createTestcaseTemplate</code>	17
3.3.17	Method: <code>createTCERTemplate</code>	18
3.3.18	Method: <code>createExecutionResultTemplate</code>	18
3.3.19	Method: <code>createBuildRecordTemplate</code>	19
3.3.20	Method: <code>createConfigurationTemplate</code>	20
3.3.21	Method: <code>createTSERTemplate</code>	20
3.3.22	Method: <code>createTestsuiteResultTemplate</code>	20
3.3.23	Method: <code>createTestsuiteTemplate</code>	21
3.3.24	Method: <code>createResource</code>	22
3.3.25	Method: <code>createBuildRecord</code>	22
3.3.26	Method: <code>createConfiguration</code>	23
3.3.27	Method: <code>updateResourceByID</code>	23
3.3.28	Method: <code>linkListTestcase2Resource</code>	24
3.3.29	Method: <code>linkListTestcase2Testplan</code>	24
3.3.30	Method: <code>linkListTestcase2Testsuite</code>	25
3.3.31	Method: <code>addTestsuite2Testplan</code>	25
4	logger.py	26
4.1	Method: <code>config</code>	26
4.2	Method: <code>log</code>	26
4.3	Method: <code>log_warning</code>	27
4.4	Method: <code>log_error</code>	27
5	robotlog2rqm.py	28
5.1	Function: <code>get_from_tags</code>	28
5.2	Function: <code>convert_to_datetime</code>	28
5.3	Function: <code>process_config_file</code>	29
5.4	Function: <code>is_valid_config</code>	29
5.5	Function: <code>process_suite_metadata</code>	29
5.6	Function: <code>process_metadata</code>	30
5.7	Function: <code>process_suite</code>	30
5.8	Function: <code>process_test</code>	31
5.9	Function: <code>RobotLog2RQM</code>	31
6	rqmtool.py	33
6.1	Function: <code>write_json_file</code>	33
6.2	Function: <code>write_csv_file</code>	33
6.3	Function: <code>write_output_file</code>	34
6.4	Function: <code>normalize_custom_attributes</code>	34
6.5	Function: <code>RQMTool</code>	34
7	Appendix	35
8	History	36

Chapter 1

Introduction

RobotLog2RQM facilitates the import of Robot Framework result file(s) in ***.xml** format into IBM® Rational® Quality Manager (RQM) resources.

It introduces the **CRQM Class**, offering the capability to interact with various RQM resources, including test plans, test cases, builds, and more, through the [RqmAPI](#) to:

- retrieve RQM resources: obtain resources by a given ID or retrieve all available entities of a specified resource type.
- update RQM resources: modify existing resources by providing the relevant ID.
- create new RQM resources: generate new resources using predefined templates located in the [RQM_templates](#) folder.

So that **RobotLog2RQM** tool can:

- create all required resources (*Test Case Excuton Record*, *Test Case Execution Result*, ...) for new test cases on RQM.
- link all test cases to provided test plan.
- add new test results for existing test cases on RQM.
- update existing test cases on RQM.

Chapter 2

Description

2.1 Get Robot Framework XML result

In order to manage test cases and their results Rational Quality Manager (RQM), certain traceable information, such as version, test case ID, component, etc., is required.

This enables the **robotframework-robotlog2rqm** tool to associate the imported test results with specific elements (Test Case) or link them to other entities (Build Record, Test Environment) in RQM.

These information should be provided in **Metadata** (for the whole testsuite/execution info: version, build, ...) and **[Tags]** information (for specific test case info: component, test case ID, requirement ID, ...) of Robot Framework test case. Then when executing Robot Framework test case(s), the generated Robot Framework result file (default is *output.xml*) will contain all of them and ready for importing.

Sample Robot Framework test case with the necessary information for importing to RQM:

```
*** Settings ***
Metadata  project      ROFW      # Test Environment on RQM for linking
Metadata  version_sw   SW_VERSION_0.1 # Build Record on RQM for linking
Metadata  machine      ${COMPUTERNAME} # Hostname attribute in RQM Test Case Result
Metadata  component    Import_Tools # Component attribute in RQM Test Case
Metadata  team-area    Internet Team RQM # team-area (case-sensitive) on RQM for linking

*** Test Cases ***
Testcase 01
[Documentation] This test is traceable with provided tcid
[Tags] TCID-1001 FID-112 FID-111 robotfile=https://github.com/test-fullautomation
Log      This is Testcase 01

Testcase 02
[Documentation] This new test case will be created if -createmissing argument
... is provided when importing
[Tags] FID-113 robotfile=https://github.com/test-fullautomation
Log      This is Testcase 02
```

Listing 2.1: Sample Robot Framework test case



Hint

In case you are using RobotFramework AIO, above highlighted **Metadata** definitions are not required because they have been handled by **RobotFramework_TestsuitesManagement** library within **Suite Setup**.

2.2 Tool features

After getting the Robot Framework **.xml* result file(s), you can use the **robotframework-robotlog2rqm** tool to import them into RQM.

Its usage and features are described as following sections.

2.2.1 Usage

Use below command to get tools's usage:

```
RobotLog2DB -h
```

The tool's usage should be showed as below:

```
usage: RobotLog2RQM (RobotXMLResult to RQM importer) [-h] [-v] [--testsuite TESTSUITE] [↔
↔ --recursive] [--createmissing] [--updatetestcase] [--dryrun] [--stream STREAM] [↔
↔ --baseline BASELINE]
↔                                     resultxmlfile host project user ↔
↔ password testplan

RobotLog2RQM imports XML result files (default: output.xml) generated by the Robot ↔
↔ Framework into an IBM Rational Quality Manager.

positional arguments:
  resultxmlfile          absolute or relative path to the xml result file or directory of ↔
↔ result files to be imported.
  host                   RQM host url.
  project                project on RQM.
  user                   user for RQM login.
  password               password for RQM login.
  testplan               testplan ID for this execution.

optional arguments:
  -h, --help             show this help message and exit
  -v, --version           Version of the RobotLog2RQM importer.
  --testsuite TESTSUITE  testsuite ID for this execution. If 'new', then create a new testsuite for this ↔
↔ execution.
  --recursive            if set, then the path is searched recursively for log files to be↔
↔ imported.
  --createmissing         if set, then all testcases without tcid are created when ↔
↔ importing.
  --updatetestcase       if set, then testcase information on RQM will be updated bases on↔
↔ robot testfile.
  --dryrun               if set, then verify all input arguments (includes RQM ↔
↔ authentication) and show what would be done.
  --stream STREAM        project stream. Note, requires Configuration Management (CM) to ↔
↔ be enabled for the project area.
  --baseline BASELINE    project baseline. Note, requires Configuration Management (CM), ↔
↔ or Baselines Only to be enabled for the project area.
```

As above instruction, **robotframework-robotlog2rqm** tool requires 5 positional arguments consists of:

- The Robot Framework result file/folder `resultxmlfile`
- The RQM authentication `host`, `project`, `user`, `password`
- The RQM `testplan` ID which will contains all importing test results

2.2.2 Basic import feature

Use the below command for the simple import the `output.xml` file to RQM project **ROBFW-AIO** which is hosted at <https://sample-rqm-host.com>

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720
```

When command is executed, the tool will process with following steps:

- Login the RQM server with the provided credential, then verify the existences of given `project`, `testplan` on RQM
- Create RQM **Build Record** and **Test Environment** (if already provided in Robot Framework test case and not existing on RQM)

- Create new RQM **Test Case Execution Record - TCER** (if it is not existing) bases on test case ID (defined `TCID-xxx` in `[Tags]` of Robot Framework Test Cases) and `testplan` ID
- Create new RQM **Test Case Execution Result** which contents the detail and result state of Robot Framework test case
- Link all test case(s) to provided `testplan`

2.2.3 Verify the given arguments

In case you just want to verify whether the given `*.xml` file/folder and the RQM authentication in arguments are corrected or not, the optional argument `--dryrun` will help to do it.

In the dryrun mode, **robotframework-robotlog2rqm** will not create any resources on RQM, it just verify:

- The given Robot Framework result file/folder is valid or not
- The given RQM authentication is correct or not
- The given RQM project and testplan are existing or not

2.2.4 Import multiple *.xml result files

robotframework-robotlog2rqm accepts the first argument `resultxmlfile` can be a single file or the folder that contains multiple Robot Framework result files.

When the folder is used, **robotframework-robotlog2rqm** will only search for `*.xml` file under given directory and exclude any file within subdirectories as default.

In case you have result file(s) under the subdirectory of given folder and want these result files will also be imported, the optional argument `--recursive` should be used when executing **robotframework-robotlog2rqm** command.

When `--recursive` argument is set, **robotframework-robotlog2rqm** will walk through the given directory and its subdirectories to discover and collect all available `*.xml` for importing.

For example: your result folder has a structure as below:

```
logFolder
|_____ result_1.xml
|_____ result_2.xml
|_____ subFolder_1
|           |_____ result_sub_1.xml
|           |_____ subSubFolder
|                   |_____ result_sub_sub_1.xml
|_____ subFolder_2
|           |_____ result_sub_2.xml
```

- Without `--recursive`: only `result_1.xml` and `result_2.xml` are found for importing.
- With `--recursive`: all `result_1.xml`, `result_2.xml`, `result_sub_1.xml`, `result_sub_2.xml` and `result_sub_sub_1.xml` will be imported.

2.2.5 Create missing Test Case on RQM

By default, **robotframework-robotlog2rqm** tool will not touch (create, update) any RQM Test Case.

If the `TCID-xxx` information is missing in `[Tags]` section of Robot Framework Test Case, an error message will be raised for that specific importing Test Case, and the tool will proceed with the next Test Cases accordingly

```
ERROR: There is no 'tcid' information for importing test 'Testcase 01'.
```

So that, in order to import those missing `TCID-xxx` Test Cases, the optional arguments `--createmissing` should be provided in the **robotframework-robotlog2rqm** arguments.

When `--createmissing` is used, **robotframework-robotlog2rqm** will help to create RQM Test Cases bases on the defined information in Robot Framework Test Cases. It obtains the new Test Case ID and uses it for linking to related RQM resources **TCER**, **Test Case Execution Result** as basic feature.

The new IDs for the created Test Cases are also displayed in the execution log. You can copy these IDs and update the `TCID-xxx` information in Robot Framework Test Cases for the next execution. This information will then be available in the generated `*.xml` result file for importing.

Please refer [Robot Framework Test Case Information on RQM](#) section for details on how the defined information in Robot Framework Test Cases is reflected in RQM.

2.2.6 Update existing Test Case on RQM

In case the Test Case is existing on RQM, but you want to update its attribute(s) such as **Component**, **Description**, ... the optional argument `--update testcase` should be used.

robotframework-robotlog2rqm will update RQM Test Case resource bases on the defined information in Robot Framework Test Case before creating its result.

Please refer [Robot Framework Test Case Information on RQM](#) section for details on how the defined information in Robot Framework Test Cases is reflected in RQM.

2.2.7 Test suite feature

In addition to managing the imported test results by testplan, the **robotframework-robotlog2rqm** tool also supports the use of testsuite with the optional argument `--testsuite TESTSUITE`.

This argument provides two main features:

- **Importing to an existing testsuite:** If you specify the ID of an existing testsuite using the `--testsuite TESTSUITE` argument, the tool will import the test results directly into that testsuite.

Example: testsuite with id `1234` is already existing

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720 ↵
↩ --testsuite 1234
```

- **Creating a new testsuite and then importing:** If you set the value to `new` using the `--testsuite new` argument, the tool will create a new testsuite and then import the test results into this newly created testsuite.

Example:

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720 ↵
↩ --testsuite new
```

When using the `--testsuite` argument, the tool will follow these additional steps besides the basic steps which is described in above [Basic import feature](#) section.

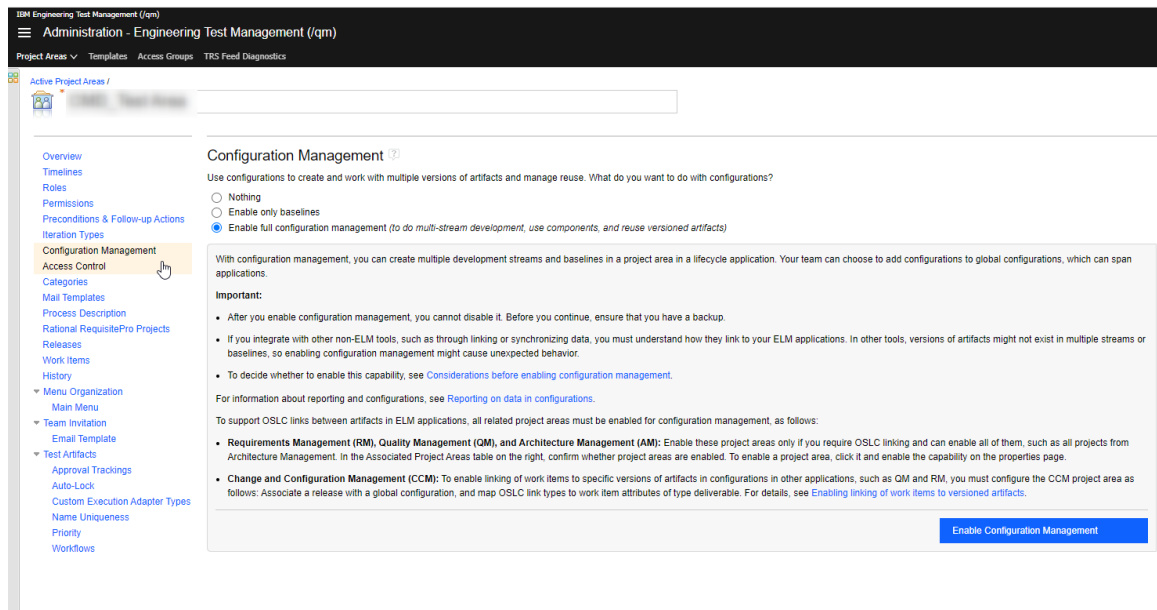
- Verify the existence of given `testsuite`. Or create the new **Test Suite** if `new` value is used.
- Create a **Test Suite Execution Record** for given (or newly created) `testsuite`.
- Create a **Test Suite Result** which contains all **Test Case Execution Result**.
- Link all test cases to the given (or newly created) `testsuite`.
- Add that `testsuite` to the given `testplan`.

2.2.8 Project with Configuration Management (CM)

When Configuration Management (CM) is enabled for the project area, it is necessary to specify the project stream or baseline when executing import tool to interact with the proper stream/baseline resources.

Otherwise, the tool will not be able to correctly associate the imported test results.

Configuration Management (CM) is only enabled by Project Administrator as below figure:



The following arguments are used to provide this information:

- `--stream STREAM`: specifies the project `STREAM`.
A project stream represents a branch of work within a project area. When CM is enabled, it is crucial to provide this option to correctly associate the imported test results with the appropriate stream.
Example:

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720↔
↔ --stream "Develop_stream"
```

- `--baseline BASELINE`: specifies the project `BASELINE`.
A baseline captures the state of project artifacts at a particular point in time. When CM or Baselines Only is enabled, this option is required to ensure that the test results are linked to the correct baseline.
Example:

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720↔
↔ --baseline "Baseline_v1"
```

Note

Only one option (`--stream` or `--baseline`) should be used at a time.
If both options are provided, the `--stream` option will take precedence.

2.2.9 Naming convention of generated resources

This feature provides flexibility in defining how new resources such as test execution results, test cases, and test suite are named, allowing users to customize the naming of resources generated during the import of test results to RQM. The optional argument `--naming_config` is used to provide the user's definitions. It takes a path to a JSON file as value. The JSON file specifies the format and structure of the names for different types of resources using predefined placeholders `{resource_type}`.

Supported placeholders:

- `{testplan}`: Name of the test plan.
- `{build}`: Name of the build record.
- `{environment}`: Name of the test environment.
- `{testsuite}`: Name of the test suite (if `--testsuite` is used).

- **{testcase}**: Name of the test case, only available for test case relevant resources: **testcase**, **tcer** and **testresult**

Example JSON file (`naming_convention.json`):

```
{
  "testcase": "{testcase} (imported by RobotLog2RQM tool)", // customize test case name
  "tcer": "{testcase}_{environment}", // Name TCER with environment as suffix
  "testresult": "{testcase}_{build}", // Name Test result with build as suffix
  "testsuite": "{testplan}_{testsuite}", // Name testsuite with test plan as prefix
  "tser": "{testsuite}_{environment}",
  "suiteresult": "{testsuite}_{environment}_{build}"
}
```

Example Usage:

```
RobotLog2RQM output.xml https://sample-rqm-host.com ROBFW-AIO test_user test_pw 720 ↔
↪ --naming_config path/to/naming_convention.json
```

Default Behavior:

If the `--naming_config` argument is not used, the tool will use default naming convention as defined by the **robotframework-robotlog2rqm** tool:

```
{
  "testcase"      : "{testcase}",
  "tcer"          : "TCER: {testcase}",
  "testresult"    : "Execution result: {testcase}",
  "testsuite"     : "{testsuite}",
  "tser"          : "TSER: {testsuite}",
  "suiteresult"   : "Testsuite result: {testsuite}"
}
```

2.3 Robot Framework Test Case Information on RQM:

For more detail about the mapping between the defined information from Robot Framework Test Case to Robot Framework result (*output.xml*) file and their reflections on RQM WebApp, please refer below mapping table:

Table: Mapping RQM - Robot

CHAPTER 2. DESCRIPTION

2.3. ROBOT FRAMEWORK TEST CASE INFORMATION ON RQM:

RQM data		Robot Framework	
Resource	Attribute/ Field	Testsuite/Testcase	Output.xml
Build Record	Title	Metadata version_sw Build	//suite/metadata/item[@name="version_sw"]
Test Environment	Title	Metadata project ↔ ↔ Environment	//suite/metadata/item[@name="project"]
Test Case	ID	[Tags] tcid-xxx	//suite/test/tags/tag[@text="tcid-xxx"]
	Name	tesname	//suite/test/@name
	Team Area	Metadata team-area ↔ ↔ Team_Area	//suite/metadata/item[@name="team-area"]
	Description	test doc - [Documentation]	//suite/test/doc/@text
	Owner	provided user in cli	
	Component/ Categories	Metadata component ↔ ↔ Component	//suite/metadata/item[@name="component"]
	Requirement ID	[Tags] fid-yyy	//suite/test/tags/tag[@text="fid-yyy"]
	Robot File	[Tags] robotfile-zzz	//suite/test/tags/tag[@text="robotfile-zzz"]
Test Case Execution Record (TCER)	Owner	provided user in cli	
	Team Area	Metadata team-area ↔ ↔ Team_Area	//suite/metadata/item[@name="team-area"]
	Test Plan	Interaction URL to provided testplan in cli	
	Test Case	Interaction URL to provided test case ID: provided tcid in [Tags]: tcid-xxx or generated tcid when using -createmissing	//suite/test/tags/tag[@text="tcid-xxx"]
	Test Environment	Metadata project ↔ ↔ Environment	//suite/metadata/item[@name="project"]
Test Result	Owner	provided user in cli	
	Tested By	provided user in cli - userid must be used	
	Team Area	Metadata team-area ↔ ↔ Team_Area	//suite/metadata/item[@name="team-area"]
	Actual Result	Test case result (PASSED, FAILED, UNKNOWN)	//suite/test/status/@status
	Host Name	Metadata machine %{↔ ↔ COMPUTERTNAME}	//suite/metadata/item[@name="machine"]
	Test Plan	Interaction URL to provided testplan in cli	
	Test Case	Interaction URL to provided test case ID: provided tcid in [Tags]: tcid-xxx or generated tcid when using -createmissing	//suite/test/tags/tag[@text="tcid-xxx"]
	Test Case Execution Record	Interaction URL to TCER ID	
	Build	Metadata version_sw8Build	//suite/metadata/item[@name="version_sw"]
	Start Time	Test case start time	//suite/test/status/@starttime

2.4 Additional Tool: RQMTool

The **robotframework-robotlog2rqm** package now includes the **RQMTool** submodule, which provides a standalone CLI for fetching test cases and test suites from IBM Rational Quality Manager (RQM) test plans.

2.4.1 Purpose

RQMTool enables:

- Fetching test cases and/or test suites from a given RQM test plan.
- Exporting fetched artifacts as CSV or JSON.
- Performing dry-run validation of inputs and RQM connectivity without fetching.

2.4.2 Usage

The RQMTool can be executed as a Python module:

```
python -m RobotLog2RQM.rqmttool --host <\textless{}\>RQM_SERVER_URL<\textgreater;{}> ↵
    ↵ --project <\textless{}\>PROJECT_AREA<\textgreater;{}> \
        --user <\textless{}\>USERNAME<\textgreater;{}> --password <\textless{}\>PASSWORD<\textgreater;{}> \
        --testplan <\textless{}\>TESTPLAN_ID<\textgreater;{}> [↵
    ↵ --types <\textless{}\>artifact_types<\textgreater;{}>] \
        [--format <\textless{}\>csv|json<\textgreater;{}>] [↵
    ↵ --output-dir <\textless{}\>DIR<\textgreater;{}>] \
        [--basename <\textless{}\>BASENAME<\textgreater;{}>] [↵
    ↵ --dryrun]
```

Arguments description:

- **-host (required)**: Base URL of the RQM server.
- **-project (required)**: Name of the RQM project area.
- **-user (required)**: RQM username.
- **-password (required)**: RQM password.
- **-testplan (required)**: ID of the test plan to fetch.
- **-types (optional)**: Comma-separated list of artifact types to fetch (`testcase`, `testsuite`). Default: both.
- **-format (optional)**: Output format: `csv` or `json`. Default: `csv`.
- **-output-dir (optional)**: Directory to save exported files. Default: current directory.
- **-basename (optional)**: Base name for output files. Default: `testplan_export`.
- **-dryrun (optional)**: Validate inputs and RQM connection without performing fetch.

2.4.3 Examples

Fetch all test cases and test suites from test plan ID 720 and export to CSV:

```
python -m RobotLog2RQM.rqmtool --host https://sample-rqm-host.com \
    --project ROBFW-AIO \
    --user test_user \
    --password test_pw \
    --testplan 720 \
    --types testcase,testsuite \
    --format csv
```

Perform a dry-run to check connection and arguments:

```
python -m RobotLog2RQM.rqmtool --host https://sample-rqm-host.com \  
                                --project ROBFW-AIO \  
                                --user test_user \  
                                --password test_pw \  
                                --testplan 720 \  
                                --dryrun
```

2.4.4 Notes

- CSV export automatically generates filenames including test plan ID.
- JSON export contains only selected artifact types.
- Dry-run mode does not fetch or save any data, only validates arguments and RQM access.



Tip

RQMTTool can be used independently to fetch test cases from a testplan and even execute them selectively, or it can be combined with the main **robotframework-robotlog2rqm** import tool to retrieve test case information before importing Robot Framework results into RQM.

Chapter 3

CRQM.py

3.1 Function: get_xml_tree

Parse xml object from file.

Arguments:

- `file_name`
/ *Condition*: required / *Type*: str /
Path to file or file-like object.
- `bdt_d_validation`
/ *Condition*: optional / *Type*: bool /
If True, validate against a DTD referenced by the document.

Returns:

- `oTree`
/ *Type*: `xml.etree._ElementTree` object /
The xml etree object.

3.2 Class: Identifier

Identifier class used to identify RQM resource with name and id

3.3 Class: CRQMClient

CRQMClient class uses RQM REST APIs to get, create and update resources (testplan, testcase, test result, ...) on RQM - Rational Quality Manager

Resource type mapping:

- `buildrecord`: Build Record
- `configuration`: Test Environment
- `testplan`: Test Plan
- `testsuite`: Test Suite
- `suiteexecutionrecord`: Test Suite Execution Record (TSER)
- `testsuitelog`: Test Suite Log
- `testcase`: Test Case
- `executionworkitem`: Test Execution Record (TCER)
- `executionresult`: Execution Result

3.3.1 Method: login

Log in RQM by provided user & password.

Arguments:

(no arguments)

Returns:

- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method `login` was successful or not.

3.3.2 Method: verifyProjectName

Verify the project name by searching it in *project-areas* XML response.

Arguments:

(no arguments)

Returns:

- `bSuccess`
/ *Type*: bool /
Indicates if the computation of the method `verifyProjectName` was successful or not.

3.3.3 Method: disconnect

Disconnect from RQM.

Arguments:

(no arguments)

Returns:

(no returns)

3.3.4 Method: config

Configure RQMClient with testplan ID, build, configuration, createmissing, ...

- Verify the existence of provided testplan ID.
- Verify the existences of provided build and configuration names before creating new ones.

Arguments:

- `plan_id`
/ *Condition*: optional / *Type*: str /
Testplan ID of RQM project for importing result(s).
- `build_name`
/ *Condition*: optional / *Type*: str / *Default*: None /
The *Build Record* for linking result(s). Set it to *None* if not be used, the empty name " will lead to error.
- `config_name`
/ *Condition*: optional / *Type*: str / *Default*: None /
The *Test Environment* for linking result(s). Set it to *None* if not be used, the empty name " may lead to error.
- `createmissing`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If *True*, the testcase without *tcid* information will be created on RQM.

- `updatetestcase`
/ Condition: optional / Type: bool / Default: False /
 If *True*, the information of testcase on RQM will be updated bases on robot testfile.
- `suite_id`
/ Condition: optional / Type: str / Default: None /
 Testsuite ID of RQM project for importing result(s).
- `stream`
/ Condition: optional / Type: str / Default: None /
 Project stream which is enabled in Configuration Management (CM).
- `baseline`
/ Condition: optional / Type: str / Default: None /
 Project baseline which is enabled in Configuration Management (CM).
- `naming_convention`
/ Condition: optional / Type: dict / Default: None /
 Naming convention for the creating new resources (testcase, TCER, ...).
- `fetch_team_areas`
/ Condition: optional / Type: bool / Default: True /
 If *True*, fetch team-areas information for further process.

Returns:*(no returns)***3.3.5 Method: userURL**

Return interaction URL of provided userID

Arguments:

- `userID`
/ Condition: required / Type: str /
 The user ID.

Returns:

- `userURL`
/ Type: str /
 The interaction URL of provided userID.

3.3.6 Method: integrationURL

Return interaction URL of provided resource and ID. The provided ID can be internalID (contains only digits) or externalID.

Arguments:

- `resourceType`
/ Condition: required / Type: str /
 The RQM resource type (e.g. "testplan", "testcase", ...).
- `id`
/ Condition: optional / Type: str / Default: None /
 The ID of given resource.

- If given: the specified url to resource ID is returned.
- If None: the url to resource type (to get all entity) is returned.

- `forceinternalID`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If *True*, force to return the url of resource as internal ID.

Returns:

- `integrationURL`
/ *Type*: str /
The interaction URL of provided resource and ID.

3.3.7 Method: webIDfromResponse

Get internal ID (number) from response of POST method.

Note: Only *executionresult* has response text. Other resources has only response header.

Arguments:

- `response`
/ *Condition*: required / *Type*: str /
The xml response from POST method for parsing ID information.
- `tagID`
/ *Condition*: optional / *Type*: str / *Default*: 'rqm:resultId' /
Tag name which contains ID information.

Returns:

- `resultId`
/ *Type*: str /
The internal ID (as number).

3.3.8 Method: webIDfromGeneratedID

Return web ID (ns2:webId) from generate ID by get resource data from RQM.

Note:

- This method is only used for generated *testcase*, *executionworkitem* and *executionresult*.
- *buildrecord* and *configuration* does not have *ns2:webId* in response data.

Arguments:

- `resourceType`
/ *Condition*: required / *Type*: str /
The RQM resource type.
- `generateID`
/ *Condition*: required / *Type*: str /
The Slug ID which is returned in *Content-Location* from POST response.

Returns:

- `webID`
/ *Type*: str /
The web ID (as number).

3.3.9 Method: getResourceByID

Return data of provided resource and ID by GET method

Arguments:

- `resourceType`
/ *Condition*: required / *Type*: str /
The RQM resource type.
- `id`
/ *Condition*: required / *Type*: str /
ID of resource.

Returns:

- `res`
/ *Type*: *Response* object /
Response data of GET request.

3.3.10 Method: getAllByResource

Return all entries (in all pages) of provided resource by GET method.

Arguments:

- `resourceType`
/ *Condition*: required / *Type*: str /
The RQM resource type.

Returns:

- `dReturn`
/ *Type*: dict /
A dictionary which contains response status, message and data.
Example:

```
{
    'success' : False,
    'message' : '',
    'data'    : {}
}
```

3.3.11 Method: getAllBuildRecords

Get all available build records of project on RQM and store them into *dBuildVersion* property.

Arguments:

(no arguments)

Returns:

(no returns)

3.3.12 Method: getAllConfigurations

Get all available configurations of project on RQM and store them into *dConfiguration* property.

Arguments:

(no arguments)

Returns:

(no returns)

3.3.13 Method: getAllTeamAreas

Get all available team-areas of project on RQM and store them into *dTeamAreas* property.

Example:

```
{
  'teamA' : '{host}/qm/process/project-areas/{project-id}/team-areas/{teamA-id}',
  'teamB' : '{host}/qm/process/project-areas/{project-id}/team-areas/{teamB-id}'
}
```

Arguments:

(no arguments)

Returns:

(no returns)

3.3.14 Method: getTestArtifactsFromResource

Get all test cases and test suites associated with a given test plan.

Arguments:

- resource_type
/ Condition: required / Type: str /
The RQM test resource (*testplan* or *testsuite*) to get test artifact(s).
- resource_id
/ Condition: required / Type: str /
The RQM test plan/suite to get test artifact(s).
- artifact_types
/ Condition: required / Type: list /
List of artifact types (*testcase*, *testsuite*) for fetching.

Returns:

- / Type: dict /
A dictionary containing fetched artifacts:

```
{
  'testcase': [{'id': ..., 'name': ..., 'url': ...}, ...],
  'testsuite': [{'id': ..., 'name': ..., 'url': ...}, ...]
}
```

3.3.15 Method: addTeamAreaNode

Append *team-area* node which contains URL to given team-area into xml template.

Note: *team-area* information is case-casesensitive

Arguments:

- root
/ Condition: required / Type: *Element* object /
The xml root object.
- sTeam
/ Condition: required / Type: str /
Team name to be added.

Returns:

- `root`
/ *Type*: str /
The xml root object with addition *team-area* node.

3.3.16 Method: createTestcaseTemplate

Return testcase template from provided information.

Arguments:

- `testcaseName`
/ *Condition*: required / *Type*: str /
Testcase name.
- `sDescription`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testcase description.
- `sComponent`
/ *Condition*: optional / *Type*: str / *Default*: " /
Component which testcase is belong to.
- `sFID`
/ *Condition*: optional / *Type*: str / *Default*: " /
Function ID (requirement ID) for linking.
- `sTeam`
/ *Condition*: optional / *Type*: str / *Default*: " /
Team name for linking.
- `sRobotFile`
/ *Condition*: optional / *Type*: str / *Default*: " /
Link to robot file on source control.
- `sTestType`
/ *Condition*: optional / *Type*: str / *Default*: " /
Test type information.
- `sASIL`
/ *Condition*: optional / *Type*: str / *Default*: " /
ASIL information.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testcase owner.
- `sTCtemplate`
/ *Condition*: optional / *Type*: str / *Default*: None /
Existing testcase template as xml string.
If not provided, template file under *RQM_templates* is used as default.

Returns:

- `sTCxml`
/ *Type*: str /
The xml testcase template as string.

3.3.17 Method: createTCERTemplate

Return testcase execution record template from provided information.

Arguments:

- `testCaseID`
/ *Condition*: required / *Type*: str /
Testcase ID for linking.
- `testCaseName`
/ *Condition*: required / *Type*: str /
Testcase name.
- `testplanID`
/ *Condition*: required / *Type*: str /
Testplan ID for linking.
- `confID`
/ *Condition*: optional / *Type*: str / *Default*: " /
Configuration - *Test Environment* for linking.
- `sTeam`
/ *Condition*: optional / *Type*: str / *Default*: " /
Team name for linking.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testcase owner.

Returns:

- `sTCERxml`
/ *Type*: str /
The xml testcase execution record template as string.

3.3.18 Method: createExecutionResultTemplate

Return testcase execution result template from provided information.

Arguments:

- `testCaseID`
/ *Condition*: required / *Type*: str /
Testcase ID for linking.
- `testCaseName`
/ *Condition*: required / *Type*: str /
Testcase name.
- `testplanID`
/ *Condition*: required / *Type*: str /
Testplan ID for linking.
- `TCERID`
/ *Condition*: required / *Type*: str /
Testcase execution record (TCER) ID for linking.

- `resultState`
/ *Condition*: required / *Type*: str /
Testcase result status.
- `startTime`
/ *Condition*: required / *Type*: str /
Testcase start time.
- `endTime`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testcase end time.
- `duration`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testcase duration.
- `testPC`
/ *Condition*: optional / *Type*: str / *Default*: " /
Test PC which executed testcase.
- `testBy`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID who executed testcase.
- `lastlog`
/ *Condition*: optional / *Type*: str / *Default*: " /
Traceback information (for Failed testcase).
- `buildrecordID`
/ *Condition*: optional / *Type*: str / *Default*: " /
Build Record ID for linking.
- `sTeam`
/ *Condition*: optional / *Type*: str / *Default*: " /
Team name for linking.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testcase owner.

Returns:

- `sTCResultxml`
/ *Type*: str /
The xml testcase result template as string.

3.3.19 Method: createBuildRecordTemplate

Return build record template from provided build name.

Arguments:

- `buildName`
/ *Condition*: required / *Type*: str /
Build Record name.

Returns:

- `sBuildxml`
/ *Type*: str /
The xml build template as string.

3.3.20 Method: createConfigurationTemplate

Return configuration - Test Environment template from provided configuration name.

Arguments:

- `buildName`
/ *Condition*: required / *Type*: str /
Configuration - *Test Environment* name.

Returns:

- `sEnvironmentxml`
/ *Type*: str /
The xml test environment template as string.

3.3.21 Method: createTSERTemplate

Return testsuite execution record (TSER) template from provided configuration name.

Arguments:

- `testsuiteID`
/ *Condition*: required / *Type*: str /
Testsuite ID.
- `testsuiteName`
/ *Condition*: required / *Type*: str /
Testsuite name.
- `testplanID`
/ *Condition*: required / *Type*: str /
Testplan ID for linking.
- `confID`
/ *Condition*: optional / *Type*: str / *Default*: " /
Configuration - *Test Environment* ID for linking.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testsuite owner.

Returns:

- `sTSxml`
/ *Type*: str /
The xml testsuite template as string.

3.3.22 Method: createTestsuiteResultTemplate

Return testsuite execution result template from provided configuration name.

Arguments:

- `testsuiteID`
/ *Condition*: required / *Type*: str /
Testsuite ID.

- `testsuiteName`
/ *Condition*: required / *Type*: str /
Testsuite name.
- `TSERID`
/ *Condition*: required / *Type*: str /
Testsuite execution record (TSER) ID for linking.
- `lTCER`
/ *Condition*: required / *Type*: str /
List of testcase execution records (TCER) for linking.
- `lTCResults`
/ *Condition*: required / *Type*: str /
List of testcase results for linking.
- `startTime`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testsuite start time.
- `endTime`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testsuite end time.
- `duration`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testsuite duration.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testsuite owner.

Returns:

- `sTSResultxml`
/ *Type*: str /
The xml testsuite result template as string.

3.3.23 Method: createTestsuiteTemplate

Return testcase template from provided information.

Arguments:

- `testsuiteName`
/ *Condition*: required / *Type*: str /
Testsuite name.
- `sDescription`
/ *Condition*: optional / *Type*: str / *Default*: " /
Testsuite description.
- `sOwnerID`
/ *Condition*: optional / *Type*: str / *Default*: " /
User ID of testsuite owner.

- `stStemplate`
/ Condition: optional / Type: str / Default: None /
 Existing testsuite template as xml string.
 If not provided, template file under *RQM_templates* is used as default.

Returns:

- `stSxml`
/ Type: str /
 The xml testsuite template as string.

3.3.24 Method: createResource

Create new resource with provided data from template by POST method.

Arguments:

- `resourceType`
/ Condition: required / Type: str /
 Resource type.
- `content`
/ Condition: required / Type: str /
 The xml template as string.

Returns:

- `returnObj`
/ Type: dict /
 A dictionary reponse which contains status, ID, status_code and error message.
 Example:

```
{
    'success' : False,
    'id': None,
    'message': '',
    'status_code': ''
}
```

3.3.25 Method: createBuildRecord

Create new build record.

Arguments:

- `sBuildSWVersion`
/ Condition: required / Type: str /
 Build version - *Build Record* name.
- `forceCreate`
/ Condition: optional / Type: bool / Default: False /
 If True, force to create new build record without existing verification.

Returns:

- `returnObj`
/ *Type*: dict /
A dictionary reponse which contains status, ID, status_code and error message.
Example:

```
{
    'success' : False,
    'id': None,
    'message': '',
    'status_code': ''
}
```

3.3.26 Method: createConfiguration

Create new configuration - test environment.

Arguments:

- `sConfigurationName`
/ *Condition*: required / *Type*: str /
Configuration - *Test Environment* name.
- `forceCreate`
/ *Condition*: optional / *Type*: str / *Default*: False /
If True, force to create new Test Environment without existing verification.

Returns:

- `returnObj`
/ *Type*: dict /
A dictionary reponse which contains status, ID, status_code and error message.
Example:

```
{
    'success' : False,
    'id': None,
    'message': '',
    'status_code': ''
}
```

3.3.27 Method: updateResourceByID

Update data of provided resource and ID by PUT method.

Arguments:

- `resourceType`
/ *Condition*: required / *Type*: str /
Resource type.
- `id`
/ *Condition*: required / *Type*: str /
Resource id.
- `content`
/ *Condition*: required / *Type*: str /
The xml template as string.

Returns:

- `res`
/ *Type*: *Response* object /
Response object from PUT request.

3.3.28 Method: linkListTestcase2Resource

Link list of test cases to a given resource (testplan or testsuite).

Arguments:

- `resource_type`
/ *Condition*: required / *Type*: str /
Type of resource: 'testplan' or 'testsuite'
- `resource_id`
/ *Condition*: required / *Type*: str /
ID of the target resource
- `lTestcases`
/ *Condition*: optional / *Type*: list / *Default*: None /
List of testcase IDs. If None, use `self.lTestcaseIDs`

Returns:

- `returnObj`
/ *Type*: dict /
Response dictionary which contains status and error message.
Example:

```
{
    'success' : False,
    'message': ''
}
```

3.3.29 Method: linkListTestcase2Testplan

Link list of test cases to provided testplan ID.

Arguments:

- `testplanID`
/ *Condition*: required / *Type*: str /
Testplan ID to link given testcase(s).
- `lTestcases`
/ *Condition*: optional / *Type*: list / *Default*: None /
List of testcase(s) to be linked with given testplan.
If not provide, `lTestcaseIDs` property will be used as list of testcase.

Returns:

- `returnObj`
/ *Type*: dict /
Response dictionary which contains status and error message.
Example:

```
{
    'success' : False,
    'message': ''
}
```

3.3.30 Method: linkListTestcase2Testsuite

Link list of test cases to provided testsuite ID

Arguments:

- testsuiteID
/ *Condition*: required / *Type*: str /
Testsuite ID to link given testcase(s).
- lTestcases
/ *Condition*: optional / *Type*: list / *Default*: None /
List of testcase(s) to be linked with given testplan.
If not provide, *lTestcaseIDs* property will be used as list of testcase.

Returns:

- returnObj
/ *Type*: dict /
Response dictionary which contains status and error message.
Example:

```
{
    'success' : False,
    'message': ''
}
```

3.3.31 Method: addTestsuite2Testplan

Add testsuite ID to provided testplan ID

Arguments:

- testplanID
/ *Condition*: required / *Type*: str /
Testplan ID to link given testsuite ID.
- testsuiteID
/ *Condition*: optional / *Type*: str / *Default*: None /
Testsuite to be linked with given testplan.
If not provide, *testsuite.id* value will be used as id of testsuite.

Returns:

- returnObj
/ *Type*: dict /
Response dictionary which contains status and error message.
Example:

```
{
    'success' : False,
    'message': ''
}
```

Chapter 4

logger.py

Logger class for logging message.

4.1 Method: `config`

Configure Logger class.

Arguments:

- `output_console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Write message to console output.
- `output_logfile`
/ *Condition*: optional / *Type*: str / *Default*: None /
Path to log file output.
- `dryrun`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If set, a prefix as 'dryrun' is added for all messages.

Returns:

(no returns)

4.2 Method: `log`

Write log message to console/file output.

Arguments:

- `msg`
/ *Condition*: optional / *Type*: str / *Default*: " /
Message which is written to output.
- `color`
/ *Condition*: optional / *Type*: str / *Default*: None /
Color style for the message.
- `indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.

Returns:

(no returns)

4.3 Method: `log_warning`

Write warning message to console/file output.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Warning message which is written to output.
- `indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.

Returns:

(no returns)

4.4 Method: `log_error`

Write error message to console/file output.

- `msg`
/ *Condition*: required / *Type*: str /
Error message which is written to output.
- `fatal_error`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If set, tool will terminate after logging error message.
- `indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.

Returns:

(no returns)

Chapter 5

robotlog2rqm.py

5.1 Function: get_from_tags

Extract testcase information from tags.

Example: TCID-xxxx, FID-xxxx, ...

Arguments:

- `lTags`
/ *Condition*: required / *Type*: list /
List of tag information.
- `reInfo`
/ *Condition*: required / *Type*: str /
Regex to get the expectedated info (ID) from tag info.

Returns:

- `lInfo`
/ *Type*: list /
List of expected information (ID)

5.2 Function: convert_to_datetime

Convert time string to datetime.

Arguments:

- `time`
/ *Condition*: required / *Type*: str /
String of time.

Returns:

- `dt`
/ *Type*: *datetime* object /
Datetime object.

5.3 Function: process_config_file

Parse and validate content of configuration file

Arguments:

- `path_file`
/ *Condition*: required / *Type*: str /
Path to the configuration json file.

Returns:

- `dConfig`
/ *Type*: dict /
Content of json file.

5.4 Function: is_valid_config

Validate the json configuration base on given schema.

Default schema supports below information:

```
NAMING_CONVENTION_SCHEMA = {
    "testcase"      : str,
    "tcer"          : str,
    "testresult"    : str,
    "testsuite"     : str,
    "tser"          : str,
    "suiteresult"   : str
}
```

Arguments:

- `dConfig`
/ *Condition*: required / *Type*: dict /
Json configuration object to be verified.
- `dSchema`
/ *Condition*: optional / *Type*: dict / *Default*: CONFIG_SCHEMA /
Schema for the validation.
- `bExitOnFail`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, exit tool in case the validation is fail.

Returns:

- `bValid`
/ *Type*: bool /
True if the given json configuration data is valid.

5.5 Function: process_suite_metadata

Try to find metadata information from all suite levels.

Metadata at top suite level has a highest priority.

Arguments:

- `suite`
/ *Condition*: required / *Type*: *TestSuite* object /
Robot suite object.
- `default_metadata`
/ *Condition*: optional / *Type*: dict / *Default*: `DEFAULT_METADATA` /
Initial Metadata information for updating.

Returns:

- `dMetadata`
/ *Type*: dict /
Dictionary of Metadata information.

5.6 Function: process_metadata

Extract metadata from suite result bases on `DEFAULT_METADATA`.

Arguments:

- `metadata`
/ *Condition*: required / *Type*: dict /
Robot metadata object.
- `default_metadata`
/ *Condition*: optional / *Type*: dict / *Default*: `DEFAULT_METADATA` /
Initial Metadata information for updating.

Returns:

- `dMetadata`
/ *Type*: dict /
Dictionary of Metadata information.

5.7 Function: process_suite

Process robot suite for importing to RQM.

Arguments:

- `RQMClient`
/ *Condition*: required / *Type*: *RQMClient* object /
RQMClient object.
- `suite`
/ *Condition*: required / *Type*: *TestSuite* object /
Robot suite object.
- `log_indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Indent for logging message.

Returns:

(no returns)

5.8 Function: process_test

Process robot test for importing to RQM.

Arguments:

- `RQMClient`
/ *Condition*: required / *Type*: *RQMClient* object/
RQMClient object.
- `test`
/ *Condition*: required / *Type*: *TestCase* object/
Robot test object.
- `log_indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Indent for logging message.

Returns:

(no returns)

5.9 Function: RobotLog2RQM

Import robot results from output.xml to RQM - IBM Rational Quality Manager.

Flow to import Robot results to RQM:

1. Process provided arguments from command line
2. Login Rational Quality Management (RQM)
3. Parse Robot results
4. Import results into RQM
5. Link all executed testcases to provided testplan/testsuite ID

Arguments:

- `args`
/ *Condition*: required / *Type*: *ArgumentParser* object /
Argument parser object which contains:
 - *resultxmlfile* : path to the xml result file or directory of result files to be imported.
 - *host* : RQM host url.
 - *project* : RQM project name.
 - *user* : user for RQM login.
 - *password* : user password for RQM login.
 - *testplan* : RQM testplan ID.
 - *testsuite* : testsuite ID for this execution. If 'new', then create a new testsuite for this execution.
 - *recursive* : if True, then the path is searched recursively for log files to be imported.
 - *createtestcase* : if True, then all testcases without tcid are created when importing.
 - *updatetestcase* : if True, then testcases information on RQM will be updated bases on robot testfile.
 - *naming_config* : configuration json file for naming conventions when creating RQM resources.
 - *dryrun* : if True, then verify all input arguments (includes RQM authentication) and show what would be done.

- *stream* : project stream. Note, requires Configuration Management (CM) to be enabled for the project area.
- *baseline* : project baseline. Note, requires Configuration Management (CM), or Baselines Only to be enabled for the project area.

System Message

WARNING/2 in <string>, line 268

Bullet list ends without a blank line; unexpected unindent. backrefs:

Returns:*(no returns)*

Chapter 6

rqmtool.py

6.1 Function: write_json_file

Write data to a JSON file.

Arguments:

- `file_name`
/ *Condition*: required / *Type*: str /
Path of the JSON file to write.
- `data`
/ *Condition*: required / *Type*: dict /
Data to export.

Returns:

(no returns)

6.2 Function: write_csv_file

Write data to a CSV file for a specific artifact type.

Arguments:

- `file_name`
/ *Condition*: required / *Type*: str /
Path of the CSV file to write.
- `data`
/ *Condition*: required / *Type*: dict /
Data dictionary containing artifacts.
- `artifact_type`
/ *Condition*: required / *Type*: str /
Artifact type (*testcase* or *testsuite*) to export.

Returns:

(no returns)

6.3 Function: write_output_file

Write data to output files (JSON or CSV) according to specified options.

Arguments:

- data
/ *Condition*: required / *Type*: dict /
Data dictionary containing artifacts.
- output_dir
/ *Condition*: optional / *Type*: str /
Directory to save output files. Default is current directory.
- basename
/ *Condition*: optional / *Type*: str /
Base name for output files. Default is "testplan_export".
- extension
/ *Condition*: optional / *Type*: str /
Output format: *json* or *csv*. Default is *csv*.
- artifact_types
/ *Condition*: optional / *Type*: list /
Artifact types to export. Default is all supported types.

Returns:

(no returns)

6.4 Function: normalize_custom_attributes

Ensure all artifacts across projects have consistent keys. Missing custom attributes will be filled with empty strings.

Arguments:

- data
/ *Condition*: required / *Type*: dict /
Dictionary containing artifact data fetched from RQM, where each key represents an artifact type (e.g., *testcase*, *testsuite*) and the value is a list of dictionaries holding artifact details.
- artifact_types
/ *Condition*: required / *Type*: list /
List of artifact types to process (e.g., [*'testcase'*, *'testsuite'*]). Each type key in data will be normalized to ensure all items have the same set of attributes.

Returns:

- data
/ *Type*: dict /
Normalized dictionary with consistent keys across all artifacts. Any missing custom attributes are added with an empty string (") as value.

6.5 Function: RQMTool

Main entry point for RQMTool CLI.

Arguments:

(no arguments)

Returns:

(no returns)

Chapter 7

Appendix

About robotframework-robotlog2rqm:

Author

Tran Duy Ngoan (Ngoan.TranDuy@vn.bosch.com)

Version

1.8.1 (09.06.2026)

Short Description

Imports robot results to IBM Rational Quality Manager (RQM)

Homepage

[robotframework-robotlog2rqm](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 8

History

0.1.0	07/2022
<i>Initial version</i>	
1.1.1	01.08.2022
<i>Rework repository's document bases on GenPackageDoc</i>	
1.1.2	25.08.2022
- Correct indent of sourcode's docstring. - Update new style for history.	
1.1.3	13.10.2022
- Fix findings and enhance README and document files - Change argument name 'outputfile' to 'resultxmlfile'	
1.1.4	10.11.2022
<i>Rename package to RobotLog2RQM</i>	
1.2.0	09.01.2023
- Rework optional arguments and improve logging messages - Update README and document for publishing pypi	
1.2.1	14.06.2023
<i>Update README: fix links issue and update installation section</i>	
1.2.2	06.03.2024
<i>Fix findings in documentation</i>	
1.2.3	14.03.2024
<i>Add support for basic authentication as an alternative to SSO system</i>	
1.2.4	11.06.2024
<i>Fix issue when test result has SKIP status</i>	
1.3.0	19.06.2024
<i>Add new feature of test suite with new optional argument <code>--testsuite TEST-SUITE</code></i>	
1.3.1	03.07.2024
<i>Add link to Test Environment for TSER when using testsuite feature</i>	
1.4.0	24.07.2024

CHAPTER 8. HISTORY

<i>Add support project with enabled Configuration Management (multiple streams, baselines)</i>	
1.4.1	20.09.2024
<i>Add new feature of defining the naming convention of generated resources</i>	
1.4.2	18.02.2025
<i>Add missing build information when creating the testsuite result</i>	
1.5.0	28.10.2025
<i>Added RQMTool: additional support for fetching RQM test cases and test suites from a given test plan</i>	
1.6.0	10.11.2025
- Update RQMTool to get all custom attributes from test case. - Update RQMTool to support Configuration Management (multiple streams, baselines). - Update RQMTool to support for fetching test cases from a given test suite (-testsuite)	
1.7.0	07.01.2026
<i>Added support for encoded password (-password-enc) and password file (-password-file)</i>	
1.7.1	03.02.2026
<i>Update RobotLog2RQM import tool to avoid duplicate creation of test case in the given test plan/suite.</i>	
1.8.0	24.02.2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	
1.8.1	09.06.2026
<i>Fix issue with duplicate creation of test case in the given test plan/suite.</i>	

RobotLog2RQM.pdf*Created at 07.09.2026 - 09:47:17**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.12 TestResultDBAccess

TestResultDBAccess

v. 0.2.1

Tran Duy Ngoan

30.06.2026

CONTENTS

CONTENTS

Contents

1	Introduction	1
2	Description	2
2.0.1	DBAccessFactory	2
2.0.2	DBAccess Interface	2
2.0.3	DirectDBAccess	2
2.0.4	RestApiDBAccess	2
3	db_access_interface.py	3
3.1	Method: connect	3
3.2	Method: disconnect	3
3.3	Method: commit	3
3.4	Method: arGetCategories	3
3.5	Method: bExistingResultID	3
3.6	Method: sGetLatestFileID	3
3.7	Method: sCreateNewTestResult	3
3.8	Method: nCreateNewFile	3
3.9	Method: vCreateNewHeader	4
3.10	Method: nCreateNewSingleTestCase	4
3.11	Method: nCreateNewTestCase	4
3.12	Method: vCreateAbortReason	4
3.13	Method: vCreateCCRdata	4
3.14	Method: vCreateTags	4
3.15	Method: vCreateReanimation	4
3.16	Method: vSetCategory	4
3.17	Method: vUpdateFileEndTime	4
3.18	Method: vUpdateResultEndTime	4
3.19	Method: vUpdateEvtbl	4
3.20	Method: vUpdateEvtbls	4
3.21	Method: vFinishTestResult	5
4	direct_db_access.py	6
4.1	Method: connect	6
4.2	Method: commit	6
4.3	Method: disconnect	7
4.4	Method: cleanAllTables	7
4.5	Method: sCreateNewTestResult	7
4.6	Method: nCreateNewFile	8

CONTENTS

CONTENTS

4.7	Method: <code>vCreateNewHeader</code>	9
4.8	Method: <code>nCreateNewSingleTestCase</code>	11
4.9	Method: <code>nCreateNewTestCase</code>	12
4.10	Method: <code>vCreateTags</code>	13
4.11	Method: <code>vSetCategory</code>	14
4.12	Method: <code>vUpdateStartEndTime</code>	14
4.13	Method: <code>arGetCategories</code>	14
4.14	Method: <code>vCreateAbortReason</code>	15
4.15	Method: <code>vCreateReanimation</code>	15
4.16	Method: <code>vCreateCCRdata</code>	15
4.17	Method: <code>vFinishTestResult</code>	16
4.18	Method: <code>vUpdateEvtbls</code>	16
4.19	Method: <code>vUpdateEvtbl</code>	16
4.20	Method: <code>vEnableForeignKeyCheck</code>	16
4.21	Method: <code>sGetLatestFileID</code>	17
4.22	Method: <code>vUpdateFileEndTime</code>	17
4.23	Method: <code>vUpdateResultEndTime</code>	17
4.24	Method: <code>bExistingResultID</code>	17
4.25	Method: <code>arGetProjectVersionSWByID</code>	18
5	<code>rest_api_db_access.py</code>	19
5.1	Method: <code>get_certs_file</code>	19
5.2	Method: <code>encrypt_password</code>	19
5.3	Method: <code>connect</code>	19
5.4	Method: <code>disconnect</code>	20
5.5	Method: <code>arGetCategories</code>	20
5.6	Method: <code>bExistingResultID</code>	20
5.7	Method: <code>sGetLatestFileID</code>	21
5.8	Method: <code>arGetProjectVersionSWByID</code>	21
5.9	Method: <code>sCreateNewTestResult</code>	21
5.10	Method: <code>nCreateNewFile</code>	22
5.11	Method: <code>vCreateNewHeader</code>	23
5.12	Method: <code>nCreateNewSingleTestCase</code>	25
5.13	Method: <code>nCreateNewTestCase</code>	26
5.14	Method: <code>vCreateAbortReason</code>	26
5.15	Method: <code>vCreateCCRdata</code>	27
5.16	Method: <code>vCreateTags</code>	27
5.17	Method: <code>vCreateReanimation</code>	27
5.18	Method: <code>vSetCategory</code>	28
5.19	Method: <code>vUpdateFileEndTime</code>	28
5.20	Method: <code>vUpdateResultEndTime</code>	28
5.21	Method: <code>vFinishTestResult</code>	29
5.22	Method: <code>vUpdateEvtbl</code>	29
5.23	Method: <code>vUpdateEvtbls</code>	29
6	<code>DBAccessFactory.py</code>	30

<u>CONTENTS</u>	<u>CONTENTS</u>
7 Appendix	31
8 History	32
C	

Chapter 1

Introduction

The **TestResultDBAccess** package is designed to streamline the process of importing test data into the database of a test result web application.

This package offers two distinct access methods: **Direct Access** and **REST API Access**.

Users can seamlessly choose between these methods based on their preferences and requirements.

- **Direct Access:** This method provides a direct connection to the database using the SQL protocol, allowing efficient data retrieval and import operations.
- **REST API Access:** This method interacts with the web application via HTTP/HTTPS protocol, providing flexibility and compatibility with modern web architectures.

The package's DB Access Factory facilitates the creation of instances for both **Direct Access** and **REST API Access**, ensuring a smooth and adaptable workflow for importing test data into the target database.

Whether users prefer direct database interaction or web-based API access, **TestResultDBAccess** offers a comprehensive solution to meet diverse testing needs.

Chapter 2

Description

The `TestResultDBAccess` package offers a flexible and efficient way to interact with the database of a test result web application.

Users can choose between direct database access and REST API access, depending on their specific needs and preferences.

2.0.1 DBAccessFactory

The **DBAccessFactory** class is responsible for creating instances of the **DBAccess** interface based on the specified access method.

It provides the following method:

- **create(access_method: str): DBAccess** - Creates an instance of **DBAccess** based on the specified access method (`rest` or `db`).

2.0.2 DBAccess Interface

The **DBAccess** interface defines a set of methods for interacting with the database. These methods include:

- **connect(host: str, user: str, passwd: str, database: str)** - Establishes a connection to the database.
- **disconnect()** - Destroys the connection to the database.
- Various methods for retrieving, creating, updating, and finishing test result data in the database, such as **arGetCategories()**, **sGetLatestFileID()**, **vCreateTags()**, **vUpdateResultEndTime()**, and **vFinishTestResult()**.

2.0.3 DirectDBAccess

The **DirectDBAccess** class implements the **DBAccess** interface for direct database access. It provides methods for establishing and destroying connections, as well as implementing the various data manipulation methods defined in **DBAccess**.

2.0.4 RestApiDBAccess

The **RestApiDBAccess** class implements the **DBAccess** interface for REST API access to the database. Similar to **DirectDBAccess**, it provides methods for connecting, disconnecting, and implementing data manipulation methods through the REST API.

Chapter 3

db__access__interface.py

Abstract base class defining the interface for database access.

This interface defines methods for connecting to and disconnecting from the database, as well as methods for retrieving, creating, updating, and calling stored procedures in the database.

3.1 Method: connect

Connects to the database using the provided connection parameters.

3.2 Method: disconnect

Disconnects from the database.

3.3 Method: commit

Commits a transaction (only applicable for DirectDBAccess with transaction support).

3.4 Method: arGetCategories

Retrieves categories from the database.

3.5 Method: bExistingResultID

Checks if the given result ID exists in the database.

3.6 Method: sGetLatestFileID

Retrieves the latest file ID from the database.

3.7 Method: sCreateNewTestResult

Creates a new test result record in the database.

3.8 Method: nCreateNewFile

Creates a new file record in the database.

3.9 Method: vCreateNewHeader

Creates a new file header record in the database.

3.10 Method: nCreateNewSingleTestCase

Creates a new single test case record in the database.

3.11 Method: nCreateNewTestCase

Creates new test case(s) record in the database.

3.12 Method: vCreateAbortReason

Creates a new abort reason record in the database.

3.13 Method: vCreateCCRdata

Creates new CCR data in the database.

3.14 Method: vCreateTags

Creates new tags in the database.

3.15 Method: vCreateReanimation

Updates an existing test result with reanimation data in the database.

3.16 Method: vSetCategory

Updates an existing test result with category information in the database.

3.17 Method: vUpdateFileEndTime

Updates an existing file record with end time in the database.

3.18 Method: vUpdateResultEndTime

Updates an existing test result record with end time in the database.

3.19 Method: vUpdateEvtbl

Calls a stored procedure to update event tables in the database.

3.20 Method: vUpdateEvtbls

Calls a stored procedure to update event tables (plural) in the database.

3.21 Method: vFinishTestResult

Calls a stored procedure to finish a test result in the database.

Chapter 4

direct_db_access.py

DirectDBAccess class play a role as mysqlclient and provide methods to interact with TestResultWebApp's database.

4.1 Method: connect

Connect to the database with provided authentication and db info.

Arguments:

- `host`
/ *Condition*: required / *Type*: str /
URL which is hosted the TestResultWebApp's database.
- `user`
/ *Condition*: required / *Type*: str /
User name for database authentication.
- `passwd`
/ *Condition*: required / *Type*: str /
User's password for database authentication.
- `database`
/ *Condition*: required / *Type*: str /
Database name.
- `charset`
/ *Condition*: optional / *Type*: str / *Default*: 'utf8' /
The connection character set.
- `use_unicode`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, CHAR and VARCHAR and TEXT columns are returned as Unicode strings, using the configured character set.

Returns:

(no returns)

4.2 Method: commit

Commit changes within transaction.

Arguments:

(no arguments)

Returns:*(no returns)*

4.3 Method: disconnect

Disconnect from TestResultWebApp's database.

Arguments:*(no arguments)***Returns:***(no returns)*

4.4 Method: cleanAllTables

Delete all table data. Please be careful before calling this method.

Arguments:*(no arguments)***Returns:***(no returns)*

4.5 Method: sCreateNewTestResult

Creates a new test result in `tbl_result`. This is the main table which is linked to all other data by means of `test_result_id`.

Arguments:

- `_tbl_prj_project`
/ *Condition*: required / *Type*: str /
Project information.
- `_tbl_prj_variant`
/ *Condition*: required / *Type*: str /
Variant information.
- `_tbl_prj_branch`
/ *Condition*: required / *Type*: str /
Branch information.
- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_result_interpretation`
/ *Condition*: required / *Type*: str /
Result interpretation.
- `_tbl_result_time_start`
/ *Condition*: required / *Type*: str /
Test result start time as format %Y-%m-%d %H:%M:%S.
- `_tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Test result end time as format %Y-%m-%d %H:%M:%S.

- `_tbl_result_version_sw_target`
/ *Condition*: required / *Type*: str /
Software version information.
- `_tbl_result_version_sw_test`
/ *Condition*: required / *Type*: str /
Test version information.
- `_tbl_result_version_target`
/ *Condition*: required / *Type*: str /
Hardware version information.
- `_tbl_result_jenkinsurl`
/ *Condition*: required / *Type*: str /
Jenkins's url in case test result is executed by jenkins.
- `_tbl_result_reporting_qualitygate`
/ *Condition*: required / *Type*: str /
Quality gate information for reporting.

Returns:

- `_tbl_test_result_id`
/ *Type*: str /
test_result_id of new test result.

4.6 Method: nCreateNewFile

Create new file entry in `tbl_file` table.

Arguments:

- `_tbl_file_name`
/ *Condition*: required / *Type*: str /
File name information.
- `_tbl_file_tester_account`
/ *Condition*: required / *Type*: str /
Tester account information.
- `_tbl_file_tester_machine`
/ *Condition*: required / *Type*: str /
Test machine information.
- `_tbl_file_time_start`
/ *Condition*: required / *Type*: str /
Test file start time as format %Y-%m-%d %H:%M:%S.
- `_tbl_file_time_end`
/ *Condition*: required / *Type*: str /
Test file end time as format %Y-%m-%d %H:%M:%S.
- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to `tbl_result` table.

- `_tbl_file_origin`
/ *Condition*: required / *Type*: str /
Origin (test framework) of test file. Default is "ROBFW"

Returns:

- `iInsertedID`
/ *Type*: int /
ID of new entry.

4.7 Method: vCreateNewHeader

Create a new header entry in `tbl_file_header` table which is linked with the file.

Arguments:

- `_tbl_file_id`
/ *Condition*: required / *Type*: int /
File ID information.
- `_tbl_header_testtoolconfiguration_testtoolname`
/ *Condition*: required / *Type*: str /
Test tool name.
- `_tbl_header_testtoolconfiguration_testtoolversionstring`
/ *Condition*: required / *Type*: str /
Test tool version.
- `_tbl_header_testtoolconfiguration_projectname`
/ *Condition*: required / *Type*: str /
Project name.
- `_tbl_header_testtoolconfiguration_logfileencoding`
/ *Condition*: required / *Type*: str /
Encoding of logfile.
- `_tbl_header_testtoolconfiguration_pythonversion`
/ *Condition*: required / *Type*: str /
Python version info.
- `_tbl_header_testtoolconfiguration_testfile`
/ *Condition*: required / *Type*: str /
Test file name.
- `_tbl_header_testtoolconfiguration_logfilepath`
/ *Condition*: required / *Type*: str /
Path to log file.
- `_tbl_header_testtoolconfiguration_logfilemode`
/ *Condition*: required / *Type*: str /
Mode of log file.
- `_tbl_header_testtoolconfiguration_ctrlfilepath`
/ *Condition*: required / *Type*: str /
Path to control file.

- `_tbl_header_testtoolconfiguration_configfile`
/ *Condition*: required / *Type*: str /
Path to configuration file.
- `_tbl_header_testtoolconfiguration_confname`
/ *Condition*: required / *Type*: str /
Configuration name.
- `_tbl_header_testfileheader_author`
/ *Condition*: required / *Type*: str /
File author.
- `_tbl_header_testfileheader_project`
/ *Condition*: required / *Type*: str /
Project information.
- `_tbl_header_testfileheader_testfiledate`
/ *Condition*: required / *Type*: str /
File creation date.
- `_tbl_header_testfileheader_version_major`
/ *Condition*: required / *Type*: str /
File major version.
- `_tbl_header_testfileheader_version_minor`
/ *Condition*: required / *Type*: str /
File minor version.
- `_tbl_header_testfileheader_version_patch`
/ *Condition*: required / *Type*: str /
File patch version.
- `_tbl_header_testfileheader_keyword`
/ *Condition*: required / *Type*: str /
File keyword.
- `_tbl_header_testfileheader_shortdescription`
/ *Condition*: required / *Type*: str /
File short description.
- `_tbl_header_testexecution_useraccount`
/ *Condition*: required / *Type*: str /
Tester account who run the execution.
- `_tbl_header_testexecution_computername`
/ *Condition*: required / *Type*: str /
Machine name which is executed on.
- `_tbl_header_testrequirements_documentmanagement`
/ *Condition*: required / *Type*: str /
Requirement management information.
- `_tbl_header_testrequirements_testenvironment`
/ *Condition*: required / *Type*: str /
Requirement environment information.

- `_tbl_header_testbenchconfig_name`
/ *Condition*: required / *Type*: str /
Test bench configuration name.
- `_tbl_header_testbenchconfig_data`
/ *Condition*: required / *Type*: str /
Test bench configuration data.
- `_tbl_header_preprocessor_filter`
/ *Condition*: required / *Type*: str /
Preprocessor filter information.
- `_tbl_header_preprocessor_parameters`
/ *Condition*: required / *Type*: str /
Preprocessor parameters definition.

Returns:*(no returns)*

4.8 Method: nCreateNewSingleTestCase

Create single testcase entry in `tbl_case` table immediately.**Arguments:**

- `_tbl_case_name`
/ *Condition*: required / *Type*: str /
Test case name.
- `_tbl_case_issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `_tbl_case_tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).
- `_tbl_case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.
- `_tbl_case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `_tbl_case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repeatition count.
- `_tbl_case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.
- `_tbl_case_time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format `%Y-%m-%d %H:%M:%S`.

- `_tbl_case_result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `_tbl_case_result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `_tbl_case_result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `_tbl_case_counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `_tbl_case_lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to file in `tbl_result` table.
- `_tbl_file_id`
/ *Condition*: required / *Type*: int /
Test file ID for linking to file in `tbl_file` table.

Returns:

- `iInsertedID`
/ *Type*: int /
ID of new entry.

4.9 Method: nCreateNewTestCase

Create bulk of test case entries: new test cases are buffered and inserted as bulk.

Once `__NUM_BUFFERED_ELEMENTS_FOR_EXECUTE` is reached, the creation query is executed.

Arguments:

- `_tbl_case_name`
/ *Condition*: required / *Type*: str /
Test case name.
- `_tbl_case_issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `_tbl_case_tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).
- `_tbl_case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.

- `_tbl_case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `_tbl_case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repeatition count.
- `_tbl_case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.
- `_tbl_case_time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format %Y-%m-%d %H:%M:%S.
- `_tbl_case_result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `_tbl_case_result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `_tbl_case_result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `_tbl_case_counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `_tbl_case_lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for linking to file in `tbl_result` table.
- `_tbl_file_id`
/ *Condition*: required / *Type*: int /
Test file ID for linking to file in `tbl_file` table.

Returns:*(no returns)***4.10 Method: vCreateTags**

Create tag entries.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

- `_tbl_usr_result_tags`
/ *Condition*: required / *Type*: str /
User tags information.

Returns:*(no returns)*

4.11 Method: vSetCategory

Create category entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `tbl_result_category_main`
/ *Condition*: required / *Type*: str /
Category information.

Returns:*(no returns)*

4.12 Method: vUpdateStartEndTime

Create start-end time entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_result_time_start`
/ *Condition*: required / *Type*: str /
Result start time as format %Y-%m-%d %H:%M:%S.
- `_tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Result end time as format %Y-%m-%d %H:%M:%S.

Returns:*(no returns)*

4.13 Method: arGetCategories

Get existing categories.

Arguments:*(no arguments)***Returns:**

- `arCategories`
/ *Type*: list /
List of existing categories.

4.14 Method: vCreateAbortReason

Create abort reason entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_abort_reason`
/ *Condition*: required / *Type*: str /
Abort reason.
- `_tbl_abort_message`
/ *Condition*: required / *Type*: str /
Detail message of abort.

Returns:

(no returns)

4.15 Method: vCreateReanimation

Create reanimation entry.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `_tbl_num_of_reanimation`
/ *Condition*: required / *Type*: int /
Counter of target reanimation during execution.

Returns:

(no returns)

4.16 Method: vCreateCCRdata

Create CCR data per test case.

Arguments:

- `_tbl_test_case_id`
/ *Condition*: required / *Type*: int /
test case ID.
- `lCCRdata`
/ *Condition*: required / *Type*: list /
list of CCR data.

Returns:

(no returns)

4.17 Method: vFinishTestResult

Finish upload:

- First do bulk insert of rest of test cases if buffer is not empty.
- Then set state to "new report".

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

4.18 Method: vUpdateEvtbls

Call `update_evtbls` stored procedure.

Arguments:

(no arguments)

Returns:

(no returns)

4.19 Method: vUpdateEvtbl

Call `update_evtbl` stored procedure to update provided `test_result_id`.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

4.20 Method: vEnableForeignKeyCheck

Switch `foreign_key_checks` flag.

Arguments:

- `enable`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, enable foreign key constraint.

Returns:

(no returns)

4.21 Method: sGetLatestFileID

Get latest file ID from tbl_file table.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

- `_tbl_file_id`
/ *Type*: int /
File ID.

4.22 Method: vUpdateFileEndTime

Update test file end time.

Arguments:

- `_tbl_file_id`
/ *Condition*: required / *Type*: int /
File ID to be updated.
- `_tbl_file_time_end`
/ *Condition*: required / *Type*: str /
File end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

4.23 Method: vUpdateResultEndTime

Update test result end time.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be updated.
- `_tbl_result_time_end`
/ *Condition*: required / *Type*: str /
Result end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

4.24 Method: bExistingResultID

Verify the given test result UUID is existing in tbl_result table or not.

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be verified.

Returns:

- `bExisting`
/ *Type*: bool /
True if test result UUID is already existing.

4.25 Method: `arGetProjectVersionSWByID`

Get the project and version_sw information of given *test_result_id*

Arguments:

- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be get the information.

Returns:

- / *Type*: tuple /
None if test result UUID is not existing, else the tuple which contains project and version_sw: (project, variant)
is returned.

Chapter 5

rest_api_db_access.py

RestApiDBAccess class provide methods to interact with TestResultWebApp's REST APIs.

This class implements the **DBAccessInterface** and extends it. It includes methods for connecting to the database, handling API requests, creating, updating, and calling stored procedures in the database via RESTful API calls.

5.1 Method: get_certs_file

Retrieves SSL certificates and saves them to a temporary file for request's verification.

Returns:

/ *Type*: str /

The path to the temporary file containing SSL certificates.

5.2 Method: encrypt_password

Encrypts a password using a public key.

Arguments:

- password

/ *Type*: str /

The password to encrypt.

- pubkey

/ *Type*: str /

The public key used for encryption.

Returns:

/ *Type*: str /

The encrypted password.

5.3 Method: connect

Connects to the database via REST API using the provided credentials.

Arguments:

- host

/ *Condition*: required / *Type*: str /

URL which is hosted the TestResultWebApp's REST APIs.

- `user`
/ *Condition*: required / *Type*: str /
User name for database authentication.
- `passwd`
/ *Condition*: required / *Type*: str /
User's password for database authentication.
- `database`
/ *Condition*: required / *Type*: str /
Database name.

Returns:

(no returns)

5.4 Method: disconnect

Disconnect from TestResultWebApp's database.

Arguments:

(no arguments)

Returns:

(no returns)

5.5 Method: arGetCategories

Get existing categories.

Arguments:

(no arguments)

Returns:

- `arCategories`
/ *Type*: list /
List of existing categories.

5.6 Method: bExistingResultID

Verify the given test result UUID is existing or not.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be verified.

Returns:

/ *Type*: bool /
True if given `result_id` is already existing.

5.7 Method: sGetLatestFileID

Get latest file ID of all result or given `result_id`.

Arguments:

- `result_id`
/ *Condition*: optional / *Type*: str /
Test result ID. If used, the latest file id of given `result_id` is returned.

Returns:

/ *Type*: int /
Latest file ID.

5.8 Method: arGetProjectVersionSWByID

Get the project and version_sw information of given `result_id`

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be get the information.

Returns:

- / *Type*: tuple /
None if test result UUID is not existing, else the tuple which contains project and version_sw: (project, variant) is returned.

5.9 Method: sCreateNewTestResult

Creates a new test result.

Arguments:

- `project`
/ *Condition*: required / *Type*: str /
Project information.
- `variant`
/ *Condition*: required / *Type*: str /
Variant information.
- `branch`
/ *Condition*: required / *Type*: str /
Branch information.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `result_interpretation`
/ *Condition*: required / *Type*: str /
Result interpretation.

- `result_start_time`
/ *Condition*: required / *Type*: str /
Test result start time as format %Y-%m-%d %H:%M:%S.
- `result_end_time`
/ *Condition*: required / *Type*: str /
Test result end time as format %Y-%m-%d %H:%M:%S.
- `result_version_sw_target`
/ *Condition*: required / *Type*: str /
Software version information.
- `result_version_sw_test`
/ *Condition*: required / *Type*: str /
Test version information.
- `result_version_hw`
/ *Condition*: required / *Type*: str /
Hardware version information.
- `result_build_url`
/ *Condition*: required / *Type*: str /
Link to the execution of test result (Jenkins, Gitlab CI/CD, ...).
- `result_report_qualitygate`
/ *Condition*: required / *Type*: str /
Quality gate information for reporting.

Returns:

- `result_id`
/ *Type*: str /
`test_result_id` of new test result.

5.10 Method: nCreateNewFile

Create new result file.

Arguments:

- `file_name`
/ *Condition*: required / *Type*: str /
File name information.
- `file_tester_account`
/ *Condition*: required / *Type*: str /
Tester account information.
- `file_tester_machine`
/ *Condition*: required / *Type*: str /
Test machine information.
- `file_time_start`
/ *Condition*: required / *Type*: str /
Test file start time as format %Y-%m-%d %H:%M:%S.

- `file_time_end`
/ *Condition*: required / *Type*: str /
Test file end time as format %Y-%m-%d %H:%M:%S.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result to which this result file belongs.
- `file_origin`
/ *Condition*: required / *Type*: str /
Origin (test framework) of test file."

Returns:

- / *Type*: int /
ID of new entry.

5.11 Method: vCreateNewHeader

Create a new result file header.

Arguments:

- `file_id`
/ *Condition*: required / *Type*: int /
File ID information.
- `testtoolconfiguration_testtoolname`
/ *Condition*: required / *Type*: str /
Test tool name.
- `testtoolconfiguration_testtoolversionstring`
/ *Condition*: required / *Type*: str /
Test tool version.
- `testtoolconfiguration_projectname`
/ *Condition*: required / *Type*: str /
Project name.
- `testtoolconfiguration_logfileencoding`
/ *Condition*: required / *Type*: str /
Encoding of logfile.
- `testtoolconfiguration_pythonversion`
/ *Condition*: required / *Type*: str /
Python version info.
- `testtoolconfiguration_testfile`
/ *Condition*: required / *Type*: str /
Test file name.
- `testtoolconfiguration_logfilepath`
/ *Condition*: required / *Type*: str /
Path to log file.
- `testtoolconfiguration_logfilemode`
/ *Condition*: required / *Type*: str /
Mode of log file.

- `testtoolconfiguration_ctrlfilepath`
/ *Condition*: required / *Type*: str /
Path to control file.
- `testtoolconfiguration_configfile`
/ *Condition*: required / *Type*: str /
Path to configuration file.
- `testtoolconfiguration_confname`
/ *Condition*: required / *Type*: str /
Configuration name.
- `testfileheader_author`
/ *Condition*: required / *Type*: str /
File author.
- `testfileheader_project`
/ *Condition*: required / *Type*: str /
Project information.
- `testfileheader_testfiledate`
/ *Condition*: required / *Type*: str /
File creation date.
- `testfileheader_version_major`
/ *Condition*: required / *Type*: str /
File major version.
- `testfileheader_version_minor`
/ *Condition*: required / *Type*: str /
File minor version.
- `testfileheader_version_patch`
/ *Condition*: required / *Type*: str /
File patch version.
- `testfileheader_keyword`
/ *Condition*: required / *Type*: str /
File keyword.
- `testfileheader_shortdescription`
/ *Condition*: required / *Type*: str /
File short description.
- `testexecution_useraccount`
/ *Condition*: required / *Type*: str /
Tester account who run the execution.
- `testexecution_computername`
/ *Condition*: required / *Type*: str /
Machine name which is executed on.
- `testrequirements_documentmanagement`
/ *Condition*: required / *Type*: str /
Requirement management information.

- `testrequirements_testenvironment`
/ *Condition*: required / *Type*: str /
Requirement environment information.
- `testbenchconfig_name`
/ *Condition*: required / *Type*: str /
Test bench configuration name.
- `testbenchconfig_data`
/ *Condition*: required / *Type*: str /
Test bench configuration data.
- `preprocessor_filter`
/ *Condition*: required / *Type*: str /
Preprocessor filter information.
- `preprocessor_parameters`
/ *Condition*: required / *Type*: str /
Preprocessor parameters definition.

Returns:*(no returns)***5.12 Method: nCreateNewSingleTestCase**

Create single test case.

Arguments:

- `case_name`
/ *Condition*: required / *Type*: str /
Test case name.
- `case_issue`
/ *Condition*: required / *Type*: str /
Test case issue ID.
- `case_tcid`
/ *Condition*: required / *Type*: str /
Test case ID (used for testmanagement tool).
- `case_fid`
/ *Condition*: required / *Type*: str /
Test case requirement (function) ID.
- `case_testnumber`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `case_repeatcount`
/ *Condition*: required / *Type*: int /
Test case repeatition count.
- `case_component`
/ *Condition*: required / *Type*: str /
Component which test case is belong to.

- `case_time_start`
/ *Condition*: required / *Type*: str /
Test case start time as format %Y-%m-%d %H:%M:%S.
- `case_result_main`
/ *Condition*: required / *Type*: str /
Test case main result.
- `case_result_state`
/ *Condition*: required / *Type*: str /
Test case completion state.
- `case_result_return`
/ *Condition*: required / *Type*: int /
Test case result code (as integer).
- `case_counter_resets`
/ *Condition*: required / *Type*: int /
Counter of target reset within test case execution.
- `case_lastlog`
/ *Condition*: required / *Type*: str /
Traceback information when test case is failed.
- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result to which this test case belongs.
- `file_id`
/ *Condition*: required / *Type*: int /
ID of result file to which this test case belongs.

Returns:

/ *Type*: int /
ID of new entry.

5.13 Method: nCreateNewTestCase

Alias for `nCreateNewSingleTestCase`, used in older import tools to import a bulk of test cases at once.

5.14 Method: vCreateAbortReason

Create abort reason entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `abort_reason`
/ *Condition*: required / *Type*: str /
Abort reason.

- `abort_message`
/ *Condition*: required / *Type*: str /
Detail message of abort.

Returns:

(no returns)

5.15 Method: vCreateCCRdata

Create CCR data per test case.

Arguments:

- `_tbl_test_case_id`
/ *Condition*: required / *Type*: int /
test case ID.
- `lCCRdata`
/ *Condition*: required / *Type*: list /
list of CCR data.

Returns:

(no returns)

5.16 Method: vCreateTags

Create tag entries.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `tags`
/ *Condition*: required / *Type*: str /
User tags information.

Returns:

(no returns)

5.17 Method: vCreateReanimation

Create reanimation entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `num_of_reanimation`
/ *Condition*: required / *Type*: int /
Counter of target reanimation during execution.

Returns:

(no returns)

5.18 Method: vSetCategory

Create category entry.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.
- `category_main`
/ *Condition*: required / *Type*: str /
Category information.

Returns:

(no returns)

5.19 Method: vUpdateFileEndTime

Update test file end time.

Arguments:

- `file_id`
/ *Condition*: required / *Type*: int /
File ID to be updated.
- `time_end`
/ *Condition*: required / *Type*: str /
File end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

5.20 Method: vUpdateResultEndTime

Update test result end time.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
Result UUID to be updated.
- `time_end`
/ *Condition*: required / *Type*: str /
Result end time as format %Y-%m-%d %H:%M:%S.

Returns:

(no returns)

5.21 Method: vFinishTestResult

Update state of given test result to "new report".

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

5.22 Method: vUpdateEvtbl

Call `update_evtbl` stored procedure to update given `result_id`.

Arguments:

- `result_id`
/ *Condition*: required / *Type*: str /
UUID of test result.

Returns:

(no returns)

5.23 Method: vUpdateEvtbls

Call `update_evtbls` stored procedure.

Arguments:

(no arguments)

Returns:

(no returns)

Chapter 6

DBAccessFactory.py

Chapter 7

Appendix

About this package:

Table 7.1: Package setup

Setup parameter	Value
Name	TestResultDBAccess
Version	0.2.1
Date	30.06.2026
Description	Interfaces to access TestResultWebApp database
Package URL	python-testresultdbaccess
Author	Tran Duy Ngoan
Email	Ngoan.TranDuy@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 8

History

0.1.0	22.04.2024
<i>Initial version</i>	
0.1.1	15.05.2024
<i>Fixed issues of workflow to publish package to pypi</i>	
0.1.2	17.05.2024
<i>Added requirements_ linux.sh file, which helps to install Linux dependency packages</i>	
0.1.3	17.05.2024
<i>Corrected link to package on pypi</i>	
0.1.4	01.08.2024
- Fixed typos and renamed files properly - Fixed wam issue with test account due to ssl verification	
0.1.5	10.04.2025
<i>Add step to get cookies by using webdriver in case of failure kerberos request</i>	
0.1.6	11.04.2025
<i>Add step to try the requests without verification</i>	
0.2.0	07.05.2026
<i>Add missing dependencies for this package</i>	
0.2.1	30.06.2026
<i>Fix typo and console log messages</i>	

TestResultDBAccess.pdf*Created at 07.09.2026 - 09:47:20**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.13 RobotLog2DB

RobotLog2DB

v. 1.6.0

Tran Duy Ngoan

24.02.2026

CONTENTS

CONTENTS

Contents

1	Introduction	1
2	Description	2
2.1	Get Robot Framework XML result	2
2.1.1	Robot Framework test case Settings	2
2.1.2	Sample Robot Framework Test Case	3
2.1.3	Execute Robot Framework test case(s) to get result file	4
2.2	Tool features	5
2.2.1	Usage	5
2.2.2	Verify provided arguments	6
2.2.3	Searching *.xml result file(s)	6
2.2.4	Handle essential information for TestResultWebApp	6
2.2.5	Append to existing execution result	8
2.2.6	Switch Database Access Interface	9
2.3	Display on WebApp	10
3	robotlog2db.py	11
3.1	Function: collect_xml_result_files	11
3.2	Function: validate_xml_result	11
3.3	Function: is_valid_uuid	12
3.4	Function: is_valid_config	12
3.5	Function: get_from_tags	13
3.6	Function: get_branch_from_swversion	13
3.7	Function: format_time	14
3.8	Function: retrieve_result_starttime	14
3.9	Function: retrieve_result_endtime	14
3.10	Function: process_suite_metadata	14
3.11	Function: process_metadata	15
3.12	Function: process_suite	15
3.13	Function: process_test	16
3.14	Function: process_config_file	16
3.15	Function: normalize_path	17
3.16	Function: RobotLog2DB	17
3.17	Class: Logger	18
3.17.1	Method: config	18
3.17.2	Method: log	18
3.17.3	Method: log_warning	19

CONTENTS	CONTENTS
3.17.4 Method: log_error	19
4 Appendix	20
5 History	21
B	

Chapter 1

Introduction

robotframework-robotlog2db is a command-line tool that enables you to import Robot Framework XML result files into TestResultWebApp's database for presenting an overview about the whole test execution and details of each test result.

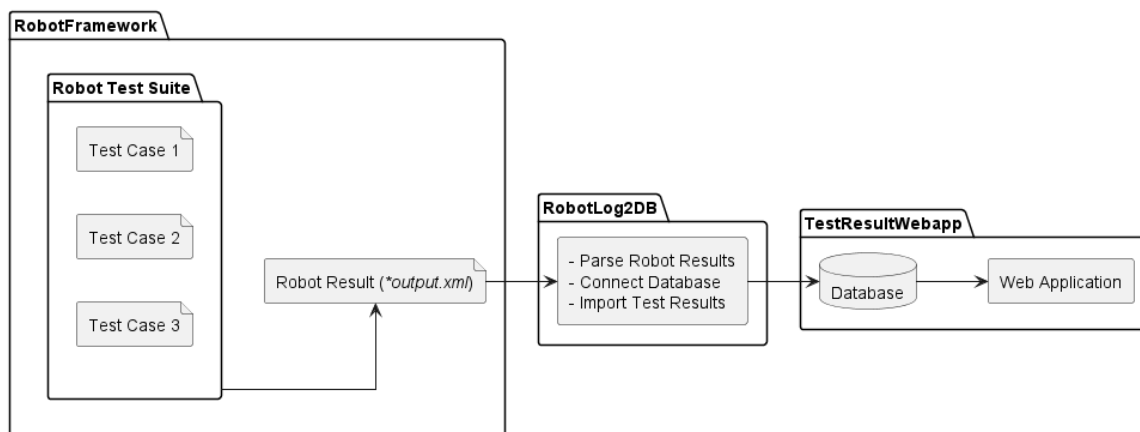


Figure 1.1: Tool data flow

robotframework-robotlog2db tool requires several arguments, including the location of the [Robot Framework XML result file\(s\)](#) to parse all information of test execution result and TestResultWebApp's database credential for importing that result.

[TestResultWebApp](#) requires some mandatory information to manage and display the test result properly. Therefore, they should be provided in Robot Framework test case before execution, then they will be available in XML result file for importing.

However, you can use optional arguments of **robotframework-robotlog2db** tool to provide those data if they are missing or you want to overwrite them with the expected values.

Finally, **robotframework-robotlog2db** also allows you to append existing results in the database, which is helpful when you need to update previous test results or add the missing XML result file(s) from previous tool execution.

Chapter 2

Description

TestResultWebApp requires **project/variant**, **software version** as essential data to manage results in database and some optional information such as **component**, ... to visualize test data properly on web application.

These data are not the mandatory fields of Robot Framework test case or XML result file. So that, they should be defined as `Metadata` and `[Tags]` in Robot Framework test case, then the generated XML result file includes all necessary information of test execution for importing to TestResultWebApp's database.

The next section [Get Robot Framework XML result](#) will introduce you the Robot Framework test case settings to make necessary information available for displaying on TestResultWebApp.

In case you have already had the Robot Framework XML file(s) without those definitions, do not worry because **robotframework-robotlog2db** will handle those data with default values.

Futhermore, you can specify them with your expected values by providing as command line arguments when executing **robotframework-robotlog2db** tool. The detail of those command line arguments are described in [Specify essential information with optional arguments](#) section.

2.1 Get Robot Framework XML result

2.1.1 Robot Framework test case Settings

The below document is recommended Robot Framework test case `Settings` before execution. So that, the generated `*.xml` result file will contain all necessary information.

For the whole test execution:

- Project/Variant

```
Metadata    project    ${Project_name}
```

- Versions

```
Metadata    version_hw    ${Software_version}
Metadata    version_hw    ${Hardware_version}
Metadata    version_test  ${Test_version}
```

For the Suite/File information:

- Description/Documentation

```
Documentation  ${Suite_description}
```

- Author

```
Metadata    author    ${Author_name}
```

- Component

```
Metadata    component    ${Component_name}
```

- Test Tool - Test framework and Python version, e.g **Robot Framework 3.2rc2 (Python 3.9.0 on win32)**

```
Metadata    testtool    ${Test_tool}
```

- Test Machine

```
Metadata    machine    ${COMPUTERNAME}
```

- Tester

```
Metadata    tester    ${USER}
```

For test case information:

- Issue ID

```
[Tags]      ISSUE-${ISSUE_ID}
```

- Testcase ID

```
[Tags]      TCID-${TC_ID}
```

- Requirement ID

```
[Tags]      FID-${REQ_ID}
```

2.1.2 Sample Robot Framework Test Case

Sample Robot Framework test case with the necessary information for importing to TestResultWebApp's database:

```
*** Settings ***
# Test execution level
Metadata    project      ROBFW          # Project/Variant
Metadata    version_sw   SW_VERSION_0.1  # Software version
Metadata    version_hw   HW_VERSION_0.1  # Hardware version
Metadata    version_test TEST_VERSION_0.1 # Test version

# File/Suite level
Documentation    This is description for robot test file
Metadata    author      Tran Duy Ngoan (RBVH/ECM1)
Metadata    component    Import_Tools
Metadata    testtool     Robot Framework 4.1.3 (Python 3.9.16 on win32)
Metadata    machine      ${COMPUTERNAME}
Metadata    tester       ${USER}

*** Test Cases ***
Testcase 01
[Tags]      ISSUE-001    TCID-1001    FID-112    FID-111
Log         This is Testcase 01

Testcase 02
[Tags]      ISSUE-RTC-003    TCID-1002    FID-113
Log         This is Testcase 01
```

Listing 2.1: Sample Robot Framework testcase

Hint

Instead of setting `Metadata` in Robot Framework test case, you can also specify them as the `--metadata` [command line argument](#) when executing Robot Framework test cases. Furthermore, in case you are using RobotFramework AIO, above highlighted `Metadata` definitions are not required because they have been handled by `RobotFramework_TestsuitesManagement` library within `Suite Setup`.

2.1.3 Execute Robot Framework test case(s) to get result file

Now, execute your Robot Framework test case(s) with your IDE or [using command line](#) to get the Robot result file

```
robot your_testcases.robot
```

Or with python module

```
python -m robot your_testcases.robot
```

The default name of Robot result file is `output.xml`. Its filename can be changed by specifying other filename with argument `--output (-o)` when executing Robot Framework test case(s)

```
robot your_testcases.robot --output path/to/robot_result.xml
```

2.2 Tool features

After getting the Robot Framework **.xml* result file(s), you can use the **robotframework-robotlog2db** tool to import the test results into TestResultWebApp's database.

Its usage and enhance features are described as below.

2.2.1 Usage

Use below command to get tools's usage:

```
RobotLog2DB -h
```

The tool's usage should be showed as below:

```
usage: RobotLog2DB (RobotXMLResult to TestResultWebApp importer) [-h] [-v]
                        [--recursive] [--dryrun] [--append] [--UUID UUID]
                        [--variant VARIANT] [--versions VERSIONS] [--config CONFIG]
                        resultxmlfile server user password database

RobotLog2DB imports XML result files (default: output.xml) generated by the
Robot Framework into a WebApp database.

positional arguments:
  resultxmlfile          absolute or relative path to the result file or directory
                        of result files to be imported.
  server                 server which hosts the database (IP or URL).
  user                   user for database login.
  password               password for database login.
  database               database schema for database login.

optional arguments:
  -h, --help             show this help message and exit
  -v, --version           version of the RobotLog2DB importer.
  --recursive            if set, then the path is searched recursively for output
                        files to be imported.
  --dryrun               if set, then verify all input arguments (includes DB
                        connection) and show what would be done.
  --append               is used in combination with --UUID <UUID>.If set, allow to
                        append new result(s) to existing execution result UUID in
                        --UUID argument.
  --UUID UUID            UUID used to identify the import and version ID on webapp.
                        If not provided RobotLog2DB will generate an UUID for the
                        whole import.
  --variant VARIANT      variant name to be set for this import.
  --versions VERSIONS    metadata: Versions (Software;Hardware;Test) to be set for
                        this import (semicolon separated).
  --config CONFIG        configuration json file for component mapping information.
  --interface {db,rest} database access interface.
  --testrunurl TESTRUNURL
                        link to test execution job: Jenkins job, Gitlab CI/CD pipeline, ...
```

As above instruction, **robotframework-robotlog2db** tool requires 5 positional arguments which contains all required information for importing.

The below command is simple usage with all required arguments to import robot results into TestResultWebApp's database:

```
RobotLog2DB <output.xml> <localhost> <db\_user> <db\_pw> <db\_name>
```

Besides the executable file **robotframework-robotlog2db** you can also run tool as a Python module

```
python -m RobotLog2DB <output.xml> <localhost> <db\_user> <db\_pw> <db\_name>
```

Beside the required arguments, there are optional arguments which provides some enhance features as the following sections.

2.2.2 Verify provided arguments

Sometimes, we just want to validate the `*.xml` and database connection without changing anything in the database, the optional argument `--dryrun` can be used in this case.

When executing in dryrun mode, **robotframework-robotlog2db** will:

- Verify the provided Robot Framework `*.xml` result file is valid or not.
- Verify the database connection with provided credential.
- Verify other information which given in optional arguments.
- Just print all test cases will be imported without touching database.

This feature will helps you to ensure that there is no error when executing **robotframework-robotlog2db** tool (normal mode) to create new record(s) and update TestResultWebApp's database.

2.2.3 Searching *.xml result file(s)

The first argument `resultxmlfile` of **robotframework-robotlog2db** can be a single file or the folder that contains multiple result files.

When the folder is used, **robotframework-robotlog2db** will only search for `*.xml` file(s) under given directory and exclude any file within subdirectories as default.

In case you have result file(s) under the subdirectory of given folder and want these result files will also be imported, the optional argument `--recursive` should be used when executing **robotframework-robotlog2db** command.

When `--recursive` argument is set, **robotframework-robotlog2db** will walk through the given directory and its subdirectories to discover and collect all available `*.xml` for importing.

For example: your result folder has a structure as below:

```
logFolder
|_____ result_1.xml
|_____ result_2.xml
|_____ subFolder_1
|           |_____ result_sub_1.xml
|           |_____ subSubFolder
|                   |_____ result_sub_sub_1.xml
|_____ subFolder_2
|           |_____ result_sub_2.xml
```

- Without `--recursive`: only **result_1.xml** and **result_2.xml** are found for importing.
- With `--recursive`: all **result_1.xml**, **result_2.xml**, **result_sub_1.xml**, **result_sub_2.xml** and **result_sub_sub_1.xml** will be imported.

2.2.4 Handle essential information for TestResultWebApp

Default values

TestResultWebApp requires **Project**, **Software version** to manage the execution results and **Component** to group test cases in the displayed charts.

In case above information is missing in [test case settings](#) during the test case execution, that leads to the missing information in the `output.xml` result file. So, this missing information will be set to default values when importing with **robotframework-robotlog2db** tool:

- **Project**: will be set to default value `ROBFW` if not defined.
- **Software version**: will be set to execution time `%Y%m%d_%H%M%S` as default value.
- **Component**: will be set to default value `unknown` if not defined.

Specify essential information with optional arguments

You can also provide essential information in command line when executing the **robotframework-robotlog2db** tool with below optional arguments:

**Warning**

These below settings may overwrite the existing information of `project`, `version_sw` and `component` which have been defined in Robot Framework test case in [above section](#).

So, **only** use these arguments in case you want to define the missing information or overwrite the existing values with your expected values for the importing result.

- `--variant VARIANT` To specify the **Project/Variant** information.
- `--versions VERSIONS` To specify the **Software, Hardware** and **Test** versions information.
- `--config CONFIG` To provide a configuration `*.json` file as `CONFIG` argument. Currently, the configuration `*.json` supports below settings:
 - `"variant"` to specify the **Project/Variant** as `string` value.
 - `"version_sw"` to specify the **Software version** information as `string` value.
 - `"version_hw"` to specify the **Hardware under-test version** as `string` value.
 - `"version_test"` to specify the **Test version** as `string` value.

**Notice**

These above settings with `--config CONFIG` will have lower priority than the commandline arguments `--variant VARIANT` and `--versions VERSIONS`

- `"testtool"` to specify the **Test toolchain** as `string` value.
- `"tester"` to specify the **Test user** as `string` value.
- `"components"` to specify the **Component** information which will be displayed on [TestResultWebApp](#). Value can be:

- * `string`: to specify the same **Component** for all test casea within this execution.

```
{
  "components" : "atest",
  ...
}
```

- * `object`: to define the mapping json object between **Component** info as key and a directory path (or list of directory paths) to Robot Framework test file (`*.robot` file) as value.

```
{
  "components" : {
    "cli"      : "robot/cli",
    "core"     : "robot/core",
    "external" : "robot/external",
    "keywords" : "robot/keywords",
    "libdoc"   : "robot/libdoc",
    "connectivity" : [
      "selftest/serial",
      "selftest/ssh",
      "selftest/tcpip"
    ]
  },
  ...
}
```

As above sample configuration, the `"components"` key contains the mappings for individual components, such as **cli**, **core**, **external**, **keywords** and **libdoc**, where the value is the directory path for the corresponding Robot Framework files.

Additionally, the **connectivity** key is an example of a mapping where the value is a list of directory paths, indicating that the **connectivity** component is composed of all Robot Framework files located in multiple directories, namely **selftest/serial**, **selftest/ssh** and **selftest/tcpip**.

In other words, the component mapping can be explained as below:

- Test case(s) under **robot/cli** folder is belong **cli** component

- Test case(s) under **robot/core** folder is belong **core** component
- ...
- **connectivity** component contains all test case(s) which is located under **selftest/serial**, **selftest/ssh** and **selftest/tcpip** folders.

In case the given configuration **.json* is not valid or unsupported key is used, the corresponding error will be raised.

2.2.5 Append to existing execution result

robotframework-robotlog2db also allows you to append new test result(s) (missing from previous import, on other test setup, ...) into the existing execution result (identified by the **UUID**) in TestResultWebApp's database. The combination of optional arguments `--UUID <UUID>` and `--append` should be used in this case.

The `--append` makes **robotframework-robotlog2db** run in append mode and the `--UUID <UUID>` is used to specify the existing UUID of execution result to be appended.

For example, the result with UUID **c7991c07-4de2-4d65-8568-00c5556c82aa** is already existing in TestResultWebApp's database and you want to append result(s) in **output.xml** into that execution result.

The command will be used as below:

```
python -m RobotLog2DB output.xml localhost testuser testpw testdb --UUID ↩
↪ c7991c07-4de2-4d65-8568-00c5556c82aa --append
```

If the argument `--UUID <UUID>` is used without `--append`:

- An error will be thrown and the import job is terminated immediately if the provided **UUID** is already existing

```
FATAL ERROR: Execution result with UUID 'c7991c07-4de2-4d65-8568-00c5556c82aa' is ↩
↪ already existing.
      Please use other UUID (or remove '--UUID' argument from your command) ↩
↪ for new execution result.
      Or add '--append' argument in your command to append new result(s) to ↩
↪ this existing UUID.
```

- The importing execution result will have an identifier as the provided **UUID** if that **UUID** is not existing

If the argument `--append` is used and:

- given UUID in `--UUID <UUID>` argument is existing: the new result(s) will be appended to that **UUID**
- given UUID in `--UUID <UUID>` argument is not existing: tool will be terminated immediately with below error

```
FATAL ERROR: Execution result with UUID 'c7991c07-4de2-4d65-8568-00c5556c82aa' is not↩
↪ existing for appending.
      Please use an existing UUID to append new result(s) to that UUID.
      Or remove '--append' argument in your command to create new execution ↩
↪ result with given UUID.
```

- `--UUID <UUID>` is not provided: tool will be terminated immediately with below error

```
FATAL ERROR: '--append' argument should be used in combination with '--UUID <UUID>' ↩
↪ argument
```

Notice

When using append mode and `project` / `variant`, `version_sw` are provided within `--variant VARIANT`, `--versions VERSIONS` or `--config CONFIG` arguments, they will be validated with the existing values in database.

An error will be raised in case the given value is not matched with the existing ones. E.g:

```
FATAL ERROR: Given version software 'my_version' is different with existing ↩
↪ value 'SW01' in database.
```

2.2.6 Switch Database Access Interface

In addition to direct database connections — which can sometimes be restricted or prohibited due to security concerns — the **robotframework-robotlog2db** tool also provides users with an alternative way (interface) to import test data into the database, namely the REST API.

`--interface INTERFACE`: An optional argument to specify the database backend interface. The default value is `db` (direct connection to MySQL database). It can also be set to `rest` to import data into the database via a REST API.

Direct Connection Backend (interface="db")

The default mode of operation for the import tool is the direct connection to the MySQL database. This means that when you run the tool without specifying the `--interface` option, it automatically uses the `db` interface, establishing a direct connection to the MySQL database.

You can also explicitly specify the `db` interface by using the `--interface` option with the value `db`.

```
RobotLog2DB <output.xml> <db\_host> <db\_user> <db\_password> <db\_name> --interface db
```

This method provides flexibility, allowing users to choose between interfaces explicitly or rely on the default behavior.

REST API Backend (interface="rest")

The 'rest' interface allows users to import data into the database through a REST API. This mode is particularly useful in scenarios where direct database connections are restricted or not feasible, such as when the database is secured and direct connections are not allowed.

```
RobotLog2DB <output.xml> <api\_endpoint> <api\_user> <api\_password> <db\_name> ↔  
↪ --interface rest
```

Additional Notes



This feature offers easy switching between database backends, making it transparent and convenient for end-users.

When using the REST API interface, ensure that you have the necessary credentials and permissions to access the REST API endpoints.

2.3 Display on WebApp

As soon as the *output.xml* file(s) is imported successfully to database, the result for that execution will be available on [TestResultWebApp](#).

Above test case settings in Robot Framework test case will be reflected on **Dashboard** (General view) and **Data table** (Detailed view) as below figures:

Execution result metadata:

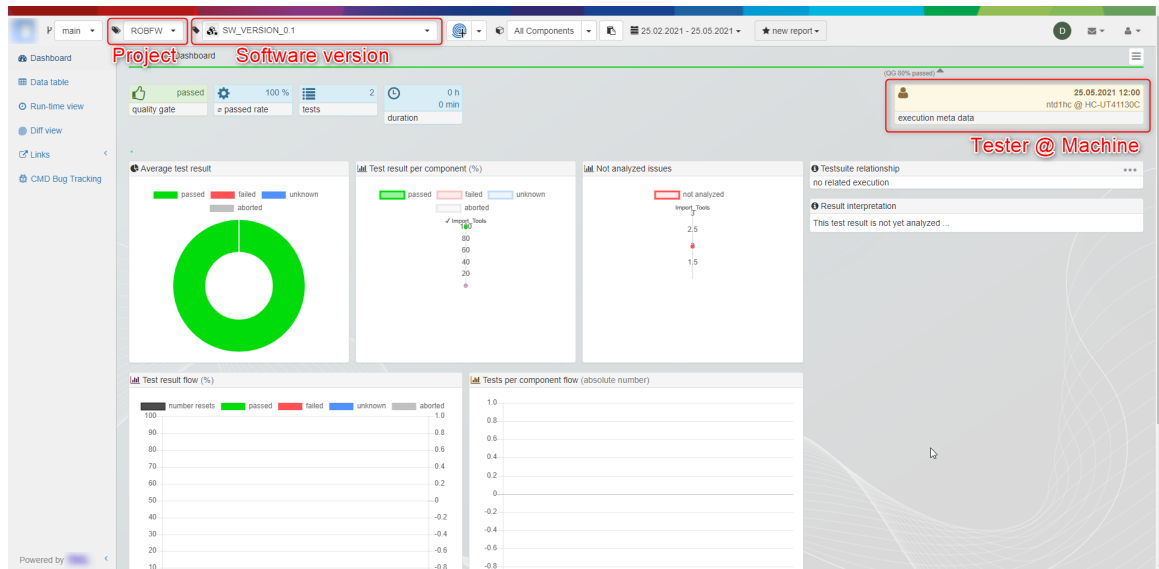


Figure 2.1: Dashboard view

Suite/File metadata and test case information:

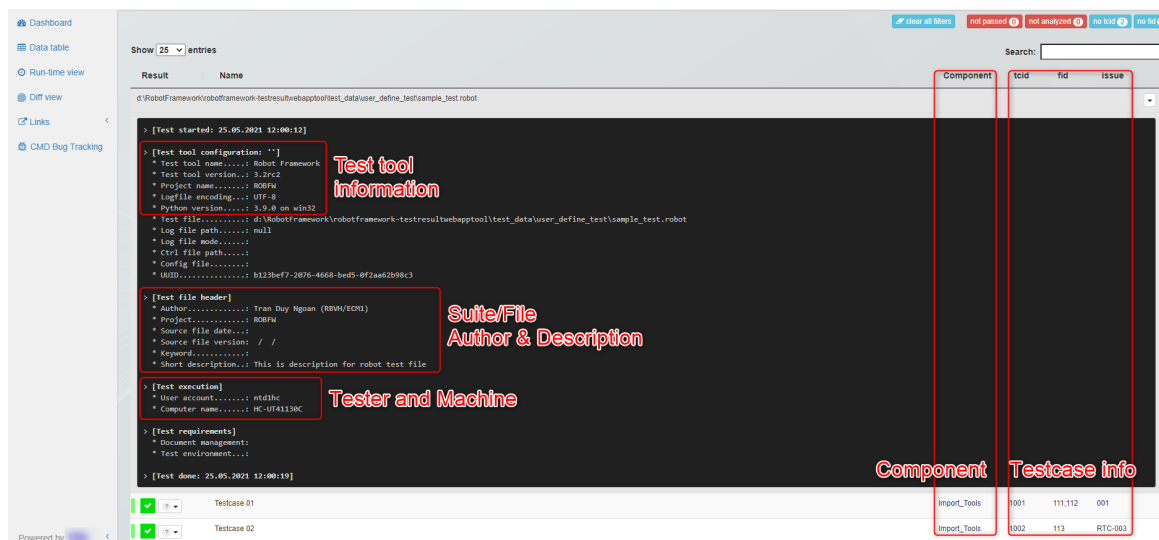


Figure 2.2: Datatable view

Chapter 3

robotlog2db.py

3.1 Function: collect_xml_result_files

Collect all valid Robot xml result file in given path.

Arguments:

- `path`
/ *Condition*: required / *Type*: str /
Path to Robot result folder or file to be searched.
- `search_recursive`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If set, the given path is searched recursively for xml result files.

Returns:

- `lFoundFiles`
/ *Type*: list /
List of valid xml result file(s) in given path.

3.2 Function: validate_xml_result

Verify the given xml result file is valid or not.

Arguments:

- `xml_result`
/ *Condition*: required / *Type*: str /
Path to Robot result file.
- `xsd_schema`
/ *Condition*: optional / *Type*: str / *Default*: <installed_folder>/xsd/robot.xsd /
Path to Robot schema *.xsd file.

System Message

WARNING/2 in <string>, line 45
[Inline emphasis start-string without end-string.](#)

- `exit_on_failure`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If set, exit with fatal error if the schema validation of given xml file failed.

Returns:

- / *Type*: bool /

True if the given xml result is valid with the provided schema *.xsd.

System Message

WARNING/2 in <string>, line 57

[Inline emphasis start-string without end-string.](#)

3.3 Function: is_valid_uuid

Verify the given UUID is valid or not.

Arguments:

- `uuid_to_test`
/ *Condition*: required / *Type*: str /
UUID to be verified.
- `version`
/ *Condition*: optional / *Type*: int / *Default*: 4 /
UUID version.

Returns:

- `bValid`
/ *Type*: bool /
True if the given UUID is valid.

3.4 Function: is_valid_config

Validate the json configuration base on given schema.

Default schema supports below information:

```
CONFIG_SCHEMA = {
    "components": [str, dict],
    "variant"    : str,
    "version_sw": str,
    "version_hw": str,
    "version_test": str,
    "testtool"   : str,
    "tester"    : str
}
```

Arguments:

- `dConfig`
/ *Condition*: required / *Type*: dict /
Json configuration object to be verified.
- `dSchema`
/ *Condition*: optional / *Type*: dict / *Default*: CONFIG_SCHEMA /
Schema for the validation.

- `bExitOnFail`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, exit tool in case the validation is fail.

Returns:

- `bValid`
/ *Type*: bool /
True if the given json configuration data is valid.

3.5 Function: get_from_tags

Extract testcase information from tags.

Example: TCID-xxxx, FID-xxxx, ...

Arguments:

- `lTags`
/ *Condition*: required / *Type*: list /
List of tag information.
- `reInfo`
/ *Condition*: required / *Type*: str /
Regex to get the expected info (ID) from tag info.

Returns:

- `lInfo`
/ *Type*: list /
List of expected information (ID)

3.6 Function: get_branch_from_swversion

Get branch name from software version information.

Convention of branch information in suffix of software version:

- All software version with .0F is the main/feature branch. The leading number is the current year. E.g. 17.0F03
- All software version with .1S, .2S, ... is a stabi branch. The leading number is the year of branching out for stabilization. The number before "S" is the order of branching out in the year.

Arguments:

- `sw_version`
/ *Condition*: required / *Type*: str /
Software version.

Returns:

- `branch_name`
/ *Type*: str /
Branch name.

3.7 Function: format_time

Format the given time string to TestResultWebApp's format for importing to db.

Arguments:

- `stime`
/ *Condition*: required / *Type*: str /
String of time.

Returns:

- `sFormattedTime`
/ *Type*: str /
TestResultWebApp's time as format `%Y-%m-%d %H:%M:%S`.

3.8 Function: retrieve_result_starttime

Retrieve starttime infomration from given result object (TestSuite or TestCase). In case the starttime in given suite is 'N/A', it will try to get this information from its children suite/test.

Arguments:

- `stime`
/ *Condition*: required / *Type*: *TestSuite* or *TestCase* object /
Result object to retrieve starttime.

Returns:

- / *Type*: str /
Start time of given result.

3.9 Function: retrieve_result_endtime

Retrieve endtime information from given result object (TestSuite or TestCase). In case the endtime in given suite is 'N/A', it will try to get this information from its children suite/test.

Arguments:

- `stime`
/ *Condition*: required / *Type*: *TestSuite* or *TestCase* object /
Result object to retrieve endtime.

Returns:

- / *Type*: str /
End time of given result.

3.10 Function: process_suite_metadata

Try to find metadata information from all suite levels.

Metadata at top suite level has a highest priority.

Arguments:

- `suite`
/ *Condition*: required / *Type*: *TestSuite* object /
Robot suite object.
- `default_metadata`
/ *Condition*: optional / *Type*: dict / *Default*: `DEFAULT_METADATA` /
Initial Metadata information for updating.

Returns:

- `dMetadata`
/ *Type*: dict /
Dictionary of Metadata information.

3.11 Function: process_metadata

Extract metadata from suite result bases on `DEFAULT_METADATA`.

Arguments:

- `metadata`
/ *Condition*: required / *Type*: dict /
Robot metadata object.
- `default_metadata`
/ *Condition*: optional / *Type*: dict / *Default*: `DEFAULT_METADATA` /
Initial Metadata information for updating.

Returns:

- `dMetadata`
/ *Type*: dict /
Dictionary of Metadata information.

3.12 Function: process_suite

Process to the lowest suite level (test file):

- Create new file and its header information
- Then, process all child test cases

Arguments:

- `db`
/ *Condition*: required / *Type*: *CDataBase* object /
CDataBase object.
- `suite`
/ *Condition*: required / *Type*: *TestSuite* object /
Robot suite object.
- `_tbl_test_result_id`
/ *Condition*: required / *Type*: str /
UUID of test result for importing.

- `root_metadata`
/ *Condition*: required / *Type*: dict /
Metadata information from root level.
- `dConfig`
/ *Condition*: required / *Type*: dict / *Default*: None /
Configuration data which is parsed from given json configuration file.

Returns:*(no returns)***3.13 Function: process_test**

Process test case data and create new test case record.

Arguments:

- `db`
/ *Condition*: required / *Type*: *CDataBase* object /
CDataBase object.
- `test`
/ *Condition*: required / *Type*: *TestCase* object /
Robot test object.
- `file_id`
/ *Condition*: required / *Type*: int /
File ID for mapping.
- `test_result_id`
/ *Condition*: required / *Type*: str /
Test result ID for mapping.
- `metadata_info`
/ *Condition*: required / *Type*: dict /
Metadata information.
- `test_number`
/ *Condition*: required / *Type*: int /
Order of test case in file.

Returns:*(no returns)***3.14 Function: process_config_file**

Parse information from configuration file:

- `component`:

```
{
    "components" : {
        "componentA" : "componentA/path/to/testcase",
        "componentB" : "componentB/path/to/testcase",
        "componentC" : [
            "componentC1/path/to/testcase",
```

```

        "componentC2/path/to/testcase"
    ]
}
}

```

Then all testcases which their paths contain componentA/path/to/testcase will be belong to componentA, ...

- variant, version_sw: configuration file has low priority than command line.

Arguments:

- config_file
/ *Condition*: required / *Type*: str /
Path to configuration file.

Returns:

- dConfig
/ *Type*: dict /
Configuration object.

3.15 Function: normalize_path

Normalize path file.

Arguments:

- sPath
/ *Condition*: required / *Type*: str /
Path file to be normalized.
- sNPath
/ *Type*: str /
Normalized path file.

3.16 Function: RobotLog2DB

Import robot results from output.xml to TestResultWebApp's database.

Flow to import Robot results to database:

1. Process provided arguments from command line.
2. Parse Robot results.
3. Connect to database.
4. Import results into database.
5. Disconnect from database.

Arguments:

- args
/ *Condition*: required / *Type*: *ArgumentParser* object /
Argument parser object which contains:
 - resultxmlfile : path to the xml result file or directory of result files to be imported.

- *server* : server which hosts the database (IP or URL).
- *user* : user for database login.
- *password* : password for database login.
- *database* : database name.
- *recursive* : if True, then the path is searched recursively for log files to be imported.
- *dryrun* : if True, then verify all input arguments (includes DB connection) and show what would be done.
- *append* : if True, then allow to append new result(s) to existing execution result UUID which is provided by --UUID argument.
- *UUID* : UUID used to identify the import and version ID on TestResultWebApp.
- *variant* : variant name to be set for this import.
- *versions* : metadata: Versions (Software;Hardware;Test) to be set for this import.
- *config* : configuration json file for component mapping information.
- *testrunurl*: link to test execution job: Jenkins job, Gitlab CI/CD pipeline, ...

Returns:*(no returns)*

3.17 Class: Logger

Logger class for logging message.

3.17.1 Method: config

Configure Logger class.

Arguments:

- `output_console`
/ *Condition*: optional / *Type*: bool / *Default*: True /
Write message to console output.
- `output_logfile`
/ *Condition*: optional / *Type*: str / *Default*: None /
Path to log file output.
- `dryrun`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If set, a prefix as 'dryrun' is added for all messages.

Returns:*(no returns)*

3.17.2 Method: log

Write log message to console/file output.

Arguments:

- `msg`
/ *Condition*: optional / *Type*: str / *Default*: " /
Message which is written to output.
- `color`
/ *Condition*: optional / *Type*: str / *Default*: None /
Color style for the message.

- `indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.

Returns:

(no returns)

3.17.3 Method: log_warning

Write warning message to console/file output.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Warning message which is written to output.

Returns:

(no returns)

3.17.4 Method: log_error

Write error message to console/file output.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Error message which is written to output.
- `fatal_error`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If set, tool will terminate after logging error message.

Returns:

(no returns)

Chapter 4

Appendix

About robotframework-robotlog2db:

Author

Tran Duy Ngoan (Ngoan.TranDuy@vn.bosch.com)

Version

1.6.0 (24.02.2026)

Short Description

Imports robot results to TestResultWebApp database

Homepage

[robotframework-robotlog2db](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 5

History

0.1.0	07/2022
<i>Initial version</i>	
1.2.1	22.08.2022
<i>Rework repository's document bases on GenPackageDoc</i>	
1.2.2	13.10.2022
- Fix findings and enhance README and document files - Change argument name 'outputfile' to 'resultxmlfile'	
1.2.3	10.11.2022
<i>Rename package to RobotLog2DB</i>	
1.2.4	18.11.2022
<i>Add -append argument which allow to append into existing UUID</i>	
1.3.0	06.12.2022
- Rework optional arguments in command line - Improve coding: logging messages, shorten variable names, ...	
1.3.1	09.01.2023
<i>Improve messages when verify configuration json file</i>	
1.3.2	28.02.2023
- Rename key to 'components' in configuration json file - Change implementation of append mode to raise errors without proper given UUID - Enhance console log with component and append mode information - Add more supported keys in configuration json file	
1.3.3	01.03.2023
<i>Add a validation for xmlresultfile with robot schema</i>	
1.3.4	13.03.2023
- Add a validation for existing project/variant and version_sw in db when using append mode - Remove db maxlength handlers: truncations and validation	
1.3.5	21.03.2023
<i>Support 4 byte characters for importing to db</i>	
1.3.6	05.04.2023

CHAPTER 5. HISTORY

- Enhance console log with test case counter and component statistics - Add try/except for database access	
1.3.7	14.06.2023
Update README with new instruction for installation and image's links	
1.3.8	21.06.2023
Fix issue with suite.starttime at root suite is N/A	
1.3.9	23.06.2023
- Retrieve start/end time for suite - Update result schema to support Robotframework Version 6.1	
1.4.0	19.09.2023
Adaption for Robotframework 6.1 with change of TestSuite.source datatype	
1.4.1	15.03.2024
Update robot.xsd schema to support the new log level USER of Robotframework	
1.5.0	22.04.2024
Add optional argument <code>-interface</code> which allow to switch between db and rest interfaces	
1.5.1	14.06.2024
Fix issue when test result has SKIP status	
1.5.2	28.06.2024
Add optional argument <code>-testrunurl</code> to specify link to test job such as Jenkins or Gitlab CI/CD	
1.5.3	21.11.2024
Add support for robotframework result file version 7.x.x (result schema version 5)	
1.5.4	04.02.2025
Add support for robotframework (extended version) result file with thread feature	
1.6.0	24.02.2026
Usage of Setuptools replaced by usage of PIP/TOML/Setuptools	

RobotLog2DB.pdf

Created at 07.09.2026 - 09:47:22

by GenPackageDoc v. 0.44.0 / 05.06.2026

10.14 RobotFramework_DoIP

RobotFramework_DoIP

v. 0.2.0

Hua Van Thong

23.02.2026

Contents

1	Introduction	1
1.1	Overview	1
1.2	Abbreviations	1
1.3	Terms and definitions	2
1.3.1	Diagnostic Power Mode	2
1.3.2	Transport protocol	2
1.3.3	Diagnostics tester	2
1.3.4	Diagnostics gateway	2
1.3.5	Logical addresses	2
2	Description	3
2.1	System Overview	3
2.2	Multiple Connections	4
2.3	DoIP application scenarios	4
2.3.1	Example of Diagnostic Process	4
2.3.2	Example of Vehicle Identification	6
3	The Ecu Simulator	7
3.1	Initialize	7
3.2	Start	8
3.3	Example	9
4	DoipKeywords.py	11
4.1	Class: DoipDeviceManager	11
4.2	Class: DoipDevice	11
4.3	Class: DoipKeywords	11
4.3.1	Keyword: connect_to_ecu	11
4.3.2	Keyword: send_diagnostic_message	12
4.3.3	Keyword: receive_diagnostic_message	12
4.3.4	Keyword: reconnect_to_ecu	13
4.3.5	Keyword: disconnect	13
4.3.6	Keyword: await_vehicle_announcement	13
4.3.7	Keyword: get_entity	14
4.3.8	Keyword: request_entity_status	15
4.3.9	Keyword: request_vehicle_identification	15
4.3.10	Keyword: request_alive_check	15
4.3.11	Keyword: request_activation	16
4.3.12	Keyword: request_diagnostic_power_mode	16

CONTENTS	CONTENTS
4.3.13 Keyword: build_payload	16
5 __init__.py	18
6 Appendix	19
7 History	20
B	

Chapter 1

Introduction

1.1 Overview

robotframework-doip is a Robot Framework library specifically designed for interacting with Electronic Control Units (ECUs) using the Diagnostics over Internet Protocol (DoIP).

At its core, DoIP serves as a communication bridge between external diagnostic tools and a vehicle's ECUs. This library, RobotFrameworkDoIP, provides a set of keywords that enable users to perform diagnostic operations and engage with ECUs, facilitating automated testing processes and interaction with vehicles through the DoIP protocol.

The **robotframework-doip** sources can be found in repository **robotframework-doip**: [DoIP](#)

1.2 Abbreviations

Table 1.1: Abbreviation

Abbreviation / Acronym	Description
ARP	Address Resolution Protocol
DHCP	Diagnostic Host Configuration Protocol
EID	Entity identifier
GID	Group identifier
ICMP	Internet Control Message Protocol
IP	Internet Protocol
IPv4	Internet Protocol version 4
IPv6	Internet Protocol version 6
TCP	Transmission Control Protocol
TCP/IP	A family of communication protocols used in computer networks
VIN	Vehicle Identification Number
UDP	User Datagram Protocol

1.3 Terms and definitions

1.3.1 Diagnostic Power Mode

The vehicle internal power supply status affects the diagnostic capabilities of all ECUs on the in-vehicle network and identifies the status of all ECUs in all gateway subnetworks that allow diagnostic communication. Its purpose is to provide external test equipment with whether it can be performed on the connected vehicle.

1.3.2 Transport protocol

Transport protocols reside on top of the IP protocol. Transport protocols run over the best-effort IP layer to provide a mechanism for applications to communicate with each other without directly interacting with the IP layer.

The important thing here is that DoIP does not represent a diagnostic protocol according to ISO 13400 but rather an expanded transport protocol. This means that the transmission of diagnostic packets is defined in DoIP, but the contained diagnostic services continue to be specified and described by diagnostic protocols such as KWP2000 and UDS.

1.3.3 Diagnostics tester

A diagnostic tester / diagnostic client enables the sending of diagnostic requests. Testers can take the form of external devices, such as in repair shops, or on-board testers in the vehicle. The receiving ECU must process the diagnostic requests and return an associated diagnostic response to the tester.

1.3.4 Diagnostics gateway

The gateway assumes the role of the intermediary. Requests of the tester are forwarded to internal networks so that a desired ECU can receive and process them. As soon as a response from the requested ECU is available, the gateway routes this back to the tester.

1.3.5 Logical addresses

A diagnostic gateway always requires two pieces of information in order to forward diagnostic requests and responses.

- First, it requires a logical address that uniquely identifies the ECU to be diagnosed in the vehicle, equivalent to the ecu logical address parameter.
- Second, the gateway must know which messages on the respective bus system or network will be used to send diagnostic requests and to receive diagnostic responses, equivalent to client logical addressp parameter.

Both pieces of information must be available for an ECU for it to be accessible via the gateway.

Chapter 2

Description

2.1 System Overview

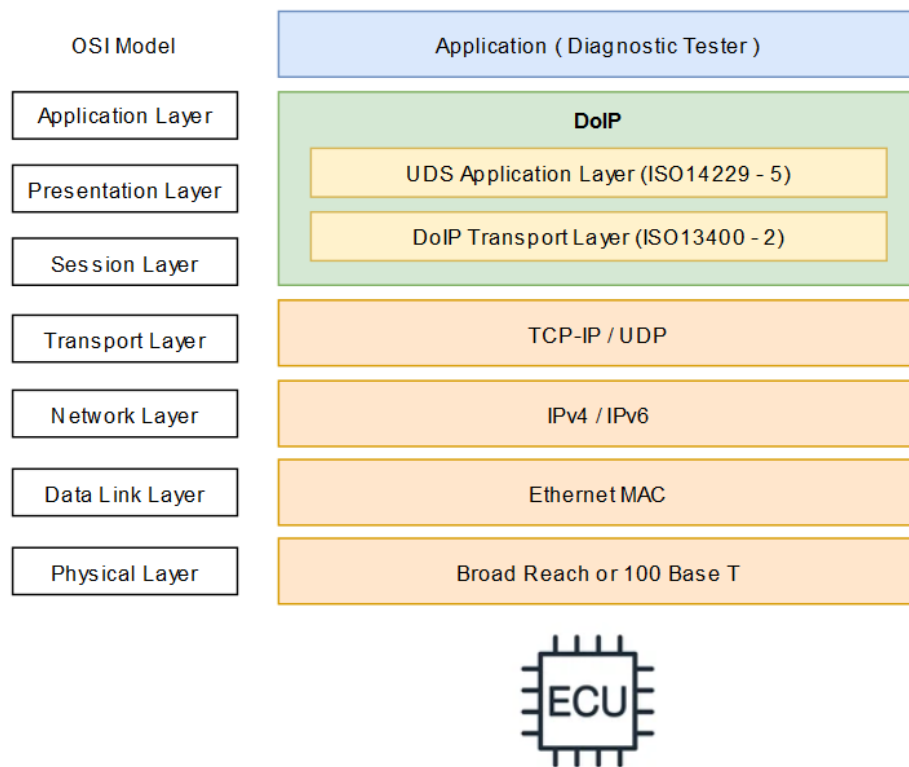


Figure 2.1: System overview

- The DoIP (Diagnostics over Internet Protocol) protocol is a standard for vehicle diagnostics that allows communication between diagnostic tester devices and electronic control units (ECU) over Ethernet networks.
- DoIP is a standardized diagnostic transport protocol according to ISO 13400.
 - DoIP Transport Layer (ISO 13400-2) is equipped with features to establish and maintain connection between external tester device and DoIP gateway inside the vehicle.
 - UDS application layer (ISO 14229-5) is the application profile that implements UDS on IP.
- The overall goal of the protocol is to encapsulate diagnostics messages of protocol standards like Unified Diagnostic Services (UDS) and route them to and from the ECU.

- The DoIP gateway or server can be a part of the ECU. A vehicle can have multiple DoIP entities and multiple testing devices and ECUs can route their traffic via a single DoIP entity.
- DoIP uses both User Datagram Protocol (UDP) and Transmission Control Protocol (TCP) for specific phases of the underlying layer. The initial announcement and identification messages are over UDP, after which the communication switches over to TCP.

2.2 Multiple Connections

The **robotframework-doip** can be extended to manage multiple connections simultaneously. This is beneficial when working with complex vehicle systems or simultaneously testing multiple ECUs.

Example

```
Test user can connect multiple connection
    Log    Connect to ECU 1

    Connect To ECU        device_name= ${DEVICE_NAME_1}
    ...                   ecu_ip_address= ${SUT_IP_ADDRESS_1}
    ...                   ecu_logical_address= ${SUT_LOGICAL_ADDRESS_1}
    ...                   client_ip_address= ${TB_IP_ADDRESS_1}
    ...                   client_logical_address= ${TB_LOGICAL_ADDRESS_1}
    ...                   activation_type= ${ACTIVATION_TYPE_1}

    Send Diagnostic Message    1001    device_name= ${DEVICE_NAME_1}
    ${resp}=    Receive Diagnostic Message    device_name= ${DEVICE_NAME_1}
    Log To Console    ${resp}
    Disconnect    device_name= ${DEVICE_NAME_1}

    Log    Connect to ECU 2
    Connect To ECU        device_name= ${DEVICE_NAME_2}
    ...                   ecu_ip_address= ${SUT_IP_ADDRESS_2}
    ...                   ecu_logical_address= ${SUT_LOGICAL_ADDRESS_2}
    ...                   client_ip_address= ${TB_IP_ADDRESS_2}
    ...                   client_logical_address= ${TB_LOGICAL_ADDRESS_2}
    ...                   activation_type= ${ACTIVATION_TYPE_2}

    Send Diagnostic Message    1001    device_name= ${DEVICE_NAME_2}
    ${resp}=    Receive Diagnostic Message    device_name= ${DEVICE_NAME_2}
    Log To Console    ${resp}
    Disconnect    device_name= ${DEVICE_NAME_2}
```

2.3 DoIP application scenarios

This section will provide some features along with examples through system above:

- Vehicle identification and announcement: Is necessary to detect who is participating in the DoIP communication.
- Request diagnostic message: Request for diagnostic information, which is crucial for diagnosing vehicle issues and ensuring effective communication within the DoIP network.
- Routing Activation: Allows that single Diagnostic Message pathes are activated or not to treat different protocols different (like UDS and OBD) and to also treat single testers different.
- Entity DoIP (Gateway): Provides general information of the single DoIP entity. Usually used by the testers to get the current DoIP protocol relevant information from the single DoIPEntities.
- Alive mechanism: Is used to maintain different tester connections.

2.3.1 Example of Diagnostic Process

Vehicle Diagnostics

- **Use Case:**
A vehicle diagnostic tool needs to establish a DoIP connection to communicate with an Electronic Control Unit

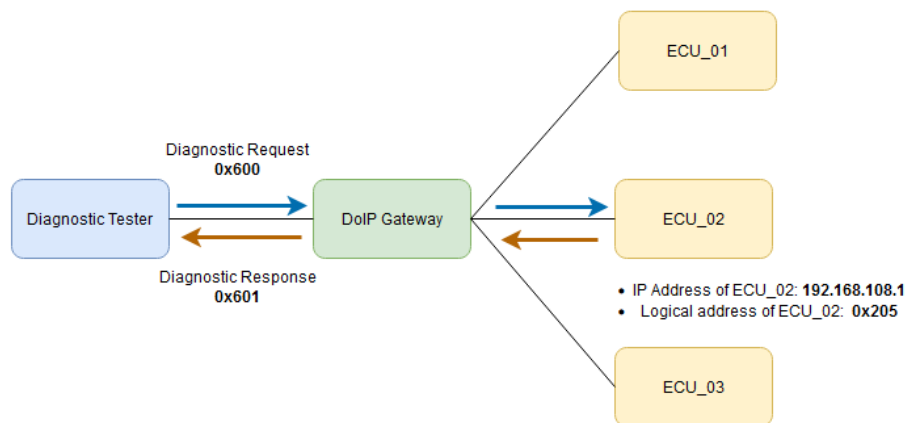


Figure 2.2: Vehicle diagnostic process demonstration

(ECU) within a vehicle for diagnostic purposes.

- **Scenario:**

The diagnostic tool initiates a connection to the ECU's IP address (192.168.108.1) and logical address 517 (0x205) using the **robotframework-doip** library.

It sends a diagnostic message (600) to the ECU, receives a response, logs the response to the console, and then disconnects from the ECU.

```

*** Settings ***
Library      RobotFramework_DoIP

*** Test Cases ***
Test Establish an DoIP connection
    # Establish an connection to ecu ip address and ecu logical address
    Connect To ECU      192.168.108.1      517
    Send Diagnostic Message      600
    ${res}= Receive Diagnostic Message
    Log To Console      ${resp}
    Disconnect
  
```

Remote Testing

- **Use Case:**

A remote testing environment requires a connection between a target tester and an ECU located in a different location.

- **Scenario:**

The target tester, located at IP address 192.168.108.20, establishes a DoIP connection to the ECU at IP address 192.168.108.1 and logical address 517 (0x205). Additionally, the target tester specifies its own logical address 1895 (0x767) using the **robotframework-doip** library.

It sends a diagnostic message (600) to the ECU, receives a response, logs the response to the console, and then disconnects from the ECU.

```

*** Settings ***
Library      RobotFramework_DoIP

*** Test Cases ***
Test Establish an DoIP connection between specific tester and target ECU
    # Establish an connection to ecu ip address and ecu logical address
    Connect To ECU      192.168.108.1      517      client_ip_address=192.168.108.20      ↔
    ↩ client_logical_address=1895
  
```

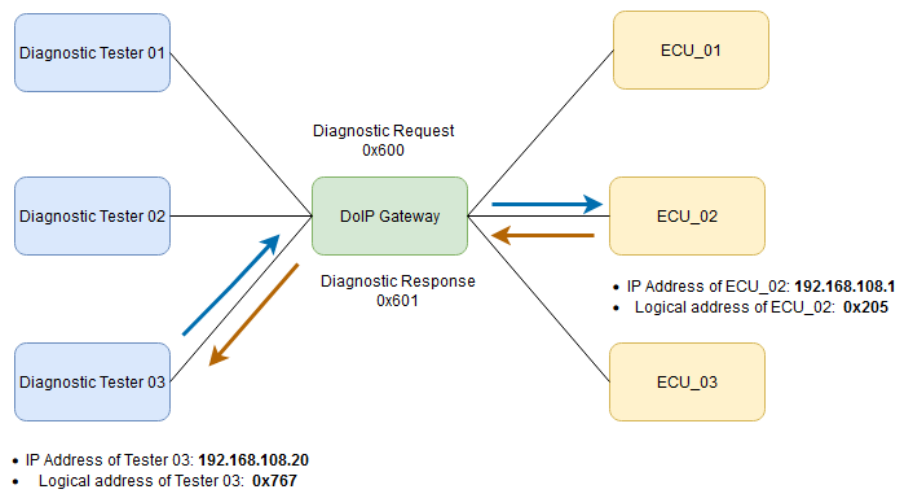


Figure 2.3: Remote diagnostic process demonstration

```
Send Diagnostic Message      600
${res}= Receive Diagnostic Message
Log To Console      ${resp}
Disconnect
```

2.3.2 Example of Vehicle Identification

- **Use Case:**
A diagnostic tool needs to request vehicle identification information from an Electronic Control Unit (ECU) within a vehicle.
- **Scenario:**
The diagnostic tool initiates a DoIP connection to the ECU's IP address (192.168.108.1) and logical address (205) using the **robotframework-doip** library. It then sends a request for vehicle identification information to the ECU.

```
*** Settings ***
Library      RobotFramework_DoIP

*** Test Cases ***
Test Request Vehicle Identification
    Connect To ECU      192.168.108.1      205
    Request Vehicle Identification
```

Chapter 3

The Ecu Simulator

This chapter provides a detailed explanation of the utilization of the ECU simulator through DoIP base on doipclient library. It serves for development or testing scenarios where a physical device is not available.

The ECU simulator is designed to receive messages and respond accordingly to the following types of messages:

- Alive Check Request
- Diagnostic Power Mode Request
- Doip Entity Status Request
- Routing Activation Request
- Vehicle Identification Request

3.1 Initialize

This function sets up an instance of an ECU, initializes its attributes with default values, and includes placeholders for various properties that can be customized based on specific requirements.

```
def __init__(self, ecu_type, ip_address, tcp_port, udp_port):
    # Initialize ECU attributes with default values
    self.ecu_type = ecu_type
    self.ip_address = ip_address
    self.tcp_port = tcp_port
    self.udp_port = udp_port
    self.tcp_socket = None
    self.udp_socket = None
    # Set default values for various ECU properties
    # These values might be placeholders and can be updated based on your actual ↩
    ↩ requirements
    self._ecu_logical_address = 3584
    self._client_logical_address = 3584
    self._logical_address = 55
    self._response_code = doip_message.RoutingActivationResponse.ResponseCode.Success
    self._diagnostic_power_mode = doip_message.DiagnosticPowerModeResponse.↩
    ↩ DiagnosticPowerMode.Ready
    self._node_type = 1
    self._max_concurrent_sockets = 16
    self._currently_open_sockets = 1
    self._max_data_size = None
    self._vin = '19676527011956855057'
    self._eid = b'11111'
    self._gid = b'22222'
    self._further_action_required = doip_message.VehicleIdentificationResponse.↩
    ↩ FurtherActionCodes.NoFurtherActionRequired
    self._vin_sync_status = doip_message.VehicleIdentificationResponse.↩
    ↩ SynchronizationStatusCodes.Synchronized
```

3.2 Start

This method is responsible for initializing and setting up TCP and UDP sockets, binding them to specific IP addresses and ports, and then starting separate threads to handle the communication on these sockets concurrently.

```
def start(self):
    # Create TCP socket
    self.tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    self.tcp_socket.bind((self.ip_address, self.tcp_port))
    self.tcp_socket.listen(5)

    # Create UDP socket
    self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    self.udp_socket.bind((self.ip_address, self.udp_port))

    # Start listening on separate threads
    tcp_thread = threading.Thread(target=self.listen_tcp)
    udp_thread = threading.Thread(target=self.listen_udp)

    tcp_thread.start()
    udp_thread.start()
```

Explanation:

1. TCP Socket Setup

- A TCP socket is created using the socket module with the `socket.AF_INET` family (IPv4) and `socket.SOCK_STREAM` type (TCP).
- The TCP socket is bound to the specified IP address `self.ip_address` and TCP port `self.tcp_port`.
- The TCP socket is set to listen for incoming connections with a backlog of 5 connections.

2. UDP Socket Setup

- A UDP socket is created using the same socket module with the `socket.AF_INET` family (IPv4) and `socket.SOCK_DGRAM` type (UDP).
- The UDP socket is bound to the specified IP address `self.ip_address` and UDP port `self.udp_port`.

3. Thread Creation

- Two separate threads `tcp_thread` and `udp_thread` are created using the threading module.
- The target parameter of each thread is set to point to specific methods `self.listen_tcp` and `self.listen_udp`, suggesting that these methods likely contain the logic for handling TCP and UDP communication.

4. Thread Start

- Both threads are started concurrently using the start method, allowing the ECU to handle TCP and UDP communication simultaneously.

3.3 Example

We have provided an example demonstrating the usage of the ECU simulator in the file located at `test_ecu_simulator.py`

```
if __name__ == "__main__":
    # Create and start instances of different ECUs using the factory pattern and ↵
    ↵ abstract class
    factory = ECUFactory()

    positive_ecu = factory.create_ecu(ECUType.POSITIVE_ECU, POSITIVE_ECU_IP, ↵
    ↵ POSITIVE_TCP_PORT, POSITIVE_UDP_PORT)
    negative_ecu = factory.create_ecu(ECUType.NEGATIVE_ECU, NEGATIVE_ECU_IP, ↵
    ↵ NEGATIVE_TCP_PORT, NEGATIVE_UDP_PORT)
    # Start positive and negative ECUs
    positive_ecu.start()
    negative_ecu.start()
```

In the given example, an instance of the ECU is created in `ecu_simulator.py` by specifying the ECU's IP address, TCP port, and UDP port. Subsequently, the start method is invoked to initiate its operation.

Output:

```
TCP Server 172.17.0.5 listening on port 13400
UDP Server 172.17.0.5 listening on port 13400
TCP Server 172.17.0.5 listening on port 12346
UDP Server 172.17.0.5 listening on port 12347
```

Now you can execute the test by running the file located at `test_ecu_simulator.py`

```
def test_positive_ecu_simulator():
    try:
        ip = '172.17.0.5'
        ecu_logical_address = 57344

        # Create a DoIPClient instance for positive ECU simulator
        doip = DoIPClient(ip, ecu_logical_address, activation_type=None)

        # Test various interactions
        print(doip.request_diagnostic_power_mode())
        print(doip.request_entity_status())
        print(doip.request_alive_check())
        print(doip.request_activation(1))
        print(doip.get_entity())
        print(doip.request_vehicle_identification(vin="1" * 17))
        print(doip.request_vehicle_identification(eid=b"1" * 6))

    except Exception as e:
        print(f"Error during positive ECU simulation: {e}")
```

Output:

```

# Diagnostic power mode response
DiagnosticPowerModeResponse (0x4004): { diagnostic_power_mode : DiagnosticPowerMode.↵
↵ Ready }

# Entity status response
EntityStatusResponse (0x4002): { node_type : 1, max_concurrent_sockets : 16, ↵
↵ currently_open_sockets : 1, max_data_size : None }

# Alive check response
AliveCheckResponse (0x8): { source_address : 3584 }

# Routing activation response
RoutingActivationResponse (0x6): { client_logical_address : 3584, logical_address : ↵
↵ 55, response_code : ResponseCode.Success, reserved : 0, vm_specific : None }

# Get entity response
(('172.17.0.5', 13400), VehicleIdentificationResponse(b'19676527011956855', 3584, b'↵
↵ 11111\x00', b'22222', 0, 0))

# Vehicle identification response
VehicleIdentificationResponse (0x4): { vin: "19676527011956855", logical_address : ↵
↵ 3584, eid : b'11111\x00', gid : b'22222', further_action_required : ↵
↵ FurtherActionCodes.NoFurtherActionRequired, vin_sync_status : ↵
↵ SynchronizationStatusCodes.Synchronized }
VehicleIdentificationResponse (0x4): { vin: "19676527011956855", logical_address : ↵
↵ 3584, eid : b'11111\x00', gid : b'22222', further_action_required : ↵
↵ FurtherActionCodes.NoFurtherActionRequired, vin_sync_status : ↵
↵ SynchronizationStatusCodes.Synchronized }

```


Chapter 4

DoipKeywords.py

4.1 Class: DoipDeviceManager

4.2 Class: DoipDevice

4.3 Class: DoipKeywords

4.3.1 Keyword: connect_to_ecu

Description:

Establishing a **DoIP** connection to an (ECU) within the context of automotive communication.

Parameters:

- `ecu_ip_address` (required, type: `str`): The IP address of the ECU to establish a connection. This should be a string representing an IPv4 address like `192.168.1.1` or an IPv6 address like `2001:db8::`.
- `ecu_logical_address` (required, type: *any*): The logical address of the ECU.
- `tcp_port` (optional, default: `TCP_DATA_UNSECURED`, type: `int`): The TCP port used for unsecured data communication.
- `udp_port` (optional, default: `UDP_DISCOVERY`, type: `int`): The UDP port used for ECU discovery.
- `activation_type` (optional, type: `RoutingActivationRequest.ActivationType`): The type of activation, which can be the default value (`ActivationTypeDefault`) or a specific value based on application-specific settings.
- `protocol_version` (optional, default: `0x02`, type: `int`): The version of the protocol used for the connection.
- `client_logical_address` (optional, type: `int`): The logical address that this **DoIP** client will use to identify itself. Per the spec, this should be `0x0E00` to `0x0FFF`. Can typically be left as default.
- `client_ip_address` (optional, type: `str`): If specified, attempts to bind to this IP as the source for both UDP and TCP communication. Useful if you have multiple network adapters. Can be an **IPv4** or **IPv6** address just like `ecu_ip_address`, though the type should match.
- `use_secure` (optional, type: `Union[bool, ssl.SSLContext]`): Enables **TLS**. If set to `True`, a default SSL context is used. For more control, a preconfigured SSL context can be passed directly. Untested. Should be combined with changing `tcp_port` to `3496`.
- `auto_reconnect_tcp` (optional, type: `bool`): Attempt to automatically reconnect TCP sockets that were closed by peer.
- `device_name` (optional, type: `str`): Name of **DoIP** device.

Return:

None

Exception:

raises `ConnectionError`: Failed to establish a DoIP connection

Usage:

Explicitly specifies all establishing a connection

- `Connect To ECU | 172.17.0.111 | 1863 |`
- `Connect To ECU | 172.17.0.111 | 1863 | client_ip_address=172.17.0.5 | client_logical_add`
|
- `Connect To ECU | 172.17.0.111 | 1863 | client_ip_address=172.17.0.5 | client_logical_add`
| `activation_type=0 |`

4.3.2 Keyword: send_diagnostic_message**Description:**

Send a raw diagnostic payload (ie: **UDS**) to the ECU.

Parameters:

- `diagnostic_payload` (type: `str`): UDS payload to transmit to the ECU
- `timeout` (type: `int` (s)): send diagnostic time out (default: `A_PROCESSING_TIME`)
- `device_name` (optional, type: `str`): Name of **DoIP** device

Return:

None

Exception:

- raises `ConnectionRefusedError`: DoIP connection attempt failed
- raises `IOError`: DoIP negative acknowledgement received

Usage:

Explicitly specifies all diagnostic message properties

- `Send Diagnostic Message | 1040 |`
- `Send Diagnostic Message | 1040 | timeout=10 |`

4.3.3 Keyword: receive_diagnostic_message**Description:**

Receive a raw diagnostic payload (ie: **UDS**) from the ECU.

Parameters:

- `timeout` (optional, default: `None`, type: `int` (s)): time waiting diagnostic message.
- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

- raises `ConnectionRefusedError`: DoIP connection attempt failed
- raises `IOError`: DoIP negative acknowledgement received

Usage:

Explicitly specifies all diagnostic message properties

- Receive Diagnostic Message |
- Receive Diagnostic Message | timeout=10 |

4.3.4 Keyword: reconnect_to_ecu**Description:**

Attempts to re-establish the connection. Useful after an ECU reset

Parameters:

- `close_delay` (type: int (s), default: `A_PROCESSING_TIME`): Time to wait between closing and re-opening socket.
- `device_name` (optional, type: str): Name of DoIP device

Return:

None

Exception:

raises `ConnectionRefusedError`: DoIP connection attempt failed

Usage:

Explicitly specifies all diagnostic message properties

- Reconnect To Ecu |
- Reconnect To Ecu | close_delay=10 |

4.3.5 Keyword: disconnect**Description:**

Close the **DoIP** client

Parameters:

- `device_name` (optional, type: str): Name of **DoIP** device

Return:

None

Exception:

- raises `ConnectionRefusedError`: DoIP connection attempt failed
- raises `ConnectionAbortedError`: close DoIP connection aborted

Usage:

Explicitly specifies all diagnostic message properties

- Disconnect

4.3.6 Keyword: await_vehicle_announcement**Description:**

When an ECU first turns on, it's supposed to broadcast a Vehicle Announcement Message over UDP 3 times to assist **DoIP** clients in determining ECU IP's and Logical Addresses. Will use an IPv4 socket by default, though this can be overridden with the `ipv6` parameter.

Parameters:

- `udp_port` (optional, type: `int`): The UDP port to listen on. Per the spec this should be 13400, but some VM's use a custom one.
- `timeout` (optional, type: `float`): Maximum amount of time to wait for message
- `ipv6` (optional, type: `bool`): Bool forcing IPV6 socket instead of IPV4 socket
- `source_interface` (optional, default: `None`, type: `str`): Interface name (like "eth0") to bind to for use with IPV6. Defaults to `None` which will use the default interface (which may not be the one connected to the ECU). Does nothing for IPV4, which will bind to all interfaces uses `INADDR_ANY`.
- `device_name` (optional, type: `str`): Name of DoIP device

Return:

- return: IP Address of ECU and `VehicleAnnouncementMessage` object
- rtype: tuple

Exception:

- raises `TimeoutError`: If vehicle announcement not received in time

Usage:

Explicitly specifies all diagnostic message properties

- `Await Vehicle Announcement`
- `Await Vehicle Announcement | timeout=10`

4.3.7 Keyword: `get__entity`**Description:**

Sends a `VehicleIdentificationRequest` and awaits a `VehicleIdentificationResponse` from the ECU, either with a specified VIN, EIN, or nothing. Equivalent to the `request_vehicle_identification()` method but can be called without instantiation.

Parameters:

- `udp_port` (optional, type: `int`): The UDP port to listen on. Per the spec this should be 13400, but some VM's use a custom one.
- `timeout` (optional, type: `float`): Maximum amount of time to wait for message
- `ipv6` (optional, type: `bool`): Bool forcing IPV6 socket instead of IPV4 socket
- `source_interface` (optional, default: `None`, type: `str`): Interface name (like "eth0") to bind to for use with IPV6. Defaults to `None` which will use the default interface (which may not be the one connected to the ECU). Does nothing for IPV4, which will bind to all interfaces uses `INADDR_ANY`.
- `device_name` (optional, type: `str`): Name of DoIP device

Return:

- return: IP Address of ECU and `VehicleAnnouncementMessage` object
- rtype: tuple

Exception:

raises `TimeoutError`: If vehicle announcement not received in time

Usage:

- `Get Entity |`
- `Get Entity | ecu_ip_address=172.17.0.111 |`
- `Get Entity | ecu_ip_address=172.17.0.111 | protocol_version=0x02`

4.3.8 Keyword: request__entity__status

Description:

Request that the ECU send a **DoIP** Entity Status Response

Parameters:

- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

None

Usage:

- `Request Entity Status`

4.3.9 Keyword: request__vehicle__identification

Description:

Sends a `VehicleIdentificationRequest` and awaits a `VehicleIdentificationResponse` from the ECU, either with a specified VIN, EIN, or nothing.

Parameters:

- `eid` (optional, type: `bytes`): EID of the Vehicle
- `vin` (optional, type: `str`): VIN of the Vehicle
- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

None

Usage:

- `Request Vehicle Identification`
- `Request Vehicle Identification | eid=0x123456789abc`
- `Request Vehicle Identification | vin=0x123456789abc`

4.3.10 Keyword: request__alive__check

Description:

Request that the ECU send an alive check response.

Parameters:

- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

None

Usage:

- `Request Vehicle Identification`
- `Request Vehicle Identification | eid=0x123456789abc`
- `Request Vehicle Identification | vin=0x123456789abc`

4.3.11 Keyword: request_activation

Description:

Requests a given activation type from the ECU for this connection using payload type 0x0005.

Parameters:

- `activation_type` (required, type: `RoutingActivationRequest.ActivationType`): The type of activation to request - see Table 47 ("Routing activation request activation types") of ISO-13400, but should generally be 0 (default) or 1 (regulatory diagnostics).
- `vm_specific` (optional, type: `int`): 4 byte long int
- `disable_retry` (optional, type: `bool`): Disables retry regardless of `auto_reconnect_tcp` flag. This is used by activation requests during connect/reconnect.
- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

None

Usage:

- Request Routing Activation | `${0x02}`
- Request Routing Activation | `vm_specific=`
- Request Routing Activation | `vin=0x123456789abc`

4.3.12 Keyword: request_diagnostic_power_mode

Description:

Request that the ECU send a Diagnostic Power Mode response

Parameters:

- `device_name` (optional, type: `str`): Name of DoIP device

Return:

None

Exception:

None

Usage:

- Request Diagnostic Power Mode

4.3.13 Keyword: build_payload

Description:

Build payload

Parameters:

- `request` (required, type: `hex`): hex data
- `device_name` (optional, type: `str`): Name of **DoIP** device

Return:

return bytes data of the request

Exception:

*CHAPTER 4. DOIPKEYWORDS.PY**4.3. CLASS: DOIPKEYWORDS*

- `raise ValueError: if request is None`

Usage:

- Build payload by protocol version | `${request}`
- Build payload by protocol version | `hex_value`

Chapter 5

`__init__.py`

RobotFrameworkDoIP is a Robot Framework library aimed to provide DoIP protocol for diagnostic message.

Chapter 6

Appendix

About robotframework-doip:

Author

Hua Van Thong (thong.huavan@vn.bosch.com)

Version

0.2.0 (23.02.2026)

Short Description

Robot Framework library for interacting with Electronic Control Units (ECUs)

Homepage

[robotframework-doip](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

`>=3.11`

License

Apache-2.0

Chapter 7

History

0.1.0	09/2023
<i>Initial version</i>	
0.1.1	12/2023
<i>Add ecu simulator to use for self test</i>	
0.1.2	4/2024
<i>Update the documentation for DoIP</i>	
0.1.3	4/2024
<i>Publish package to PyPi</i>	
0.1.4	4/2024
<i>Add dependency for package</i>	
0.1.5	10/2024
<i>Add feature that supports working with multiple connections</i>	
0.1.6	3/2025
<i>Add feature that supports sending diag message without waiting any response</i>	
0.2.0	02/2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	

RobotFramework_DoIP.pdf*Created at 07.09.2026 - 09:47:25**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.15 PyTestLog2DB

PyTestLog2DB

v. 0.4.2

Tran Duy Ngoan

01.07.2026

Contents

1	Introduction	1
2	Description	2
2.1	Execute pytest test case(s) to get result file	2
2.2	Tool features	2
2.2.1	Usage	2
2.2.2	Verify provided arguments	3
2.2.3	Searching *.xml result file(s)	3
2.2.4	Handle essential information for TestResultWebApp	4
2.2.5	Append to existing execution result	5
2.2.6	Switch Database Access Interface	6
2.3	Display on TestResultWebApp	8
3	pytestlog2db.py	9
3.1	Function: collect_xml_result_files	9
3.2	Function: validate_xml_result	9
3.3	Function: is_valid_uuid	10
3.4	Function: is_valid_config	10
3.5	Function: parse_pytest_xml	11
3.6	Function: get_branch_from_swversion	11
3.7	Function: get_test_result	12
3.8	Function: process_component_info	12
3.9	Function: process_config_file	12
3.10	Function: process_test	13
3.11	Function: process_suite	14
3.12	Function: PyTestLog2DB	14
3.13	Class: Logger	15
3.13.1	Method: config	15
3.13.2	Method: log	15
3.13.3	Method: log_warning	16
3.13.4	Method: log_error	16
4	Appendix	17
5	History	18

Chapter 1

Introduction

PyTestLog2DB is a command-line tool that enables you to import [pytest](#) XML result files into TestResultWebApp's database for presenting an overview about the whole test execution and detail of each test result.

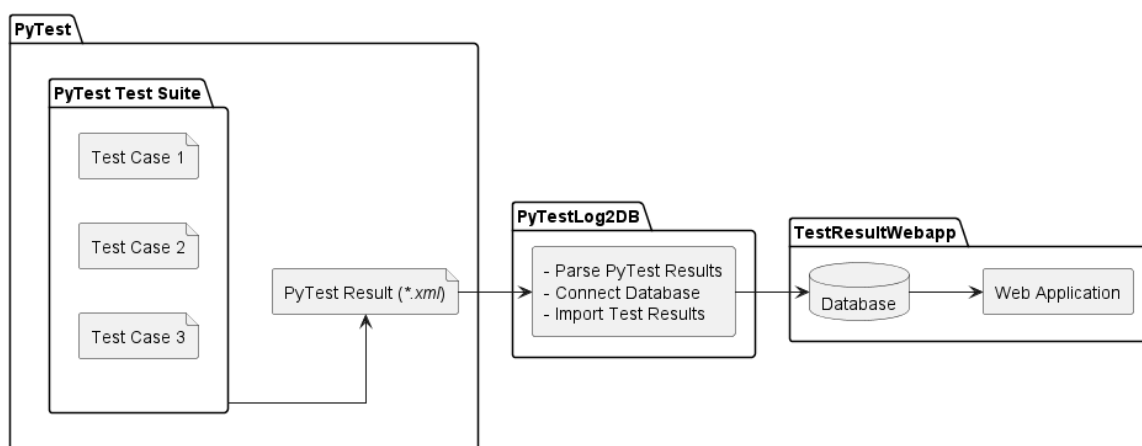


Figure 1.1: Tool data flow

PyTestLog2DB tool requires serveral arguments, including the location of the pytest XML result file(s) to parse all information of test execution result and TestResultWebApp's database credential for importing that result.

[TestResultWebApp](#) requires some mandatory information to manage and display the test result properly, but the generated **xml* file contains only the basic test result information. So that, **PyTestLog2DB** tool will set those required information with [default values](#).

Besides, you can also use optional arguments of **PyTestLog2DB** tool to provide missing information or you want to overwrite them with the expected values.

Finally, **PyTestLog2DB** also allows you to append existing results in the database, which is helpful when you need to update previous test results or add the missing XML result file(s) from previous tool execution.

Chapter 2

Description

2.1 Execute pytest test case(s) to get result file

When executing pytest test case(s), the test result is only displayed on console log without generating any result file by default.

In order to get the result **.xml* (JUnit XML format) files, the optional argument `--junit-xml=<log>` needs to be specified when executing the pytest test case(s).

The example pytest command to get the ***.xml** result file:

```
pytest --junit-xml=path/to/result.xml pytest/folder
```

2.2 Tool features

2.2.1 Usage

Use below command to get tool's usage:

```
PyTestLog2DB -h
```

The tool's usage should be showed as below:

```
usage: PyTestLog2DB (PyTestXMLReport to TestResultWebApp importer) [-h] [-v]
                        [--recursive] [--dryrun] [--append] [--UUID UUID]
                        [--variant VARIANT] [--versions VERSIONS] [--config CONFIG]
                        resultxmlfile server user password database

PyTestLog2DB imports pytest JUnit XML report file(s) generated by pytest into a WebApp ↩
↪ database.

positional arguments:
  resultxmlfile          absolute or relative path to the pytest JUnit XML report
                        file or directory of report files to be imported.
  server                 server which hosts the database (IP or URL).
  user                   user for database login.
  password               password for database login.
  database               database schema for database login.

optional arguments:
  -h, --help             show this help message and exit
  -v, --version           version of the PyTestLog2DB importer.
  --recursive            if set, then the path is searched recursively for output
                        files to be imported.
  --dryrun               if set, then verify all input arguments (includes DB connection)
                        and show what would be done.
  --append               is used in combination with --UUID <UUID>. If set, allow to append
                        new result(s) to existing execution result UUID in --UUID argument.
  --UUID UUID            UUID used to identify the import and version ID on webapp.
```

```

↪ import.                                If not provided PyTestLog2DB will generate an UUID for the whole ↪
--variant VARIANT                        variant name to be set for this import.
--versions VERSIONS                      metadata: Versions (Software;Hardware;Test) to be set for this ↪
↪ import                                (semicolon separated)
--config CONFIG                          configuration json file for component mapping information.
--interface {db,rest}                   database access interface.
--testrunurl TESTRUNURL                  link to test execution job: Jenkins job, Gitlab CI/CD pipeline, ...

```

The below command is simple usage with all required arguments to import pytest results into TestResultWebApp's database:

```
PyTestLog2DB <resultxmlfile> <server> <user> <password> <database>
```

Besides the executable file, you can also run tool as a Python module

```
python -m PyTestLog2DB <resultxmlfile> <server> <user> <password> <database>
```

2.2.2 Verify provided arguments

Sometimes, we just want to validate the **.xml* and database connection without changing anything in the database, the optional argument `--dryrun` can be used in this case.

When executing in dryrun mode, **PyTestLog2DB** will:

- Verify the provided pytest **.xml* file is valid or not.
- Verify the database connection with provided credential.
- Verify other information which given in optional arguments.
- Just print all test cases will be imported without touching database.

This feature will helps you to ensure that there is no error when executing **PyTestLog2DB** tool (normal mode) to create new record(s) and update TestResultWebApp's database.

2.2.3 Searching *.xml result file(s)

The first argument `resultxmlfile` of **PyTestLog2DB** can be a single file or the folder that contains multiple result files.

When the folder is used, **PyTestLog2DB** will only search for **.xml* file(s) under given directory and exclude any file within subdirectories as default.

In case you have result file(s) under the subdirectory of given folder and want these result files will also be imported, the optional argument `--recursive` should be used when executing **PyTestLog2DB** command.

When `--recursive` argument is set, **PyTestLog2DB** will walk through the given directory and its subdirectories to discover and collect all available **.xml* for importing.

For example: your result folder has a structure as below:

```

logFolder
|_____ result_1.xml
|_____ result_2.xml
|_____ subFolder_1
|           |_____ result_sub_1.xml
|           |_____ subSubFolder
|                   |_____ result_sub_sub_1.xml
|_____ subFolder_2
|           |_____ result_sub_2.xml

```

- Without `--recursive`: only **result_1.xml** and **result_2.xml** are found for importing.
- With `--recursive`: all **result_1.xml**, **result_2.xml**, **result_sub_1.xml**, **result_sub_2.xml** and **result_sub_sub_1.xml** will be imported.

2.2.4 Handle essential information for TestResultWebApp

Default values

TestResultWebApp requires **Project**, **Software version** to manage the execution results and **Component** to group test cases in the displayed charts.

But the ***.xml** report file which is generated by pytest contains only the testcase result(s) and no data for the information which is required by TestResultWebApp.

So that, the missing information will be set to default values when importing with **PyTestLog2DB** tool:

- **Project:** `PyTest`
- **Software version:** the execution time `%Y%m%d_%H%M%S` as default value. E.g `20221128_143547`
- **Hardware version:** empty string
- **Test version:** empty string
- **Component:** `unknown`
- **Test tool:** the combination of current Python and pytest version. E.g `PyTest 6.2.5 (Python 3.9.0)`
- **Tester:** The current user

Specify essential information with optional arguments

You can also provide essential information in command line when executing **PyTestLog2DB** tool with below optional arguments:

- `--variant VARIANT` To specify the **Project/Variant** information.
- `--versions VERSIONS` To specify the **Software, Hardware** and **Test** versions information.
- `--config CONFIG` To provide a configuration ***.json** file as `CONFIG` argument. Currently, the configuration ***.json** supports below settings:

- `"variant"` to specify the **Project/Variant** as `string` value.
- `"version_sw"` to specify the **Software version** information as `string` value.
- `"version_hw"` to specify the **Hardware under-test version** as `string` value.
- `"version_test"` to specify the **Test version** as `string` value.

Notice
! These above settings with `--config CONFIG` will have lower priority than the commandline arguments `--variant VARIANT` and `--versions VERSIONS`

- `"testtool"` to specify the **Test toolchain** as `string` value.
- `"tester"` to specify the **Test user** as `string` value.
- `"components"` to specify the **Component** information which will be displayed on **TestResultWebApp**. Value can be:

- * `string`: to specify the same **Component** for all test casea within this execution.

```
{
  "components" : "atest",
  ...
}
```

- * `object`: to define the mapping json object between **Component** info as key and a test case `classname` (list of `classname`) (***.robot** file) as value.

```
{
  "components": {
    "Testsuite1": "test-data.test-tsclass.TestSuite1",
    "Testsuite2": "test-data.test-tsclass.TestSuite2",
    "Others"     : [
      "test-data.test-tsl",

```



```

        "test-data.test-ts2"
    ]
    },
    ...
}

```

As above sample configuration, the `"components"` key contains the mappings for individual components, such as `Testsuite1` and `Testsuite2`, where the value is the `classname` (part of `classname`) of pytest test case(s).

Additionally, the `Others` key is an example of a mapping where the value is a list of test case `classname`, indicating that the `Others` component is composed of all pytest test case(s) which its `classname` contains `test-data.test-ts1` and `test-data.test-ts2`.

In other words, the component mapping can be explained as below:

- Test case(s) which its `classname` contains `test-data.test-tsclass.TestSuite1` is belong `Test-suite1` component
- Test case(s) which its `classname` contains `test-data.test-tsclass.TestSuite2` is belong `Test-Suite2` component
- `Others` component consists of all test case(s) which its `classname` contains `test-data.test-ts1` and `test-data.test-ts2`.

Hint

The `classname` of test case in the generated pytest result `*.xml` file is the combination of directory, test file and class of defined pytest test case.

Therefore, you can use directory information for component mapping to group all test cases in a folder as the same component.

In addition, you can use the optional argument `--junit-prefix=str` when executing pytest to set the prefix to the `classname` of the test case in the result `*.xml` file. After that, you can use that prefix information for component mapping when importing the result `*.xml` file with the **PyTestLog2DB** tool.

In case the given configuration `*.json` is not valid or unsupported key is used, the corresponding error will be raised.

2.2.5 Append to existing execution result

PyTestLog2DB also allows you to append new test result(s) (missing from previous import, on other test setup, ...) into the existing execution result (identified by the **UUID**) in `TestResultWebApp`'s database. The combination of optional arguments `--UUID <UUID>` and `--append` should be used in this case.

The `--append` makes **PyTestLog2DB** run in append mode and the `--UUID <UUID>` is used to specify the existing **UUID** of execution result to be appended.

For example, the result with **UUID** `c7991c07-4de2-4d65-8568-00c5556c82aa` is already existing in `TestResultWebApp`'s database and you want to append result(s) in `output.xml` into that execution result.

The command will be used as below:

```
python -m PyTestLog2DB output.xml localhost testuser testpw testdb --UUID ↵
↳ c7991c07-4de2-4d65-8568-00c5556c82aa --append
```

If the argument `--UUID <UUID>` is used without `--append`:

- An error will be thrown and the import job is terminated immediately if the provided **UUID** is already existing

```

FATAL ERROR: Execution result with UUID 'c7991c07-4de2-4d65-8568-00c5556c82aa' is ↵
↳ already existing.
      Please use other UUID (or remove '--UUID' argument from your command) ↵
↳ for new execution result.
      Or add '--append' argument in your command to append new result(s) to ↵
↳ this existing UUID.

```

- The importing execution result will have an identifier as the provided **UUID** if that **UUID** is not existing

If the argument `--append` is used and:

- given UUID in `--UUID <UUID>` argument is existing: the new result(s) will be appended to that UUID
- given UUID in `--UUID <UUID>` argument is not existing: tool will be terminated immediately with below error

```
FATAL ERROR: Execution result with UUID 'c7991c07-4de2-4d65-8568-00c5556c82aa' is not↵
  ↳ existing for appending.
    Please use an existing UUID to append new result(s) to that UUID.
    Or remove '--append' argument in your command to create new execution ↵
  ↳ result with given UUID.
```

- `--UUID <UUID>` is not provided: tool will be terminated immediately with below error

```
FATAL ERROR: '--append' argument should be used in combination with '--UUID <UUID>' ↵
  ↳ argument
```

Notice

When using append mode and `project` / `variant`, `version_sw` are provided within `--variant VARIANT`, `--versions VERSIONS` or `--config CONFIG` arguments, they will be validated with the existing values in database.

An error will be raised in case the given value is not matched with the existing ones. E.g:

```
FATAL ERROR: Given version software 'my_version' is different with existing ↵
  ↳ value 'SW01' in database.
```

2.2.6 Switch Database Access Interface

In addition to direct database connections — which can sometimes be restricted or prohibited due to security concerns — the **PyTestLog2DB** tool also provides users with an alternative way (interface) to import test data into the database, namely the REST API.

`--interface INTERFACE`: An optional argument to specify the database backend interface. The default value is `db` (direct connection to MySQL database). It can also be set to `rest` to import data into the database via a REST API.

Direct Connection Backend (interface="db")

The default mode of operation for the import tool is the direct connection to the MySQL database. This means that when you run the tool without specifying the `--interface` option, it automatically uses the `db` interface, establishing a direct connection to the MySQL database.

You can also explicitly specify the `db` interface by using the `--interface` option with the value `db`.

```
PyTestLog2DB <output.xml> <db\_host> <db\_user> <db\_password> <db\_name> --interface ↵
  ↳ db
```

This method provides flexibility, allowing users to choose between interfaces explicitly or rely on the default behavior.

REST API Backend (interface="rest")

The 'rest' interface allows users to import data into the database through a REST API. This mode is particularly useful in scenarios where direct database connections are restricted or not feasible, such as when the database is secured and direct connections are not allowed.

```
PyTestLog2DB <output.xml> <api\_endpoint> <api\_user> <api\_password> <db\_name> ↵
  ↳ --interface rest
```



Additional Notes

This feature offers easy switching between database backends, making it transparent and convenient for end-users.

When using the REST API interface, ensure that you have the necessary credentials and permissions to access the REST API endpoints.

2.3 Display on TestResultWebApp

As soon as the **.xml* file(s) is imported successfully to database, the result for that execution will be available on [TestResultWebApp](#).

Dashboard view:

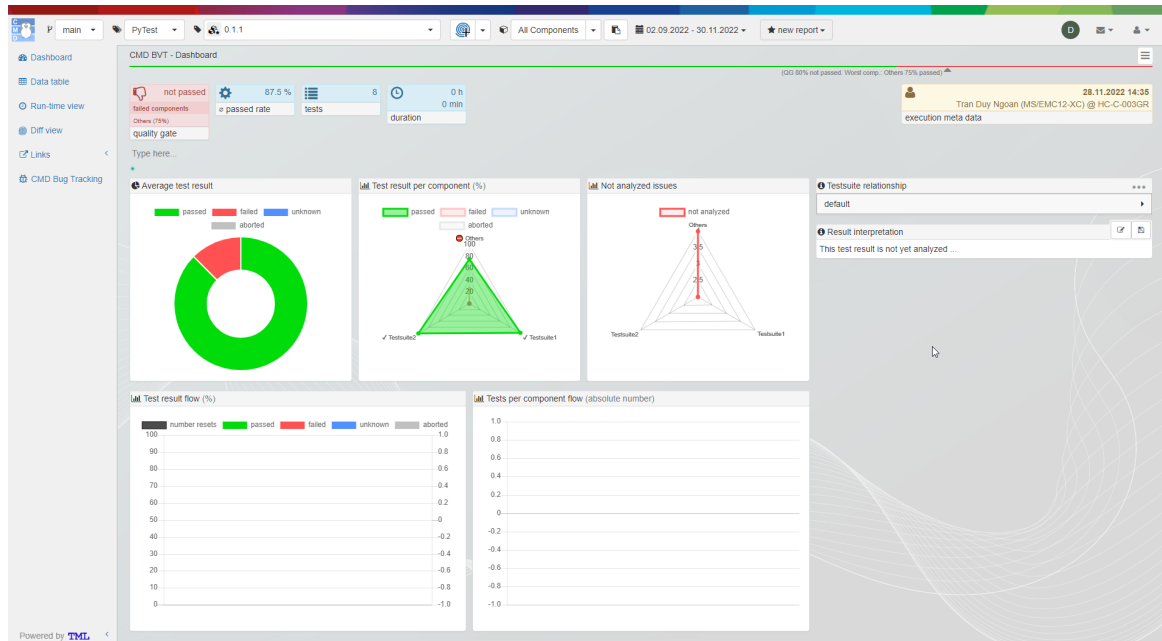


Figure 2.1: Dashboard view

Datatable view:

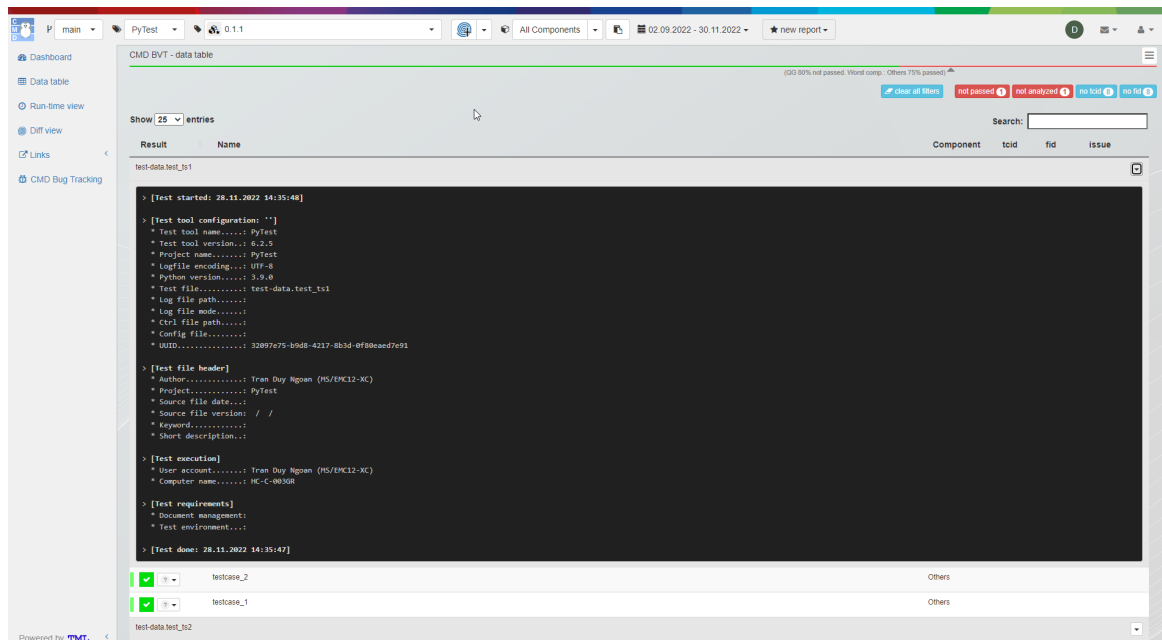


Figure 2.2: Datatable view

Chapter 3

pytestlog2db.py

3.1 Function: collect_xml_result_files

Collect all valid Robot xml result file in given path.

Arguments:

- `path`
/ *Condition*: required / *Type*: str /
Path to Robot result folder or file to be searched.
- `search_recursive`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If set, the given path is searched recursively for xml result files.

Returns:

- `lFoundFiles`
/ *Type*: list /
List of valid xml result file(s) in given path.

3.2 Function: validate_xml_result

Verify the given xml result file is valid or not.

Arguments:

- `xml_result`
/ *Condition*: required / *Type*: str /
Path to PyTest result file.
- `xsd_schema`
/ *Condition*: optional / *Type*: str / *Default*: <installed_folder>/xsd/junit.xsd /
Path to Robot schema *.xsd file.

System Message

WARNING/2 in <string>, line 45
[Inline emphasis start-string without end-string.](#)

- `exit_on_failure`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If set, exit with fatal error if the schema validation of given xml file failed.

Returns:

- / *Type*: bool /
True if the given xml result is valid with the provided schema *.xsd.

System Message

WARNING/2 in <string>, line 57
[Inline emphasis start-string without end-string.](#)

3.3 Function: is_valid_uuid

Verify the given UUID is valid or not.

Arguments:

- `uuid_to_test`
/ *Condition*: required / *Type*: str /
UUID to be verified.
- `version`
/ *Condition*: optional / *Type*: int / *Default*: 4 /
UUID version.

Returns:

- `bValid`
/ *Type*: bool /
True if the given UUID is valid.

3.4 Function: is_valid_config

Validate the json configuration base on given schema.

Default schema supports below information:

```
CONFIG_SCHEMA = {
    "components": [str, dict],
    "variant"    : str,
    "version_sw": str,
    "version_hw": str,
    "version_test": str,
    "testtool"   : str,
    "tester"     : str
}
```

Arguments:

- `dConfig`
/ *Condition*: required / *Type*: dict /
Json configuration object to be verified.
- `dSchema`
/ *Condition*: optional / *Type*: dict / *Default*: CONFIG_SCHEMA /
Schema for the validation.

- `bExitOnFail`
/ *Condition*: optional / *Type*: bool / *Default*: True /
If True, exit tool in case the validation is fail.

Returns:

- `bValid`
/ *Type*: bool /
True if the given json configuration data is valid.

3.5 Function: parse_pytest_xml

Parse and merge all given pytest *.xml result files into one result file. Besides, *starttime* and *endtime* are also calculated and added in the merged result.

System Message

WARNING/2 in <string>, line 136
[Inline emphasis start-string without end-string.](#)

Arguments:

- `xmlfiles`
/ *Condition*: required / *Type*: str /
Path to pytest *.xml result file(s).

System Message

WARNING/2 in <string>, line 144
[Inline emphasis start-string without end-string.](#)

Returns:

- `oMergedTree`
/ *Type*: `etree._Element` object /
The result object which is parsed from provided pytest *.xml result file(s).

System Message

WARNING/2 in <string>, line 152
[Inline emphasis start-string without end-string.](#)

3.6 Function: get_branch_from_swversion

Get branch name from software version information.

Convention of branch information in suffix of software version:

- All software version with .0F is the main/feature branch. The leading number is the current year. E.g. 17.0F03
- All software version with .1S, .2S, ... is a stabi branch. The leading number is the year of branching out for stabilization. The number before "S" is the order of branching out in the year.

Arguments:

- `sw_version`
/ *Condition*: required / *Type*: str /
Software version.

Returns:

- `branch_name`
/ *Type*: str /
Branch name.

3.7 Function: get_test_result

Get test result from provided Testcase object.

Arguments:

- `oTest`
/ *Condition*: required / *Type*: `etree._Element` object /
Testcase object.

Returns:

/ *Type*: tuple /
Testcase result which contains `result_main`, `lastlog` and `result_return`.

3.8 Function: process_component_info

Return the component name bases on provided testcase's classname and component mapping.

Arguments:

- `dConfig`
/ *Condition*: required / *Type*: dict /
Configuration which contains the mapping between component and testcase's classname.
- `sTestClassname`
/ *Condition*: required / *Type*: str /
Testcase's classname to get the component info.

Returns:

- `sComponent`
/ *Type*: tuple /
Component name maps with given testcase's classname. Otherwise, "unknown" will be return as component name.

3.9 Function: process_config_file

Parse information from configuration file:

- `component`:


```

    {
        "components" : {
            "componentA" : "componentA/path/to/testcase",
            "componentB" : "componentB/path/to/testcase",
            "componentC" : [
                "componentC1/path/to/testcase",
                "componentC2/path/to/testcase"
            ]
        }
    }

```

Then all testcases which their paths contain componentA/path/to/testcase will be belong to componentA, ...

Arguments:

- `config_file`
/ *Condition*: required / *Type*: str /
Path to configuration file.

Returns:

- `dConfig`
/ *Type*: dict /
Configuration object.

3.10 Function: process_test

Process test case data and create new test case record.

Arguments:

- `db`
/ *Condition*: required / *Type*: *CDataBase* object /
CDataBase object.
- `test`
/ *Condition*: required / *Type*: *etree._Element* object /
Robot test object.
- `file_id`
/ *Condition*: required / *Type*: int /
File ID for mapping.
- `test_result_id`
/ *Condition*: required / *Type*: str /
Test result ID for mapping.
- `component_name`
/ *Condition*: required / *Type*: str /
Component name which this test case is belong to.
- `test_number`
/ *Condition*: required / *Type*: int /
Order of test case in file.
- `start_time`
/ *Condition*: required / *Type*: *datetime* object /
Start time of testcase.

Returns:

/ *Type*: float /
Duration (in second) of test execution.

3.11 Function: process_suite

Process to the lowest suite level (test file):

- Create new file and its header information
- Then, process all child test cases

Arguments:

- db
/ *Condition*: required / *Type*: *CDataBase* object /
CDataBase object.
- suite
/ *Condition*: required / *Type*: *etree._Element* object /
Robot suite object.
- _tbl_test_result_id
/ *Condition*: required / *Type*: str /
UUID of test result for importing.
- dConfig
/ *Condition*: required / *Type*: dict / *Default*: None /
Configuration data which is parsed from given json configuration file.

Returns:

(no returns)

3.12 Function: PyTestLog2DB

Import pytest results from *.xml file(s) to TestResultWebApp's database.

Flow to import PyTest results to database:

1. Process provided arguments from command line.
2. Parse PyTest results.
3. Connect to database.
4. Import results into database.
5. Disconnect from database.

Arguments:

- args
/ *Condition*: required / *Type*: *ArgumentParser* object /
Argument parser object which contains:
 - *resultxmlfile* : path to the xml result file or directory of result files to be imported.
 - *server* : server which hosts the database (IP or URL).

- *user* : user for database login.
- *password* : password for database login.
- *database* : database name.
- *recursive* : if True, then the path is searched recursively for log files to be imported.
- *dryrun* : if True, then verify all input arguments (includes DB connection) and show what would be done.
- *append* : if True, then allow to append new result(s) to existing execution result UUID which is provided by --UUID argument.
- *UUID* : UUID used to identify the import and version ID on TestResultWebApp.
- *variant* : variant name to be set for this import.
- *versions* : metadata: Versions (Software;Hardware;Test) to be set for this import.
- *config* : configuration json file for component mapping information.
- *testrunurl*: link to test execution job: Jenkins job, Gitlab CI/CD pipeline, ...

Returns:*(no returns)*

3.13 Class: Logger

Logger class for logging message.

3.13.1 Method: config

Configure Logger class.

Arguments:

- *output_console*
/ *Condition*: optional / *Type*: bool / *Default*: True /
Write message to console output.
- *output_logfile*
/ *Condition*: optional / *Type*: str / *Default*: None /
Path to log file output.
- *indent*
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.
- *dryrun*
/ *Condition*: optional / *Type*: bool / *Default*: True /
If set, a prefix as 'dryrun' is added for all messages.

Returns:*(no returns)*

3.13.2 Method: log

Write log message to console/file output.

Arguments:

- *msg*
/ *Condition*: optional / *Type*: str / *Default*: "" /
Message which is written to output.

- `color`
/ *Condition*: optional / *Type*: str / *Default*: None /
Color style for the message.
- `indent`
/ *Condition*: optional / *Type*: int / *Default*: 0 /
Offset indent.

Returns:

(no returns)

3.13.3 Method: log_warning

Write warning message to console/file output.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Warning message which is written to output.

Returns:

(no returns)

3.13.4 Method: log_error

Write error message to console/file output.

Arguments:

- `msg`
/ *Condition*: required / *Type*: str /
Error message which is written to output.
- `fatal_error`
/ *Condition*: optional / *Type*: bool / *Default*: False /
If set, tool will terminate after logging error message.

Returns:

(no returns)

Chapter 4

Appendix

About PyTestLog2DB:

Author

Tran Duy Ngoan (Ngoan.TranDuy@vn.bosch.com)

Version

0.4.2 (01.07.2026)

Short Description

Imports pytest result(s) to TestResultWebApp database

Homepage

[python-pytestlog2db](#)

Documentation

[Readme](#)
[Main Documentation](#)

Sources

[Repository](#)

Issues

[Issues](#)

Python required

≥ 3.11

License

Apache-2.0

Chapter 5

History

0.1.0	11/2022
<i>Initial version</i>	
0.1.1	22.11.2022
<i>Initial implementation of PyTestLog2DB tool</i>	
0.1.2	01.12.2022
<i>Add tool's document</i>	
0.1.4	24.02.2023
- Rename renamed key "components" in configuration json - Change behaviour of append mode without UUID to raise a fatal error - Improve console log for append mode and matched component for testcase - Add command line arguments <code>--versions</code> and <code>--variant</code>	
0.1.5	27.02.2023
<i>Change behaviour of append mode with not existing UUID to raise a fatal error</i>	
0.1.6	01.03.2023
<i>Add a validation for xmlresultfile with junitxml schema</i>	
0.1.7	10.03.2023
- Add a validation for existing project/variant and version_sw in db when using append mode - Remove db maxlength handlers: truncations and validation	
0.1.8	21.03.2023
<i>Support 4 byte characters for importing to db</i>	
0.1.9	20.04.2023
- Enhance console log with test case counter and component statistics - Add try/except for database access	
0.2.6	08.05.2023
<i>Update README for publishing package to PyPI</i>	
0.2.8	14.12.2023
<i>Fix missing return of test duration when failed to import testcase</i>	
0.3.0	07.05.2024
<i>Add optional argument <code>--interface</code> which allow to switch between <code>db</code> and <code>rest</code> interfaces</i>	

CHAPTER 5. HISTORY

0.3.1	20.05.2024
<i>Fix type error of lastlog when import with rest interface</i>	
0.3.2	28.06.2024
<i>Add optional argument <code>--testrunurl</code> to specify link to test job such as Jenkins or Gitlab CI/CD</i>	
0.3.3	24.07.2024
<i>Fix issue with parsing timestamps from pytest v8.3.1 result files</i>	
0.3.4	02.08.2024
<i>Fix issue when import datetime with timezone for old version of mysql server</i>	
0.4.0	23.03.2026
<i>Usage of Setuptools replaced by usage of PIP/TOML/Setuptools</i>	
0.4.1	30.06.2026
<i>Use <code>importlib.metadata</code> as alternative to deprecated <code>pkg_resources</code></i>	
0.4.2	01.07.2026
<i>Fix issue when classname is not provided in junitxml result file, use suite name instead</i>	

PyTestLog2DB.pdf*Created at 07.09.2026 - 09:47:27**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.16 TestResultWebApp

TestResultWebApp

v. 0.1.3

Thomas Pollerspöck

18.10.2022

Contents

1	Introduction	1
2	Description	2
2.1	TestResultWebApp Architecture	2
2.2	Import Data	2
2.3	Data Visualization	3
2.3.1	Main menu	3
2.3.2	Dashboard View	3
2.3.3	DataTable View	7
2.3.4	Runtime View	11
2.3.5	Diff View	12
2.4	Developer guidance	14
2.4.1	How to run new TestResultWebApp instance	14
3	Appendix	17
4	History	18

Chapter 1

Introduction

TestResultWebApp is a web-based application which is developed and used at  **BOSCH** since 2016 and was published as open source on Github in 2020.

TestResultWebApp is used for visualizing and tracking test execution results. It provides charts from an overview of the test result to the detail of all included test cases.

It also provides tools to control the quality of developing software version (under testing) by the a graphical comparison of test results from different test executions.

TestResultWebApp is highly modular written. Therefore it is also easy to extend it in order to add new graphics or evaluations of the test result data.

TestResultWebApp uses bootstrap, jquery, nodejs and mysql. For the charts is uses chartjs and D3.

Chapter 2

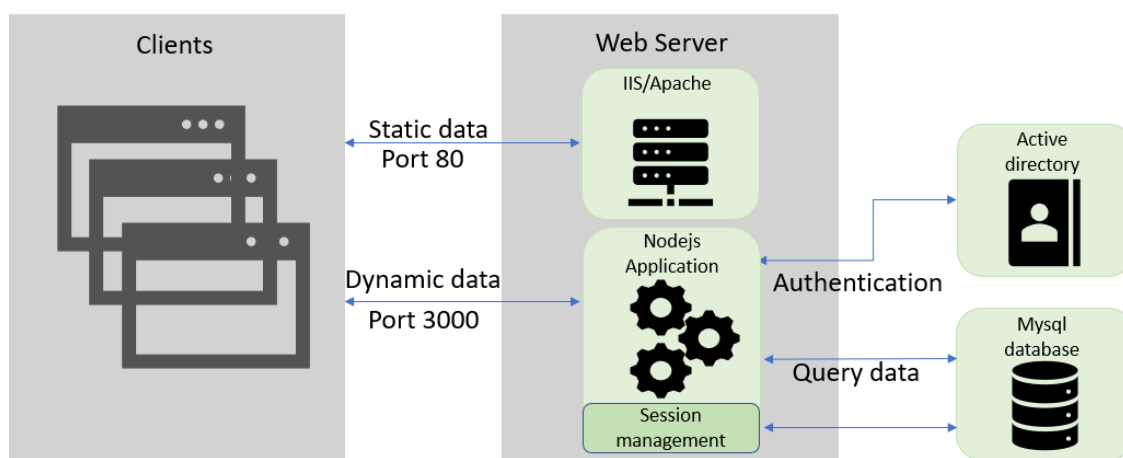
Description

2.1 TestResultWebApp Architecture

The TestResultWebApp application includes:

- Database: [MySQL](#) - which contains all test execution results. For details of all tables in the database, please refer the [database model](#).
- Active Directory: for the authentication.
- Web server: which hosts the static data and runs the [Node.js](#) application for providing the dynamic data.
- Web client: is written in javascript which also use some libraries such as [jQuery](#), [bootstrap](#), [Chart.js](#), [D3](#) for data visualization.

Please refer below architecture for more detail:



2.2 Import Data

The data which will be visualized on WebApp comes directly from the database. For this the test execution result data must be transformed first into the defined [database model](#) then imported into the database.

The data base model is generic, so test result data can be any. Only a test result importer must transform and import the data.

We provide already the [RobotLog2DB](#) which helps to import the Robot Framework result file(s) *output.xml* to the database of the TestResultWebApp.

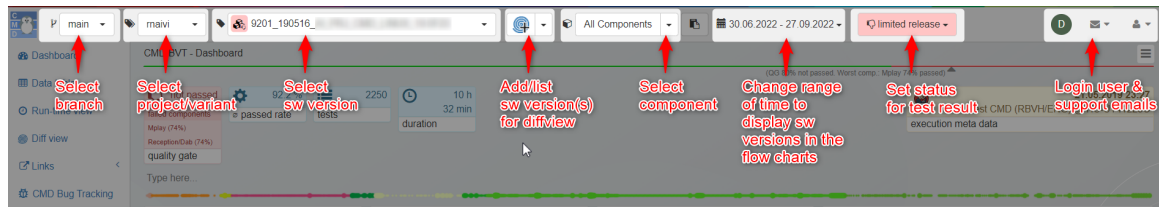
You will need to provide only the Robot Framework result file(s) and database's credential information, that Robot-Log2DB will parse the test execution result data and interact with the provided database to import the data.

Please refer [RobotLog2DB's usage](#) and [its document](#) for more detail.

2.3 Data Visualization

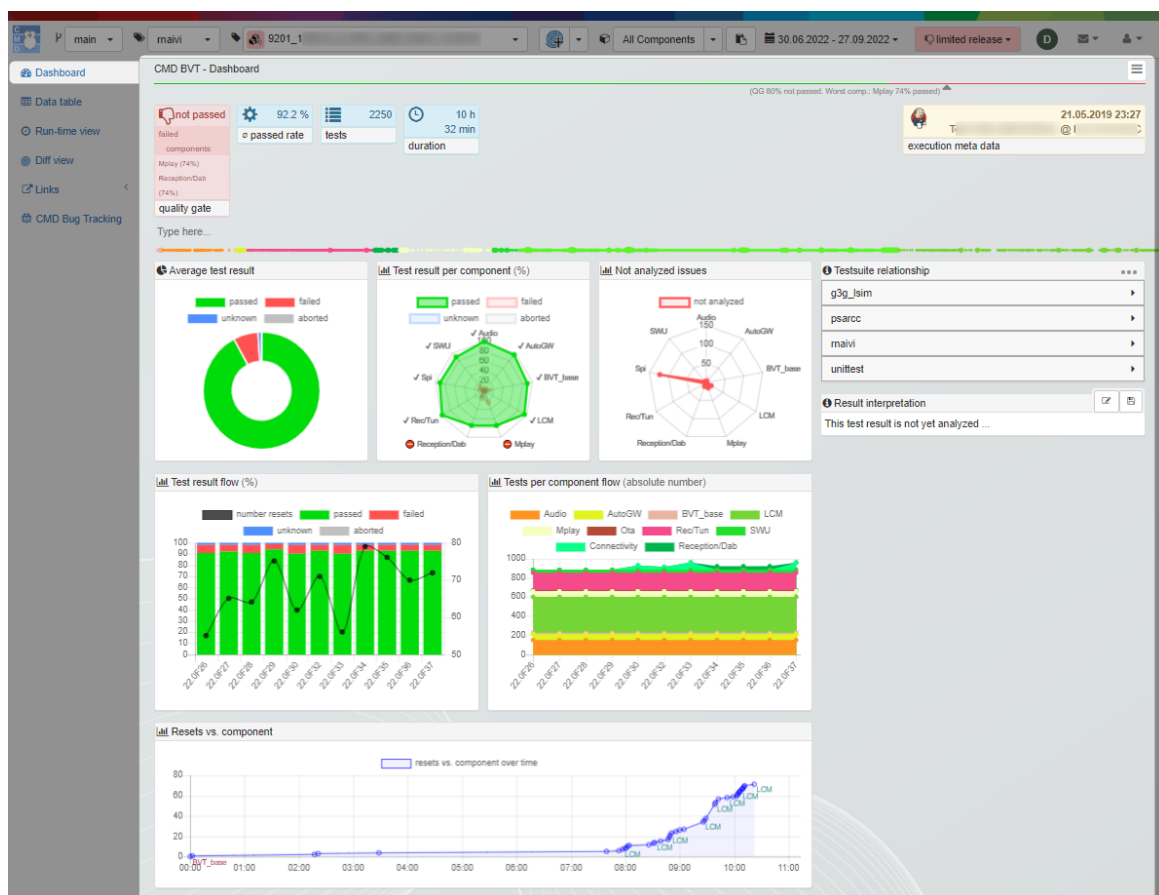
2.3.1 Main menu

The WebApp provides a main menu for selecting a specific **branch**, **project/variant** and **software version** to be displayed.



2.3.2 Dashboard View

The Dashboard view does not only provide an overview of test execution results but also shows the correlation between components within the result and relationship with other test execution results.

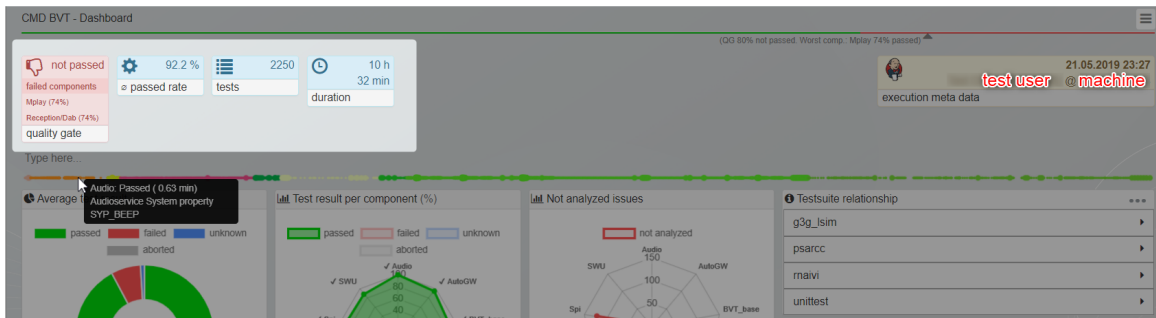


Result overview

At the top left corner of the Dashboard view you find the test execution result statistics:

CHAPTER 2. DESCRIPTION

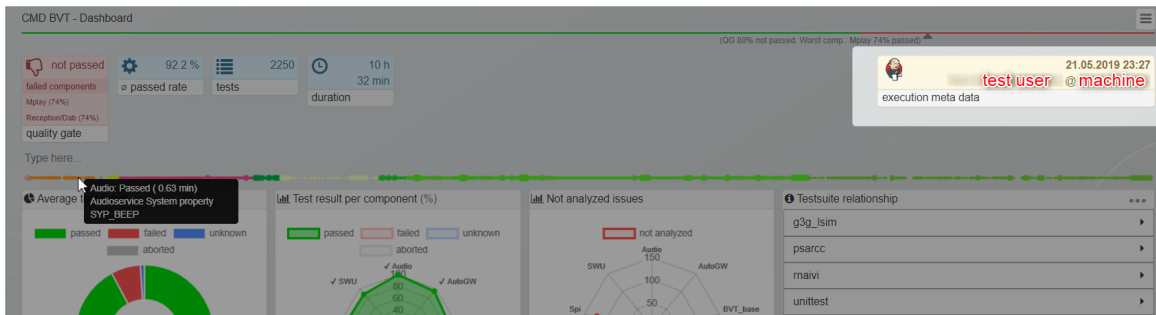
2.3. DATA VISUALIZATION



Which contains:

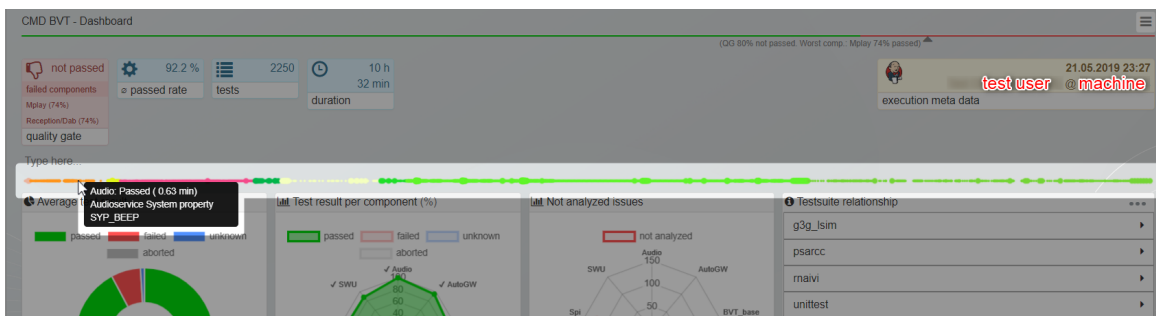
- Overall status (you can define a quality gate).
- Passed rate.
- Total number of executed test cases.
- Test execution duration.

On other right-hand side, there is information about the execution environment:



- Execution time (start of test execution)
- Test machine
- Test user
- Jenkins link (embedded URL in the Jenkins's icon)

Below them is the test execution result timeline:



It provides:

- The timeline of the executed test cases which are grouped by components (different color). Left side is start of testing. Right side the end.
- How much time is consumed by the individual test case by the distance to the next test at the time line or the detail pop-up when hovering on the dot at the timeline.
- Test status result: A small dot for **passed** status and a big dot for others.

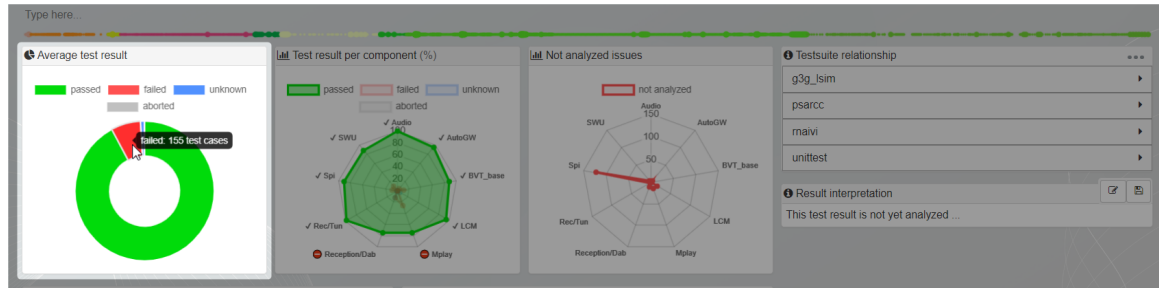
CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION

Average test result

This chart will give you the detail of the test result with the percentage (number of test cases will be shown when moving the mouse over the pie chart) of each result status (passed, failed, unknown or aborted) of the execution.

So that, you can qualify this test execution result is good or not.

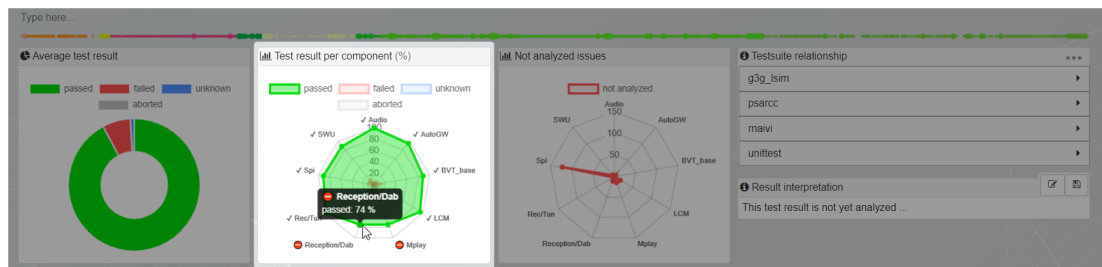


Component's correlation

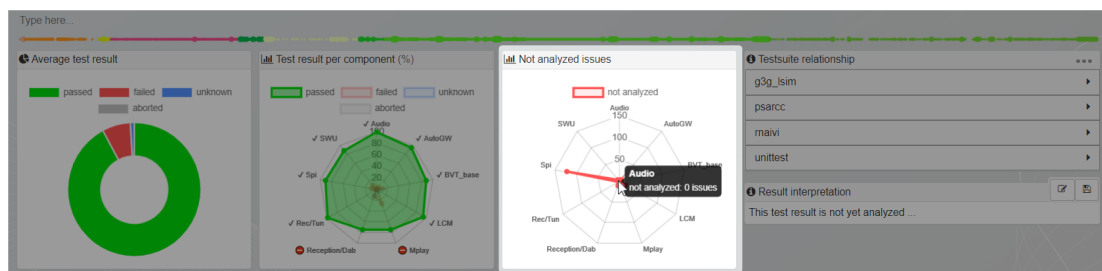
The next charts will help you to get the correlation between components within the test result, so that you can know which component(s) impact(s) the test result.

- **Test result per component** chart: provides a fast overview of test result percentages over all components. Here you can quickly see the quality of the system under test. Ideally the spider chart should be fully green. This means that all component tests result in 100% passed.

You can also define a quality gate (default 80%) which results in a "minus" in front of the component name if the quality gate is not reached.



- **Not analyzed issues** chart: you can know how many test cases of components are issued without analysis. The Datatable view provides a process to set failed test cases to "analyzed". Ideally this spider chart should have a red dot in the center. This means all failed test cases are analyzed.



Relationships with other test execution results

- **Testsuite relationship**: will let you know all the related test results (grouped by project/variant) of the current selected version.

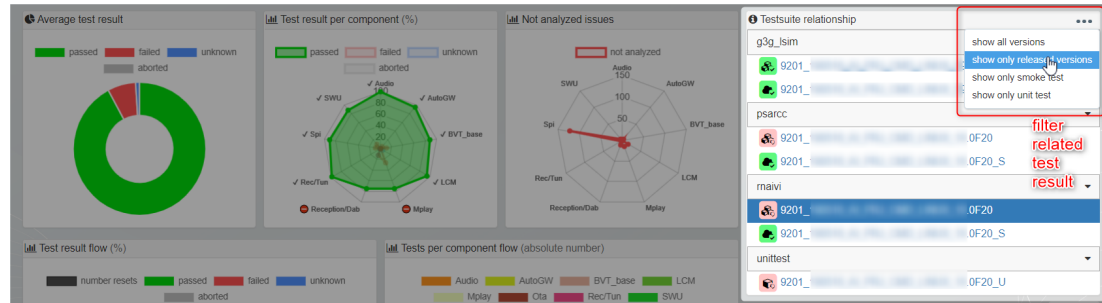
This allows to quickly go to related test results to have a fast comparison about the quality of the selected version across all projects/variants.

CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION

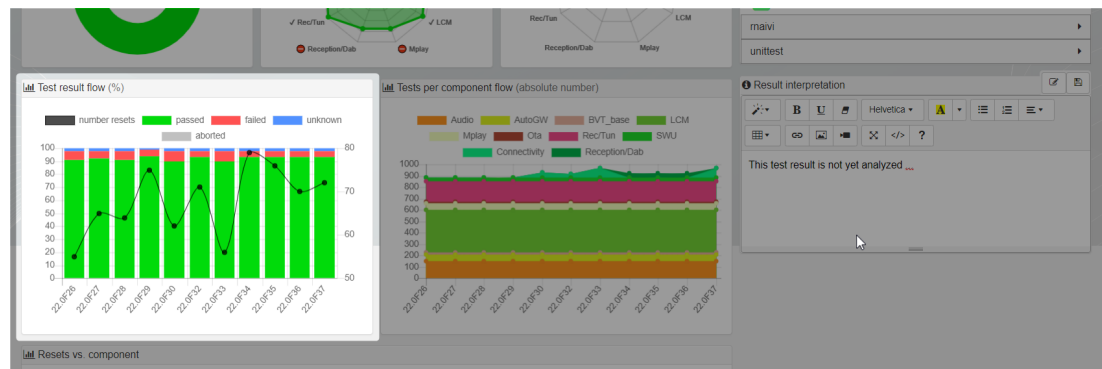
For example: the selected version have been executed for 4 projects/variants: *g3g_lsim*, *psarcc*, *rnaivi* and *unittest*. Each variant (except the *unittest*) has 2 test results (one for the smoke test and one for the whole test execution result).

Then, all the related test execution results will be displayed as below:



There is also the context menu (...) that allows you to filter the related test results.

- **Test result flow chart:** provides the picture of quality change (percentages of each status) between versions. So that, you can understand the quality of testing software is being improved or vice versa.



- **Tests per component flow chart:** provides the change of number test cases per component between versions. You can aware the number of test cases per component and how many test cases are added or removed (per component or the whole test result) from those versions.

This allows to quickly verify if all expected tests are executed, or if expected tests were not executed.

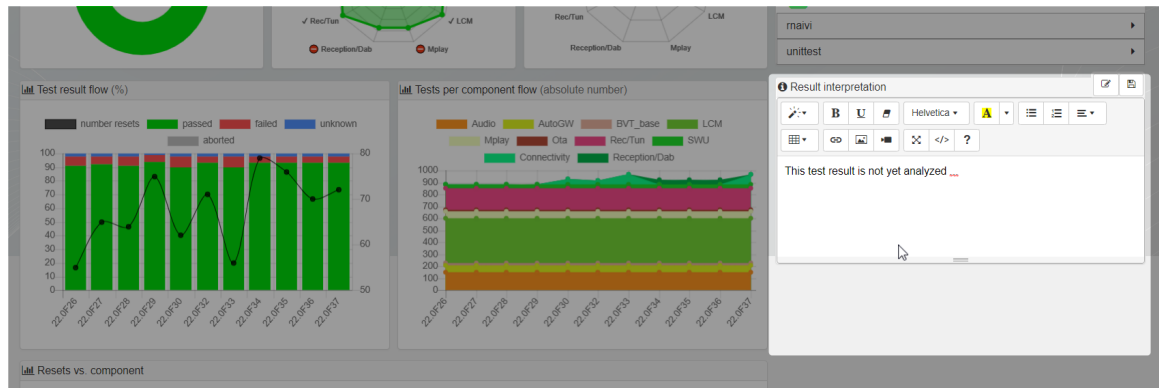
**Notice**

The versions which are displayed in **Test result flow** and **Tests per component flow** charts are the executed test results within the selected range of time in [the main menu](#) (default is last 90 days).

Result interpretation

You can give more information, provide an analysis, take notes, ... on the execution result by leaving comments in the **Result interpretation** section.

As soon as the Result interpretation is saved, that information will be updated to the database. So that, other users who browse to this test result can see the analysis/comment for reference.



Resets vs. component

The **Resets vs. component** chart will help you to know the interaction of component tests with the DUT (device under test) by providing the reset counter per component during the execution.

You can have awareness that:

- When the DUT has been reset.
- Which component tests were executed when a reset happened.
- How many reset has been done during each component and the whole test execution.



2.3.3 DataTable View

The Datatable view provides the summary table which contains all detail information of each test case (grouped by component) within the test execution.

CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION

Result	Name	Component	tcid	fid	issue
✓	D:/JenkinsClient/workspace/man_bvt_AIVI_cmd_fa/al_audio/_tmtest/components/AMCommandPlugin/testcases/connectUTS_02_027.tml	Audio	TC1458	SWF-8093	
✓	INVALID sink connection	Audio	TC1412	SWF-8093	
✓	Connect to AUX_2	Audio	TC1480	SWF-8093	
✓	change source from AUX_2 to TUNER_AM and sink internal Amp	Audio	TC1415	SWF-8093	RTC-122345
✓	change source from TUNER_AM to TUNER_FM and sink internal Amp	Audio	TC1421	SWF-8093	
✓	change source from AUX_2 to MEDIA_PLAYER and sink internal Amp	Audio	TC3005	SWF-8093	
✓	change source from AUX_2 to AUX_2 and sink internal Amp	Audio	TC1440	SWF-8093	
✓	INVALID sink connection	Audio	TC1483	SWF-8093	
✓	Connect to TUNER_AM	Audio	TC1483	SWF-8093	
✓	change source from TUNER_AM to TUNER_FM and sink internal Amp	Audio	TC1483	SWF-8093	
✓	change source from TUNER_AM to MEDIA_PLAYER and sink internal Amp	Audio	TC1482	SWF-8093	

Showing 1 to 10 of 2250 entries

Previous 1 2 3 4 5 ... 225 Next

Besides, you can also:

- determine how many test cases (entries) are displayed in the table page.

CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION

CMD BVT - data table

(GG 90% not passed, Worst comp.: Mplay 74% passed)

clear all filters not passed 175 not analyzed 175 no fid 1555 no fid 1707

Show 10 entries

Result Name

D:\JenkinsClient\workspace\man_bvt_AIVI_cmd_fs\ai_audio_tmtest\components\AMCommandPlugin\tests\testcases\connect\UTS_02_027.tml

Component	tcid	fid	issue
Audio	TC1458	SWF-6093	
Audio	TC1412	SWF-6093	
Audio	TC1480	SWF-6093	
Audio	TC1415	SWF-6093	RTG-122349
Audio	TC1421	SWF-6093	
Audio	TC3005	SWF-6093	
Audio	TC1440	SWF-6093	
Audio	TC1483	SWF-6093	
Audio	TC1483	SWF-6093	
Audio	TC1482	SWF-6093	

Showing 1 to 10 of 2250 entries

Previous 1 2 3 4 5 ... 225 Next

- apply filters to display such as not passed test cases.

CMD BVT - data table

(GG 80% not passed, Worst comp.: Mplay 74% passed)

clear all filters not passed 175 not analyzed 175 no fid 1555 no fid 1707

Show 10 entries

Result Name

D:\JenkinsClient\workspace\man_bvt_AIVI_cmd_fs\ai_automotive_id\ai_gateway\tml\tests\tests\CMD\Audio\TC\tml\Audio_Negative_Boundary.tml

Component	tcid	fid	issue
AutomotiveGateway			
AutomotiveGateway			
AutomotiveGateway			
AutomotiveGateway			
LCM	TC6314	SWF-784	
LCM	TC6256	SWF-7266	
LCM	TC3458	SWF-784	
LCM	TC6309	SWF-7266	
LCM	TC6178	SWF-784	
LCM	TC6177	SWF-784	

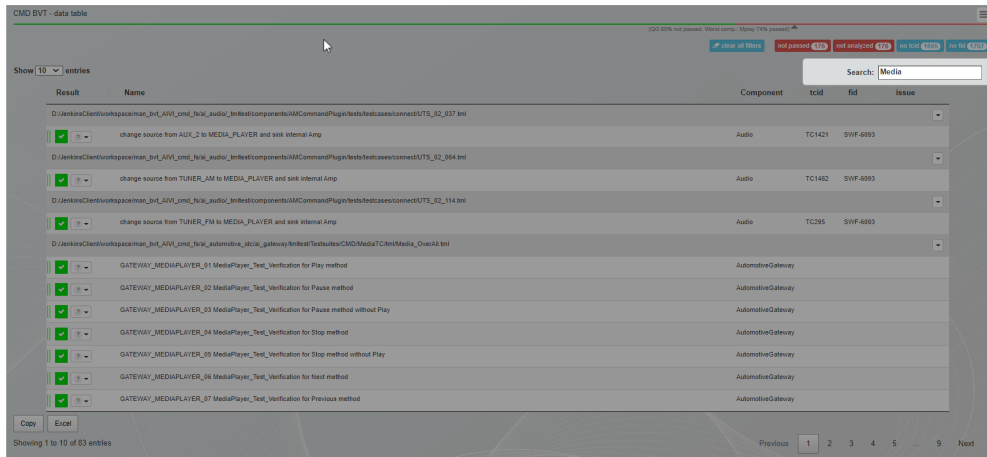
Showing 1 to 10 of 176 entries

Previous 1 2 3 4 5 ... 18 Next

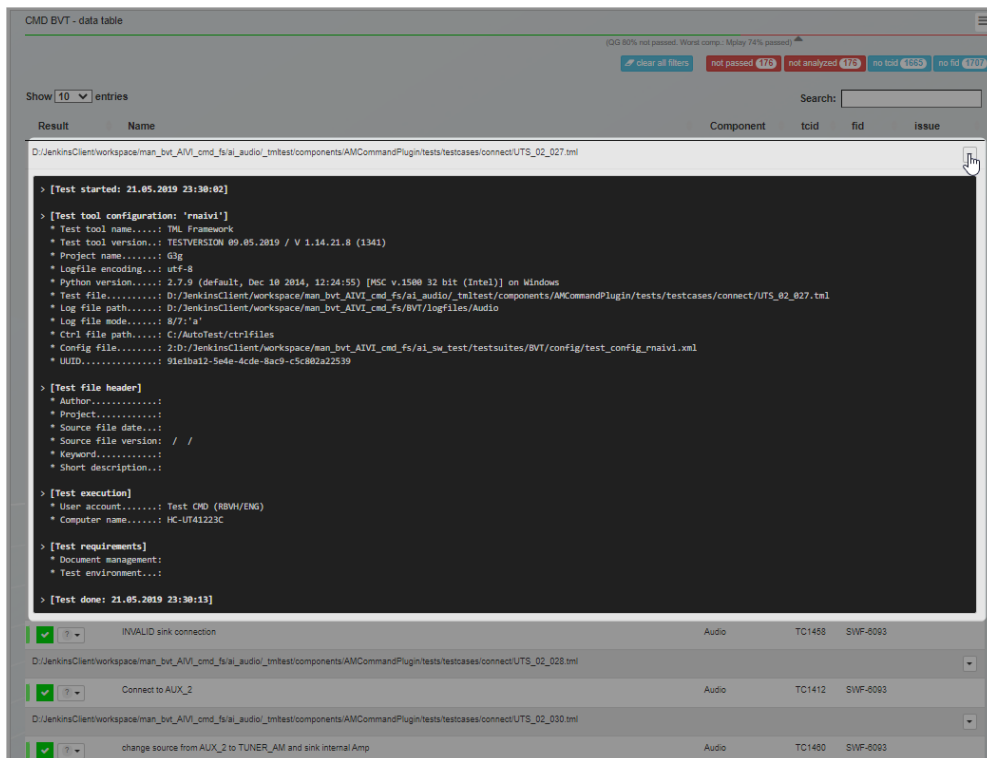
CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION

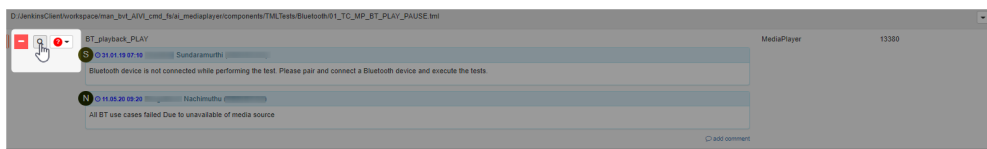
- search for the specific test case for reference.



- get more information about test environment, configurations.



- get traceback information for not passed test case(s). An pop-up will be displayed which show all recorded traceback (error) information.



- give comment to the test case, link an issue ticket with the test case or watch the history of the user interaction on the test case.

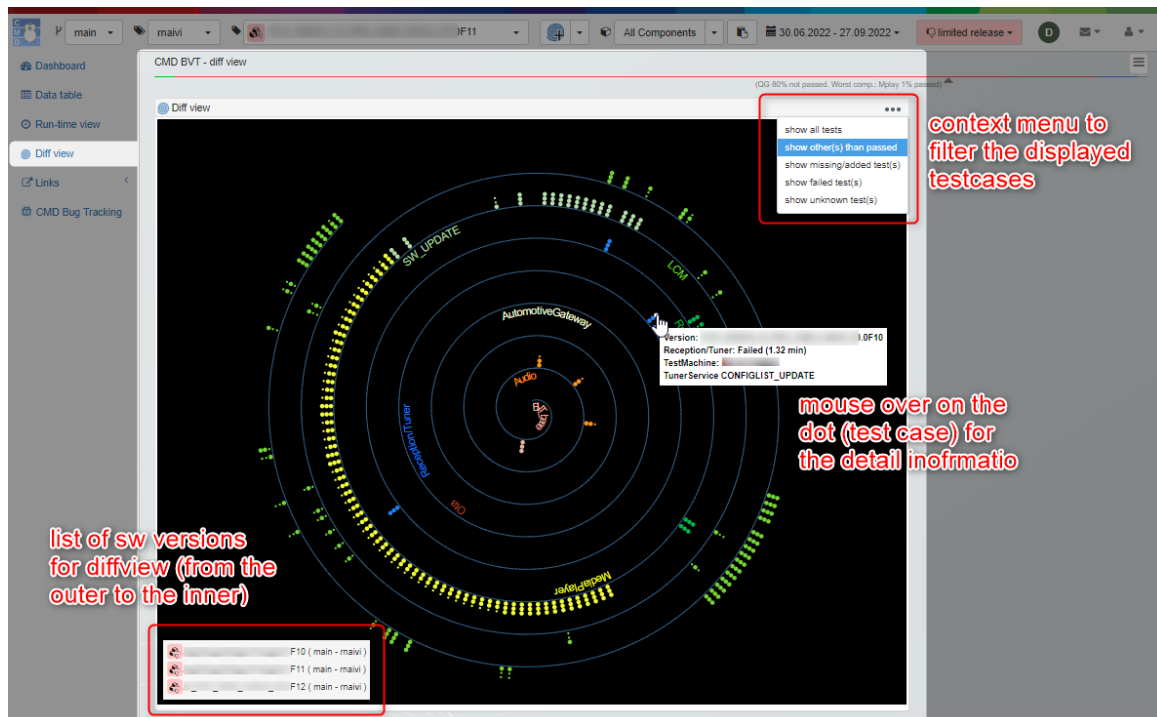
2.3.5 Diff View

The diffview contains a spiral chart which displays the differences between the software versions you want to compare. The center of the spiral is the start time of the test execution, the end of the spiral the end time. You find also the name of the component in the spiral. By default only changed or **failed** test results are displayed.

So that you can:

- see a test case result change (e.g from **passed** to **failed**).
- quickly find new or removed test case(s).
- recognize unstable test case(s)/component(s).

By default, without adding the version for diffview, the current selected version and its around versions (the previous and the next version if existing) will be chosen for the diffview as below:

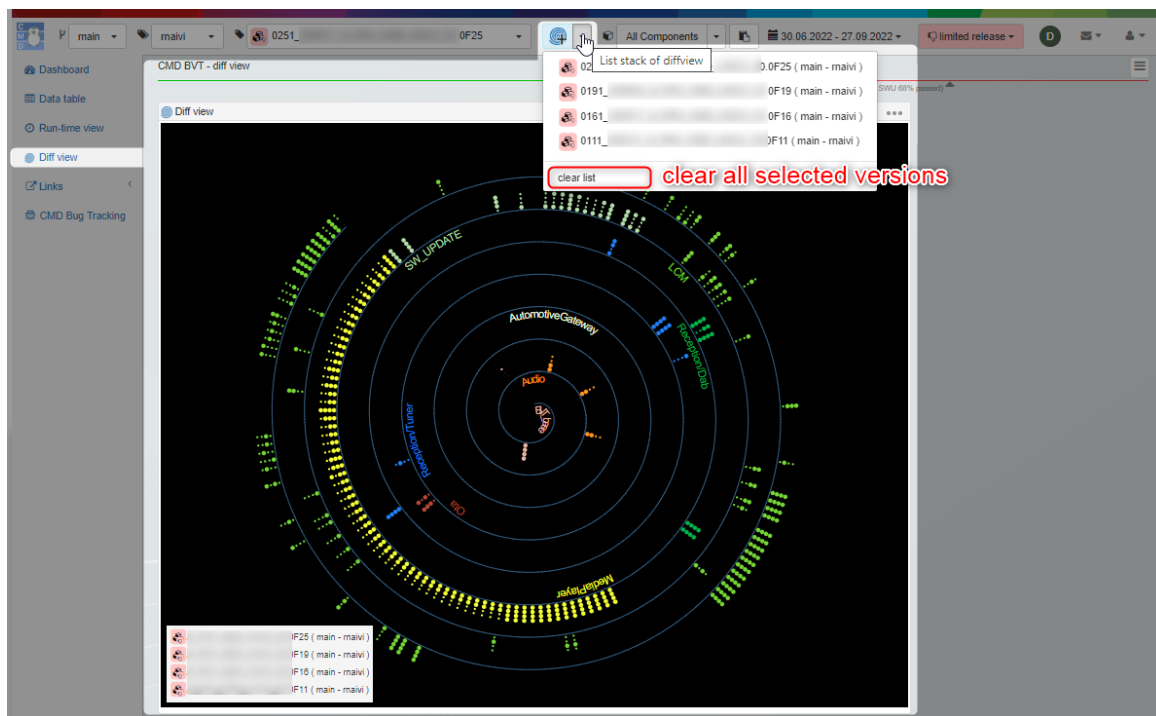


In case you want to add specific versions for the diffview, select the version from the version select box then click the add button  to add it to the list of versions to diff.

The dropdown button is used for viewing your selected versions. You can also clear your selection with **clear list** option.

CHAPTER 2. DESCRIPTION

2.3. DATA VISUALIZATION



As soon as a new version is added for diffview, the spiral chart is updated immediately. Dependent on the number of executed test cases and browser rendering can need some time.

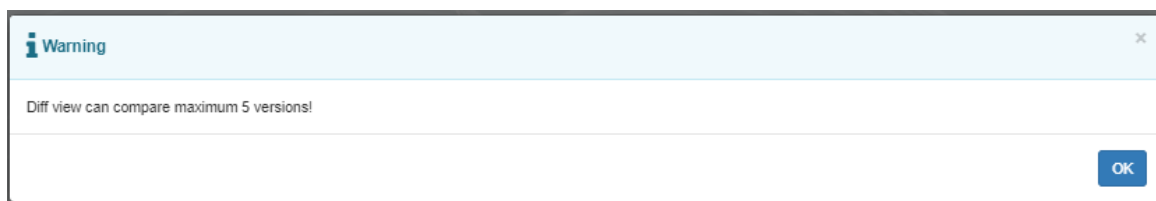
The dots which present the test cases along the spiral line (small dot is **passed** and bigger one for other status) are interactable. It means that you can:

- move the mouse over the dots to see the test case information.
- click on the failed test case (bigger dot) for the traceback information.

Notice:

You can only select maximum of **5** versions for diffview.

If the maximum of selected versions is reached and you click on the button to add more, a warning message will be displayed to prevent that action.



2.4 Developer guidance

Notice

In order to run up the TestResultWebApp, it requires some knowledge about:

- Web server: setup and run web server for web hosting.
- Nodejs platform and Express framework: adapt the sourcecode with your environemnt: domain, configurations, ...
- Mysql database: schema, tables, SQL scripting. We propose to use [MySQL Workbench](#) tool for working with Mysql database.

2.4.1 How to run new TestResultWebApp instance

1. Precondition:

- [Node.js](#) should be installed.
- [MySQL server](#) should be installed.
- A cloned package's resource from [Github repo](#)

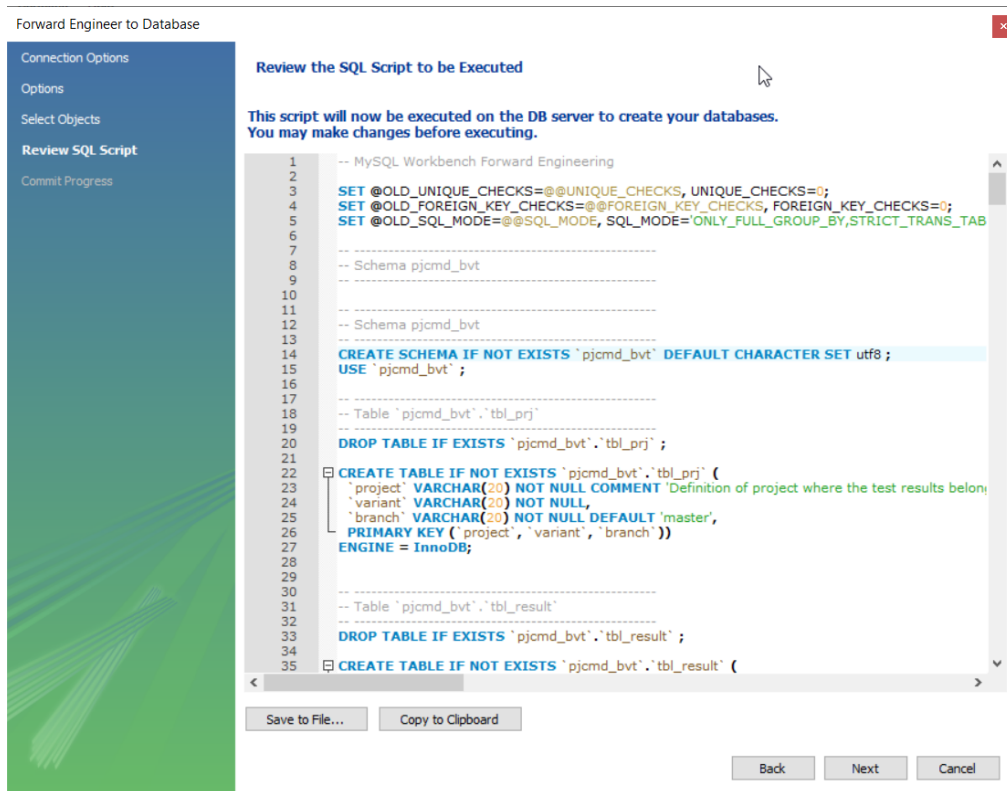
```
git clone https://github.com/test-fullautomation/testresultwebapp.git
```

- The port which will run Node.js application (default is **3000**) is opened (enable) in firewall.

2. Setup mysql database:

- create *user*, *password* and *schema* then update them as `global.mysqlOptions` in `webapp/web_server/lib/global.js` file.
- create required tables in your database schema with MySQL Workbench.
 - open the datamodel which is located at `mysql_server/datamodel/test_result.mwb`
 - export all defined tables in datamodel to your schema, use **Foreward Enginneer to Database** feature of MySQL Workbench.

```
EER Diagram > Database > Forward Engineer... > your schema
```

**Notice**

If you have created your schema with other name than the default name *pjcmd_bvt*, you should replace the schema name at the **Review SQL Script** step:



- * copy SQL script to text editor such as VsCode
- * replace *pjcmd_bvt* which your new schema name
- * paste SQL script back to the **Forward Engineer to Database** tool

before moving to next step to execute SQL script.

- create all store procedures by loading all SQL scripts under `mysql_server/TMLdb_sproc/` folder then execute them.

Notice

If you have created your schema with other name than the default name *pjcmd_bvt*, you should replace the schema name before executing SQL scripts.

3. Import the initial data to the database with [RobotLog2DB tool](#).
4. Adapt Web server:
 - `web_server/lib/global.js`: for domain, database's configuration, keys for authentication, ldap server for authentication, ...
 - `web_server/test.js`: for listening port of nodejs application.
5. Adapt Web client:
 - `web_client/dashboard/dist/js/common/global.js`: for domain.
 - `web_client/dashboard/dist/js/common/communication.js`: for listening port of Node.js application's API.
6. Start nodejs application by command:


```
node testdb.js
```

7. Start web server for hosting the static files:

You can use any web server [Apache](#), [IIS](#) or [Nginx](#) for hosting the static files under `web_client/dashboard/` folder when running as production.

For development, you can use directly `express.static` which is supported by Express framework for hosting static files. The `web_server/test.js` file should be modified to add this setting.

```
'use strict';

var global = require('./lib/global');
var path = require('path');

...

//for local GUI tests deliver static HTML
//content from port 3000
app.use(express.static(path.join(__dirname, "../web_client/dashboard/")));

var session = require('express-session');
var MySQLStore = require('express-mysql-session')(session);
...
```

8. Now open your favourite browser, go to the domain of webapp and enjoy.

Chapter 3

Appendix

About TestResultWebApp:

Author

Thomas Pollerspöck (Thomas.Pollerspoeck@de.bosch.com)

Version

0.1.3 (18.10.2022)

Short Description

Web based display of test results

Homepage

testresultwebapp

Documentation

Readme
Main Documentation

Sources

Repository

Issues

Issues

Chapter 4

History

0.1.0	07/2022
<i>Initial version</i>	
0.1.1	09/2022
<i>Update README file and package's document</i>	
0.1.2	05/10/2022
<i>Fix findings with package's document</i>	

TestResultWebApp.pdf*Created at 07.09.2026 - 09:47:30**by GenPackageDoc v. 0.44.0 / 05.06.2026*

10.17 VSCodeWelcome

VSCodeWelcome

v. 0.0.2

Mai Minh Tri

06.2025

Contents

1	Introduction	1
2	Description	2
2.1	Getting Started	2
2.2	Usage	2
2.2.1	How to Install	2
2.2.2	How to use	2
2.3	Feedback	2
2.4	About	2
2.4.1	Maintainers	2
2.4.2	Contributors	3
2.5	License	3
3	Appendix	4
4	History	5

Chapter 1

Introduction

The **VSCoDeWelcome** is for this and that. That has to be introduced here.

The sources of **VSCoDeWelcome** are available in [GitHub](#).

Information about how to install the **VSCoDeWelcome** can be found in the [README](#).

Chapter 2

Description

2.1 Getting Started

VSCoDeWelcome is a VSCodium extension that enhances the user experience by displaying a RobotFramework-AIO introduction for new users.

2.2 Usage

2.2.1 How to Install

To install the **Robotframework Welcome** extension:

1. Download the extension package or clone the repository.
2. Run the build script `build_vsix.sh` to generate the VSIX file.
3. Open Visual Studio Code or VSCodium.
4. Select the generated `robotframework-welcome-x.x.x.vsix` file to install the extension.

2.2.2 How to use

To customize the URL displayed on the welcome page:

1. Look for the setting named `robotframeworkWelcome.welcomeUrl` in the `settings.json` of `vscode`.
2. Enter the desired URL (e.g., <https://your-custom-url.com>) or a local file path (e.g., `path/to/file.html`).
3. Save the changes to the `package.json` file.

The status bar (bottom-right) now includes a "Welcome" button, allowing you to easily reopen the redesigned welcome page at any time.

2.3 Feedback

If you have any problem when using the library or think there is a better solution for any part of the library, I'd love to know it, as this will all help me to improve the library. Please don't hesitate to contact me.

Do share your valuable opinion, I appreciate your honest feedback!

2.4 About

2.4.1 Maintainers

[Mai Minh Tri](#)

2.4.2 Contributors

Mai Minh Tri

Thomas Pollerspöck

2.5 License

Copyright 2020-2023 Robert Bosch GmbH

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Chapter 3

Appendix

About VSCodeWelcome:

Author

Mai Minh Tri (Tri.MaiMinh@vn.bosch.com)

Version

0.0.2 (06.2025)

Short Description

RobotFramework_AIO Welcome Page for VSCode

Homepage

vscode-welcome

Documentation

Readme
Main Documentation

Sources

Repository

Issues

Issues

Chapter 4

History

0.0.1	06.2025
Initial version	
0.0.2	11.2025
- Redesigned the welcome page with a new look. - Added functionality to load URLs or local HTML files into the webview. - Added features to open files or folders from the welcome page. - Included library documentation for RobotFramework AIO keywords. - Added a "Welcome" button to the status bar (bottom-right), enabling quick and seamless access to the redesigned welcome page at any time.	

10.18 VSCodeJsonp

VSCodeJsonp

v. 0.0.3

Tran Duy Ngoan

04.2026

CONTENTS	CONTENTS
Contents	
1 Introduction	1
2 Description	2
3 Appendix	3
4 History	4
A	

Chapter 1

Introduction

The **VSCodeJsonp** is for this and that. That has to be introduced here.

The sources of **VSCodeJsonp** are available in [GitHub](#).

Informations about how to install the **VSCodeJsonp** can be found in the [README](#).

Chapter 2

Description

The **VSCodeJsonp** does this and that. That has to be explained here.

Chapter 3

Appendix

About VSCodeJsonp:

Author

Tran Duy Ngoan (Ngoan.TranDuy@vn.bosch.com)

Version

0.0.3 (04.2026)

Short Description

JSONP extension for VSCode

Homepage

vscode-jsonp

Documentation

Readme
Main Documentation

Sources

Repository

Issues

Issues

Chapter 4

History

0.0.1	05.2023
Initial version	

Chapter 11

Appendix

11.1 Installed Python Modules

This chapter contains a list of installed Python modules together with their version numbers.

Based on: Python 3.13.2 (main, Feb 5 2025, 19:11:32) [Clang 19.1.6]

Identified 94 packages.

Appium-Python-Client : 6.0.0	markdownify : 1.2.3	robotframework-extensions-collection : 0.12.0
argparse-addons : 0.12.0	MarkupSafe : 3.0.3	robotframework-prometheus : 0.9.0
attrs : 25.4.0	mdurl : 0.1.2	robotframework-pythonlibcore : 4.6.0
bcrypt : 5.0.0	mysqlclient : 2.2.7	robotframework-qconnect-base : 1.4.0
beautifulsoup4 : 4.15.0	odxtools : 11.5.4	robotframework-qconnect-winapp : 1.1.0
bincopy : 20.1.1	outcome : 1.3.0.post0	robotframework-robotlog2db : 1.6.0
build : 1.6.0	packaging : 26.3	robotframework-robotlog2rqm : 1.8.1
can-isotp : 2.0.7	paramiko : 5.0.0	RobotFramework_TestsuitesManagement : 0.11.0
certifi : 2025.11.12	pip : 26.2.1	RobotFramework-UDS : 0.2.0
cfci : 2.0.0	prometheus_client : 0.26.0	RobotFrameworkAIO : 0.39.1
charset-normalizer : 3.5.1	pyparser : 2.23	rpds-py : 2026.6.3
colorama : 0.4.6	pydotdict : 3.3.11	selenium : 4.39.0
cryptography : 46.0.3	pyelftools : 0.33	setuptools : 84.0.0
dasbus : 1.7	Pygments : 2.21.0	six : 1.17.0
decorator : 5.2.1	PyNaCl : 1.6.2	sniffio : 1.3.1
deprecation : 2.1.0	py pandoc : 1.17	sortedcontainers : 2.4.0
docutils : 0.22.4	py pandoc_binary : 1.17	soupsieve : 2.9.2
doipclient : 1.2.2	pyproject_hooks : 1.2.0	tabulate : 0.10.0
dotdict : 0.1	pyserial : 3.5	TestResultDBAccess : 0.2.1
EventBusClient : 0.2.0	PySocks : 1.7.1	trio : 0.34.0
GenPackageDoc : 0.44.0	pyspnego : 0.12.2	trio-websocket : 0.12.2
gssapi : 1.12.0	PyTestLog2DB : 0.4.2	typing_extensions : 4.16.0
h11 : 0.16.0	python-can : 4.6.1	udsoncan : 1.26.1
humanfriendly : 10.0	python-dateutil : 2.9.0.post0	urllib3 : 2.5.0
idna : 3.11	PythonExtensionsCollection : 0.17.1	websocket-client : 1.9.2
invoke : 3.0.3	referencing : 0.37.0	wheel : 0.48.0
Jinja2 : 3.1.6	regex : 2026.9.3	wrapt : 1.17.3
JsonPreprocessor : 0.12.0	requests : 2.32.5	wsproto : 1.3.2
jsonschema : 4.26.0	requests-kerberos : 0.15.0	
jsonschema-specifications : 2025.9.1	rich : 15.0.0	
krb5 : 0.10.0	robotframework : 7.4.2	
lxml : 6.1.3	robotframework-dbus : 0.2.0	
markdown-it-py : 4.2.0	robotframework-doip : 0.2.0	

Additionally required Python packages can be installed in this way:

1. Windows:

Use the following command in **Command Prompt**:

```
"%RobotPythonPath%\python.exe" -m pip install --proxy <proxy address> <packagename>
```

Or, in **Windows PowerShell**:

```
& "$env:RobotPythonPath\python.exe" -m pip install --proxy <proxy address> <packagename>
```

2. Linux:

```
"${RobotPythonPath}/python3" -m pip install --proxy <proxy address> <packagename>
```

The full path and name of the Python interpreter is required in these command lines because the **RobotFramework AIO** installer does not modify the environment of the computer (except the setup of some environment variables).

The proxy address is an option and depends on the conditions under which your company grants the access to the internet.